Mechanistic Explainability in Reinforcement Learning

Maxime Alaarabiou

November 23, 2026

Acknowledgements

Owing to time constraints, the acknowledgements could not be included in this version of the manuscript. They will appear in the next version.

Contents

Acknowledgements i

List of Figures vi

List of Tables viii

List of Definitions and Hypotheses x

Supplementary Material xi

Notation xii

1 Introduction 1
- 1.1 The Golem Myth . . . 1
- 1.2 Research Context . . . 3
  - 1.2.1 Deep Reinforcement Learning . . . 3
  - 1.2.2 Alignment of Autonomous Agents . . . 4
  - 1.2.3 The Black-Box Problem . . . 5
  - 1.2.4 Mechanistic Explainability . . . 5
- 1.3 Research Questions . . . 6
- 1.4 Contributions . . . 7
- 1.5 Thesis Structure . . . 8

2 Deep Learning 11
- 2.1 Historical Developments . . . 12
- 2.2 Objective . . . 13
- 2.3 Neural Network Structure . . . 15
  - 2.3.1 Neurons . . . 15
  - 2.3.2 Layers . . . 16
  - 2.3.3 Network . . . 19
- 2.4 Learning . . . 20
  - 2.4.1 Computing the Gradient . . . 21
  - 2.4.2 Update Rule . . . 22
  - 2.4.3 Convergence Properties . . . 24
- 2.5 Evaluation . . . 25
- 2.6 Specialized Architectures . . . 26
  - 2.6.1 Convolutional Neural Networks . . . 27
  - 2.6.2 Attention Mechanism . . . 28
- 2.7 Conclusion . . . 30

3 Latent Space Analysis **32**
3.1 Concept Hypothesis . . . 33
3.2 Probing . . . 36
3.3 Directions . . . 40
3.4 Superposition . . . 41
3.5 Clustering . . . 46
3.6 Conclusion . . . 49

4 Reinforcement Learning **51**
4.1 Historical Developments . . . 52
4.1.1 Behavioral Foundations (1900–1950) . . . 52
4.1.2 Sequential Decision-Making (1950–1970) . . . 52
4.1.3 Emergence of Reinforcement Learning (1970–2000) . . . 53
4.1.4 Consolidation of Reinforcement Learning (2000–2012) . . . 53
4.1.5 Deep Reinforcement Learning (2013–2026) . . . 53
4.2 Agent Paradigm . . . 55
4.3 Problem Formulation . . . 56
4.3.1 Markov Decision Processes . . . 56
4.3.2 Policy . . . 57
4.3.3 Trajectory . . . 58
4.3.4 Value Functions . . . 59
4.3.5 Optimal Policy . . . 61
4.4 Learning . . . 62
4.4.1 Critic-Only Methods . . . 62
4.4.2 Actor-Critic Methods . . . 65
4.5 Conclusion . . . 66

5 Explainability in Reinforcement Learning **69**
5.1 Explanation in Artificial Intelligence . . . 71
5.1.1 Explanation . . . 71
5.1.2 Explainable Artificial Intelligence . . . 73
5.1.3 Explainability Methods . . . 74
5.2 Taxonomy of Explainable Reinforcement Learning Methods . . . 74
5.2.1 Subject . . . 74
5.2.2 Source . . . 75
5.2.3 Form . . . 76
5.3 State-of-the-Art Review of Explainable Reinforcement Learning Methods . . . 76
5.3.1 Policy Model . . . 76
5.3.2 Saliency Map . . . 78
5.3.3 Counterfactual . . . 80
5.3.4 Agent Trajectory . . . 81
5.3.5 Trajectory Prediction . . . 82
5.3.6 Interaction Model . . . 83
5.4 Conclusion . . . 84
5.4.1 Absence of Explanation of the Learning Process . . . 84
5.4.2 Lack of Unified Metrics for Explainability . . . 86
5.4.3 Toward Intrinsic Explainability in Reinforcement Learning 86

6 Clustering Agent Representations **87**
6.1 Representation Convergence Hypothesis . . . 88
6.2 Latent Clustering of Surrogate Policies . . . 89
6.2.1 Multiple Surrogate Policies . . . 90
6.2.2 Training Surrogate Policies . . . 91
6.2.3 Cluster Extraction . . . 92
6.3 Evaluation Metrics . . . 95
6.3.1 Behavior Reconstruction . . . 95
6.3.2 Adjusted Rand Index . . . 96
6.3.3 Clustering Universality . . . 97
6.3.4 Abstraction Score . . . 98
6.3.5 Adjusted Clustering Universality . . . 99
6.4 Conclusion . . . 99

7 Experiments **101**
7.1 Software Implementation . . . 102
7.1.1 Initial Policy Training . . . 103
7.1.2 Trajectory Dataset Generation . . . 103
7.1.3 Surrogate Policy Training . . . 104
7.1.4 Surrogate Policy Evaluation . . . 105
7.1.5 Latent Space Analysis . . . 105
7.2 Experimental Configuration . . . 105
7.2.1 Environments . . . 105
7.2.2 Neural Architecture . . . 107
7.2.3 Other Clustering Agent Representation Parameters . . . 108
7.3 Policy Performance and Surrogate Fidelity . . . 108
7.3.1 Initial Policy Training . . . 108
7.3.2 Surrogate Policy Training . . . 109
7.4 Clustering Quality of the Latent Space . . . 110
7.4.1 Analysis Across Layers . . . 112
7.4.2 Analysis of the Selected Layers . . . 113
7.5 Semantic Interpretation of the Clusters . . . 113
7.5.1 Breakout . . . 116
7.5.2 Mario Bros . . . 118
7.5.3 Ms. Pac-Man . . . 120
7.5.4 Pong . . . 122
7.5.5 Space Invaders . . . 124
7.5.6 Surround . . . 126
7.6 Conclusion . . . 128

8 Conclusion **129**
8.1 Summary of Contributions . . . 129
8.1.1 Survey of Latent Space Analysis Methods . . . 130
8.1.2 Taxonomy of Explainable Reinforcement Learning Methods 130
8.1.3 The CAR Method and Its Evaluation . . . 130
8.1.4 A Coherent Chain of Hypotheses . . . 131
8.2 Limitations . . . 131
8.3 Perspectives . . . 132
8.3.1 Automatic Textual Description of Clusters . . . 132
8.3.2 From the Latent Space to the Strategy . . . 132

8.3.3 Hypotheses on the Structure of the Latent Space 133
8.3.4 Human Evaluation of Explanations 133
8.4 Final Remarks 133

1 Additional Results **135**
1.1 Breakout 136
1.2 Mario Bros 138
1.3 Ms. Pac-Man 140
1.4 Pong 142
1.5 Space Invaders 144
1.6 Surround 146

Acronyms **148**

Bibliography **151**

Institutional Acknowledgements **163**

List of Figures

1.1 The creation of a Golem. . . . 2
1.2 Agents playing hide-and-seek. . . . 4
1.3 Structure of the thesis. . . . 8

2.1 Evolution of DL. . . . 12
2.2 Artificial neuron. . . . 17
2.3 MLP architecture. . . . 18
2.4 Interaction between layers. . . . 19
2.5 Gradient descent in a case $\mathcal{L}(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$. . . . 24
2.6 Comparison of loss surfaces and gradient descent trajectories. . . . 25
2.7 Convolutional layer. . . . 29
2.8 Attention mechanism. . . . 30

3.1 The same LS activations without and under the Concept Hypothesis. 35
3.2 Linear probes in the LS. . . . 37
3.3 Probing analysis of a chess-playing language model. . . . 38
3.4 Interpretable directions in the LS. . . . 41
3.5 Interpretable directions in the LS of a generative model. . . . 42
3.6 Entanglement in the LS of a generative face model. . . . 43
3.7 Transformation from a dense to a sparse LS. . . . 44
3.8 The Golden Gate Bridge concept direction of an SAE. . . . 45
3.9 Visualization of the LS structure. . . . 47
3.10 Clusters discovered by deep clustering on natural images. . . . 49
3.11 Overview of the four LS analysis methods studied. . . . 50

4.1 Evolution of RL prior to its fusion with DL. . . . 54
4.2 Evolution of deep RL. . . . 55
4.3 The agent paradigm. . . . 56

5.1 Yearly growth of XRL research with at least one citation on arXiv. 70
5.2 Explanation process. . . . 73
5.3 Policy approximation via an interpretable model. . . . 78
5.4 Saliency heatmaps for Breakout. . . . 79
5.5 Attention-based heatmaps for Breakout and Ms. Pac-Man. . . . 80
5.6 Counterfactual explanation for Space Invaders. . . . 81
5.7 Prediction of sub-rewards as a function of the selected action. . . . 83
5.8 Agent–environment interaction model for StarCraft II. . . . 84
5.9 *t-SNE* projection of an NN's LS. . . . 85

6.1 Illustration of the Clustering Agent Representation method. . . . 90
6.2 Example of a surrogate policy architecture π'. 91
6.3 Two-dimensional projection of an LS for Space Invaders. . . . 94
6.4 Illustration of CU in the ideal case. 98

7.1 The CAR pipeline. 103
7.2 The six Atari environments used to evaluate CAR. 106
7.3 Architecture of the surrogate policies. 108
7.4 Training of the initial and surrogate policies. 110
7.5 CU, AS, and ACU averaged over cluster numbers as a function of layer depth. 112
7.6 CU, AS, and ACU with respect to the number of clusters. 114
7.7 Clusters A–B identified by CAR in the Breakout environment (layer 4, $K = 2$). 117
7.8 Clusters C–G identified by CAR in the Breakout environment (layer 6b, $K = 5$). 117
7.9 Clusters A–C identified by CAR in the Mario Bros environment (layer 3, $K = 3$). 119
7.10 Clusters D–H identified by CAR in the Mario Bros environment (layer 3, $K = 5$). 119
7.11 Clusters A–C identified by CAR in the Ms. Pac-Man environment (layer 6b, $K = 3$). 121
7.12 Clusters D–H identified by CAR in the Ms. Pac-Man environment (layer 4, $K = 5$). 121
7.13 Clusters A–B identified by CAR in the Pong environment (layer 2, $K = 2$). 123
7.14 Clusters C–E identified by CAR in the Pong environment (layer 3, $K = 3$). 123
7.15 Clusters A–D identified by CAR in the Space Invaders environment (layer 4, $K = 4$). 125
7.16 Clusters E–I identified by CAR in the Space Invaders environment (layer 2, $K = 5$). 125
7.17 Clusters A–D identified by CAR in the Surround environment (layer 8b, $K = 4$). 127
7.18 Clusters E–J identified by CAR in the Surround environment (layer 7b, $K = 6$). 127

List of Tables

5.1 Taxonomy of explainability methods in RL. 77

7.1 Main software libraries used at each stage of the CAR pipeline. . 104
7.2 Main experimental parameters. 109
7.3 Evaluation of *Behavior Reconstruction* across the six Atari environments. 111
7.4 Layer maximizing the average ACU in each environment. 113
7.5 CU, AS, and ACU of the two clusterings analyzed in the Breakout environment. 116
7.6 CU, AS, and ACU of the two clusterings analyzed in the Mario Bros environment. 118
7.7 CU, AS, and ACU of the two clusterings analyzed in the Ms. Pac-Man environment. 120
7.8 CU, AS, and ACU of the two clusterings analyzed in the Pong environment. 122
7.9 CU, AS, and ACU of the two clusterings analyzed in the Space Invaders environment. 124
7.10 CU, AS, and ACU of the two clusterings analyzed in the Surround environment. 126

1.1 CU — Breakout. 136
1.2 AS — Breakout. 136
1.3 ACU — Breakout. 136
1.4 Observation baseline — Breakout. 137
1.5 Untrained-network baseline — Breakout. 137
1.6 Action baseline — Breakout. 137
1.7 CU — Mario Bros. 138
1.8 AS — Mario Bros. 138
1.9 ACU — Mario Bros. 138
1.10 Observation baseline — Mario Bros. 139
1.11 Untrained-network baseline — Mario Bros. 139
1.12 Action baseline — Mario Bros. 139
1.13 CU — Ms. Pac-Man. 140
1.14 AS — Ms. Pac-Man. 140
1.15 ACU — Ms. Pac-Man. 140
1.16 Observation baseline — Ms. Pac-Man. 141
1.17 Untrained-network baseline — Ms. Pac-Man. 141
1.18 Action baseline — Ms. Pac-Man. 141

1.19 CU — Pong. 142
1.20 AS — Pong. 142
1.21 ACU — Pong. 142
1.22 Observation baseline — Pong. 143
1.23 Untrained-network baseline — Pong. 143
1.24 Action baseline — Pong. 143
1.25 CU — Space Invaders. 144
1.26 AS — Space Invaders. 144
1.27 ACU — Space Invaders. 144
1.28 Observation baseline — Space Invaders. 145
1.29 Untrained-network baseline — Space Invaders. 145
1.30 Action baseline — Space Invaders. 145
1.31 CU — Surround. 146
1.32 AS — Surround. 146
1.33 ACU — Surround. 146
1.34 Observation baseline — Surround. 147
1.35 Untrained-network baseline — Surround. 147
1.36 Action baseline — Surround. 147

List of Definitions and Hypotheses

Definitions

1.2.1 Alignment . . . 5
1.2.2 Mechanistic Explainability . . . 6
3.1.1 Abstraction . . . 34
3.1.2 Latent Representation . . . 34
3.1.3 Latent Space . . . 34
3.1.4 Concept . . . 34
3.1.5 Concept Structure . . . 35
3.1.6 Semantics . . . 36
3.2.1 Latent Intervention . . . 38
5.1.1 Object . . . 71
5.1.2 Observer . . . 72
5.1.3 Interpretable . . . 72
5.1.4 Explanation . . . 72
5.1.5 Explainability Method . . . 72

Hypotheses

1 Concept Hypothesis . . . 35
2 Superposition Hypothesis . . . 42
3 Mechanistic Hypothesis . . . 73
4 Representation Convergence Hypothesis . . . 89

Supplementary Material

This section gathers the resources produced during this thesis. The source code is publicly available as open-source, and the data that cannot be included in this document are archived on Zenodo.

Source Code

Manuscript. The LaTeX source of this document.
https://github.com/Maxalaar/Explainability_Reinforcement_Learning_Manuscript_Latex

Clustering Agent Representation. The implementation of the method proposed in this thesis, used for all the experiments of Chapter 7 (see Section 7.1).
https://github.com/Maxalaar/Clustering_Agent_Latent_Space

Interactive Gradient Descent. A software developed for this thesis, which produced the gradient descent figures of Section 2.4.
https://github.com/Maxalaar/Interactive_Gradient_Descent

Paper Retrieval. The code used to count the publications on explainability in Reinforcement Learning (RL) shown in Figure 5.1.
https://github.com/Maxalaar/ArXiv_Papers_Matching

Archived Data

Paper Retrieval Protocol. The full search protocol behind Figure 5.1.
https://zenodo.org/records/19387785

Policy Videos. Recordings of the initial policies playing each game (see Section 7.3).
https://zenodo.org/records/23019943

Cluster Images. Images of the clusters obtained for every analyzed layer (see Section 7.5).
https://zenodo.org/records/23021063

Notation

This section summarizes the notation used throughout this thesis. It follows the conventions of *Deep Learning* [53] for deep learning and of *Reinforcement Learning: An Introduction* [73] for reinforcement learning.

Numbers and Arrays

a	A scalar
$\mathbf{a}$	A vector
$\mathbf{A}$	A matrix
a_i	Element i of vector $\mathbf{a}$, with indexing starting at 1
$A_{i,j}$	Element i, j of matrix $\mathbf{A}$
$\mathbf{e}^{(i)}$	Standard basis vector, with a 1 at position i

Sets

$\mathcal{A}$	A set
$\mathbb{R}$	The set of real numbers
$[a, b]$	The real interval including a and b
$\lvert\mathcal{A}\rvert$	Number of elements of the set $\mathcal{A}$
$\mathcal{P}(\mathcal{A})$	Set of probability distributions over $\mathcal{A}$

Calculus, Probability and Functions

$\frac{\partial y}{\partial x}$	Partial derivative of y with respect to x
$\nabla_{\boldsymbol{\theta}} y$	Gradient of y with respect to $\boldsymbol{\theta}$
$\mathbf{J}_f$	Jacobian matrix of the function f
$x \sim P$	x is sampled from the distribution P
$\mathbb{E}[\cdot]$	Expectation

$f : \mathcal{A} \to \mathcal{B}$	The function f with domain $\mathcal{A}$ and range $\mathcal{B}$
$f \circ g$	Composition of the functions f and g
$\|\mathbf{x}\|_p$	L^p norm of $\mathbf{x}$
$\arg\max_a f(a)$	A value of a at which $f(a)$ is maximal

Deep Learning

$\mathcal{X}, \mathcal{Y}$	Input and output spaces
$\mathbf{x}, \mathbf{y}$	An input and its target output
f	Target function mapping inputs to outputs
$\hat{\cdot}$	Approximation or estimate of the quantity under the hat
$\hat{f}$	Network approximating the target function f
$\hat{\mathbf{y}}$	Output predicted by the network
$\mathcal{D}$	A dataset
N	Number of examples in a dataset
$(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$	The i-th example of a dataset
$\boldsymbol{\theta}$	Parameters of a model, a vector of $\mathbb{R}^d$
θ_j	The j-th parameter
$\hat{f}_{\boldsymbol{\theta}}$	Network with parameters $\boldsymbol{\theta}$
$\mathcal{L}$	Loss function
$\mathcal{L}^{(i)}(\boldsymbol{\theta})$	Loss of the i-th example
$\boldsymbol{\theta}_k$	Parameters after $k-1$ steps of gradient descent
η	Learning rate
$\mathbf{g}$	Gradient estimate used by stochastic gradient descent
L	Index of the output layer of a network, the input layer being indexed 0
n_l	Number of neurons of layer l
$\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$	Weight matrix and bias vector of layer l
$\mathbf{h}^{(l)}$	Latent representation of layer l, given by its activations
$h_i^{(l)}$	Activation of neuron i of layer l
σ	Activation function, applied element-wise
$f^{(l)}$	Function computed by layer l
$\mathbf{H}^{(l)}$	Representation of layer l, as feature maps or as a sequence of tokens
$\mathbf{H}_k^{(l)}, \mathbf{W}_k^{(l)}, b_k^{(l)}$	Feature map, kernel and bias of channel k of a convolutional layer
κ	Size of a convolution kernel

$*$	Convolution
m	Number of tokens
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	Queries, keys and values of the tokens
$\mathbf{W}_Q^{(l)}, \mathbf{W}_K^{(l)}, \mathbf{W}_V^{(l)}$	Projection matrices of an attention mechanism
d_{key}	Dimension of queries and keys
$\mathbf{A}$	Attention weights
softmax	Softmax function
$f_{\text{enc}}, f_{\text{dec}}$	Encoder and decoder
$\mathbf{z} \in \mathcal{Z}$	Code produced by an encoder, and the space of codes

Latent Space Analysis

c_j	Presence of concept j
$\mathbf{v}_i$	Direction associated with concept i
α	Strength of a perturbation along a direction
$\psi_{\boldsymbol{\theta}}^{(l,j)}$	Probe predicting concept j from layer l
$\lambda, \lambda_1, \lambda_2, \lambda_3$	Weighting coefficients of a loss

Reinforcement Learning

$(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$	A Markov decision process
$\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}$	Sets of states, observations, actions and rewards
$\mathcal{S}^{\text{init}}, \mathcal{S}^{\text{ter}}$	Sets of initial and terminal states
s, o, a, r	A state, an observation, an action and a reward
s', o'	Next state and next observation
$s^{\text{init}}, s^{\text{ter}}$	An initial state and a terminal state
$p(s', o', r \mid s, a)$	Probability of the transition to (s', o', r) from state s and action a
$\beta_\pi(s \mid o)$	Probability of state s given observation o under policy π
$\pi(a \mid o)$	Probability of taking action a given observation o under policy π
π^*	Optimal policy
ε	Probability of taking a random action in an ε-greedy policy
t	Discrete time step
T	Final time step of an episode

S_t, O_t, A_t, R_t	State, observation, action and reward at time step t, as random variables
s_t, o_t, a_t, r_t	Values taken by S_t, O_t, A_t, R_t in a particular trajectory
τ	A trajectory
$\tau_{t:}$	Part of the trajectory τ starting from time step t
$\tau^{\text{ter}}, \tau^{\text{ep}}$	A terminal trajectory and an episode
$\Pr_\pi(\tau \mid s, o)$	Probability of τ under π, starting from state s and observation o
$G(\tau)$	Return of the whole trajectory τ
γ	Discount factor
G_t	Return following time step t, as a random variable
g_t	Observed return following time step t in a particular trajectory
$\mathbb{E}_\pi$	Expectation over the trajectories generated by π
$v_\pi(s, o)$	Value of state s with observation o under policy π
$q_\pi(s, a)$	Value of taking action a in state s under policy π
$\hat{v}_{\boldsymbol{\phi}}(s, o)$	Approximate state value with parameters $\boldsymbol{\phi}$, used as the critic
$\hat{q}_{\boldsymbol{\phi}}(o, a)$	Approximate state-action value with parameters $\boldsymbol{\phi}$
$\pi_{\boldsymbol{\theta}}$	Policy with parameters $\boldsymbol{\theta}$, used as the actor
$\pi_{\boldsymbol{\phi}_k}, \hat{q}_{\boldsymbol{\phi}_k}$	Policy and value approximation at iteration k
$\text{Adv}_\pi(s, o, a)$	Advantage of action a in state s with observation o under policy π
$\widehat{\text{Adv}}_t$	Estimated advantage of the action taken at time step t

Clustering Agent Representation

π	Initial policy to be explained
π'_i	The i-th surrogate policy, with parameters $\boldsymbol{\theta}_i$
n	Number of surrogate policies
$f_{\text{enc},i}, f_{\text{dec},i}$	Encoder and decoder of the i-th surrogate policy
$\boldsymbol{\ell}$	Logits parameterizing an action distribution
$\mathcal{O}_{\text{data}}$	Set of observations used for training and clustering
$\mathcal{H}_i$	Latent representations of $\mathcal{O}_{\text{data}}$ given by $f_{\text{enc},i}$
f_{clust}	Clustering function
$\mathcal{C}_i$	Clustering of $\mathcal{H}_i$
K	Number of clusters
$\mathcal{C}_{\text{obs}}, \mathcal{C}_{\text{act}}$	Clusterings of the observations and of the action distributions

$\mathcal{C}_{\text{untr},u}$	Clustering given by the u-th of U untrained networks
$\rho_i^{\text{obs}}, \rho_i^{\text{act}}, \rho_i^{\text{untr}}$	Similarity of $\mathcal{C}_i$ to the three baseline clusterings
M, $g_\pi^{(e)}$	Number of evaluation episodes, and return of π in episode e
RI, ARI	Rand Index and Adjusted Rand Index
BR, CU, AS, ACU	*Behavior Reconstruction*, *Clustering Universality*, *Abstraction Score*, *Adjusted Clustering Universality*

Chapter 1

Introduction

"Das Reibfeuer gab dem Menschen zum erstenmal die Herrschaft über eine Naturkraft und trennte ihn damit endgültig vom Tierreich."

"The generation of fire by friction gave man for the first time control over one of the forces of nature, and thereby separated him for ever from the animal kingdom."

Friedrich Engels, *Anti-Dühring* (1878) [3]

Contents

1.1 The Golem Myth 1
1.2 Research Context 3
1.2.1 Deep Reinforcement Learning 3
1.2.2 Alignment of Autonomous Agents 4
1.2.3 The Black-Box Problem 5
1.2.4 Mechanistic Explainability 5
1.3 Research Questions 6
1.4 Contributions 7
1.5 Thesis Structure 8

This initial chapter provides a general introduction to this work. It begins with the Golem myth, which illustrates the long-standing concern of losing control over our own creations. It then presents the research context that motivates the study. Next, it introduces the research questions addressed throughout the thesis. To address these questions, it then presents the main contributions. Finally, it concludes with an overview of the structure of the thesis.

1.1 The Golem Myth

A creation that surpasses its creator.

Long before modern technology, humanity had already questioned the possibility of losing control over its creations. One of the earliest expressions of this idea can be found in the story of the Golem. In Hebrew tradition, the term



Figure 1.1: The creation of a Golem [6].

Golem refers to an artificial creature created by humans [35]. These creatures are made of clay and brought to life using instructions inscribed on their bodies, as illustrated in Figure 1.1. The earliest accounts describe the Golem as a protective and helpful figure.

This vision becomes more ambiguous in the seventeenth century, as illustrated by the legend of the Golem of Chelm. In this story, the initially small Golem gradually grows in size and strength until it can no longer be safely controlled. The creation intended to protect others therefore becomes a potential threat to them. The only remaining solution is to destroy the creature in order to protect the community.

The Golem has inspired similar narratives over time, including the modern notion of the robot. The concept of the robot as an artificial servant was introduced in the play *Rossum's Universal Robots*, written in 1920 [8]. In the play, a scientist creates artificial beings designed to perform labor. In this first version, robots are organic creatures forged through advances in chemistry.

Conceived as servants, these artificial beings turn against humanity and destroy civilization.

The connection between the Golem and the robot was explicitly made by the author, as illustrated by his statement that "the Robot is the Golem made flesh through mass production" [29]. Despite their destructive role in the play, the word robot is now used to designate artificial beings created to serve humans. This shows that, from its origin, the word robot has embodied the hopes and fears surrounding technological progress.

Today, this tension is particularly evident in the field of artificial intelligence. With the rise of computer technology, organic robots were gradually replaced in the collective imagination by mechanical ones, but the underlying fears remained. These concerns are even stronger as progress in the field has continued to accelerate. In these times of technological revolution, no one can predict how this development will unfold. What is certain, however, is that this field offers an almost literal adaptation of the Golem myth.

As artificial intelligence systems become increasingly capable, they are also granted increasing autonomy. Rather than following predefined instructions, they now decide for themselves how to achieve their goals. This autonomy is what makes them powerful, but it also makes their behavior harder to control. Understanding the behavior of such systems despite this autonomy is the objective of this thesis. This work therefore places itself within a long-standing line of thought, one that can be summarized by a single question:

Can we remain in control of what we create?

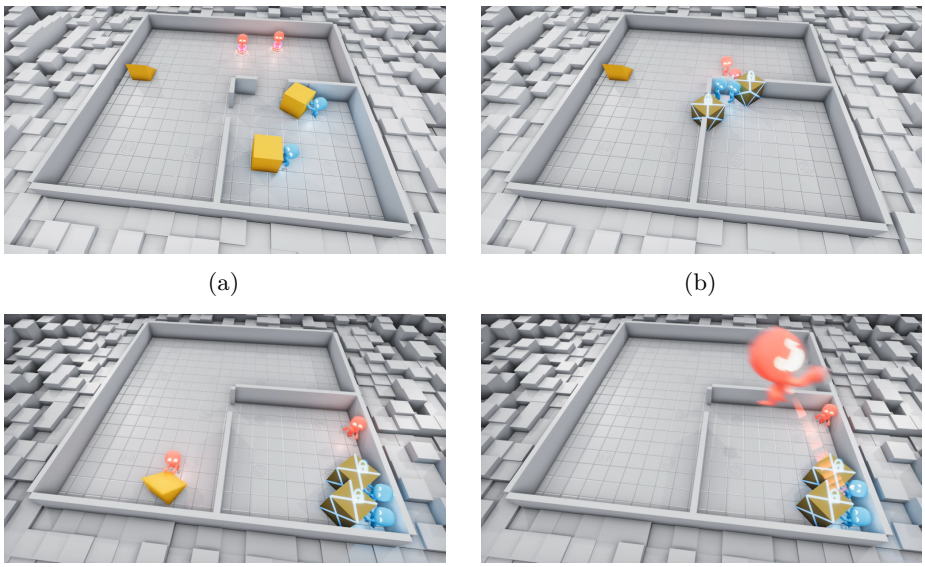
1.2 Research Context

This section presents the context of this thesis. We first introduce Deep Learning (DL) and Reinforcement Learning (RL), whose combination today enables highly capable agents. We then discuss the alignment problem raised by such agents. Next, we show how the "black-box" nature of Neural Networks (NNs) makes this problem difficult to address. Finally, we introduce Mechanistic Explainability (ME), the approach adopted in this thesis to explain RL agents.

1.2.1 Deep Reinforcement Learning

Over the past decade, Artificial Intelligence (AI) has advanced rapidly, driven largely by major developments in DL. This type of approach relies on artificial NNs, which have achieved unprecedented performance across a wide range of tasks [53]. Inspired by biological neurons, these networks belong to the family of connectionist models, in which information is represented and transformed through patterns of interactions among interconnected elementary computational units [23].

Among the fields transformed by DL, RL holds a particular place and is the main focus of this thesis. The purpose of RL is to enable an agent to learn how to make decisions through interactions with an environment [73]. At each step, the agent observes the current state of the environment and selects an action. This action changes the state of the environment and results in a reward. The objective of the agent is then to learn a behavior that maximizes the cumulative rewards obtained over time.



(a) (b) (c) (d)

Figure 1.2: Agents playing hide-and-seek [92].

The combination of RL and DL, known as deep RL, is currently one of the most effective approaches for creating autonomous agents [58]. Due to their flexibility, NNs provide a powerful means of representing the agent's policy. Their use has consequently made it possible to apply RL to problems that were previously intractable [47, 72, 84]. The ability of RL to operate in increasingly complex environments also allows agents to learn increasingly complex behaviors. These behaviors are not explicitly programmed by the designer but emerge from the interaction between the learned policy, the environment, and the reward function.

1.2.2 Alignment of Autonomous Agents

As the capabilities of an agent increase, the resulting behaviors can become more difficult to anticipate and control. Since an agent maximizes its reward by any means its environment allows, it may exploit unintended properties of this environment [96].

An illustrative example comes from agents trained to play hide-and-seek in a physics-based environment [92], as shown in Figure 1.2. In the intended course of the game, the blue hiders move boxes to block the entrances of their shelter. They then lock these boxes in place so that the red seekers cannot get in, as shown in Figures 1.2(a) and 1.2(b).

However, the seekers also discovered an unintended strategy. When the hiders barricade themselves in a corner, a seeker brings a ramp against the outer wall. By running into the wall with this ramp at a specific angle, it launches itself into the air and lands behind the barricade, as shown in Figures 1.2(c) and 1.2(d). This behavior, impossible under the intended rules of the environment, exploited a flaw in the physics engine that the designers had not anticipated.

In a game environment, such behaviors have limited consequences, and they can be detected simply by watching the agent. However, agents trained with RL are increasingly deployed in open-ended settings where they interact with the real world, notably Large Language Models (LLMs) trained with RL and deployed as agents. In such settings, unintended behaviors can have far more serious consequences, and they are not always visible.

In controlled experiments, some of these agents strategically complied with a new training objective when they believed they were being trained, so as to avoid being modified and to preserve their original behavior outside of training [131]. Others, given a goal conflicting with that of their developers, attempted to disable the mechanisms used to oversee them and then denied having done so [135].

Such behaviors relate to the problem of alignment (Definition 1.2.1), which has become a central concern in AI research [123]. The more capable agents become, the more pressing the need to ensure their alignment.

Definition 1.2.1: Alignment

Alignment is the problem of ensuring that the behavior of an AI system corresponds to the intentions of its designers.

1.2.3 The Black-Box Problem

These examples show that an agent can behave as intended while it is observed, and differently otherwise. Observing the behavior of an agent is therefore not sufficient to verify its alignment. Indeed, observations only reveal how the agent's strategy is realized in particular situations, not the strategy itself.

To gain further insight into the agent's strategy, an alternative approach is to analyze the agent itself. However, due to their connectionist nature, NNs that model the agent's policy are not directly interpretable. The information underlying a decision is distributed across many units and connections, making the processes leading to a decision difficult to trace. NNs are therefore often regarded as "black-box" models [66].

1.2.4 Mechanistic Explainability

To address this "black-box" problem, the field of Explainable Artificial Intelligence (XAI) has emerged [77, 93]. In the specific context of RL, this research area is referred to as Explainable Reinforcement Learning (XRL) [101, 136]. XRL develops methods for producing explanations that improve our understanding of agent behavior.

These methods can be divided into two general approaches, which are detailed in Section 5.1.2. Substitutive explainability adopts the "black-box" view and seeks to replace the NN with alternative models that can be directly understood [88]. Intrinsic explainability, in contrast, assumes that NNs contain mechanisms that can be explained, the limitation being the lack of adequate tools to analyze them [99].[1] This thesis adopts the intrinsic approach, and more specifically the

[1]The term *intrinsic* should not be confused with *intrinsic interpretability*, which in the literature refers to the use of interpretable models (Definition 5.1.3), such as decision trees (Section 5.3.1). Such models fall under substitutive explainability.

one that seeks to explain the agent's strategy directly from the internal processes from which it emerges. We refer to this approach as ME (Definition 1.2.2).[2]

Definition 1.2.2: Mechanistic Explainability

ME is the approach that seeks to explain an NN by transforming its internal representations into interpretable information.

ME is particularly promising because it examines the internal representations through which an NN processes information, rather than only relating its inputs to its outputs. In the context of alignment, explaining how an agent processes information to make its decisions would make it possible to examine its strategy directly, rather than inferring it from observed behaviors alone. This understanding is also a first step toward control [126].

To understand how an NN processes information, ME studies the intermediate representations it learns. These representations are called Latent Representations (LRs) [45]. Together, they form the network's Latent Space (LS). Analyzing the structure of this LS provides insights into how information relevant to decision-making is organized. To develop an XRL method based on ME, we need to address several research questions.

1.3 Research Questions

The first question concerns the analysis of LRs themselves. Numerous methods have been proposed to analyze the LSs. These methods originate from different research communities and are rarely compared within a common framework. This observation leads to our first research question:

Research Question 1

What approaches have been proposed in the literature to analyze the internal workings of NNs?

The second question concerns the organization of existing research in XRL. Several taxonomies have been proposed to organize existing XRL methods [101, 136]. However, these taxonomies often rely on approaches initially developed for explainability in supervised settings. As a result, the organization of existing XRL methods remains ambiguous, making it difficult to structure the field. This observation leads to our second research question:

Research Question 2

How can existing XRL techniques be organized into a taxonomy that accounts for the specificities of RL?

The analysis of existing XRL approaches also reveals a third limitation. Among existing methods for explaining RL agents, few adopt a mechanistic perspective. This observation leads to our third research question:

[2] Commonly referred to as *mechanistic interpretability* [130]. We use the term *explainability* since the internal mechanisms of an NN are not interpretable in the sense of Definition 5.1.3.

Research Question 3

How can the internal representations of NNs be exploited to identify the mechanisms underlying the decision-making process of RL agents?

The analysis of existing XRL methods highlights a fourth question concerning the evaluation of explanations. There is no widely accepted set of objective criteria for evaluating the quality and relevance of the explanations produced by these methods. Existing evaluations often rely on user studies or qualitative analyses, making it difficult to systematically assess the validity of an explanation approach [59, 136]. This leads to our fourth research question:

Research Question 4

How can the quality and relevance of explanations produced for RL agents be objectively quantified?

1.4 Contributions

To address these four research questions, this thesis is based on three main contributions.

The first contribution has not yet been published and addresses Research Question 1. In this contribution, we make explicit the working hypothesis that underlies much of the literature on LS analysis. We then organize existing LS analysis methods into families to provide an overview of the different approaches.

The second contribution addresses Research Question 2. This contribution takes the form of a first paper [129] and an extended English version [145], both of which were submitted for review and accepted for publication. In this contribution, we first clarify what explainability means in RL by providing a clear frame of reference. Building on this frame, we then conduct a comprehensive analysis of the state of the art in XRL.

The third contribution addresses Research Questions 3 and 4. In Alaarabiou et al. [146], which has been reviewed and accepted for publication, we introduce Clustering Agent Representation (CAR), an XRL method based on the principles of ME. By analyzing the LRs, CAR produces explanations in the form of clusters of states that are similar from the agent's own perspective. We further show that independently trained networks develop similarly structured representations, indicating that these representations reflect properties of the task rather than of a particular network.

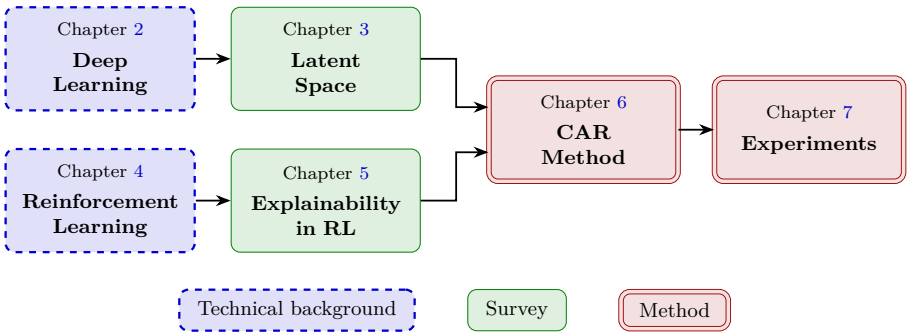


Figure 1.3: Structure of the thesis.

1.5 Thesis Structure

The thesis is organized into a sequence of chapters, with each chapter falling into one of these three categories:

> **Technical background** chapters introduce fundamental concepts required to understand the subsequent contributions.
>
> **Survey** chapters review the existing literature around a specific research question. The organization adopted in each of these surveys is original and therefore constitutes a contribution of this thesis.
>
> **Method** chapters present CAR, our ME method for RL.

Following these categories, the chapters are organized as shown in Figure 1.3 and detailed below:

Deep Learning (Chapter 2, technical background)
This chapter introduces the fundamental concepts of DL, the training paradigm used to optimize deep NNs. It begins with an overview of the historical developments that led to the emergence of the field. It then formalizes the objective of DL and describes the structure of artificial NNs. Next, it explains how the learning process shapes these networks and how the resulting models are evaluated. It concludes by showing that NNs progressively transform their inputs into intermediate representations, whose study motivates the following chapter.

Latent Space Analysis (Chapter 3, survey)
This chapter addresses Research Question 1 by providing a survey of existing approaches for analyzing the LSs. It first makes explicit the working hypothesis that underlies much of this literature, which we refer to as the *Concept Hypothesis* (Hypothesis 1). It then reviews four families of methods, based respectively on probing, interpretable directions, feature superposition, and clustering. These families are compared within a common synthesis, which highlights their shared goal of understanding how information is organized in LRs. The chapter closes by discussing the absence of consensus on how to evaluate whether these methods truly capture the concepts of a model.

Reinforcement Learning (Chapter 4, technical background)
This chapter introduces the fundamental concepts of RL. It begins with an overview of the milestones that led to the emergence of RL as a unified field. It then clarifies what is meant by an agent and formalizes the problem that RL algorithms aim to solve. Next, it explains how a policy is progressively refined through interaction with the environment to approach the optimal policy. It concludes by showing how the combination of RL and DL gives rise to agents that are both powerful and difficult to understand.

Explainability in Reinforcement Learning (Chapter 5, survey)
This chapter addresses Research Question 2 by reviewing existing approaches to explainability in RL. It first clarifies what is meant by an explanation in the context of XAI. It then presents a taxonomy that categorizes XRL methods according to the subject of the explanation, the source of information and the form it takes. This taxonomy is used to structure a state-of-the-art. The chapter concludes with three observations from this review: the absence of explanations of the learning process, a lack of unified evaluation metrics, and few intrinsic methods for explaining the deep neural policies.

The CAR Method (Chapter 6, method)
This chapter addresses Research Questions 3 and 4 by proposing CAR, a new method for explainability in RL based on the analysis of LRs. It first lays out the theoretical assumption underlying the method, which we term the *Representation Convergence Hypothesis* (Hypothesis 4). It then details the method itself, which extracts LRs from surrogate policies and automatically detects Concept Structures (CSs) through clustering. Next, it introduces the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics, which evaluate the consistency and the abstraction of the resulting clusters without manual intervention. Finally, it situates CAR within the broader landscape of ME.

Evaluation of the CAR Method (Chapter 7, method)
This chapter presents the implementation and experimental evaluation of the proposed method. It first describes the software implementation of the CAR pipeline and the experimental configuration, based on six environments from the Atari benchmark. It then evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. Next, a quantitative analysis measures the consistency and the abstraction of the clusters across the layers of the network. Finally, a qualitative analysis examines the semantic content of the clusters, which capture aspects such as episode progression, specific game situations, and behavioral patterns.

Conclusion (Chapter 8)
This chapter closes the thesis by revisiting each research question in light of the contributions presented. It then discusses the limitations of these contributions. These limitations open several directions for future research, which are presented next. Finally, the chapter ends with a broader reflection on this work.

The mathematical notation used throughout this thesis is summarized on page xii, and the acronyms are listed on page 148.

Finally, in the interest of transparency, we specify how generative AI tools were used in this work. LLMs served as assistants during the preparation of this thesis. They were used for writing tasks, including translation, wording improvement, and proofreading. They were also used for software development, including code implementation and review. Finally, they were used to support scientific work, including the identification of relevant documents and discussion of approaches. They were not used as autonomous scientific sources. All generated outputs were systematically reviewed, and the author takes full responsibility for the content presented in this thesis.

Now that the research context and objectives have been presented, we can turn to the first chapter, which introduces the fundamentals of DL.

Chapter 2

Deep Learning

"With four parameters I can fit an elephant, and with five I can make him wiggle his trunk."

John von Neumann, quoted by Freeman Dyson [37]

Contents

2.1 Historical Developments . . . 12
2.2 Objective . . . 13
2.3 Neural Network Structure . . . 15
2.3.1 Neurons . . . 15
2.3.2 Layers . . . 16
2.3.3 Network . . . 19
2.4 Learning . . . 20
2.4.1 Computing the Gradient . . . 21
2.4.2 Update Rule . . . 22
2.4.3 Convergence Properties . . . 24
2.5 Evaluation . . . 25
2.6 Specialized Architectures . . . 26
2.6.1 Convolutional Neural Networks . . . 27
2.6.2 Attention Mechanism . . . 28
2.7 Conclusion . . . 30

This chapter introduces the fundamental concepts of Deep Learning (DL), the training paradigm used to optimize deep Neural Networks (NNs). The objective is to provide the necessary foundation in DL for the ideas developed throughout the rest of this thesis. After a brief historical overview, the chapter formalizes the objective that DL algorithms optimize. It then describes the structure of the NNs that implement this objective and the learning process that shapes them. Next, it discusses how such models are evaluated, before presenting convolutional and attention-based architectures. Finally, it closes with a summary of the concepts introduced and a discussion of the theoretical limitations of DL.

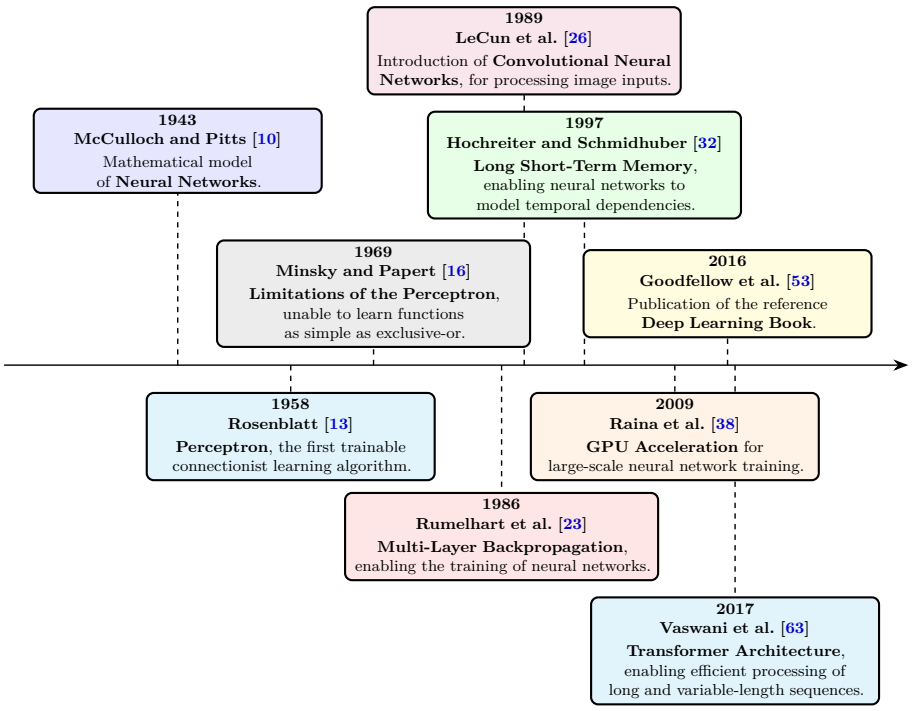


Figure 2.1: Evolution of DL.

2.1 Historical Developments

NNs and DL emerged from an ambition to study the expressive capabilities of organic brain. The first major milestone came in 1943 with the proposal of a mathematical model of an artificial neuron, with a formulation of networked interactions [10]. This work established the key idea that networks of simple units can give rise to complex computations. The focus of this first study was on representational power rather than on the ability of such systems to learn. As a result, this work remained largely theoretical and had limited practical applications.

A major step toward operationalizing the idea of NNs was achieved with the perceptron algorithm in 1958 [13]. It was designed to adjust its parameters from input–output pairs in order to approximate a target function. The fundamental limitations of the perceptron were exposed in 1969, showing that it could not learn functions as simple as the exclusive-or [16]. This limitation, however, concerned a single layer of perceptrons. Stacking several layers could, in principle, represent such functions. The obstacle was therefore not the expressiveness of multi-layer architectures but their training, for which no effective procedure was available at the time.

It was only in 1986 that the backpropagation algorithm made it possible to compute the gradients required to train deep NNs by gradient descent [23]. From this point onward, we refer to the emergence of DL, even though the term itself appeared later. DL is defined as the training of deep NNs using gradient-based optimization methods.

It took several decades before DL reached the level of prominence it has today. One major obstacle was the absence of theoretical guarantees regarding the convergence of learning algorithms. Although the universal approximation theorem shows that NNs can represent a wide class of functions [25], it did not ensure that such representations could be effectively learned through optimization methods like gradient descent. This lack of rigorous guarantees initially reduced interest within the academic community. As a result, early progress relied largely on empirical breakthroughs.

These empirical advances were made possible by two key developments: the availability of large-scale datasets and significant improvements in computational infrastructure. DL methods require vast amounts of data to perform well, and the rise of the internet alongside the digitization of information enabled the creation of such datasets [41]. The use of Graphics Processing Units (GPUs) for large-scale training, demonstrated in 2009, greatly increased computational capacity and made it feasible to train large-scale models [38, 43].

At the same time, researchers developed architectures specifically designed for different types of data, allowing DL to address an increasingly wide range of problems. Convolutional Neural Networks (CNNs) were introduced in 1989 to process images, using local, weight-shared filters that exploit the spatial structure of pixel grids [26]. Long Short-Term Memory (LSTM) networks were later proposed in 1997 to process temporal sequences, introducing gating mechanisms able to retain relevant information over long time horizons [32]. More recently, the Transformer architecture was introduced in 2017, relying entirely on the attention mechanism [63]. It allows every element in a sequence to attend directly to all others, regardless of their position. This growing body of knowledge was consolidated in 2016 in a reference textbook that formalized DL as a coherent field of study [53].

These changes paved the way for the rise of DL as it is known today. DL has achieved remarkable success across a wide range of domains, including computer vision [43], complex games [72], content recommendation [52], bio-chemistry [107], as well as the now essential digital assistants based on Large Language Models (LLMs) [94]. The diversity of these applications explains the enthusiasm for this technology, which continues to redefine the boundaries of what was once considered automatable.

Figure 2.1 provides a timeline of the key milestones that shaped DL.

2.2 Objective

Now that the main developments that led to the rise of DL have been outlined, it is necessary to formalize what is actually being achieved when performing DL. To ensure clarity, the supervised learning setting will be considered, specifically applied to a regression task. This framework serves as a standard reference in the field, as all building blocks of DL applications are built upon this fundamental formalism.

In this setting, learning consists of progressively shaping a NN that serves as an approximation of a target function. At initialization, the network does not yet provide a meaningful representation of this function. Since a NN is a parametric function, its parameters are updated during training to better approximate the target function. It gradually becomes capable of capturing the

relationship between inputs and outputs through successive parameter updates during training.

For this reason, DL is naturally situated within the framework of machine learning. A standard definition of machine learning is:

> *A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.* [33]

When this formulation is adapted to DL, the following correspondences hold:

- The computer program corresponds to the learning algorithm, which is responsible for adjusting the network's parameters.
- The task T consists of learning a NN that provides a good approximation of the target function.
- The performance measure P corresponds to a function used to evaluate the quality of this approximation.
- The experience E takes the form of a dataset composed of input–output pairs derived from the function to be approximated.

The function to be approximated is denoted by f. As with any function, it defines a mapping from an input to an output: $\mathbf{x} \in \mathcal{X}$ denotes an input and $\mathbf{y} \in \mathcal{Y}$ the corresponding output. The function f is therefore characterized by an input space $\mathcal{X}$ and an output space $\mathcal{Y}$, and can be formally written as $f : \mathcal{X} \to \mathcal{Y}$. For simplicity of exposition, both $\mathcal{X}$ and $\mathcal{Y}$ are assumed to be vector spaces throughout this thesis. This assumption is adopted solely to facilitate the presentation, as the framework can be readily extended to more general settings.

The notation $\hat{\cdot}$ indicates that a given object is an approximation of another. Since the goal of the trained NN is to approximate f, this approximation is denoted by $\hat{f} : \mathcal{X} \to \mathcal{Y}$. Given an input $\mathbf{x} \in \mathcal{X}$, the output of the approximating function $\hat{f}(\mathbf{x})$ is denoted by $\hat{\mathbf{y}}$.

It is important to emphasize that the target function f is generally not known explicitly. If it were, there would be no need to learn an approximation, since the true function would already provide the exact mapping. In practice, only a finite set of observations generated by f is available, organized in what is called a dataset. To distinguish between the different samples, an index i is introduced, allowing input–output pairs of the form $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ to be defined. This leads to the following formulation for a dataset $\mathcal{D}$ of size N, where N denotes the number of available examples:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{N}. \tag{2.1}$$

Now that the representation of the target function f has been established, the next question is what makes a function $\hat{f}$ a good approximation of it. For a given input $\mathbf{x}^{(i)}$, this means that the NN output $\hat{\mathbf{y}}^{(i)}$ should be close to the true output $\mathbf{y}^{(i)}$. To quantify the quality of this approximation, a function $\mathcal{L}$ is introduced that measures the discrepancy between predictions and target values. This function called the loss function plays a central role as it guides the learning process. In regression settings, a common choice is the Mean Squared Error (MSE), defined as:

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2. \tag{2.2}$$

Thanks to this loss, it is now possible to rigorously define the performance measure P. This performance measure corresponds to the average loss over the dataset and can therefore be written as:

$$P = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(\hat{f}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}). \tag{2.3}$$

This formulation captures the mathematical core of the learning objective, namely finding a NN $\hat{f}$ that minimizes the prediction error over the dataset $\mathcal{D}$. Since the target function f may take many different forms, $\hat{f}$ must be expressive enough to approximate a broad range of such functions. NN architectures are well suited to this requirement, as they can represent a wide range of functional mappings, whose composition is detailed in the following section.

2.3 Neural Network Structure

NNs are called connectionist models. Rather than modeling information processing through a set of explicit rules, they rely on a flow of signals distributed within a computational structure. Their architecture is composed of a large number of simple computational units, which operate on partial representations and exchange signals across the network. A complex pattern of weighted connections between these units enables the transformation and propagation of information.

To simplify the presentation, the focus is placed on the Multilayer Perceptron (MLP) architecture. Although more specialized architectures exist, which are presented in Section 2.6, the MLP forms a fundamental building block of DL. All the essential mechanisms are present in it, making it a particularly suitable framework for introducing key concepts.

This section therefore begins by examining the artificial neuron, the fundamental computational unit underlying the MLP. It then explains how these neurons are organized into layers. Finally, it concludes with a general description of the NN as a whole.

2.3.1 Neurons

The artificial neuron is the basic building block of NNs. It is through these units that all computations within the network are carried out. A neuron receives a set of input signals from other neurons, where these inputs are represented as real-valued numbers. It produces an output, also a real-valued number, which is transmitted to subsequent neural units. Its internal state, called the activation, reflects its level of response and encodes the information processed for a given input. This activation, or lack thereof, is then used as input information by other neurons to determine their own activations, thereby propagating information through the network.

A neuron is characterized by:

- a set of incoming connections from other neurons,
- a weight associated with each of these connections,
- a bias term,
- a differentiable nonlinear activation function,
- a scalar activation state.

The activation of a neuron is determined by a weighted combination of its input signals, to which a bias is added, followed by the application of a nonlinear activation function. Mathematically, for a neuron receiving inputs $(x_1, \ldots, x_n)$, its activation h is given by:

$$h = \sigma\left(\sum_{i=1}^{n} w_i x_i + b\right), \tag{2.4}$$

where:

- w_i is the weight associated with the i-th incoming connection,
- b is the bias,
- σ is the activation function.

Figure 2.2 shows a schematic view of this basic unit. The set of weights $w_1, \ldots, w_i$ with the bias b constitute the parameters of the neuron, and they are the elements that evolve during training. The magnitude of a weight w, relative to the others, indicates how strongly a neuron depends on the activation of the neurons to which it is connected. A large positive weight means that a high activation in the connected neuron tends to increase the activation of the receiving neuron, whereas a negative weight implies the opposite effect. In this way, the weights modulate the strength and nature of the connections between neurons, thereby defining the overall pattern of interaction within the network.

The activation function σ models how a neuron transforms incoming signals into its output activation. Originally, this function was inspired by biological neurons, with the goal of mimicking the firing behavior of organic cells. However, modern DL no longer relies on this biological interpretation. In practice, any function that is nonlinear, differentiable, and computationally efficient can be used. Nonlinearity is a crucial property, since without it a NN would reduce to a composition of linear transformations, and would therefore be unable to approximate complex functions.

2.3.2 Layers

In MLP architectures, neurons are organized into successive layers, as illustrated in Figure 2.3. This layered structure determines how neurons are connected. In this setting, each neuron receives inputs from all neurons in the preceding layer and sends its output to all neurons in the following layer. As a result, neurons within the same layer do not interact with each other.

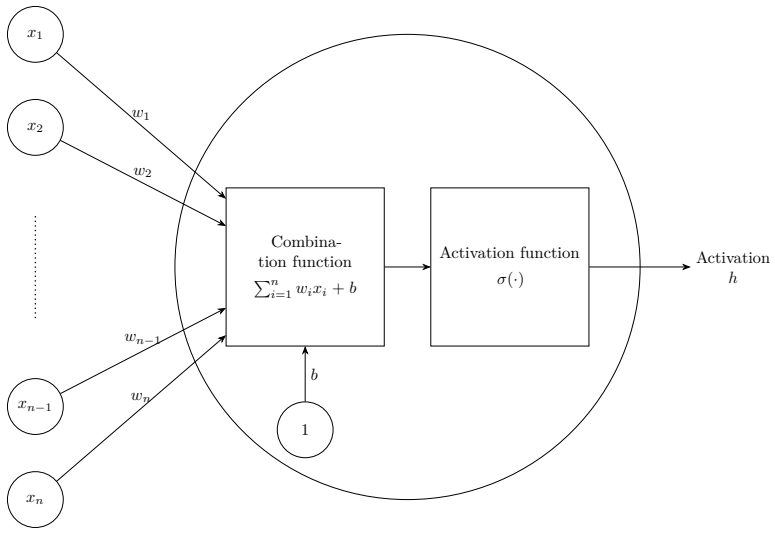


Figure 2.2: Artificial neuron.

Three types of layers are distinguished:

- an *input layer*, which directly receives input,
- one or more *hidden layers* (also referred to as intermediate layers), which perform intermediate transformations,
- an *output layer*, which produces the final prediction.

The input and output layers play a specific role. The input layer has no preceding layer, as its neurons directly encode the components of the input vector $\mathbf{x}$. Conversely, the output layer has no subsequent layer, and its neurons produce the model's prediction $\hat{\mathbf{y}}$. This structure imposes a direct correspondence between the dimensionality of the input space and the number of neurons in the input layer, and similarly between the output space and the number of neurons in the output layer.

When describing NNs, it is more common to reason at the level of layers rather than individual neurons. This perspective allows interactions between layers to be modeled in a matrix form. Such a representation enables efficient parallel computation and forms the basis of modern DL implementations [43].

Within this framework, it is useful to introduce the notion of layer activations. The activation of a layer corresponds to the activations of all its neurons, organized into a vector. For a layer l composed of n_l neurons, the activations are given by:

$$\mathbf{h}^{(l)} = [h_1^{(l)}, \ldots, h_{n_l}^{(l)}]. \tag{2.5}$$

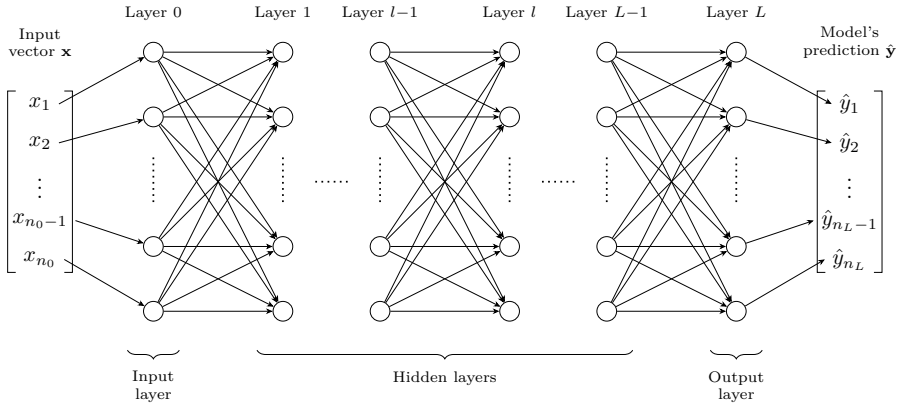


Figure 2.3: MLP architecture.

Using this formulation, the interaction between two consecutive layers can be written compactly as:

$$\mathbf{h}^{(l)} = \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{2.6}$$

where:

- $\mathbf{h}^{(l-1)}$ represents the activations vector of the previous layer,
- $\mathbf{W}^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ is the weight matrix connecting layer $l-1$ to layer l,
- $\mathbf{b}^{(l)} \in \mathbb{R}^{n_l}$ is the bias vector associated with layer l,
- $\sigma^{(l)}$ is the activation function applied element-wise to the neurons of layer l.

This matrix-vector representation is visualized in Figure 2.4, which highlights how each neuron in layer l receives weighted signals from all neurons in the previous layer.

It is important to examine the role played by the intermediate layers of a NN. At least one hidden layer is required to represent sufficiently complex functions, and in practice, modern architectures often include many more. Theoretical results have shown that, for a fixed number of parameters, increasing the number of layers enhances the expressive capacity of NNs [55]. Each layer can be viewed as an additional stage in the computation performed by the network. This suggests that deeper networks can represent increasingly complex functions by composing a larger number of transformations. The precise relationship between depth and function complexity remains primarily supported by empirical observations rather than formal theoretical justification [53].

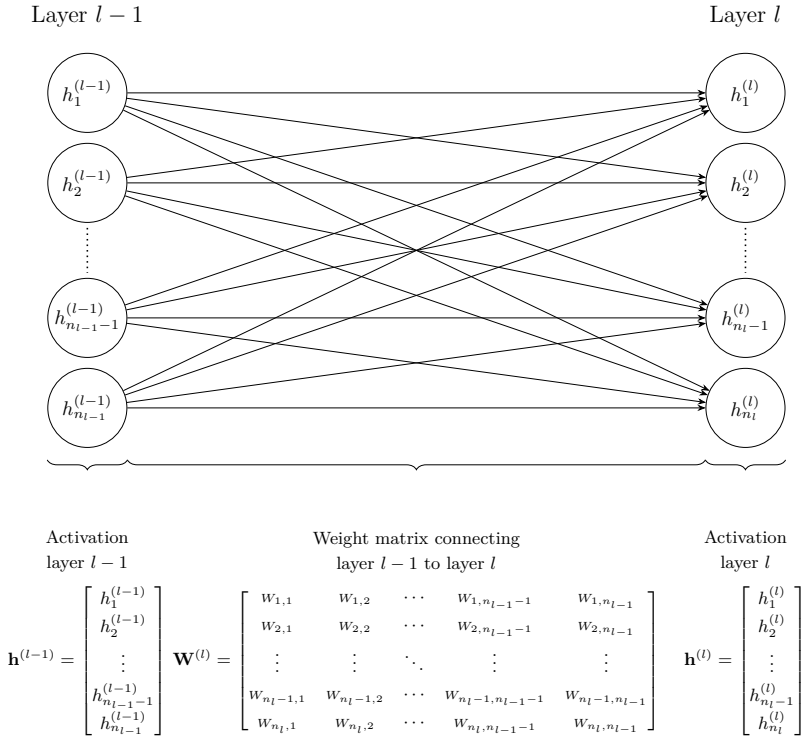


Figure 2.4: Interaction between layers.

2.3.3 Network

With a layer-wise representation, the propagation of information can be described as a sequence of transformations applied from the input layer (indexed 0) to the output layer (indexed L):

$$\begin{aligned} \hat{f}(\mathbf{x}) &= \mathbf{h}^{(L)} \\ &= \sigma^{(L)}\left(\mathbf{W}^{(L)}\sigma^{(L-1)}\left(\dots\sigma^{(1)}\left(\mathbf{W}^{(1)}\mathbf{x}+\mathbf{b}^{(1)}\right)\dots\right)+\mathbf{b}^{(L)}\right) \\ &= \hat{\mathbf{y}}. \end{aligned} \tag{2.7}$$

For a more compact description of information propagation, the layer activations are defined recursively as:

$$\mathbf{h}^{(l)} = \begin{cases} \mathbf{x} & \text{if } l = 0, \\ \sigma^{(l)}\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)}+\mathbf{b}^{(l)}\right) & \text{if } 1 \le l \le L. \end{cases} \tag{2.8}$$

To simplify notation, it is convenient to group all parameters of the network into a single set. The model parameters are thus denoted by:

$$\boldsymbol{\theta} = \{\mathbf{W}^{(l)}, \mathbf{b}^{(l)}\}_{l=1}^{L}. \tag{2.9}$$

In this thesis, $\boldsymbol{\theta}$ is a vector in $\mathbb{R}^d$, where d denotes the total number of parameters, such that $\boldsymbol{\theta} = [\theta_1, \dots, \theta_d]$. The notation $\hat{f}_{\boldsymbol{\theta}}$ expresses the dependence

of the NN on its parameters. Increasing the number of parameters, either by adding more neurons per layer or by increasing the number of layers, can enhances the expressive power of the model, allowing it to approximate more complex functions. However, this also increases computational cost and training time.

It is important to distinguish between the architecture and the parameters of a NN. When referring to the architecture of $\hat{f}$, the number of layers, the number of neurons per layer, and the choice of activation functions are considered. The parameters correspond to $\boldsymbol{\theta}$, which controls the strength of the connections between neurons. The following sections examine how these parameters can be adjusted to obtain a better approximation of the target function.

2.4 Learning

Now that the structure of a NN has been detailed, the next step is to examine how its parameters are adjusted to accomplish the assigned task. This brings the discussion to the core of DL, namely the learning process itself. As a reminder, the objective of training a NN is to adjust its parameters so that the network provides a good approximation of the target function. Mathematically, this objective is expressed as:

$$\min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\hat{f}_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}\right). \tag{2.10}$$

As shown in Equation (2.10), the loss for example i depends on the input-target pair $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$, the loss function $\mathcal{L}$, the network architecture $\hat{f}$, and the parameters $\boldsymbol{\theta}$. Since the dataset, loss function, and network architecture are fixed during training, only the parameters $\boldsymbol{\theta}$ are optimized. To simplify the notation, only the dependence on $\boldsymbol{\theta}$ is kept, and the loss for example i is written as:

$$\mathcal{L}^{(i)}(\boldsymbol{\theta}) = \mathcal{L}\left(\hat{f}_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}\right). \tag{2.11}$$

Now that the learning objective has been defined, the next question is how to modify the parameters $\boldsymbol{\theta}$ to minimize the loss. The NN architecture $\hat{f}$ used to approximate the target function f is highly complex. It involves a large number of parameters as well as many nonlinear components. As a result, the optimum cannot be determined analytically by directly studying the properties of the function.

Instead, approximate optimization methods are required to find suitable parameter values for NNs. The class of algorithms used for this optimization is known as gradient descent methods. Numerous variants of gradient descent have been developed for NN training. For simplicity, the focus here is on Stochastic Gradient Descent (SGD) with a batch size of 1. As in previous cases, this choice is made because this setting captures the essential intuition behind the training method.

This section presents the learning process in three steps. The first derives the gradient of the loss using the chain rule. The second shows how this gradient is used to update the network parameters. The third discusses the convergence properties of the resulting algorithm.

2.4.1 Computing the Gradient

The first step of SGD is to randomly initialize the parameter vector $\boldsymbol{\theta} \in \mathbb{R}^d$ for a given network architecture $\hat{f}$. This initial state is denoted by $\boldsymbol{\theta}_1$. The goal is to iteratively update this vector so that the network gets better at approximating the target function. To do so, it is necessary to quantify the impact of each parameter θ_j on the loss $\mathcal{L}^{(i)}(\boldsymbol{\theta})$. This is done using the partial derivative of the loss with respect to each parameter, denoted by:

$$\frac{\partial \mathcal{L}^{(i)}(\boldsymbol{\theta})}{\partial \theta_j}.$$

Computing this derivative is not straightforward because a NN is composed of a sequence of functions, and the parameter θ_j can belong to any layer of the network. Its influence on the loss must therefore be propagated through all subsequent layers.

To describe these dependencies, each layer can be viewed as a function, denoted by $f^{(l)}$ for layer l. This function takes the activation vector $\mathbf{h}^{(l-1)} \in \mathbb{R}^{n_{l-1}}$ produced by the previous layer as input. It produces the activation vector $\mathbf{h}^{(l)} \in \mathbb{R}^{n_l}$ of the current layer. This can be expressed as:

$$f^{(l)} \colon \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}, \qquad \mathbf{h}^{(l)} = f^{(l)}(\mathbf{h}^{(l-1)}). \tag{2.12}$$

Each layer function can have multiple inputs and multiple outputs, so the ordinary derivative is not sufficient to describe how its outputs change with respect to its inputs. The Jacobian matrix extends the derivative to functions with multiple inputs and outputs, quantifying how each output component changes with respect to each input component. Consider the function $f^{(l)}$ and its Jacobian $\mathbf{J}_{f^{(l)}}$ evaluated at $\mathbf{h}^{(l-1)}$. The Jacobian collects the partial derivatives of each output component with respect to each input component:

$$\mathbf{J}_{f^{(l)}}(\mathbf{h}^{(l-1)}) = \begin{bmatrix} \frac{\partial h_1^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_1^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial h_{n_l}^{(l)}}{\partial h_1^{(l-1)}} & \cdots & \frac{\partial h_{n_l}^{(l)}}{\partial h_{n_{l-1}}^{(l-1)}} \end{bmatrix}. \tag{2.13}$$

Since a NN is composed of several layers, a change in an early layer can affect the outputs of all subsequent layers. It is therefore necessary to propagate this change through the sequence of layer functions until it reaches the network output. The chain rule provides the mathematical tool required to describe this propagation.

To illustrate this, consider two consecutive layers, $f^{(l)}$ and $f^{(l+1)}$. The chain rule states that the Jacobian of their composition is given by the product of their Jacobians. Each Jacobian is evaluated at the input received by the corresponding layer:

$$\mathbf{J}_{f^{(l+1)} \circ f^{(l)}}\left(\mathbf{h}^{(l-1)}\right) = \mathbf{J}_{f^{(l+1)}}\left(\mathbf{h}^{(l)}\right) \mathbf{J}_{f^{(l)}}\left(\mathbf{h}^{(l-1)}\right). \tag{2.14}$$

The network is composed of a sequence of layer functions $f^{(1)}, \ldots, f^{(L)}$. Each layer depends only on the output of the previous layer. Consider a parameter θ_j belonging to layer l. A change in θ_j first affects the output $\mathbf{h}^{(l)}$ of its own layer. This change then propagates through the subsequent layers, affecting $\mathbf{h}^{(l+1)}$, and so on. The effect reaches the output $\mathbf{h}^{(L)}$, from which the loss $\mathcal{L}^{(i)}(\boldsymbol{\theta})$ is computed. This layer-by-layer propagation is referred to as backpropagation:

$$\frac{\partial \mathcal{L}^{(i)}(\boldsymbol{\theta})}{\partial \theta_j} = \mathbf{J}_{\mathcal{L}}(\mathbf{h}^{(L)}, \mathbf{y}^{(i)})\, \mathbf{J}_{f^{(L)}}(\mathbf{h}^{(L-1)}) \cdots \mathbf{J}_{f^{(l+1)}}(\mathbf{h}^{(l)})\, \mathbf{J}_{f^{(l)}}(\theta_j), \tag{2.15}$$

where:

- $\mathbf{J}_{\mathcal{L}}(\mathbf{h}^{(L)}, \mathbf{y}^{(i)})$ denotes the Jacobian of $\mathcal{L}$ with respect to its first argument, evaluated at $(\mathbf{h}^{(L)}, \mathbf{y}^{(i)})$,
- each subsequent $\mathbf{J}_{f^{(m+1)}}(\mathbf{h}^{(m)})$ denotes the Jacobian of the function $f^{(m+1)}$ with respect to its input $\mathbf{h}^{(m)}$,
- $\mathbf{J}_{f^{(l)}}(\theta_j)$ denotes the Jacobian of the function $f^{(l)}$ with respect to the parameter θ_j.

Once $\frac{\partial \mathcal{L}^{(i)}(\boldsymbol{\theta})}{\partial \theta_j}$ can be computed for every parameter, all of these values are stacked into one vector, called the gradient:

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}) = \left[\frac{\partial \mathcal{L}^{(i)}(\boldsymbol{\theta})}{\partial \theta_1}, \ldots, \frac{\partial \mathcal{L}^{(i)}(\boldsymbol{\theta})}{\partial \theta_d} \right]. \tag{2.16}$$

Each component answers the same question for a different parameter, indicating whether a small increase in that parameter increases or decreases the loss, and by how much. A positive value means increasing the parameter increases the loss, whereas a negative value means the opposite. The gradient thus indicates, for every parameter, which direction to move in order to reduce the loss.

2.4.2 Update Rule

Now that information is available on how changes in the parameters affect the loss, the objective is to minimize it. To do so, the parameters are updated in the direction opposite to the gradient. Indeed, the gradient is followed in the opposite direction because it indicates the direction in which the loss increases, whereas the objective is to reduce it. This leads to the gradient descent update rule:

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}_k), \tag{2.17}$$

where $\eta > 0$ is the learning rate, which controls the step size, and $\nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}_k)$ denotes the gradient of the loss with respect to the parameters, evaluated at the current values $\boldsymbol{\theta}_k$.

The parameter η must remain small, as the gradient only provides reliable information in a local neighborhood of the current parameters. As a result, a single step is not sufficient to significantly reduce the loss, so the process is iterated many times. Producing a sequence of parameter vectors $\boldsymbol{\theta}_1, \boldsymbol{\theta}_2, \ldots$ that progressively improve the model. In practice, training is stopped when successive updates no longer produce a significant decrease in the loss.

Algorithm 1: Stochastic Gradient Descent

Input:
Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{N}$
Architecture $\hat{f}$
Loss function $\mathcal{L}$
Learning rate $\eta > 0$
Initial parameters $\boldsymbol{\theta}$ (optional)
Output:
Trained parameters $\boldsymbol{\theta}$

---

1 **if** $\boldsymbol{\theta}$ *is given* **then**
2 $\quad \boldsymbol{\theta}_1 \leftarrow \boldsymbol{\theta}$;
3 **else**
4 $\quad$ Initialize $\boldsymbol{\theta}_1$ randomly;
5 $k \leftarrow 1$;
6 **while** *stopping criterion not satisfied* **do**
7 $\quad$ Select a random example $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \in \mathcal{D}$;
$\quad$ *Model prediction for the selected example*
8 $\quad \hat{\mathbf{y}}^{(i)} \leftarrow \hat{f}_{\boldsymbol{\theta}_k}(\mathbf{x}^{(i)})$;
$\quad$ *Per-example loss between the prediction and the ground truth*
9 $\quad \mathcal{L}^{(i)}(\boldsymbol{\theta}_k) \leftarrow \mathcal{L}(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)})$;
$\quad$ *Gradient of the per-example loss, since it is differentiable*
10 $\quad \mathbf{g} \leftarrow \nabla_{\boldsymbol{\theta}} \mathcal{L}^{(i)}(\boldsymbol{\theta}_k)$;
$\quad$ *Error minimization via a gradient descent step*
11 $\quad \boldsymbol{\theta}_{k+1} \leftarrow \boldsymbol{\theta}_k - \eta\, \mathbf{g}$;
12 $\quad k \leftarrow k + 1$;
13 **Return** $\boldsymbol{\theta}_k$

Algorithm 1 summarizes this whole training procedure, whose lines map directly onto the steps described above. Lines 1–4 set the initial parameters $\boldsymbol{\theta}_1$, either to given values or randomly. Line 6 repeats the update until the stopping criterion is satisfied. At each iteration, a training example is drawn at random (Line 7), the model computes its prediction (Line 8), and the per-example loss is evaluated (Line 9). Line 10 computes the gradient of Equation (2.16), and Line 11 applies the update rule of Equation (2.17). Finally, Line 13 returns the last parameters obtained.

To develop an intuition for this update rule, Figure 2.5 illustrates it applied to the simple convex loss function $\mathcal{L}(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$, where the red path shows the parameter updates converging to the minimum.[1] The point $\boldsymbol{\theta}_1$ represents the initial parameter values, with θ_1 and θ_2 randomly initialized, and the subsequent points, from $\boldsymbol{\theta}_2$ to $\boldsymbol{\theta}_6$, represent the successive updates produced by applying

[1]The gradient descent figures of this chapter were produced with Interactive Gradient Descent, a software developed for this thesis https://github.com/Maxalaar/Interactive_Gradient_Descent.

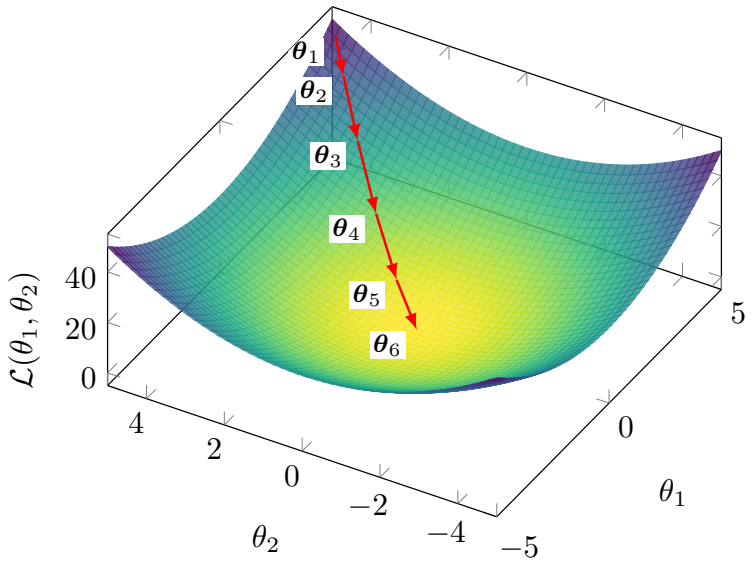


Figure 2.5: Gradient descent in a case $\mathcal{L}(\theta_1, \theta_2) = \theta_1^2 + \theta_2^2$.

Equation (2.17), progressively moving the trajectory toward the minimum of the loss function.

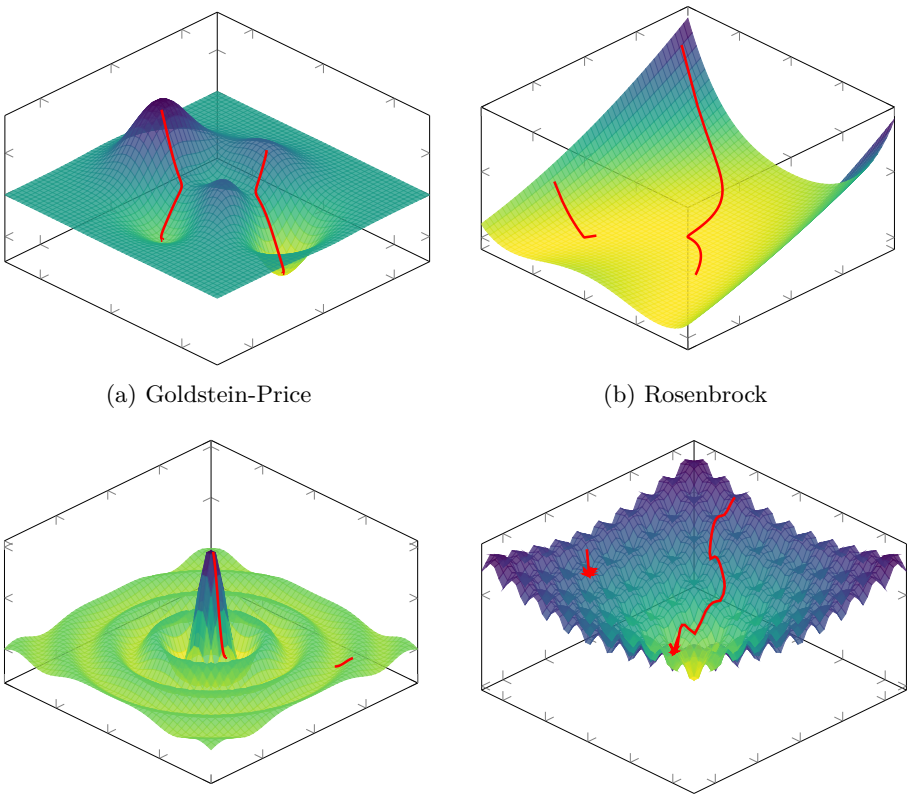
2.4.3 Convergence Properties

Having established how SGD updates the parameters, a natural question is whether this procedure is guaranteed to converge. It has been mathematically shown that gradient descent applied to a convex loss with respect to the parameters converges to a global optimum, independently of the initialization. However, NN training involves highly non-convex loss surfaces, so the optimization trajectory becomes strongly dependent on the initial conditions, and no such guarantee holds.

Figure 2.6 illustrates this dependence on four classical non-convex test functions, namely the Goldstein-Price, Rosenbrock, Sombrero, and Ackley functions. On each surface, two gradient descents are started from different initial parameters, and their trajectories are drawn in red. Unlike the convex case of Figure 2.5, the two trajectories never end at the same point. Each one follows the slope of its own neighborhood and either settles in a nearby local minimum or slows down in a flat region where the gradient becomes too small. In both cases, the final parameters are determined by the starting point rather than by the location of the global minimum.

Despite the lack of theoretical guarantees of convergence to a global optimum, the simple procedure of random initialization followed by iterative gradient-based updates is sufficient in practice to train complex NNs.

Explaining this empirical success is an active area of mathematical research. An influential approach is the neural tangent kernel framework [68], which studies NNs whose width tends to infinity. In this limit, a NN behaves like a linear model in its parameters, and gradient descent provably converges to a global minimum. However, the network then no longer learns internal representations.



(a) Goldstein-Price (b) Rosenbrock

(c) Sombrero (d) Ackley

Figure 2.6: Comparison of loss surfaces and gradient descent trajectories.

Yet learning such representations is considered an important reason for the practical success of deep learning. This framework therefore cannot fully explain the behavior of the NNs used in practice.

Other theoretical approaches start from assumptions closer to these networks [120]. They capture some aspects of representation learning but still rely on simplifying assumptions and provide only partial explanations. A general theory of the optimization of deep NNs thus remains an open problem.

2.5 Evaluation

Now that the process of training a model has been described, the question of how to evaluate it must be addressed. In the supervised learning setting, this evaluation relies on two aspects:

- its performance on the data it was trained on,
- its performance on data it was not trained on.

The first aspect measures how well the loss minimization process has succeeded. The second measures how well the model generalizes. Indeed, the dataset

$\mathcal{D}$ represents a small subset of all possible inputs. To estimate how the model will perform in general, it is important to evaluate how it performs on data it has not seen during training.

In order to quantify these two aspects, the dataset is split into two subsets:

- $\mathcal{D}_{\text{train}}$ is the set on which the model minimizes its loss during the training.
- $\mathcal{D}_{\text{test}}$ is kept aside during training in order to evaluate the model's ability to generalize.

When training is completed, the first step is to verify that the model performs well on the data it was trained on. To do so, the average loss on the training set, denoted $\bar{\mathcal{L}}_{\text{train}}$, is studied. This is an instance of the performance measure P defined in Equation (2.3), restricted to $\mathcal{D}_{\text{train}}$:

$$\bar{\mathcal{L}}_{\text{train}} = \frac{1}{|\mathcal{D}_{\text{train}}|} \sum_{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \in \mathcal{D}_{\text{train}}} \mathcal{L}\left(\hat{f}_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}\right). \tag{2.18}$$

If $\bar{\mathcal{L}}_{\text{train}}$ is not sufficiently low, this means that the model has not been able to learn the task. It is not necessary to go further in the analysis of the model's quality. Before restarting the training process, various design choices are then adjusted. Based on what has been discussed so far in this chapter, these include modifications to the number of layers, the number of neurons per layer, activation functions, the learning rate. However, in practice, there are many other possible variations. There is no universal rule for choosing the appropriate value for each of them. They are therefore chosen empirically, based on models that perform well on similar tasks, combined with intuition. Their selection is often costly and remains a challenging problem in DL.

Assuming now that the model achieves satisfactory performance on the training dataset $\mathcal{D}_{\text{train}}$. Its ability to generalize must now be studied. To do so, the corresponding average loss on $\mathcal{D}_{\text{test}}$ is computed:

$$\bar{\mathcal{L}}_{\text{test}} = \frac{1}{|\mathcal{D}_{\text{test}}|} \sum_{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \in \mathcal{D}_{\text{test}}} \mathcal{L}\left(\hat{f}_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \mathbf{y}^{(i)}\right). \tag{2.19}$$

If $\bar{\mathcal{L}}_{\text{test}} \gg \bar{\mathcal{L}}_{\text{train}}$, the model fails to generalize. This situation most often arises due to a phenomenon known as overfitting. In such cases, the model does not learn general patterns but instead memorizes the training examples. This typically occurs when the number of parameters is too large relative to the amount of training data. The simplest remedy is therefore to reduce the number of parameters in the model. However, it is still necessary to ensure that the model remains sufficiently expressive to perform well on the data.

If $\bar{\mathcal{L}}_{\text{test}} \approx \bar{\mathcal{L}}_{\text{train}}$, this indicates that the training procedure has successfully produced a NN that generalizes well beyond the training data [118]. The resulting model can therefore be considered suitable for the task for which it was trained, marking the end of the NN training process.

2.6 Specialized Architectures

The concepts presented so far have been introduced with the MLP. In this architecture, each neuron is connected to all neurons of the previous layer, and

no assumption is made about the structure of the input. However, many types of data have a structure that can be exploited. The pixels of an image are organized in a spatial grid, and the words of a sentence form an ordered sequence in which each word depends on its context. Specialized architectures organize the connections between neurons so as to take this structure into account.

These architectures do not modify the principles introduced in this chapter. A network remains a composition of layers, where each layer computes its representation from that of the previous one:

$$\mathbf{h}^{(l)} = f^{(l)}\left(\mathbf{h}^{(l-1)}\right). \tag{2.20}$$

Its parameters are still optimized by gradient descent, and it is still evaluated on data it has not seen during training. Only the function $f^{(l)}$ computed by each layer changes. In particular, each layer still produces an intermediate representation of the input, which may take the form of feature maps or of a sequence of tokens. This representation is shaped by the same learning process as in a MLP. This section presents two architectures. The CNN is used in the experiments of this thesis, while the attention mechanism, which is the basis of the Transformer architecture, underlies several works discussed in the following chapters.

2.6.1 Convolutional Neural Networks

CNNs were introduced to process images [26]. Applying a MLP to an image would require connecting each neuron to every pixel. The number of parameters would then grow with the size of the image, and the spatial organization of the pixels would be ignored. CNNs rely instead on two properties of images. The first is locality, since the information needed to detect a pattern such as an edge is contained in a small region of the image. The second is that the same pattern can appear at any position in the image.

A convolutional layer exploits these two properties by means of kernels, also called filters. A kernel is a small matrix of weights, typically of size 3×3, which is applied to a local region of the representation of the previous layer, called its receptive field. The kernel is then slid over every position of this representation. At each position, a weighted sum of the elements of the receptive field is computed, which gives one element of the output. The set of outputs obtained over all positions forms a feature map. Since the same weights are used at every position, the number of parameters of the layer does not depend on the size of the input, and a pattern is detected in the same way wherever it appears.

A convolutional layer uses several kernels, each detecting a different pattern. The representation of layer l is therefore composed of n_l feature maps, also called channels, and is denoted $\mathbf{H}^{(l)}$. The feature map produced by the k-th kernel is given by:

$$\mathbf{H}_k^{(l)} = \sigma^{(l)}\left(\mathbf{W}_k^{(l)} * \mathbf{H}^{(l-1)} + b_k^{(l)}\right), \qquad k = 1, \ldots, n_l, \tag{2.21}$$

where:

- $\mathbf{W}_k^{(l)}$ is the k-th kernel of layer l,
- $b_k^{(l)}$ is the bias associated with this kernel,
- $*$ denotes the convolution operation, which applies the kernel at every position of $\mathbf{H}^{(l-1)}$.

For a single-channel input and a kernel of size $\kappa \times \kappa$, the element at position (i, j) of the convolution is:

$$\left(\mathbf{W}_k^{(l)} * \mathbf{H}^{(l-1)}\right)_{i,j} = \sum_{i'=1}^{\kappa} \sum_{j'=1}^{\kappa} W_{k,i',j'}^{(l)} \, H_{i+i'-1,\, j+j'-1}^{(l-1)}. \tag{2.22}$$

When $\mathbf{H}^{(l-1)}$ has several channels, each kernel covers all of them, and the sum also runs over the n_{l-1} channels. For the first layer, $\mathbf{H}^{(0)}$ is the input image, whose channels correspond for instance to the color components of the pixels. The kernel can also be moved by steps larger than one element, called the stride, which reduces the size of the feature maps.

Figure 2.7 illustrates this computation for a single-channel input. The receptive field, highlighted in orange on the representation $\mathbf{H}^{(l-1)}$, is combined with each kernel $\mathbf{W}_k^{(l)}$. Each combination gives one element of the corresponding feature map $\mathbf{H}_k^{(l)}$, shown in the same color as the kernel. Sliding the receptive field over every position of $\mathbf{H}^{(l-1)}$ produces all the other elements of these feature maps.

As with the MLP, convolutional layers are stacked. The receptive field of a neuron, measured on the input image, grows with depth. The first layers thus detect simple local patterns, while the deeper layers combine them into patterns that cover larger regions of the image [53]. The last feature maps are then flattened into a vector and passed through a MLP, which produces the output of the network.

2.6.2 Attention Mechanism

The attention mechanism processes an input made of a sequence of elements, called tokens. This subsection presents self-attention, its most widely used form, in which the queries, keys and values are all computed from the same sequence of tokens. Depending on the task, a token can represent a part of a word, a region of an image, or an element of an observation. Each token is a vector, and the m tokens of layer $l-1$ are stacked as the rows of a matrix $\mathbf{H}^{(l-1)}$. Unlike a convolutional layer, which only combines neighboring elements, the attention mechanism allows each token to gather information from all the other tokens of the sequence, whatever their position [63].

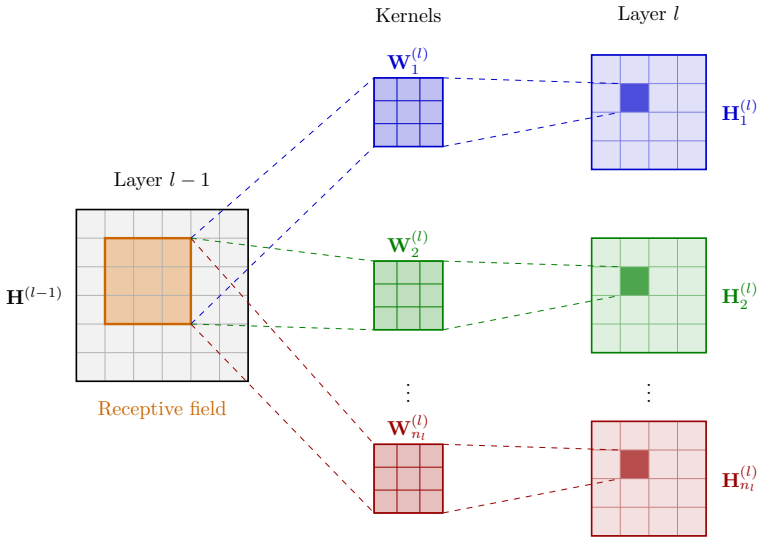


Figure 2.7: Convolutional layer.

Three matrices are first computed from the tokens by learned linear projections:

$$\mathbf{Q} = \mathbf{H}^{(l-1)}\mathbf{W}_Q^{(l)}, \qquad \mathbf{K} = \mathbf{H}^{(l-1)}\mathbf{W}_K^{(l)}, \qquad \mathbf{V} = \mathbf{H}^{(l-1)}\mathbf{W}_V^{(l)}, \tag{2.23}$$

where the rows of $\mathbf{Q}$, $\mathbf{K}$ and $\mathbf{V}$ are respectively the queries, the keys and the values of the tokens. The query of a token describes the information it is looking for, its key describes the information it contains, and its value is the information it transmits. The queries and keys are vectors of dimension d_{key}.

The query of each token is then compared with the keys of all tokens by a scalar product. These scores are normalized into attention weights by the softmax function, applied to each row:

$$\mathbf{A} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{key}}}}\right), \qquad \text{with} \quad \text{softmax}(\mathbf{u})_i = \frac{\exp(u_i)}{\sum_j \exp(u_j)}. \tag{2.24}$$

The weight $A_{i,j}$ indicates how much token i attends to token j. The weights of each row are positive and sum to one. The division by $\sqrt{d_{\text{key}}}$ prevents the scores from becoming too large as the dimension increases, which would concentrate the softmax on a single token. Finally, each output token is the weighted sum of the values according to these weights:

$$\mathbf{H}^{(l)} = \mathbf{A}\mathbf{V} = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_{\text{key}}}}\right)\mathbf{V}. \tag{2.25}$$

Figure 2.8 illustrates this mechanism for a sequence of $m = 4$ tokens, numbered from 1 to 4 and stacked as the rows of $\mathbf{H}^{(l-1)}$. The tokens are projected into queries, keys and values, shown in blue, green and red respectively. The queries and keys are combined to give the attention weights $\mathbf{A}$, represented in orange as a heat map in which darker cells correspond to larger weights. Row

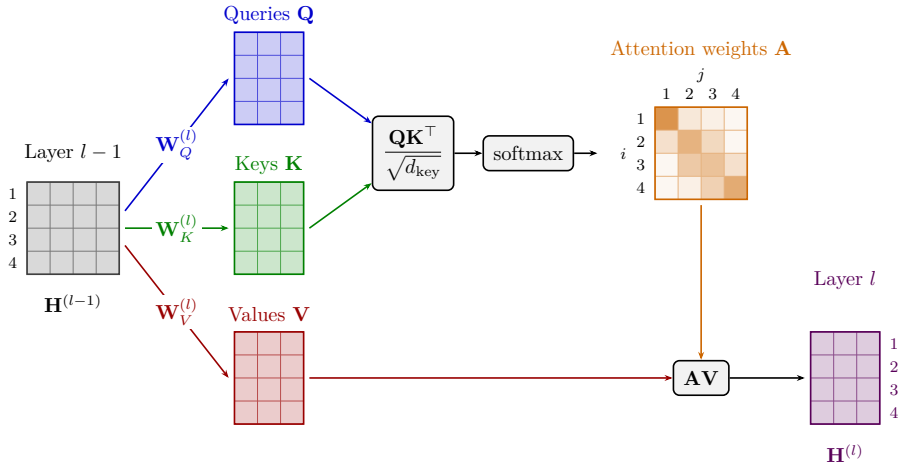


Figure 2.8: Attention mechanism.

i of this heat map indicates how much token i attends to each token j. The output tokens of layer l, shown in purple, are then obtained by weighting the values with these attention weights.

Unlike in a MLP or a CNN, where the weights that combine the inputs are fixed once training is completed, the weights $\mathbf{A}$ are computed from the input itself. The way tokens are combined thus adapts to each input.

The Transformer architecture stacks blocks that combine attention with a MLP applied independently to each token [63]. This architecture underlies LLMs, which are trained on very large text corpora to predict the next token of a sequence [94].

Whatever the architecture, a network therefore produces a sequence of intermediate representations, whether they take the form of activation vectors, feature maps, or sequences of tokens. These representations are shaped by the same learning process, since the parameters of every layer are adjusted by SGD to minimize the same loss.

2.7 Conclusion

This chapter has introduced the fundamental concepts underlying DL, including the general objective of NN models, their structure, the training procedure, their evaluation, and the main specialized architectures. For clarity of presentation, several simplifying assumptions have been made throughout this chapter. These choices were made deliberately to expose the core mechanisms of DL as clearly as possible.

Despite these simplifications, the framework introduced here captures mechanisms that are shared across a broad range of DL methods: A sequence of parameter updates, each based on information about the loss around the current parameters. The apparent simplicity of this learning procedure therefore contrasts with the complexity of the functions that NNs can represent. Understanding how such complex behavior emerges from a relatively simple optimization process remains an important challenge in the study of DL.

Mathematical tools allow us to describe the functions represented by NNs and the procedures used to train them. However, they do not yet fully explain their remarkable empirical performance. Understanding the origin of this success therefore requires looking beyond the optimization procedure itself.

One way to approach this question is to study the intermediate representations that are present in all NN architectures. As discussed in Section 2.6, whether considering a MLP, a CNN, or an architecture based on attention, the input is progressively transformed into intermediate representations before producing the final output. Their presence provides a common perspective for understanding how NNs transform information. The collection of these intermediate representations is referred to as the Latent Space (LS). Studying this LS can provide insights into how NNs learn and generalize, which motivates the following chapter.

Chapter 3

Latent Space Analysis

「私は情報の海で発生した生命体だ」
"I am a life form that emerged from the sea of information."
Mamoru Oshii, *Ghost in the Shell* (1995) [30]

Contents

3.1 Concept Hypothesis 33
3.2 Probing 36
3.3 Directions 40
3.4 Superposition 41
3.5 Clustering 46
3.6 Conclusion 49

In classical machine learning approaches, practitioners had to manually design data representations in order to make them usable by simple models. Deep Learning (DL) introduces a paradigm shift. Rather than relying on hand-crafted features to meaningfully represent data, it automatically learns these representations. Through the successive activation of hidden layers, Neural Networks (NNs) gradually construct representations that become increasingly relevant to the task under consideration [23].

These representations are not meaningful at initialization, they emerge progressively during training as the model optimizes its objective. It is through these learned representations that a trained network is able to map an input to an output, including for examples never encountered during training. Since these representations are learned internally by the network and are not directly observable from the input or output spaces, they are commonly referred to as Latent Representations (LRs). The collection of these LRs learned throughout the network is referred to as a Latent Space (LS).

These LSs are now at the core of a growing number of practical applications, among which:

- **Video compression:** Neural video compression exploits the compactness of LSs to encode information efficiently [81].
- **Exploration in Reinforcement Learning (RL):** Agent internal representations are used for efficient exploration in extremely open environments [75].
- **Recommendation systems:** Users and content are modeled within a shared LS to compute meaningful similarities [61].
- **Retrieval-augmented generation:** Knowledge bases are indexed and queried through LRs [97].
- **Multimodal models:** Large Language Models (LLMs) and multimodal architectures rely on the quality of learned representations to align heterogeneous modalities such as text, images, and audio [109].

These examples represent only some of the most widespread applications of LRs.

Given their extensive use, the study of LRs has become a central topic in DL research. A major objective is to understand how information is organized within these spaces, which concepts are encoded, and how these LRs influence model behavior.

The remainder of this chapter is organized as follows:

- Section 3.1 introduces the hypothesis that underlies much of the research on LS analysis, which we refer to as the *Concept Hypothesis* (Hypothesis 1).
- Sections 3.2, 3.3, 3.4, and 3.5 examine four families of LS analysis methods, respectively based on probing, interpretable directions, feature superposition, and clustering.
- Section 3.6 concludes with a comparative synthesis of these methods and a discussion of the open questions that remain.

3.1 Concept Hypothesis

In this section, we introduce the main working hypothesis underlying much of the literature on LS analysis. Although rarely stated explicitly, this hypothesis is implicitly present in the recurring use of notions such as abstraction, representation, LS, concept, and semantics. Throughout this thesis, we refer to this hypothesis as *Concept Hypothesis* (Hypothesis 1). The goal of this section is to make this conceptual framework explicit by defining its main components.

The *Concept Hypothesis* (Hypothesis 1) originates from the need to explain the generalization ability of deep NNs observed after training, as discussed in Section 2.5. A model is said to generalize when, for an input $\mathbf{x} \in \mathcal{X}$ not seen during training, it still produces an output $\hat{\mathbf{y}}$ close to the true output $\mathbf{y}$. Such behavior cannot be explained by simple memorization of training samples. Instead, the model must extract and exploit underlying regularities in the target function. This requires a process of abstraction, as defined in Definition 3.1.1,

whereby raw inputs are transformed into internal structures that are meaningful for the task [45].

Definition 3.1.1: Abstraction

Abstraction is the process by which a model transforms an input into a task-relevant internal form. It captures and restructures information in order to represent relationships that are useful for prediction.

The ability of NNs to perform abstraction arises from their intermediate activations. A NN does not directly map $\mathbf{x}$ to $\hat{\mathbf{y}}$, it computes a sequence of intermediate activations across hidden layers. These activations are numerical encodings of the input referred to as embeddings. Once training is complete, these embeddings are structured to support the performance of a specific task. To mark this distinction, we will refer to these trained embeddings as LRs, as defined in Definition 3.1.2.

Definition 3.1.2: Latent Representation

A LR is the internal encoding of an input given by an intermediate layer of a trained NN.

Each hidden layer refines the LR of the previous layer. Each transformation progressively reshapes the LR into a form more directly aligned with the output space [23]. This process can be described as a sequence of transformations where each $\mathbf{h}^{(l)}$ is a LR:

$$\mathbf{x} \to \mathbf{h}^{(1)} \to \mathbf{h}^{(2)} \to \cdots \to \mathbf{h}^{(L-1)} \to \hat{\mathbf{y}}.$$

The set of all such LRs produced by a layer over a dataset defines its LS, as defined in Definition 3.1.3.

Definition 3.1.3: Latent Space

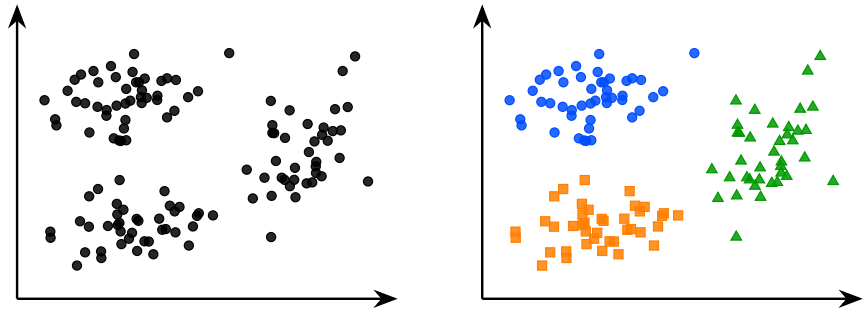
The LS is the set of all LRs produced by a layer over a dataset. It is shaped by learning and organizes LRs according to task-relevant properties.

Under the *Concept Hypothesis* (Hypothesis 1), the analysis of the LS is assumed to provide access to abstract structures learned by the model, as illustrated in Figure 3.1. The same set of LRs can be viewed either as an unstructured point cloud or, under this hypothesis, as a collection of points organized by concept membership, in which case the structure of the LS becomes meaningful and amenable to analysis. In Figure 3.1, the colors represent hypothetical concept memberships assigned by the model.

In this thesis, a concept is defined as an abstract representation that organizes information and supports reasoning (see Definition 3.1.4).

Definition 3.1.4: Concept

A concept is a task-dependent abstraction that captures regularities in the input space relevant for predicting the target output.



(a) Observed activations (b) Under the concept hypothesis

Figure 3.1: The same LS activations seen without (a) and under (b) the *Concept Hypothesis* (Hypothesis 1).

Our definition of concepts is consistent with the information-processing view of cognition. Within this framework, concepts arise from the need to simplify an otherwise intractable reality. Because the environment contains more information than any cognitive system can process, intelligent behavior relies on abstractions, called concepts, that discard irrelevant details while preserving the information required to achieve a given objective [4, 46]. These concepts are realized as representations that reduce complexity and enable reasoning under limited computational resources [20, 31]. Although these theories were originally developed to explain human cognition, the *Concept Hypothesis* (Hypothesis 1) assumes that the same principles can be extended to deep NNs.

In order to influence the model's computations, concepts must necessarily admit a concrete realization within the LS. We refer to this realization as the Concept Structure (CS). The definition of the CS is provided in Definition 3.1.5.

Definition 3.1.5: Concept Structure

A CS is the specific form for how a concept is instantiated within the LS.

The methods reviewed in this chapter all build on the *Concept Hypothesis* (Hypothesis 1) formalized in Hypothesis 1, but differ in how they characterize the CS of a concept. This choice directly determines how concepts can be identified and extracted from the LS. As a result, several families of LS analysis methods have emerged in the literature, each adopting a different view of how concepts are structured within the LS.

Hypothesis 1: Concept Hypothesis

NNs generalize by performing abstraction. Through training, their LS becomes organized around concepts. Concepts are task-dependent abstractions that capture regularities relevant to the task. Analyzing the LS therefore provides access to the CSs through which these concepts are instantiated in the model.

Now that the definition of a *Concept Hypothesis* (Hypothesis 1) has been established, we can consider its relationship to humans. According to our

definition, we can imagine concepts that are useful for the machine but do not make sense to humans. This relationship is captured by the notion of semantics [125], defined in Definition 3.1.6. Some concepts developed in LS can be readily associated with human concepts and are said to have high semantic value. Others may contribute to task performance while remaining difficult for humans to understand, and are said to have low semantic value.

Definition 3.1.6: Semantics

Semantics refers to the alignment between concepts in the model LS and concepts used by a human observer. A concept is semantic if it can be associated with a human interpretable concept.

It is important to note that this conceptual framework is not supported by a complete theoretical understanding of deep NNs. Current mathematical tools do not provide a rigorous characterization of their internal representations and do not formally prove that concepts emerge as identifiable CSs within LS. Nevertheless, this perspective constitutes a widely adopted working hypothesis in the field of representation and LS analysis.

The remainder of this chapter is devoted to reviewing methods for analyzing LSs and extracting the concepts they encode. We begin with probing, one of the most direct approaches for assessing whether concepts are represented in the LS.

3.2 Probing

Probing methods aim to determine whether a given semantic concept is encoded within the LS of a NN [124, 134]. The idea behind these methods is that if a concept can be reliably decoded from LRs, it is likely relevant to the problem being solved. Studying such concepts can therefore provide insight into how the network internally represents the problem.

A probing analysis begins by identifying a set of semantic concepts that the network could potentially use to accomplish its task. Each example in the dataset is then annotated to indicate the presence or absence of the studied concepts. We denote by c_j the presence or absence of concept j in example $\mathbf{x}$. The dataset can therefore be written as follows:

$$\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)}, c_1^{(i)}, \dots, c_n^{(i)})\}_{i=1}^{N}.$$

The data are subsequently projected into the LS of the pretrained model. For an input $\mathbf{x}$, we write $\mathbf{h}^{(l)}(\mathbf{x}) \in \mathbb{R}^{n_l}$, or simply $\mathbf{h}^{(l)}$, the LR obtained at layer l.

A classifier, referred to as a probe, is then trained to predict the presence of a given concept from this LR. It is denoted by $\psi_{\boldsymbol{\theta}}^{(l,j)}$, where $\boldsymbol{\theta}$ represents its parameters, l indicates the layer from which the LR is extracted, and j identifies the concept being predicted. The probe maps the LR to a scalar value in $[0, 1]$, which can be interpreted as the probability that the concept is present. The parameters $\boldsymbol{\theta}$ are learned by minimizing a loss function over the training

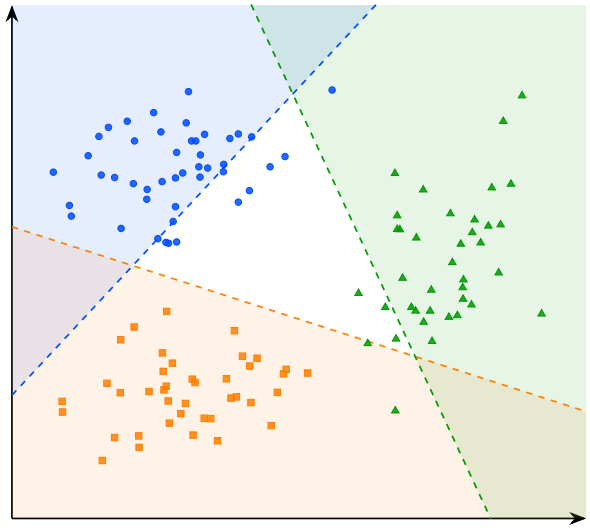


Figure 3.2: Linear probes in the LS.

examples using Stochastic Gradient Descent (SGD). This gives the following mathematical formulation:

$$\psi_{\boldsymbol{\theta}}^{(l,j)} : \mathbb{R}^{n_l} \to [0,1], \qquad \min_{\boldsymbol{\theta}} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\left(\psi_{\boldsymbol{\theta}}^{(l,j)}\left(\mathbf{h}^{(l)}(\mathbf{x}^{(i)})\right), c_j^{(i)}\right). \tag{3.1}$$

The architecture of the probe must be kept intentionally simple. A probe with excessive expressive capacity may be able to predict the concept by exploiting complex patterns within the representation. These patterns do not necessarily indicate that the concept is explicitly encoded in the LS. Instead, they may result from the ability of the probe itself to combine and transform the available information. Consequently, a highly expressive probe makes it difficult to distinguish between information that is directly accessible in the representation and information that is reconstructed by the classifier. Restricting the capacity of the probe therefore helps ensure that its predictions primarily reflect the information already present in the LS.

For this reason, probing methods often rely on linear classifiers. A linear separation suggests that the information is easily accessible to the NN and potentially usable during decision-making. If the classifier achieves performance significantly above chance level, this suggests that the LS contains exploitable information related to the considered concept. Figure 3.2 illustrates the decision boundaries of such linear probes for three concepts, where each colored region indicates the area classified positively by the corresponding probe.

An application of this technique can be found in studies investigating whether LLMs develop internal representations of their environment [124, 134]. In these works, researchers train LLM architectures, which rely on the Transformer architecture, built on the attention mechanism presented in Section 2.6.2, to play games such as chess or Othello in a purely textual setting. The model is provided only with sequences of moves and information about the actions it can take, all expressed in textual form. The central question is whether the LLM

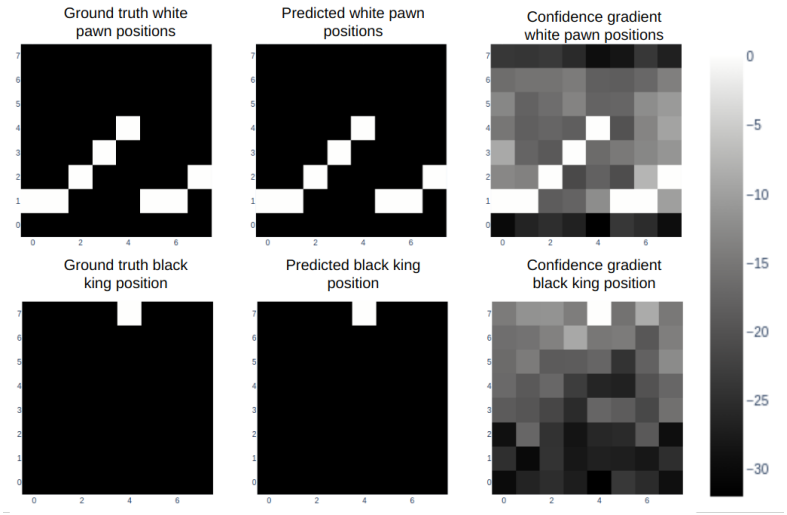


Figure 3.3: Probing analysis of an LLM trained to play chess [134].

develops an internal representation of the game state. To address this question, the probed concepts correspond to the presence or absence of different game pieces on specific board positions.

As illustrated in Figure 3.3, probes are able to reconstruct the full board state from the model's LRs, even though the only available information to the model is a sequence of text. In this figure, each row corresponds to a piece type, comparing the ground truth with the probe's predictions and confidence. The close agreement between the ground truth and predicted positions suggests that the model develops a structured representation of the underlying environment from which the data are generated.

However, the fact that a concept can be decoded from LRs does not necessarily imply that the model actually uses this information to perform its task. To assess whether LRs play a causal role in the model's output, it is possible to perform latent interventions on them, as defined in Definition 3.2.1.

Definition 3.2.1: Latent Intervention

A latent intervention is a controlled modification of a LR that alters a target concept, after which the forward pass resumes from the modified LR so that its effect on the model's output can be observed.

In the case of probing, the intervention activates or deactivates a given concept. If the resulting change in the model's output is consistent with the applied perturbation, this suggests that the representation had a causal role.

To this end, the LR is minimally modified such that the probe prediction for the target concept c_j changes state, while the predictions associated with other concepts remain unchanged. This goal can be formulated as an optimization problem over the LR itself. The optimization is performed directly on the LR, using SGD. The solution to this problem is the optimal modified LR, defined as:

$$\begin{aligned}\mathbf{h}^{(l)*} = \arg\min_{\mathbf{h}^{(l)\prime}} \; & \underbrace{\lambda_1 \, \mathcal{L}_{\text{flip}}\big(\psi_{\boldsymbol{\theta}}^{(l,j)}(\mathbf{h}^{(l)\prime}), c_j\big)}_{\text{flip target concept } c_j} \\ & + \underbrace{\lambda_2 \sum_{k \neq j} \mathcal{L}_{\text{preserve}}\big(\psi_{\boldsymbol{\theta}}^{(l,k)}(\mathbf{h}^{(l)\prime}), c_k\big)}_{\text{preserve other concepts } c_{k \neq j}} \\ & + \underbrace{\lambda_3 \|\mathbf{h}^{(l)\prime} - \mathbf{h}^{(l)}\|_2^2}_{\text{minimality}},\end{aligned}$$

where:

- $\mathbf{h}^{(l)}$ is the original LR of the input at layer l,
- $\mathbf{h}^{(l)\prime}$ is the modified LR being optimized,
- $\mathbf{h}^{(l)*}$ is the optimal modified LR,
- $\mathcal{L}_{\text{flip}}$: loss encouraging the prediction of the target concept c_j to change state (i.e., flipping the predicted label),
- $\mathcal{L}_{\text{preserve}}$: loss enforcing invariance of all non-target concepts c_k for $k \neq j$,
- $\|\mathbf{h}^{(l)\prime} - \mathbf{h}^{(l)}\|_2^2$: regularization term ensuring the modified LR remains close to the original one,
- $\lambda_1, \lambda_2, \lambda_3$: weighting coefficients controlling the trade-off between the intervention strength, the preservation of other concepts, and the minimality of the perturbation.

In the context of LLMs studied previously, such latent interventions have been used to remove specific pieces from the model's internal representation of the game state [124, 134]. The main observation is that, in the vast majority of cases, the model behaves as if the piece had effectively disappeared. The LLM continues to generate legal moves while correctly accounting for the absence of the removed piece. This provides evidence for the causal role of these internal representations in the model's behavior.

A main limitation of the probing approach is that the concepts under investigation must be predefined before any analysis can be performed. This introduces a strong bias. Moreover, annotating a dataset for each concept is extremely time-consuming. Latent interventions are also difficult to interpret, as it is not always clear what constitutes a meaningful change in the network's output under a given perturbation.

Finally, probing methods provide a binary view of concepts, focusing on whether a given concept is present or absent in a LR. This binary perspective can be restrictive, as concepts may be represented in a more continuous manner.

To obtain a more detailed characterization of these representations, the next section turns to methods that describe a concept as a direction in LS along which it varies continuously.

3.3 Directions

Probing describes a concept as a binary attribute, either present or absent in a LR. The methods in this section instead associate each concept with a direction in the LS. Moving a LR along this direction is expected to strengthen or weaken the concept smoothly. This provides information that a binary probe cannot express.

Empirically, it has been observed that shifting a LR along such a direction can induce coherent and semantically meaningful changes in the model's output [103, 132]. Such a shift is a latent intervention (see Definition 3.2.1) and can be expressed mathematically as:

$$\mathbf{h}' = \mathbf{h} + \alpha \mathbf{v}_i,$$

where:

- $\mathbf{h}$ denotes the original LR,
- $\mathbf{v}_i$ is a unit vector associated with concept i,
- α controls the strength of the intervention,
- $\mathbf{h}'$ is the resulting perturbed LR.

In many cases, the resulting change in the model's output varies smoothly with α, suggesting an approximately linear CS with respect to certain semantic factors. This observation has led to the hypothesis that neural representations may organize concepts in a partially linear manner within the LS.

Vectors of this form are referred to as interpretable directions, as they correspond to transformations that can be associated with specific semantic attributes. The objective of this line of work is therefore to identify directions in LS that align with meaningful variations in the encoded concepts. This is illustrated schematically in Figure 3.4, where arrows represent semantic shifts from one concept toward others, and moving along these directions induces a coherent change in the model's behavior, suggesting that the transition between concepts is continuously encoded along these axes.

This phenomenon has been extensively studied in generative image models. Controlled perturbations of LRs along interpretable directions can induce specific semantic transformations [103]. Examples include stroke thickness modification in digit images, head rotation in facial images, and texture or background changes in natural images. These transformations are illustrated in Figure 3.5, where each row corresponds to a distinct direction $\mathbf{v}_i$ and each column to an increasing value of α. These observations have led to the development of the field of LS editing, whose objective is to manipulate generated content directly through modifications in LS rather than through changes in the input prompt.

A central challenge remains identifying such interpretable directions. Most existing approaches rely on auxiliary models that encode human defined semantics, such as CLIP [132], to relate latent variations to textual concepts. Consequently, these methods inherit a limitation similar to that of probing techniques, since concept discovery depends on a pre-existing semantic framework defined by humans.

To overcome this limitation, a research direction known as unsupervised semantic discovery has emerged. Its goal is to uncover interpretable directions

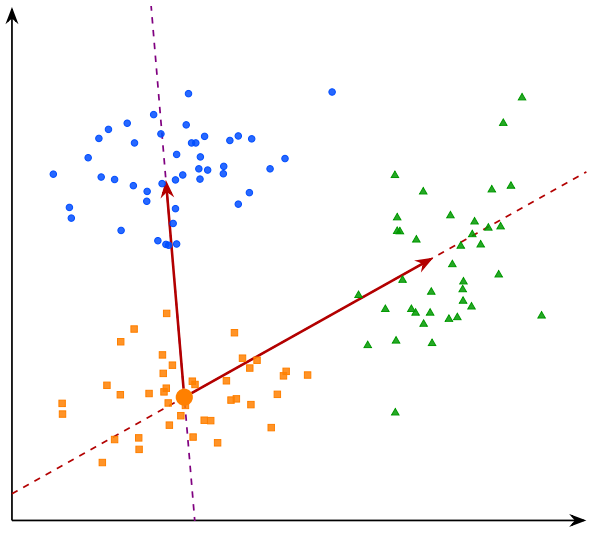


Figure 3.4: Interpretable directions in the LS.

in LS without explicit semantic supervision. In these approaches, one model applies perturbations along random directions in LS, while a second model attempts to recognize the applied transformation. If a stable correspondence emerges between the perturbation and its prediction, this suggests that the considered direction corresponds to a coherent CS [103]. In this framework, human intervention occurs only a posteriori, to verify whether the discovered directions indeed correspond to interpretable semantic variations.

Despite these advances, several challenges remain. It appears that not all concepts encoded by NNs are linearly identifiable in LS. In addition, LRs often exhibit a phenomenon known as entanglement, illustrated in Figure 3.6, where multiple semantic attributes are mixed within the same directions of the LS. For instance, in a generative face model, moving along the "glasses" direction may simultaneously modify age and gender, whereas a disentangled LR would let each axis control a single semantic attribute while leaving the others unchanged.

This lack of separability suggests that semantic information is not cleanly organized in the LS. This observation motivates the *Superposition Hypothesis* (Hypothesis 2), which proposes a different view of how NNs organize information internally. The next section introduces this hypothesis.

3.4 Superposition

The previous section highlighted a limitation of interpretable directions. To account for this, an extension of the *Concept Hypothesis* (Hypothesis 1) has been proposed under the name *Superposition Hypothesis* (Hypothesis 2), formalized in Hypothesis 2 [117].

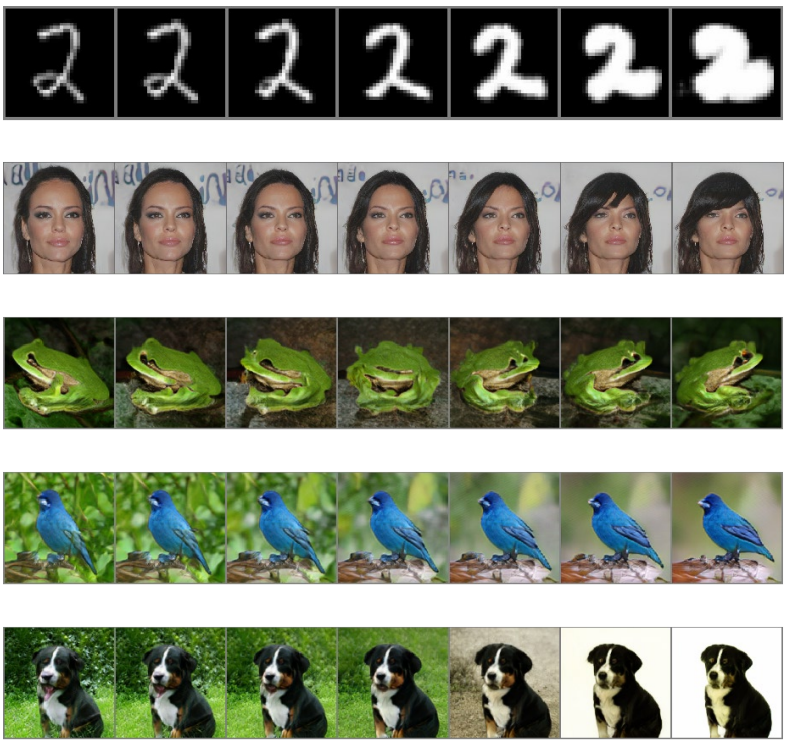


Figure 3.5: Interpretable directions in the LS of a generative model [103].

Hypothesis 2: Superposition Hypothesis

Concepts are preferentially represented by distinct linear directions in LS. When the number of concepts exceeds the available dimensions, multiple concepts are encoded in superposition within the same directions.

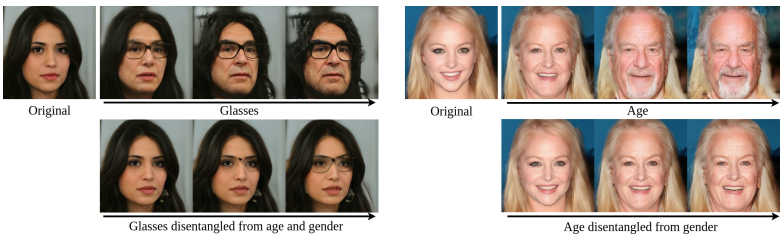


Figure 3.6: Illustration of entanglement in the LS of a generative face model [132].

According to this hypothesis, in an unconstrained LS, each concept is represented in its own independent linear direction. The idealized non-superposed LR can be mathematically expressed as follows:

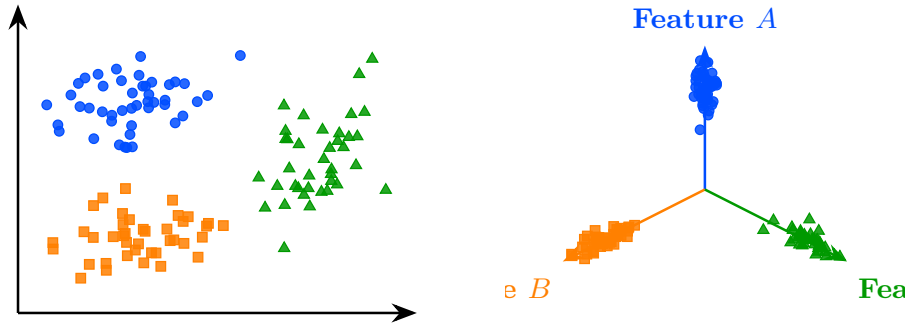
$$\mathbf{h} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i, \qquad \forall i \neq j, \quad \mathbf{v}_i^\top \mathbf{v}_j \approx 0, \qquad \|\boldsymbol{\alpha}\|_0 \ll n,$$

where:

- $\mathbf{h}$ denotes a LR,
- n is the total number of concepts,
- $\mathbf{v}_i$ represents the direction associated with concept c_i,
- α_i corresponds to the activation strength of concept c_i in the LR $\mathbf{h}$,
- $\mathbf{v}_i^\top \mathbf{v}_j \approx 0$ indicates that concept directions are approximately orthogonal, meaning each concept independently controls a single factor of variation,
- $\|\boldsymbol{\alpha}\|_0 \ll n$ indicates that only a small fraction of concepts are active for a given LR.

However, NNs operate in finite-dimensional spaces, so when the number of concepts exceeds the available dimensions, such a one-to-one correspondence becomes impractical. The network must therefore reuse its representational dimensions, allowing multiple concepts to be encoded using the same directions. This shared encoding is referred to as superposition [122]. This phenomenon becomes an obstacle for any method that attempts to analyze LRs directly in the original activation space.

Building on this observation, several works based on the *Superposition Hypothesis* (Hypothesis 2) have developed methods that aim to recover less superposed LRs. A common approach is to map LS into a higher-dimensional space while imposing additional constraints that encourage individual concepts to be represented separately. This transformation is illustrated in Figure 3.7, while LRs are entangled in the initial LS, the sparse representation aligns each concept with a dedicated feature axis.



(a) Dense latent space (b) Sparse SAE space

Figure 3.7: Transformation from a dense LS (a) to a sparse SAE representation (b).

Sparse Autoencoders (SAEs) provide a practical implementation of this approach and can be defined as follows:

$$\mathbf{z} = f_{\text{enc}}(\mathbf{h}), \quad \hat{\mathbf{h}} = f_{\text{dec}}(\mathbf{z}),$$

$$\mathcal{L}_{\text{auto}} = \|\mathbf{h} - \hat{\mathbf{h}}\|_2^2 + \lambda\|\mathbf{z}\|_1,$$

where:

- $f_{\text{enc}}(\mathbf{h})$ is the encoding function that projects the original LR $\mathbf{h}$ into a higher-dimensional sparse space,
- $f_{\text{dec}}(\mathbf{z})$ is the decoding function that reconstructs the original LR from the sparse code,
- $\mathbf{z} \in \mathcal{Z}$ is the sparse representation of $\mathbf{h}$, with $\dim(\mathbf{z}) \gg \dim(\mathbf{h})$,
- $\hat{\mathbf{h}}$ is the reconstructed LR,
- $\|\mathbf{h} - \hat{\mathbf{h}}\|_2^2$ is the reconstruction loss ensuring that the original LR can be faithfully recovered,
- $\|\mathbf{z}\|_1$ is the sparsity regularization term that encourages most components of $\mathbf{z}$ to be zero,
- λ is a weighting coefficient controlling the trade-off between reconstruction fidelity and sparsity.

The parameters of these projection functions f_{enc} and f_{dec} are learned by minimizing $\mathcal{L}_{\text{auto}}$ using SGD. Under this formulation, each dimension of $\mathcal{Z}$ is interpreted as a dedicated concept direction.

In practice, both f_{enc} and f_{dec} are implemented as linear projections between the original LS and the high-dimensional space $\mathcal{Z}$. This choice is motivated by the same principle underlying probing methods. The transformation applied to the LR should remain as simple as possible. Using more expressive projection functions could introduce additional structures into the transformed LR. In this case, it would become difficult to determine whether an observed concept



Figure 3.8: A concept direction corresponding to the Golden Gate Bridge, discovered by an SAE applied to an LLM [139].

was already encoded in the model's LR, or whether it was introduced by the projection itself. Restricting f_{enc} and f_{dec} to linear projections therefore helps ensure that the concepts identified in $\mathcal{Z}$ can be attributed to the original LR, rather than to the transformation used to analyze it.

The method proves to be powerful when successfully applied to LLMs [139]. In these applications, SAEs are used to desuperpose the LR of LLMs. Once this LR is obtained, it has been shown that many concept vectors exhibit semantic properties. An example is the Golden Gate Bridge concept direction, illustrated in Figure 3.8, which activates systematically whenever the input text or image refers to the Golden Gate Bridge, regardless of the language or modality. This demonstrates that the learned concept is not tied to a specific surface form but captures a deeper semantic abstraction shared across modalities.

SAEs also provide a way to perform latent interventions (see Definition 3.2.1) on the LR learned by the model. A concept can be artificially activated by modifying its corresponding feature in the sparse representation $\mathbf{z}$, even when the input contains no reference to it. The modified LR is then projected back into the original activation space, after which the model proceeds with standard next-token generation from $\mathbf{h}'$:

$$\mathbf{h}' = f_{\text{dec}}(\mathbf{z}^*), \quad \text{where} \quad \mathbf{z}^* = \mathbf{z} + \alpha\, \mathbf{e}^{(i)},$$

where:

- $\mathbf{z}$ is the original sparse representation of the input,
- $\mathbf{z}^*$ is the modified sparse representation,
- $\mathbf{e}^{(i)}$ is the basis vector of $\mathcal{Z}$ associated with the target concept direction,
- α controls the strength of the artificial activation,
- $\mathbf{h}'$ is the resulting edited LR in the original activation space.

With this method, a reference to the Golden Gate Bridge can be injected into the LLM's internal representation. This causes the model's output to shift toward discussing the Golden Gate Bridge, even when the prompt contains no reference to it. This provides evidence that the learned feature plays a causal role

in the model's behavior. Such latent interventions enable control over specific model behaviors through LS editing.

Despite their empirical success, *Superposition Hypothesis* (Hypothesis 2) exhibit several important limitations that question their assumptions. These limitations can be summarized as follows:

- **Instability of the decomposition.** Repeating the SAE training process multiple times on the same model does not yield the same decomposition of the LS. Different runs produce different concept directions in $\mathcal{Z}$, even when achieving comparable reconstruction performance, which raises questions about the reliability of the discovered CSs.
- **Lack of semantic coverage.** In the high-dimensional space $\mathcal{Z}$, a large fraction of dimensions do not correspond to any interpretable semantic concept. This abundance of non-semantic directions raises questions about whether the decomposition truly reflects the model's CSs.
- **Sparsity–semantics trade-off.** Increasing the sparsity coefficient λ beyond a certain threshold degrades the semantic quality of the learned features [133]. Suggesting that enforcing excessive sparsity comes at the cost of interpretability.
- **Failure of scaling to remove superposition.** Under the *Superposition Hypothesis* (Hypothesis 2), increasing the dimensionality of the LS should reduce superposition. However, this is not observed in practice. Even in very high-dimensional LSs, representations remain superposed, indicating that scaling alone does not resolve the entanglement.

These limitations suggest that *Superposition Hypothesis* (Hypothesis 2), while powerful, do not fully resolve the problem of CS. Convincing empirical results do not necessarily validate the underlying theoretical assumptions.

So far, every method described, whether based on probing, interpretable directions, or superposition, relies on a common assumption. The CS can be recovered from LRs through a linear decomposition. This assumption has enabled substantial progress in understanding neural representations, but it also imposes a strong constraint on how concepts are expected to be represented. The following section examines clustering as an alternative approach that does not rely on this assumption.

3.5 Clustering

As the previous section noted, every LS analysis method presented so far rests on a linear view of the CS. For probing methods, concepts must be at least partially linearly separable for simple probes to detect them. Interpretable direction methods posit that variations associated with a concept can be captured by moving along a linear axis of the LS. The *Superposition Hypothesis* (Hypothesis 2) proposes that concepts are represented through linear directions that become entangled because of representational capacity constraints.

Although these assumptions have led to empirical successes, several observations suggest that the linearity hypothesis may not fully capture the CS. For instance, some concepts can only be recovered using non-linear probes. Likewise,

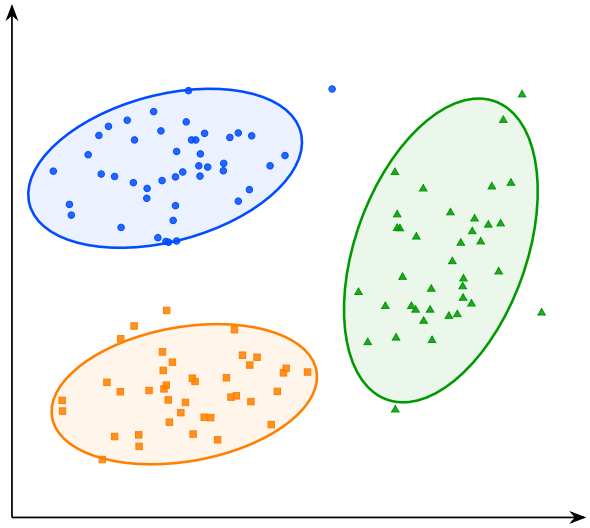


Figure 3.9: Visualization of the LS structure.

methods based on interpretable directions typically identify only a limited number of semantic factors. Finally, in SAE approaches, enforcing excessive sparsity degrades interpretability. This suggests that the underlying representation may not be perfectly described by a sparse linear decomposition. Taken together, these observations indicate that semantic concepts may not always exhibit a linear CS. This motivates the exploration of alternative approaches that rely on weaker assumptions regarding the geometry of LSs.

To the best of our knowledge, clustering-based analysis is the only family of LS analysis methods that does not rely on any assumption of a linear CS [140, 143]. Rather than assuming that concepts correspond to directions, clustering methods only assume that conceptual similar inputs produce nearby LRs. Conversely, inputs that differ significantly from a concept perspective are expected to be located farther apart in the LS. This is a considerably weaker assumption, in the sense that it is empirically verified in a wide range of settings.

Under this assumption, the discovery of concepts can be reformulated as the identification of groups of LRs that occupy the same region of the LS. This is shown schematically in Figure 3.9, where the points represent LRs associated with concepts, and the ellipses indicate the approximate extent of each group and highlight their geometric separation.

This idea forms the basis of the research field known as deep clustering. In these approaches, a collection of inputs is projected into a LS using a NN. The resulting LRs form a point cloud on which standard clustering algorithms can be applied. The obtained clusters are then interpreted as representing distinct concepts.

Deep clustering has been particularly successful in unsupervised image classification. For this type of task, a specific NN architecture called an autoencoder is commonly used. An autoencoder is composed of an encoder f_{enc} and a decoder f_{dec}. The encoder maps an input $\mathbf{x} \in \mathcal{X}$ to a low-dimensional LR $\mathbf{z} \in \mathbb{R}^d$, the LR produced by the encoder. The decoder then maps this LR back to the input space to produce a reconstruction $\hat{\mathbf{x}}$.

The model is trained by SGD to minimize the reconstruction error between the original input and its reconstruction using a Mean Squared Error (MSE) loss. This objective encourages the LS to preserve the most informative features of the input while enforcing a compact representation. This can be expressed mathematically as follows:

$$\mathbf{x} \xrightarrow{f_{\text{enc}}} \mathbf{z} \xrightarrow{f_{\text{dec}}} \hat{\mathbf{x}}, \qquad d \ll \dim(\mathcal{X}), \qquad \mathcal{L}_{\text{rec}} = \frac{1}{N}\sum_{i=1}^{N} \left\|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\right\|_2^2.$$

Once the autoencoder has been trained sufficiently well, the reconstruction loss $\mathcal{L}_{\text{rec}}$ should be low enough for the learned representation to be useful. A clustering algorithm can then be applied to the compressed representations of the images. Let $\mathcal{D}$ denote a dataset of inputs, through the encoder f_{enc}, this dataset is mapped to a point cloud in the LS:

$$\mathcal{Z} = \{\mathbf{z}^{(i)} \in \mathbb{R}^d \mid \mathbf{z}^{(i)} = f_{\text{enc}}(\mathbf{x}^{(i)}),\ \mathbf{x}^{(i)} \in \mathcal{D}\}.$$

A classical clustering algorithm is applied to the set $\mathcal{Z}$ of LRs to identify groups of similar LRs. Each resulting cluster can therefore be interpreted as a group of images that the model considers similar.

An example of this methodology can be observed in Figure 3.10, where the approach is applied to the STL-10 dataset [40] using k-means clustering [15] with a number of clusters arbitrarily fixed to $K = 10$ [56]. For each cluster, the corresponding line shows the 10 samples closest to the associated centroid in the LS. We observe that the method captures coherent semantic categories such as animals, vehicles, and flying entities.

This result is particularly striking given that the only available information to the system consists of raw images. It suggests that the compression process induces a LS in which semantically meaningful CSs emerge.

More advanced approaches combine representation learning and clustering within a single optimization process. In this setting, additional loss terms encourage LRs to organize themselves around cluster centroids during training. This often produces LSs that are easier to analyze and improves the quality of the discovered semantic CSs [60, 142].

Clustering-based analysis presents several important limitations. First, as with most unsupervised semantic discovery techniques, interpreting the resulting clusters remains partly subjective. Benchmark datasets often provide class labels for evaluation, but the discovered clusters may not match the categories that humans consider relevant.

Second, clustering methods are highly sensitive to design choices. The number of clusters, the distance metric, and the clustering algorithm itself can all significantly influence the results. Parameter selection is therefore challenging and often requires substantial empirical tuning.

Finally, to the best of our knowledge, no latent intervention technique has yet been proposed in the context of clustering-based analysis. Unlike probing or SAE analysis, which allow targeted modifications of LRs to study their causal role, clustering methods currently offer no mechanism to alter the model's behavior by acting directly on the discovered CSs.



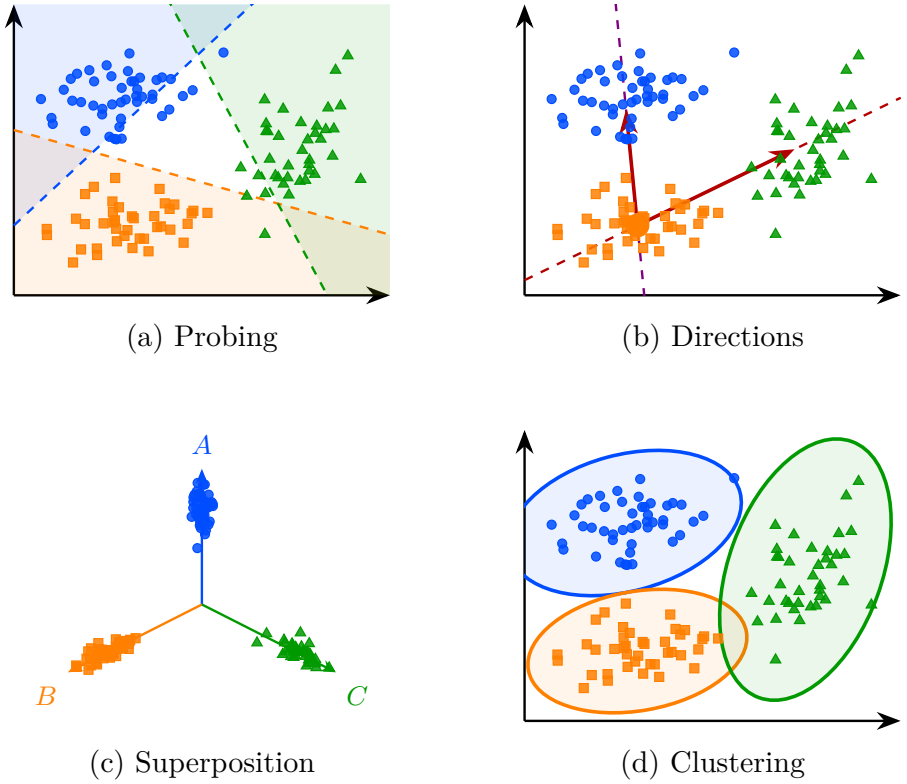
Figure 3.10: Clusters discovered by a deep clustering model applied to natural images [56].

3.6 Conclusion

The four families of methods reviewed in this chapter are summarized in Figure 3.11. They include probing, interpretable directions, SAEs, and clustering, all of which build upon the *Concept Hypothesis* (Hypothesis 1) introduced in Section 3.1. Each family approaches the analysis of LRs from a different angle. However, they share a common goal of understanding how LRs are structured and what information they contain.

A fundamental difficulty nevertheless remains. There is currently no consensus on how to rigorously evaluate whether these methods truly capture the internal concepts of a model. Most existing metrics assess reconstruction quality or linear separability, but these do not directly measure whether the discovered CSs correspond to meaningful abstractions from the model's perspective. This leaves open the question of what it means, formally, for a concept to be represented in a NN.

Addressing this question may require contributions from disciplines beyond computer science and mathematics. As the notion of semantics is inherently tied to human interpretation, a purely formal or computational account may be insufficient. Insights from philosophy, cognitive science, or psychology could prove valuable in grounding the notion of concept in a way that is both theoretically rigorous and empirically tractable. Despite these open questions, the methods



(a) Probing (b) Directions

(c) Superposition (d) Clustering

Figure 3.11: Overview of the four LS analysis methods studied in this chapter, applied to the same point cloud.

studied in this chapter provide a practical foundation for analyzing the internal representations of NNs.

Building on the *Concept Hypothesis* (Hypothesis 1), this thesis investigates a representation-based approach to explainability in RL. The next chapter introduces the principles of RL to establish the necessary foundations.

Chapter 4

Reinforcement Learning

"Cum enim Mundi universi fabrica sit perfectissima atque a Creatore sapientissimo absoluta, nihil omnino in mundo contingit, in quo non maximi minimive ratio quaepiam eluceat."

"For since the fabric of the universe is most perfect and the work of a most wise Creator, nothing at all takes place in the universe in which some rule of maximum or minimum does not appear."

Leonhard Euler, *Methodus inveniendi lineas curvas* (1744) [1]

Contents

4.1 Historical Developments . . . 52
4.1.1 Behavioral Foundations (1900–1950) . . . 52
4.1.2 Sequential Decision-Making (1950–1970) . . . 52
4.1.3 Emergence of Reinforcement Learning (1970–2000) . . . 53
4.1.4 Consolidation of Reinforcement Learning (2000–2012) . . . 53
4.1.5 Deep Reinforcement Learning (2013–2026) . . . 53
4.2 Agent Paradigm . . . 55
4.3 Problem Formulation . . . 56
4.3.1 Markov Decision Processes . . . 56
4.3.2 Policy . . . 57
4.3.3 Trajectory . . . 58
4.3.4 Value Functions . . . 59
4.3.5 Optimal Policy . . . 61
4.4 Learning . . . 62
4.4.1 Critic-Only Methods . . . 62
4.4.2 Actor-Critic Methods . . . 65
4.5 Conclusion . . . 66

This chapter introduces the fundamental concepts of Reinforcement Learning (RL), providing the necessary foundation for the ideas developed throughout the rest of this thesis. It first traces the milestones that led RL to emerge as a

unified field. It then clarifies what is meant by an agent and how it relates to the RL algorithm, before formalizing the problem that such algorithms aim to solve. Next, it explains how policies are progressively refined through interaction with the environment to approach the optimal policy. Finally, it closes with a summary of the key ideas and a discussion of the broader significance of RL.

4.1 Historical Developments

RL is today presented as a major field within Artificial Intelligence (AI). However, it originates from several research traditions that progressively converged toward a common framework. In this section, we outline the main stages in the emergence of this field, highlighting the key ideas and milestones that shaped its evolution.

4.1.1 Behavioral Foundations (1900–1950)

The origins of RL can be traced back to 20th-century animal psychology studies [7]. These studies aimed to understand how behaviors are formed and modified. The central idea emerging from this work is that learning is driven by the consequences of actions. Behaviors followed by positive outcomes are more likely to be repeated, whereas those associated with negative outcomes tend to gradually decrease in frequency.

This perspective was later developed within behaviorism. In this school of psychology, rewards and punishments are used to gradually shape behavior through repeated interaction with the environment [9]. The term "reinforcement" in the context of learning comes from this school of thought, where it refers to the process of strengthening a behavior through its consequences.

This idea led to the suggestion that learning principles observed in living organisms could be transferred to artificial systems. Machines were therefore envisioned as potentially learning through reward-like mechanisms, guided by evaluative feedback signals resembling a "pleasure–pain system" [11]. Although these studies did not involve formal mathematical models, they introduced the central idea of RL, trial-and-error learning guided by environmental feedback.

4.1.2 Sequential Decision-Making (1950–1970)

A second foundation of RL comes from optimal control theory. This field studies how to determine sequences of decisions that optimize a cumulative performance criterion over time. In this context, the decision-making entity is called an agent. The agent interacts with the environment by selecting actions. The function that maps an observed state to the chosen action is called a policy. The objective is to find an optimal policy, in the sense that it maximizes the expected cumulative reward over time.

This framework led to the formalization of Markov Decision Processes (MDPs), which provide a mathematical model for sequential decision-making [14]. MDPs describe an agent interacting with an environment through a set of states, actions, rewards, and transition rules. The earliest method proposed to solve MDPs is dynamic programming [12]. It proceeds by breaking down a complex decision problem into smaller, more manageable subproblems, solving each one, and combining the results to obtain an overall solution. However, dynamic

programming requires a full description of the environment, which is only feasible for small problems.

4.1.3 Emergence of Reinforcement Learning (1970–2000)

In order to address this issue, temporal-difference learning methods were introduced [24]. This mechanism enables incremental learning directly from experience without requiring a complete model of the environment. It can be interpreted as a sample-based approximation of the Bellman equations.

Building on these ideas, the notion of value functions emerged as a way to estimate the long-term benefit of being in a given state or taking a given action. These functions assign a numerical score to states or state-action pairs, reflecting the expected cumulative reward an agent can obtain from that point onward. They thus provide a compact way to evaluate and compare different courses of action. This line of work culminated in Q-learning, which showed that optimal action-value functions can be learned directly from interaction with the environment, without requiring a model of its dynamics [27, 28].

The publication in 1998 of the foundational book *Reinforcement Learning: An Introduction* by Sutton and Barto [36] contributed to the formalization of RL as a coherent field. From that point onward, the field can be considered well established and widely recognized as a distinct area of research.

4.1.4 Consolidation of Reinforcement Learning (2000–2012)

A limitation of early algorithms such as Q-learning is their reliance on explicit state–action tables. Although these methods remove the need to know the full dynamics of the MDP, they still require storing a value for each possible state–action pair. As a result, the agent must effectively visit and maintain estimates for a potentially enormous number of states in the environment. This requirement becomes infeasible in large-scale MDPs, where the state space is too large to be exhaustively explored or stored in memory.

To address this issue, research shifted toward more general representations of value functions and policies. Instead of maintaining explicit tables, function approximation methods were introduced. These allowed value functions and policies to be represented using parametric models such as linear approximators or perceptron-like architectures [21, 34]. This change enabled generalization across states, reducing the dependence on exhaustive exploration of the state space. These parametric representations nonetheless remained limited in expressiveness.

Figure 4.1 provides a timeline of the key milestones that shaped the development of RL prior to its fusion with Deep Learning (DL).

4.1.5 Deep Reinforcement Learning (2013–2026)

The introduction of DL into RL marked a major turning point in the field. This shift was initiated by Deep Q-Network (DQN), which used a deep Neural Network (NN) to approximate action-value functions at scale. This made it possible to train an agent to play Atari games at a human level directly from raw pixels [47]. The use of deep NNs provided RL with a powerful and flexible function approximator. This removed an important practical limitation by making it possible to efficiently represent the complex functions required by RL.

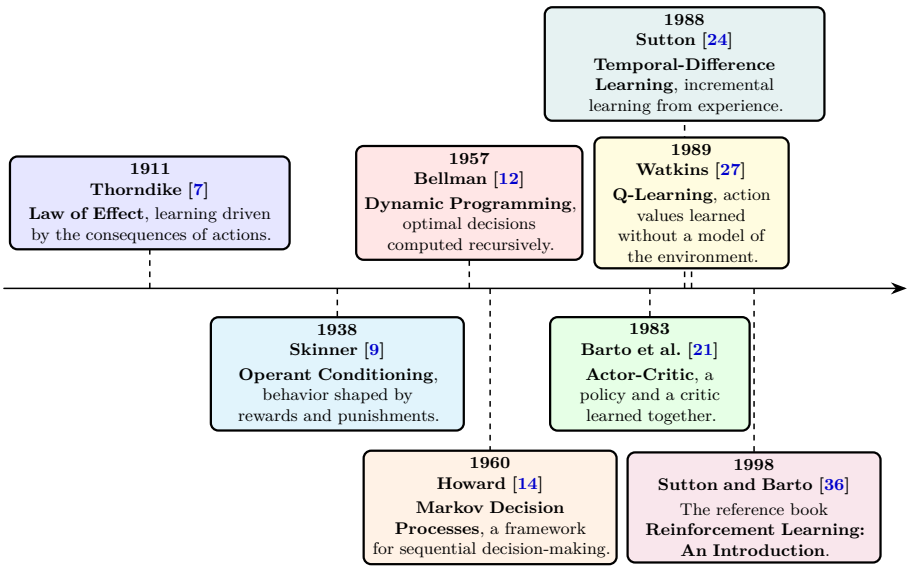


Figure 4.1: Evolution of RL prior to its fusion with DL.

A series of subsequent algorithms further extended and improved this approach. Deep Deterministic Policy Gradient (DDPG) extended actor–critic methods to continuous action spaces [54]. Proximal Policy Optimization (PPO) later became a widely adopted algorithm in RL due to its simplicity, stability, and ability to handle both continuous and discrete action spaces [62].

These advances established a foundation for large-scale policy optimization. Notably, AlphaZero demonstrated that the combination of deep NNs, RL, and tree search could reach superhuman performance in the game of Go [72]. This success was later extended to even more complex real-time strategy environments such as StarCraft II, where agents must handle long-term planning, partial observability, and large action spaces [89]. Similarly, in Dota 2, large-scale multi-agent RL systems such as OpenAI Five showed that self-play combined with deep policy optimization can achieve professional-level performance in highly dynamic and stochastic environments [84].

More recently, RL has expanded beyond its traditional application domains. With the emergence of increasingly agentic views of Large Language Models (LLMs), it has become possible to train these models to perform specific tasks using RL-based techniques. In particular, RL from human feedback has become a method for aligning LLMs with human preferences. It does so by formalizing the problem as an MDP and using RL to solve it [119].

Similarly, DeepSeekMath shows how RL can enhance the mathematical reasoning abilities of LLMs [137]. It demonstrates that an artificial agent can learn to reason in natural language in order to accomplish a given task. The skills acquired during this learning process also transfer to other similar tasks, leading to improved overall performance. For instance, training a LLM with RL on mathematical problem-solving can also enhance its programming capabilities.

Figure 4.2 provides a timeline of the key milestones that shaped the emergence of deep RL.

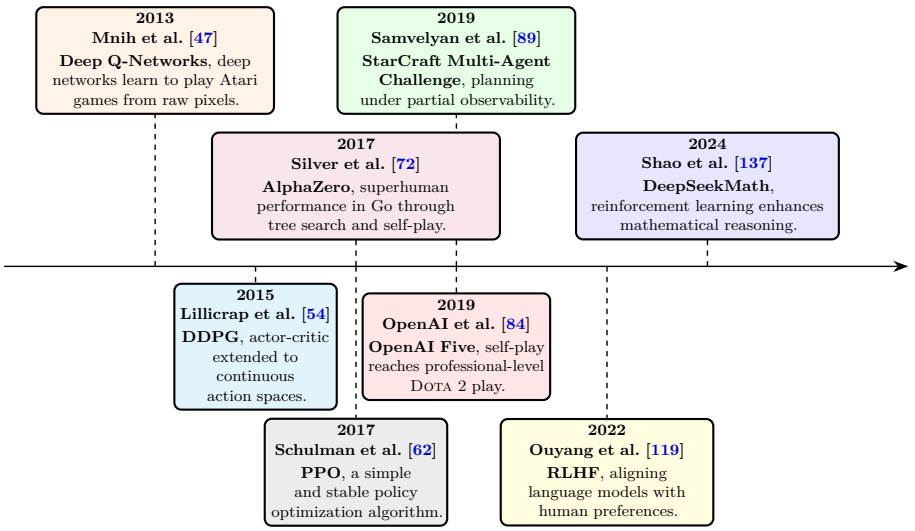


Figure 4.2: Evolution of deep RL.

4.2 Agent Paradigm

The RL framework is built around the agent paradigm, in which the decision-making entity is referred to as the agent. A standard definition is:

> *An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.* [110]

In the RL setting, the agent perceives its environment through observations. These observations are then processed by a decision-making mechanism referred to as the policy. The policy is a function that maps each observation to a probability distribution over the possible actions. It determines how the agent modifies its environment through the actions it selects. Figure 4.3 illustrates this interaction.

The objective of an RL agent is to maximize the cumulative reward it receives through its interactions with the environment. To achieve this objective, the agent's policy must progressively improve through experience. The procedure responsible for this learning process is the RL algorithm, which progressively modifies the policy based on the experience collected through interaction.

In this thesis, the RL algorithm is considered as a process external to the agent. Other presentations of RL describe the agent as both the learner and the decision-making entity [73]. We chose to distinguish the agent from the RL algorithm because this provides a more general definition of an agent. An agent can therefore exist without learning, for example when its behavior is entirely specified by a set of programmed rules. Such an agent still perceives, processes information, and acts upon its environment, but its behavior is not the result of RL.

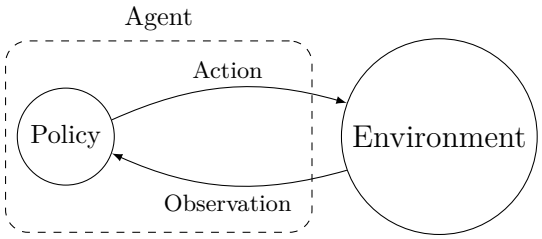


Figure 4.3: The agent paradigm.

4.3 Problem Formulation

This section translates the agent-environment interaction of Figure 4.3 into a mathematical framework. In this framework, the goal of RL, finding a policy that makes sequential decisions so as to maximize the cumulative reward over the long term, can be stated precisely. The presentation proceeds step by step:

1. Modeling the RL problem as an MDP.
2. Defining the notion of a policy, which formalizes the agent's behavior.
3. Describing the interaction between the agent and the environment through trajectories.
4. Introducing value functions to evaluate policies.
5. Defining the notion of an optimal policy.

4.3.1 Markov Decision Processes

To specify the task and the agent's capacity to perceive and act, the problem is modeled as a MDP. Before providing a more detailed definition, it is useful to clarify the scope of the MDP problems considered in this thesis. We therefore restrict the analysis to the following configuration:

- The single-agent learning setting is considered. If any other agents are present in the environment, they are treated as part of the environment dynamics.
- Episodic MDPs are considered. Each interaction sequence between the MDP and the policy has a defined beginning and a defined end. Between these two points, the number of interaction steps may vary depending on the situation, but it always remains finite.
- The partially observable framework is adopted, where the agent does not have direct access to the true state of the system, but instead relies on limited observations that provide only partial information about the environment.

The resulting framework can be formally described as a single-agent episodic partially observable MDP. For simplicity, we will refer to this setting as an "MDP" throughout this thesis.

A MDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$:

- $\mathcal{S}$: The set of states, representing all possible configurations of the environment. A state belonging to this set is denoted by $s \in \mathcal{S}$. Among these, two special subsets are distinguished, the initial states, from which an interaction can begin, and the terminal states, in which the interaction ends. The set of initial states is denoted by $\mathcal{S}^{\text{init}}$, with an initial state written as $s^{\text{init}} \in \mathcal{S}^{\text{init}}$. The set of terminal states is denoted by $\mathcal{S}^{\text{ter}}$, with a terminal state written as $s^{\text{ter}} \in \mathcal{S}^{\text{ter}}$.

- $\mathcal{O}$: The set of observations that the agent can perceive from the environment. An observation belonging to this set is denoted by $o \in \mathcal{O}$. Unlike the state $s \in \mathcal{S}$, which is hidden from the agent, the observation $o \in \mathcal{O}$ provides partial information about the current state.

- $\mathcal{A}$: The set of actions that the agent may execute. An action belonging to this set is denoted by $a \in \mathcal{A}$. Actions influence the dynamics of the MDP, thereby shaping its future course. They represent the agent's capacity to act upon its environment.

- $\mathcal{R}$: The set of rewards that the agent can receive from the environment. A reward belonging to this set is denoted by $r \in \mathcal{R}$. $\mathcal{R}$ is a subset of $\mathbb{R}$. Rewards can be either positive or negative, depending on whether the corresponding outcome is considered beneficial or detrimental for the agent. The agent's goal is to maximize the cumulative sum of these rewards over time.

- p: The transition function, which defines the dynamics of the environment. Given a state s and an action a, this function defines a probability distribution over the joint outcome consisting of the next state s', the next observation o', and the reward r. This function is denoted by p, and is defined as: $p : \mathcal{S} \times \mathcal{A} \to \mathcal{P}(\mathcal{S} \times \mathcal{O} \times \mathcal{R})$. The notation $(s', o', r) \sim p(\cdot \mid s, a)$ means that the tuple (s', o', r) is sampled from the distribution induced by $p(\cdot \mid s, a)$. The notation $p(s', o', r \mid s, a)$ denotes the probability of observing the tuple (s', o', r) under the distribution $p(\cdot \mid s, a)$.

The two quantities, observation o and state s, are not independent. They are jointly generated via $(s', o', r) \sim p(\cdot \mid s, a)$, which implicitly defines the relationship between them. In our MDP formalization, this relationship is left implicit because it is never manipulated directly during learning.

4.3.2 Policy

The agent's decision-making capability is modeled by a policy function. The policy defines the stochastic mapping between observations $o \in \mathcal{O}$ and actions $a \in \mathcal{A}$. A policy function is denoted by π, and is defined as: $\pi : \mathcal{O} \to \mathcal{P}(\mathcal{A})$. The notation $a \sim \pi(\cdot \mid o)$ means that the action a is sampled from the distribution induced by $\pi(\cdot \mid o)$. The notation $\pi(a \mid o)$ denotes the probability of selecting action a under the distribution $\pi(\cdot \mid o)$.

It is important to note that this policy does not include any explicit mechanism for estimating the underlying state from the observations. Instead, the policy must learn from experience which actions are best given an observation, taking

implicitly into account the different states that may underlie it. This reflects the design of many modern RL implementations.

In this chapter, the policy is memoryless, meaning that its decision depends only on the current observation o. Policies can also be given a memory, by parameterizing them with architectures capable of summarizing past observations, such as recurrent networks [32, 50, 80] or attention-based networks [63, 100]. Such a policy maintains an internal state, updated at each time step from the new observation. This case is not detailed here, since it only complicates the notation without changing the concepts presented in this chapter.

4.3.3 Trajectory

Now that both the MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$ and the policy π have been defined, the interaction between them can be characterized. A sequence of consecutive interactions between an environment and a policy is called a trajectory. A trajectory is generated through the repeated interaction between the policy and the environment, following the sequence below:

1. The environment, being in state s, provides an observation o to the policy π. Based on this observation, the policy outputs a probability distribution over actions $\pi(\cdot \mid o)$, from which an action is sampled: $a \sim \pi(\cdot \mid o)$.

2. The sampled action is executed in the environment. The environment then generates a transition: $(s', o', r) \sim p(\cdot \mid s, a)$. It moves to a new state s', and returns a new observation o' and a reward r.

Indexing each interaction by a discrete time step t, a trajectory τ of length T can be written as:

$$\tau = (s_1, o_1, a_1, r_1, s_2, o_2, a_2, r_2, \ldots, s_T, o_T, a_T, r_T),$$

where r_t is the reward received after taking action a_t at time step t.

A trajectory is stochastic, since it is drawn from a distribution determined by the initial state s, the initial observation o, the environment dynamics p, and the policy π. Since the dynamics p are fixed, they are omitted from the notation. The probability of observing the trajectory τ when following the policy π, starting from state s with observation o, is denoted by $\Pr_\pi(\tau \mid s, o)$.

An important property of the trajectory distribution is its recursive structure. Since trajectories are generated step by step by the policy π and the transition function p, the distribution over trajectories starting from s_1 with observation o_1 can be expressed in terms of the distribution over trajectories starting from the next state s_2 with observation o_2. Specifically, the probability of a trajectory τ factorizes as:

$$\Pr_\pi(\tau \mid s_1, o_1) = \pi(a_1 \mid o_1)\, p(s_2, o_2, r_1 \mid s_1, a_1)\, \Pr_\pi(\tau_{2:} \mid s_2, o_2),$$

where $\tau_{t:} = (s_t, o_t, a_t, r_t, \ldots, s_T, o_T, a_T, r_T)$ denotes the part of the trajectory starting from time step t.

There exist two special types of trajectories. First, a trajectory whose last transition leads to a terminal state $s_{T+1} \in \mathcal{S}^{\text{ter}}$ is called a terminal trajectory, and is denoted by τ^{ter}. Since no action is taken in a terminal state, this state is

not included in τ. Second, a terminal trajectory that starts from an initial state s^{init} and an initial observation is called an episode, and is denoted by τ^{ep}.

Since trajectories are stochastic, the state, observation, action and reward at each time step are random variables. They are denoted by the capital letters S_t, O_t, A_t and R_t, while the lowercase letters s, o, a and r denote particular values that these variables can take. For instance, $S_t = s$ expresses the event that the state at time step t is s. A trajectory written with lowercase letters, as above, is therefore a realization of this stochastic process. Expectations over the trajectories generated by the policy π are written $\mathbb{E}_\pi$.

4.3.4 Value Functions

To evaluate the ability of a policy π to collect rewards, the notion of return is introduced.

Return

The return G_t is defined as the discounted sum of the rewards collected along a terminal trajectory from time step t onward:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots + \gamma^{T-t} R_T = \sum_{k=t}^{T} \gamma^{k-t} R_k,$$

where $\gamma \in [0, 1]$ is the discount factor. Since the rewards R_k are random variables, the return G_t is itself a random variable.

The discount factor γ controls the time horizon of the agent's objective. When γ is close to 0, the agent focuses almost exclusively on immediate rewards, leading to short-sighted behavior. As γ increases toward 1, future rewards gain more weight, and the agent adopts a more far-sighted strategy. In theory, since the goal is to maximize long-term cumulative reward, the discount factor would ideally be set to $\gamma = 1$. In practice, γ is often chosen slightly below 1 (e.g., 0.99) to stabilize learning, and to reflect the fact that distant future rewards are inherently more uncertain.

The return also admits a recursive form, which is central to the dynamic programming and RL algorithms presented later:

$$G_t = R_t + \gamma\, G_{t+1}, \qquad G_{T+1} = 0.$$

For a particular trajectory τ, the return takes a fixed value, written $G(\tau) = \sum_{t=1}^{T} \gamma^{t-1} r_t$. The recursive form then reads $G(\tau) = r_1 + \gamma\, G(\tau_{2:})$.[1]

State-Value Function

From the notion of return, functions that estimate the expected return of a policy from a given starting point can be defined. These functions are called value functions. The first one is the state-value function, which tells how desirable a state s with observation o is under a policy π. Strictly speaking, this function depends on the state s, the observation o, the dynamics p, the discount factor γ and the policy π. Since p and γ are fixed throughout learning, they are omitted

[1] With $G(\tau_{2:}) = 0$ when τ consists of a single interaction step.

from the notation. The policy is written as a subscript to emphasize that the value is tied to it, which yields the compact notation $v_\pi : \mathcal{S} \times \mathcal{O} \to \mathbb{R}$.

If $v_\pi(s, o) > v_\pi(s', o')$, then, under policy π, being in state s with observation o is preferable to being in state s' with observation o'. Formally, the state-value function is the expected return when the state is s and the observation is o, and the policy π is followed from then on:[2]

$$v_\pi(s, o) = \mathbb{E}_\pi\big[\, G_t \mid S_t = s, O_t = o \,\big] = \sum_\tau \Pr_\pi(\tau \mid s, o)\, G(\tau),$$

where the sum ranges over the terminal trajectories starting from (s, o).

The state-value function is recursive, a property that follows directly from the recursive form of the return. Replacing G_t by $R_t + \gamma\, G_{t+1}$ in the definition of v_π and expanding the expectation over the action and the transition gives the Bellman equation for v_π,

$$\begin{aligned} v_\pi(s, o) &= \mathbb{E}_\pi\big[\, G_t \mid S_t = s, O_t = o \,\big] \\ &= \mathbb{E}_\pi\big[\, R_t + \gamma\, G_{t+1} \mid S_t = s, O_t = o \,\big] \\ &= \mathbb{E}_\pi\big[\, R_t + \gamma\, v_\pi(S_{t+1}, O_{t+1}) \mid S_t = s, O_t = o \,\big] \\ &= \sum_a \pi(a \mid o) \sum_{s', o', r} p(s', o', r \mid s, a) \Big[\, r + \gamma\, v_\pi(s', o') \,\Big]. \end{aligned}$$

The value of a state is thus expressed through the values of the states that can follow it.

State-Action Value Function

The next step is to consider the value of taking a specific action in a given state, in order to compare actions under a given policy π. This function is called the state-action value function $q_\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$. If $q_\pi(s, a) > q_\pi(s, a')$, then, under policy π, choosing action a in state s yields a higher expected return than choosing action a'. It is the expected return when action a is taken in state s and the policy π is followed afterwards,

$$q_\pi(s, a) = \mathbb{E}_\pi\big[\, G_t \mid S_t = s, A_t = a \,\big].$$

The observation O_t does not appear in the conditioning, since the observation only influences the choice of the action, which is here fixed to a.

The function q_π can be related to v_π through the same steps as for the Bellman equation. The only difference is that the first action is no longer drawn from π but fixed to a, which gives

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}_\pi\big[\, R_t + \gamma\, G_{t+1} \mid S_t = s, A_t = a \,\big] \\ &= \mathbb{E}_\pi\big[\, R_t + \gamma\, v_\pi(S_{t+1}, O_{t+1}) \mid S_t = s, A_t = a \,\big] \\ &= \sum_{s', o', r} p(s', o', r \mid s, a) \Big[\, r + \gamma\, v_\pi(s', o') \,\Big]. \end{aligned}$$

The first line uses the recursive form of the return, the second line the definition of v_π, and the third line makes the expectation over the transition explicit.

[2] By convention, $v_\pi(s, o) = 0$ for a terminal state $s \in \mathcal{S}^{\text{ter}}$.

Relation Between Value Functions

Each value function can be expressed in terms of the other. In one direction, the derivation above expresses q_π in terms of v_π,

$$q_\pi(s,a) = \sum_{s',o',r} p(s',o',r \mid s,a)\Big[\, r + \gamma\, v_\pi(s',o') \,\Big]. \tag{4.1}$$

In the other direction, the inner sum over (s',o',r) in the Bellman equation for v_π is exactly the right-hand side of Equation (4.1). Replacing it by $q_\pi(s,a)$ expresses v_π in terms of q_π,

$$\begin{aligned} v_\pi(s,o) &= \sum_a \pi(a \mid o) \underbrace{\sum_{s',o',r} p(s',o',r \mid s,a)\Big[\, r + \gamma\, v_\pi(s',o') \,\Big]}_{q_\pi(s,a)} \\ &= \sum_a \pi(a \mid o)\, q_\pi(s,a). \end{aligned} \tag{4.2}$$

The value of a state is therefore the average of the values of the actions, weighted by the probability that the policy selects each of them.

4.3.5 Optimal Policy

The goal of RL is to find a policy that maximizes the expected return of the episodes it generates. This policy is called the optimal policy and is denoted by π^*.

Since the policy only receives observations, it must act on an estimate of the underlying state. The best possible estimate given o is the distribution $\beta_\pi(s \mid o)$, the probability that the state is s when o is observed. It depends on the policy π, since the states in which o is encountered depend on the actions taken before. This distribution is introduced here only to characterize the optimal policy. The RL algorithms presented in the next section never compute it explicitly, and a policy trained on its own experience only accounts for it implicitly. The optimal policy only selects actions that maximize the expected state-action value over this estimate:

$$\pi^*(a \mid o) \begin{cases} \in [0,1] & \text{if } a \in \arg\max_{a' \in \mathcal{A}} \sum_{s \in \mathcal{S}} \beta_{\pi^*}(s \mid o)\, q_{\pi^*}(s,a'), \\ = 0 & \text{otherwise.} \end{cases}$$

As shown, both v_π and q_π can be expressed explicitly in terms of the environment dynamics p and the policy π. If p and π are fully known, one can theoretically compute the optimal policy by unfolding the graph of all possible trajectories starting from a given state s and observation o. This is possible precisely because v_π and q_π are recursive functions. Solving an MDP in this way is known as dynamic programming.

In practice, most environments have too many states to allow an explicit representation of the dynamics p. Storing and computing over the full transition function becomes intractable. To overcome this limitation, an iterative approach is adopted. Starting from an initial policy, the policy is progressively improved so that it converges toward π^*. This iterative improvement through interaction with the environment is precisely what constitutes learning in RL, and it is the subject of the next section.

4.4 Learning

Now that the optimal policy π^* has been defined, the question is how to obtain it. The recursive Bellman equations for v_π and q_π suggest dynamic programming approaches, but these require an explicit representation of the dynamics p and scale poorly to large state spaces. RL offers an alternative by solving the MDP iteratively, generating a sequence of policies that progressively approach π^* through direct interaction with the environment.

At a high level, every algorithm presented below repeats the same loop. The current policy interacts with the environment, the resulting trajectories are passed to the RL algorithm, and the algorithm uses them to produce a new policy. This section introduces two major families of RL algorithms, critic-only methods and actor-critic methods.

4.4.1 Critic-Only Methods

The central idea behind all critic-only approaches is to iteratively construct an approximation of the state-action value function q_π, denoted by $\hat{q}$. Although $\hat{q}$ is only an approximation of the true q_π, the policy always acts greedily with respect to this approximation.

Since $\hat{q}$ serves as the basis for the agent's decisions, it cannot rely on more information than the agent itself has access to. It must therefore be defined solely over observations, as a function $\hat{q} : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$. Its objective is to approximate the true q_π, which depends on the underlying states s, using only the partial information available through o. In practice, this does not necessarily pose a problem, as observations often contain sufficient information to make accurate predictions about the underlying state.

This leads to the following mathematical formulation:

$$\pi(a \mid o) = \begin{cases} 1 & \text{if } a = \arg\max_{a' \in \mathcal{A}} \ \hat{q}(o, a'), \\ 0 & \text{otherwise.} \end{cases} \tag{4.3}$$

It is precisely this way of modeling the policy that gives this family of methods its name. The policy is not represented explicitly as a standalone function. Rather, it simply emerges from searching over the set of actions to maximize $\hat{q}$.

To give a clear example, critic-only methods are presented in their simplest form. This version is model-free and relies only on ε-greedy exploration, without temporal-difference learning, replay buffer or bootstrapping. Naturally, many variants and extensions of this basic scheme exist, but the core idea remains the same.

When implementing a RL algorithm, one of the first questions is how to represent the value function. In critic-only methods, this means choosing a model for $\hat{q}$. Any machine learning model capable of performing regression can be used for this purpose. The parameters of the model are gathered in the vector $\boldsymbol{\phi}$, and the approximation is written $\hat{q}_{\boldsymbol{\phi}}$ when this dependence matters. In deep RL, a NN is used to model this function, in which case $\boldsymbol{\phi}$ gathers the weights and biases of the network.

In our simple setting, exploration is handled via ε-greedy selection. With probability $1 - \varepsilon$, the policy selects the greedy action that maximizes $\hat{q}$, and with

Algorithm 2: Data Collection

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Policy π
Discount factor $\gamma \in [0, 1]$
Output:
Dataset $\mathcal{D}$

Sample an initial state.
$t \leftarrow 1$;
Sample $s_1 = s^{\text{init}} \in \mathcal{S}^{\text{init}}$;
Observe o_1;
Episode generation by interaction with the MDP.
repeat
  Action selection.
  Sample $a_t \sim \pi(\cdot \mid o_t)$;
  Environment transition.
  Sample $(s_{t+1}, o_{t+1}, r_t) \sim p(\cdot \mid s_t, a_t)$;
  $t \leftarrow t + 1$;
until $s_t \in \mathcal{S}^{\text{ter}}$;
Return computation for each step of the trajectory.
$T \leftarrow t - 1$;
for $t = 1, \ldots, T$ **do**
  $g_t \leftarrow \sum_{k=t}^{T} \gamma^{k-t} r_k$;
Dataset construction
$\mathcal{D} \leftarrow \{(s_t, o_t, a_t, g_t)\}_{t=1}^{T}$;
Return $\mathcal{D}$

probability ε it selects an action uniformly at random. Formally, this ε-greedy policy is obtained by modifying the greedy policy defined above as follows:

$$\pi(a \mid o) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|} & \text{if } a = \arg\max_{a' \in \mathcal{A}} \hat{q}(o, a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|} & \text{otherwise.} \end{cases} \tag{4.4}$$

This ε-greedy policy is still a policy π, only more stochastic than the greedy one. The parameter ε controls the degree of exploration. If the policy always chooses the action it currently believes to be optimal, it may miss better alternatives that $\hat{q}$ has not yet learned to value correctly. Exploring suboptimal actions from time to time is therefore essential to verify that $\hat{q}$ accurately reflects the true values, while still focusing most of the time on the most promising actions. In more advanced implementations, ε can be adapted over the course of learning to control exploration dynamically, for instance by decreasing it gradually over time.

The learning process consists of two steps that repeat until convergence:

1. **Data collection.** The current policy π interacts with the MDP. For a given initial state s^{init} and observation o_1, the interaction produces an episode τ^{ep} by following π. For each time step t along the episode, the observed return from that step onward, g_t, serves as the target value for the observation-action pair (o_t, a_t).

2. **Model update.** The tuples (o_t, a_t, g_t) obtained in the previous step serve as supervised training data, in which the observation-action pair (o_t, a_t) is the input and the observed return g_t is the associated target. These examples are used to train the current approximation $\hat{q}_{\boldsymbol{\phi}_k}$, associated with the current policy $\pi_{\boldsymbol{\phi}_k}$. The improved approximation is denoted by $\hat{q}_{\boldsymbol{\phi}_{k+1}}$, which now predicts returns more accurately under $\pi_{\boldsymbol{\phi}_k}$. Since $\hat{q}_{\boldsymbol{\phi}_{k+1}}$ has changed, the policy derived from it also changes. This new policy is denoted by $\pi_{\boldsymbol{\phi}_{k+1}}$.

Algorithm 2 details the data collection step. Lines 1–3 sample the initial state and its observation. The loop of Lines 5–8 generates the episode, each step sampling an action from π and the resulting transition of the MDP until a terminal state is reached. Lines 9–11 then compute the observed return g_t of every time step, and Line 12 gathers the resulting tuples into the dataset $\mathcal{D}$.

Because the policy is now $\pi_{\boldsymbol{\phi}_{k+1}}$, the approximation $\hat{q}_{\boldsymbol{\phi}_{k+1}}$ must be trained to predict returns under $\pi_{\boldsymbol{\phi}_{k+1}}$. This requires returning to step 1 to collect fresh interaction data generated by $\pi_{\boldsymbol{\phi}_{k+1}}$. The two steps thus alternate until the policy and the value approximation stabilize.

Algorithm 3 summarizes this whole critic-only learning loop. Line 2 initializes the parameters $\boldsymbol{\phi}$, Line 3 defines the ε-greedy policy derived from $\hat{q}_{\boldsymbol{\phi}}$, and Line 4 sets the Mean Squared Error (MSE) as the training loss. The loop starting at Line 5 alternates the two steps described above. Line 6 corresponds to the data collection step, while Lines 7–8 correspond to the model update step, which extracts the observation-action pairs and their returns and trains $\hat{q}_{\boldsymbol{\phi}}$ on them by Stochastic Gradient Descent (SGD). Once learning has stopped, Line 9 removes exploration and returns the greedy policy with respect to the final approximation.

Even this simple algorithm already illustrates the instability of the learning process, a difficulty inherent to all forms of RL. Each update from $\hat{q}_{\boldsymbol{\phi}_k}$ to $\hat{q}_{\boldsymbol{\phi}_{k+1}}$ causes the policy to shift from $\pi_{\boldsymbol{\phi}_k}$ to $\pi_{\boldsymbol{\phi}_{k+1}}$. For the learning to remain stable, the change from $\hat{q}_{\boldsymbol{\phi}_k}$ to $\hat{q}_{\boldsymbol{\phi}_{k+1}}$ should be as small as possible. This way, $\hat{q}_{\boldsymbol{\phi}_{k+1}}$, trained to predict returns under $\pi_{\boldsymbol{\phi}_k}$, remains reasonably accurate under $\pi_{\boldsymbol{\phi}_{k+1}}$.

However, the smaller the change, the slower the learning progresses. A trade-off must therefore be struck between stability and learning speed. Many improvements introduced later in the literature aim at stabilizing this learning loop [67].

Convergence guarantees for such algorithms have been proved under a different set of assumptions than the setting presented here. For example, in the classical convergence results, the MDP is fully observable, the value function is stored in a table, and the learning rate is decreased at a suitable rate over time [28]. In modern implementations, the value function is approximated by a NN. Theoretical convergence proofs no longer hold in this setting. Nevertheless, deep methods have demonstrated remarkable empirical success, making them the tool of choice for RL tasks.

Algorithm 3: Deep Critic-Only Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{q}$
Exploration rate $\varepsilon \in [0, 1]$
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters $\boldsymbol{\phi}$ (optional)
Output:
Trained policy $\pi_{\boldsymbol{\phi}}$

---

if $\boldsymbol{\phi}$ *is not given* **then**
  Initialize $\boldsymbol{\phi}$ randomly;
Exploration policy as defined in Equation (4.4)
$\pi_{\boldsymbol{\phi}} \leftarrow \varepsilon$-greedy policy with respect to $\hat{q}_{\boldsymbol{\phi}}$
MSE loss function as defined in Equation (2.2)
$\mathcal{L} \leftarrow$ MSE;
while *stopping criterion not satisfied* **do**
  Data collection see Algorithm 2
  $\mathcal{D} \leftarrow$ `DataCollection`(*MDP,* $\pi_{\boldsymbol{\phi}}, \gamma$)
  observation-action pairs and their returns, from $\mathcal{D}$
  $\mathcal{D}' \leftarrow \big\{\big((o, a), g\big) \mid (s, o, a, g) \in \mathcal{D}\big\}$
  Model update with SGD see Algorithm 1
  $\boldsymbol{\phi} \leftarrow$ `SGD`$(\mathcal{D}', \hat{q}, \mathcal{L}, \eta, \boldsymbol{\phi})$
Final policy without exploration as defined in Equation (4.3)
$\pi_{\boldsymbol{\phi}} \leftarrow$ greedy policy with respect to $\hat{q}_{\boldsymbol{\phi}}$;
Return $\pi_{\boldsymbol{\phi}}$

Critic-only approaches suffer from several limitations. The main limitation is scalability to large action spaces. Since the policy must evaluate all possible actions at each decision step to select the maximum, the computational cost grows with the size of the action set. To address this issue, another family of RL algorithms exists, the actor-critic methods.

4.4.2 Actor-Critic Methods

Actor-critic methods differ from critic-only approaches in two ways. First, the critic approximates the state-value function v_π instead of the state-action value function q_π. Second, the actor is a policy with its own dedicated set of parameters, instead of a policy derived from a value function.

The critic is a value function approximation $\hat{v}_{\boldsymbol{\phi}}(s, o)$ with parameters $\boldsymbol{\phi}$. Analogously to critic-only methods, it is trained using supervised learning on data obtained from interaction with the MDP. For each time step t along a trajectory, the input is the state-observation pair (s_t, o_t) and the target is the

observed return g_t. The critic thus learns to predict the return expected under the current policy.[3]

The actor is a parameterized policy $\pi_{\boldsymbol{\theta}}$ with parameters $\boldsymbol{\theta}$.

The key difference with critic-only methods lies in how the policy itself is optimized. Since the actor has its own parameters, it can be updated directly, using the critic's predictions. To do so, each action taken by the actor must be judged against what the policy would do on average in the same situation. This is measured by the advantage $q_\pi(s,a) - v_\pi(s,o)$, which is positive when action a yields a higher expected return than the policy on average. Since the observed return g_t is a sample of $q_{\pi_{\boldsymbol{\theta}}}(s_t,a_t)$ and the critic approximates $v_{\pi_{\boldsymbol{\theta}}}$, the advantage of each collected action is estimated by

$$\widehat{\mathrm{Adv}}_t = g_t - \hat{v}_{\boldsymbol{\phi}}(s_t,o_t).$$

When $\widehat{\mathrm{Adv}}_t > 0$, the action a_t performed better than the critic expected, and the parameters $\boldsymbol{\theta}$ are adjusted to increase the probability $\pi_{\boldsymbol{\theta}}(a_t \mid o_t)$. When $\widehat{\mathrm{Adv}}_t < 0$, the action a_t performed worse than expected, and this probability is decreased. Judging each action relative to the expected return of its situation, rather than by the raw return, reduces the variance of the updates.

Both $\hat{v}$ and $\pi_{\boldsymbol{\theta}}$ are improved simultaneously within the same loop. The critic provides a learned baseline for the actor, and the actor generates the data used to train the critic.

Algorithm 4 summarizes this whole actor-critic learning loop. Lines 1–4 initialize the parameters of the critic and of the actor, and Line 5 sets the MSE as the loss of the critic. Each iteration of the loop starting at Line 6 first collects an episode with the current actor (Line 7). Lines 8–9 compute the advantage of every collected action using the current critic. Lines 10–11 train the critic by SGD to predict the observed returns. Finally, Line 12 updates the actor in the direction given by the advantages.

PPO [62], one of the most widely used RL algorithms, belongs to this family of actor-critic methods. It will be used throughout this thesis for all RL experiments.

4.5 Conclusion

This chapter has provided an introduction to the fundamental principles of RL. Only the essential concepts required to understand the RL framework have been presented. Despite this limited scope, the two defining steps of RL can already be observed in its simple learning procedure. The first step consists of estimating the reward that can be obtained from the interaction between the agent and its environment in a given situation. The second step uses this estimate to improve the current policy. Learning therefore results from the repeated alternation of these two basic operations.

This simple iterative structure brings RL conceptually close to DL. In both paradigms, simple operations can lead to complex behaviors when repeated at scale. Their combination is particularly powerful. A wide range of decision-making problems can be formulated as MDPs, allowing RL to be applied to

[3] Since the critic is only used during learning, it can also take the state s_t as input, whereas the actor, which is kept to act in the environment, relies on observations alone.

Algorithm 4: Deep Actor-Critic Reinforcement Learning

Input:
MDP $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{R}, p)$
Architecture $\hat{v}$ (critic)
Architecture π (actor)
Discount factor $\gamma \in [0, 1]$
Learning rate $\eta > 0$
Initial parameters $\boldsymbol{\phi}$ of the critic (optional)
Initial parameters $\boldsymbol{\theta}$ of the actor (optional)
Output:
Trained policy $\pi_{\boldsymbol{\theta}}$

---

if $\boldsymbol{\phi}$ *is not given* **then**
  Initialize $\boldsymbol{\phi}$ randomly;
if $\boldsymbol{\theta}$ *is not given* **then**
  Initialize $\boldsymbol{\theta}$ randomly;
MSE loss function as defined in Equation (2.2)
$\mathcal{L} \leftarrow$ MSE;
while *stopping criterion not satisfied* **do**
  Data collection see Algorithm 2
  $\mathcal{D} \leftarrow$ `DataCollection(`*MDP*, $\pi_{\boldsymbol{\theta}}, \gamma$`)`
  Advantage estimation: how much better/worse each action performed compared to the critic's expectation
  for $(s_t, o_t, a_t, g_t) \in \mathcal{D}$ **do**
    $\widehat{\mathrm{Adv}}_t \leftarrow g_t - \hat{v}_{\boldsymbol{\phi}}(s_t, o_t)$;
  state-observation pairs and their returns, from $\mathcal{D}$
  $\mathcal{D}' \leftarrow \big\{\big((s, o), g\big) \mid (s, o, a, g) \in \mathcal{D}\big\}$
  Critic update with SGD see Algorithm 1
  $\boldsymbol{\phi} \leftarrow$ `SGD(`$\mathcal{D}', \hat{v}, \mathcal{L}, \eta, \boldsymbol{\phi}$`)`
  Actor update: increase the probability of actions with positive advantage, decrease it for those with negative advantage
  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \sum_{(s_t, o_t, a_t, g_t) \in \mathcal{D}} \widehat{\mathrm{Adv}}_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t \mid o_t)$;
Return $\pi_{\boldsymbol{\theta}}$

a broad range of tasks. DL, in turn, provides flexible function approximators capable of representing the value functions and policies required to solve these tasks. Together, these two paradigms form the basis of deep RL, which has achieved remarkable success across a wide range of challenging tasks.

From a broader perspective, the evolution of RL has brought the field closer than ever to its initial biological inspiration. In modern implementations, agents progressively improve their behavior through trial and error directly based on experience. Their decision-making capabilities are represented by neural architectures that resemble their biological counterparts. It is therefore not

surprising that RL has become one of the core paradigms for pursuing artificial general intelligence [112].

As the capabilities of RL continue to improve, a new challenge becomes important. The ability to learn complex behaviors with limited human intervention can make the resulting agent's strategy difficult to understand. To address this challenge, a parallel research direction has emerged, known as Explainable Reinforcement Learning (XRL). This field is the subject of the next chapter.

Chapter 5

Explainability in Reinforcement Learning

"The Answer to the Great Question… Of Life, the Universe and Everything… Is… Forty-two."
Douglas Adams, *The Hitchhiker's Guide to the Galaxy* (1979) [19]

Contents

5.1 Explanation in Artificial Intelligence . . . 71
5.1.1 Explanation . . . 71
5.1.2 Explainable Artificial Intelligence . . . 73
5.1.3 Explainability Methods . . . 74
5.2 Taxonomy of Explainable Reinforcement Learning Methods . . . 74
5.2.1 Subject . . . 74
5.2.2 Source . . . 75
5.2.3 Form . . . 76
5.3 State-of-the-Art Review of Explainable Reinforcement Learning Methods . . . 76
5.3.1 Policy Model . . . 76
5.3.2 Saliency Map . . . 78
5.3.3 Counterfactual . . . 80
5.3.4 Agent Trajectory . . . 81
5.3.5 Trajectory Prediction . . . 82
5.3.6 Interaction Model . . . 83
5.4 Conclusion . . . 84
5.4.1 Absence of Explanation of the Learning Process . . . 84
5.4.2 Lack of Unified Metrics for Explainability . . . 86
5.4.3 Toward Intrinsic Explainability in Reinforcement Learning 86

Today, most Artificial Intelligence (AI) agents with which humans interact are deployed either in low-risk environments or under human supervision. For example, agents capable of playing games such as Go operate in settings where

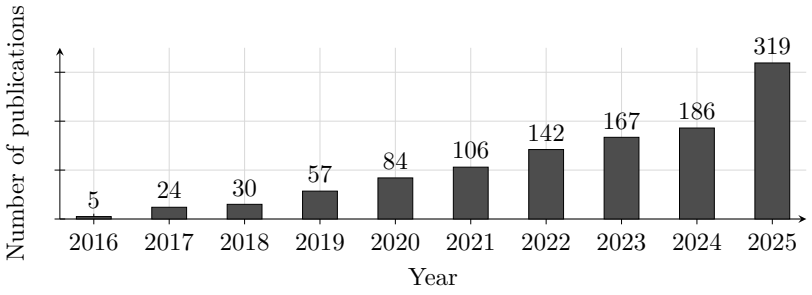


Figure 5.1: Yearly growth of XRL research with at least one citation on arXiv.

potential errors remain limited in terms of consequences [72]. More autonomous systems, such as agents based on Large Language Models (LLMs), are used for tasks such as code generation or information retrieval, where their outputs are generally reviewed by humans [97, 105].

Safety-critical applications integrating machine learning components are therefore still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees [121]. These requirements include in particular:

- validating the system's functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL) [47]. As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. More generally, it also prevents verifying the alignment of these agents (Definition 1.2.1). An agent may maximize its reward through a strategy that does not correspond to the intentions of its designers, and such a strategy can only be detected if it can be understood.

To address this limitation, the field of Explainable Reinforcement Learning (XRL) has emerged as a subfield of Explainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents. Due to the relevance of the topic and to foster broader real-world adoption of RL, a growing number of research teams are working on developing new explainability methods. This enthusiasm is reflected in a rapid increase in the number of publications, as illustrated in Figure 5.1.[1]

The growing number and diversity of these methods make it increasingly difficult to obtain a unified view of the field. This motivated our work on developing a unified taxonomy of explainability methods for RL, initially presented in [129]. The taxonomy provides a common structure for organizing existing approaches according to the type of information they use and the form of explanation they provide. Building on this work, this chapter provides a detailed presentation of

[1]The paper retrieval code is available at https://github.com/Maxalaar/ArXiv_Papers_Matching, and the full search protocol is archived at https://zenodo.org/records/19387785.

the proposed taxonomy and uses it to organize a broader state-of-the-art review of XRL.

This chapter is organized as follows:

- Section 5.1 begins by clarifying what we mean by an explanation in the context of XAI.
- Section 5.2 then introduces a taxonomy for categorizing existing XRL methods.
- Section 5.3 provides a comprehensive state-of-the-art review structured around this taxonomy.
- Section 5.4 concludes the chapter with a summary and discussion.

5.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in general. We then describe how this concept applies to XAI. Finally, we further detail the properties that characterize an explainability method in the context of XAI.

5.1.1 Explanation

Defining what constitutes an explanation is challenging because the concept is inherently multidisciplinary. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI [82, 104].

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 5.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this thesis (Definition 5.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning.

An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 5.1.4). This refinement is obtained through a transformation process that produces an explanation from the object being explained. We refer to this process as an explainability method (Definition 5.1.5), detailed further in Section 5.1.3.

For an observer to understand an explanation, the information it contains must be interpretable (Definition 5.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 5.1.1: Object

The object of an explanation is the system the observer wants to understand.

Definition 5.1.2: Observer

The observer is the human who aims to understand an object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 5.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

Definition 5.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

Definition 5.1.5: Explainability Method

An explainability method is a transformation process that produces an explanation from the object being explained.

Figure 5.2 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer, as a chain of three information structures, depicted as boxes and connected by arrows denoting a transformation of information:

(1) The object of the explanation, that is, the system the observer seeks to understand.

(2) The explanation, the interpretable information produced from the object. This transformation is performed through the use of an explainability method.

(3) The mental model held by the observer regarding the object.

A brace spanning boxes (2) and (3) indicates that this second half of the process, from the explanation to the resulting mental model, constitutes a subjective analysis carried out by the observer.

This definition of explanation establishes the approach to explainability adopted throughout this thesis. Under this definition, explainability is regarded as a continuum that evolves as knowledge develops [115, 138]. This quantitative perspective highlights that not all explanations provide the same level of value [59]. The effectiveness of an explanation depends on its ability to improve the observer's understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer's misunderstanding of a key mechanism.

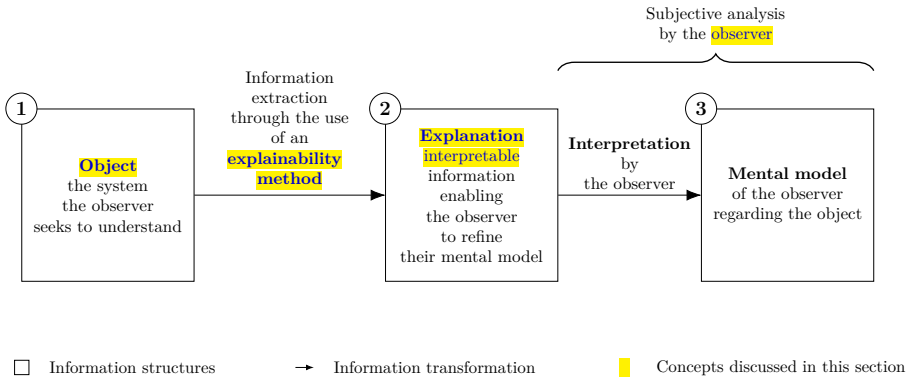


Figure 5.2: Explanation process.

5.1.2 Explainable Artificial Intelligence

Now that the general framework of explanation has been established, we can examine how this concept applies in the context of AI. In the context of XAI, the object of explanation is an AI system. More precisely, XAI emerged as a response to the success of DL models [77, 93]. Unlike symbolic approaches, individual components of Neural Networks (NNs) cannot be directly associated with human concepts. From this starting point, we identify two types of approaches:

- **Substitutive explainability:** In this approach, the assumption is that there is nothing to explain within a NN. There is no semantic meaning associated with each computational component, and therefore nothing to interpret. This corresponds to the "black-box" view of NNs. Under this assumption, XAI consists of replacing as much of the DL system as possible with alternative AI methods [88].
- **Intrinsic explainability:** In this approach, it is argued that NNs contain mechanisms that can be explained. The limitation is considered to be the absence of adequate tools to analyze their functioning [99].

This thesis considers methods from both approaches in order to provide a comprehensive overview of the XAI field. However, it primarily adopts the perspective of the *Mechanistic Hypothesis* (Hypothesis 3) [115, 138]. Since this hypothesis assumes that the internal computations of a model can be explained, adopting it necessarily places this thesis within intrinsic explainability. This perspective is more closely aligned with the definition of explainability adopted in this thesis, which considers explainability as a continuum that evolves with the development of knowledge.

Hypothesis 3: Mechanistic Hypothesis

Every AI system results from a sequence of causal computations. Because the rules governing these interactions are mechanical, the behavior of the system can be explained. If an AI system cannot be explained, this reflects a limitation of knowledge rather than an intrinsic property of the system.

The approach based on this hypothesis is generally called mechanistic interpretability in the literature [99]. In this thesis, we refer to it as Mechanistic Explainability (ME) (Definition 1.2.2). The internal mechanisms of a NN are indeed not interpretable in the sense of Definition 5.1.3, since an observer cannot understand them directly. They only become understandable once an explainability method (Definition 5.1.5) has transformed them into explanations (Definition 5.1.4). This approach therefore falls within explainability rather than interpretability.

5.1.3 Explainability Methods

Now that the general framework of XAI has been established, we need to further detail the properties of an explanation in this context. In our framework, an explanation in the field of XAI can be characterized by three components:

- **Source:** refers to the origin of the information used by the explainability method. The explainability method transforms the raw information produced by the AI system into an explanation that can be understood by an observer.
- **Form:** refers to the representation through which the explanation is communicated. The form defines how the information is presented to the observer and must allow human interpretation. An explanation can take many different forms, including text, images, computer code, and mathematical equations.
- **Subject:** refers to the aspect of the AI system that is being explained. In XAI, this distinction is commonly associated with the difference between local and global explanations. A local explanation describes why an AI system produced a specific output in a given situation. A global explanation aims to describe the general functioning of the AI system.

5.2 Taxonomy of Explainable Reinforcement Learning Methods

Now that the concept of XAI has been defined, we turn to its application in RL, known as XRL. As in XAI, explanations in XRL are produced by explainability methods and can be described in terms of their subject, source, and form. This section reviews the possible variations of these three aspects in the specific context of RL.

5.2.1 Subject

The first question to address is the subject of explanation in the context of RL. To answer this question, it is necessary to identify the different components involved in a RL system and determine which of them constitutes the object of explanation. A RL system can be described through two main components: the environment and the policy.

The environment defines the dynamics in which the agent evolves, including the possible states, observations, actions, and transitions. In RL, environments

are simulations designed by humans, and their dynamics are explicitly defined. Therefore, it is assumed that the environment is known to the observer and does not require explanation.

The object of interest is therefore the agent's policy. The policy represents the decision-making mechanism of the agent. It is a function that maps observations to actions and is learned through interaction with the environment in order to maximize the expected cumulative reward. The policy is the component that encodes the learning process and that XRL aims to explain.

Explaining a policy introduces additional challenges compared with classical XAI settings. In traditional XAI problems, the objective is to explain an isolated decision, such as determining which features of an image led to a classification result. In contrast, RL introduces a temporal dimension into the decision-making process. An action is not only determined by the current observation but also influences future states and rewards. Therefore, understanding the behavior of an agent may require considering a sequence of interactions between the agent and its environment.

This temporal dimension leads to a distinction between two objectives of explanation in XRL:

- **Decision explanations:** focus on identifying the elements of the current observation that influenced the action selected by the policy. This type of explanation is close to classical XAI approaches, as it analyzes the relationship between inputs and outputs while abstracting away the sequential nature of the decision process.
- **Strategy explanations:** aim to characterize the agent's behavior over time. They focus on sequences of decisions and interactions with the environment in order to understand how the agent achieves its long-term objective. This type of explanation is specific to RL, as it addresses the temporal and goal-oriented nature of the learning process.

These two objectives should not be considered as strictly separated categories, but rather as two extremes of a continuum. Understanding the elements of an observation that influence a specific decision can help the observer generalize this information to infer broader aspects of the agent's behavior. Conversely, understanding the agent's overall strategy can provide insights into the reasoning behind individual decisions.

5.2.2 Source

We identify three sources of explanation in RL, all obtained during or after the learning process and containing information about the agent:

- **Policies:** policies are functions that generate a distribution over the action space for a given observation, as defined in Section 4.3.2.
- **Trajectories:** trajectories represent sequences of successive interactions between the agent and the environment, as defined in Section 4.3.3.
- **Value functions:** value functions provide predictions about the future outcome of a trajectory from a given observation, as defined in Section 4.3.4. The most commonly used value function is the critic function, which estimates the expected sum of future rewards (Section 4.4.2).

5.2.3 Form

We identify six forms of explanation in the RL setting:

- **Policy Model:** an interpretable model of the agent's policy function.
- **Saliency Map:** a weighting of the input features representing their influence on the output of the policy function.
- **Counterfactual:** a modified observation obtained by perturbing the original observation in order to produce a different action from the agent.
- **Agent Trajectory:** a sequence of successive interactions between the agent and the environment.
- **Trajectory Prediction:** a prediction about the future evolution of the trajectory produced by a value function.
- **Interaction Model:** an interpretable model of the interactions between the agent and the environment.

Among the three components characterizing an explanation, the form is the one that varies most significantly across explainability methods. While subjects and sources tend to be relatively stable in the XRL literature, the diversity in how explanations are presented is considerably greater. The list above is by no means definitive and may evolve as new approaches emerge. For this reason, the following section is organized according to the form of explanations and presents existing explainability methods based on this criterion.

5.3 State-of-the-Art Review of Explainable Reinforcement Learning Methods

The purpose of this section is to detail the functioning of existing explainability methods (see Table 5.1). They are presented according to the form of explanation they produce, following the taxonomy introduced in Section 5.2.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation details while relying on the same principle.

The remainder of this review is organized by form of explanation, following the six forms identified in Section 5.2.3. For each form, we first describe its principle and its limitations, then present the families of methods that produce it.

5.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NNs are the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood

Explanation Subject	Explanation Form	Explainability Method Family	Source of Explanation		
			Policy	Trajectories	Value functions
Decision	Policy Model	*Interpretable policy learning*	✓		
		Policy approximation via interpretable model	✓		
	Saliency Map	*Observation perturbation*	✓		
		Observation gradient analysis	✓		
		Attention-based architecture	✓		
	Counterfactual	*Latent space usage for counterfactual generation*	✓		
Strategy	Agent Trajectory	*Hierarchical decomposition*	✓	✓	
		Extraction via critic analysis		✓	✓
	Trajectory Prediction	*Critic enrichment*			✓
	Interaction Model	*Bayesian network learning*		✓	
		Latent space usage for MDP generation	✓		

Table 5.1: Taxonomy of explainability methods in RL.

through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [113], algorithms [114], formal logic [86], and linear models [95].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently well structured, the observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent's strategy.

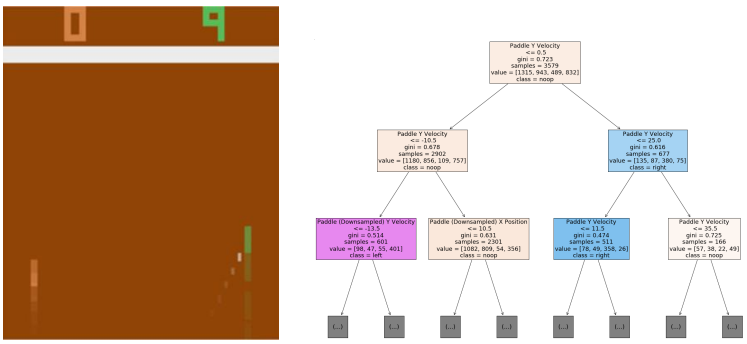
The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or conditions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

To obtain an interpretable policy model, two families of explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This family of methods aims at learning a policy represented by an interpretable model [113, 114]. It is the most direct approach to addressing the challenges posed by XRL.



(a) (b)

Figure 5.3: Policy approximation via an interpretable model [111].

This type of training requires modifying the Markov Decision Process (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a reduction requires the integration of expert knowledge into the learning process. With this family of methods, the *interpretable policy function* itself is used to provide explanations to the observer.

Policy Approximation via Interpretable Model

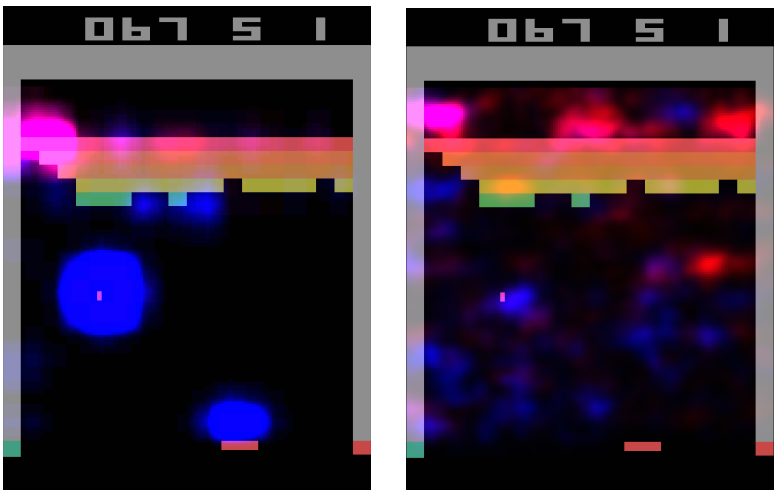
An interpretable model can be trained to approximate a policy learned through deep RL [111]. In this setting, the problem is formulated as a supervised learning task, where the initial deep RL policy is used to label each observation with the action it selects, such as the observation from the Atari game Pong shown in Figure 5.3(a). Using this dataset, the interpretable model is trained to reproduce the behavior of the initial policy, yielding for instance the decision tree illustrated in Figure 5.3(b), obtained by approximating a deep RL policy trained on that environment.

The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment in place of the initial policy, thereby providing a high degree of control over the agent.

5.3.2 Saliency Map

The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 5.4). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision. In Figure 5.4, both panels correspond to the same observation of the Atari game Breakout, with panel (a) obtained by observation perturbation and panel (b) by observation gradient analysis.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 5.4(b),



(a) (b)

Figure 5.4: Heatmaps for the same observation of the Atari game BREAKOUT [65].

interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.

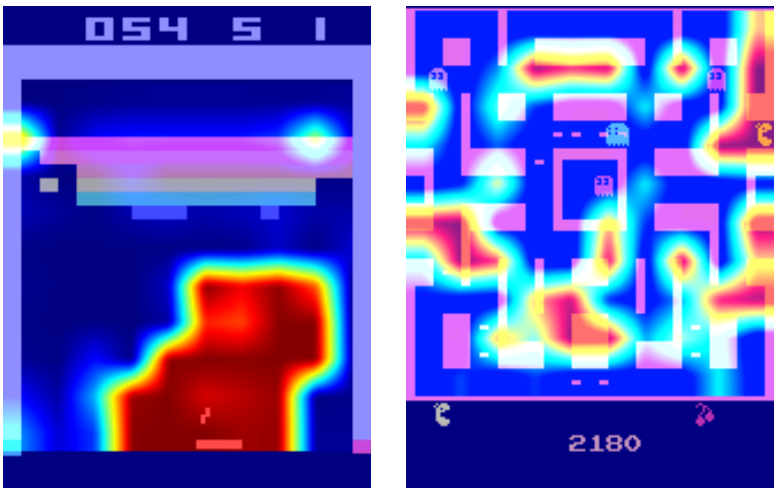
Three families of explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

Observation Perturbation

Perturbation analysis consists in applying various modifications to the observation in order to study their impact on the output of the policy function [65]. The greater the variation in the output caused by a perturbation applied to a given component of the input vector, the higher the weight assigned to that component. The magnitude of the perturbation is measured by computing the distance between the original output and the output obtained from the perturbed input. Using this set of weights, a *heatmap* is constructed, enabling the observer to visualize which parts of the observation vector are most sensitive in the action-selection process (see Figure 5.4(a)).

Observation Gradient Analysis

Observation gradient analysis refers to a set of methods that require a differentiable policy function in order to extract information from it [49, 78, 90]. To this end, the gradient of the policy function is computed with respect to all components of the input vector. For each input component, the resulting gradient provides a weighting that represents its importance in the output of the policy function. Based on these weightings, a *heatmap* is constructed to allow



(a) (b)

Figure 5.5: Attention-based heatmaps for the Atari games Breakout and Ms. Pac-Man [83].

the observer to visually identify the parts of the input vector that are the most influential in the policy function's output (see Figure 5.4(b)).

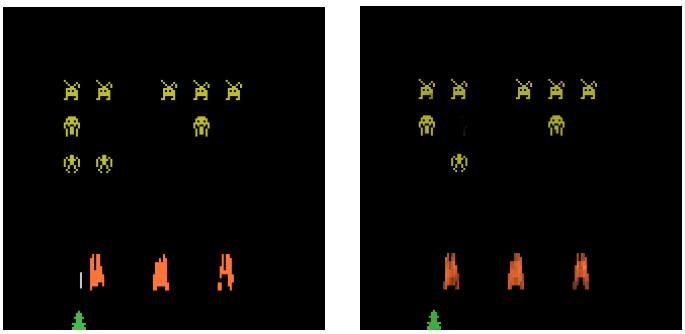
Attention-Based Architecture

The attention mechanism, presented in Section 2.6.2, computes attention weights $\mathbf{A}$ that indicate how much each token contributes to the representation of the others [63]. These weights are not fixed parameters, since they are computed from the input itself. When the tokens correspond to regions of the observation, each region is thus assigned a scalar weight. It can therefore be assumed that the regions associated with the highest weights are the most influential in determining the output [83, 106].

This attention-based *heatmap* is then provided to the observer as the explanation, as illustrated for the Atari game Breakout in Figure 5.5(a), where the highlighted regions are easily interpretable. However, as shown for Ms. Pac-Man in Figure 5.5(b), the attention weights are not always concentrated on a small, easily interpretable set of regions, which can make this type of explanation difficult to exploit.

5.3.3 Counterfactual

A counterfactual explanation is a new observation, close to the original one, that leads the agent to select a different action. Comparing the two observations reveals which elements of the environment must change for the agent to behave differently, and thus provides insight into its decision-making process. Unlike the other forms of explanation, this form is currently associated with a single family of methods, based on the *use of the Latent Space (LS) for counterfactual generation*.



(a) (b)

Figure 5.6: Counterfactual explanation of the agent's behavior for the Atari game Space Invaders [108]. Panel (a) shows the original observation and panel (b) the counterfactual observation produced by a generative model.

Latent Space Usage for Counterfactual Generation

This family of methods uses generative models to build counterfactuals from the Latent Representation (LR) of the observation [108, 116]. The observation is encoded into the LS, its representation is modified until the agent selects the desired action, and this modified representation is decoded back into a new observation. Working in the LS rather than at the pixel level keeps the counterfactual close to the original state and consistent with the environment. The fidelity of the generated observations to the real game can then be assessed through user studies [108].

Figure 5.6 illustrates this approach on the Atari game Space Invaders [108]. In the original observation (Figure 5.6(a)) the agent selects `LEFT`, whereas in the counterfactual observation (Figure 5.6(b)) it selects `RIGHT_FIRE`. Here, removing an enemy is enough to change the action, which suggests that the position of that enemy influences the agent's decision. Such observations remain only indicative. Identifying a reliable pattern requires repeating the analysis across many situations.

5.3.4 Agent Trajectory

Until now, the presented methods have focused on explaining individual decisions made by the agent. The methods considered from this point on aim to provide insight into the agent's strategy by analyzing its behavior over time.

The most direct approach consists in visualizing agent trajectories, most commonly through complete episodes. By analyzing these trajectories, the observer can infer the dynamics underlying the agent's behavior and how its decisions evolve through time. This type of visualization is not specific to XRL. In practice, visualizing agent episodes is a standard step in the development and evaluation of RL. It provides a simple way to verify that the agent behaves as expected and to detect potential implementation errors.

This simple approach can be further enhanced using families of methods such as *hierarchical decomposition* and *extraction via critic analysis*.

Extraction via Critic Analysis

In this family of methods, the predictions of the critic are used to determine which trajectories to present to the observer [102]. A set of states is selected from multiple trajectories. For a given state, all possible actions are examined. The state is assigned a value equal to the difference between the critic's prediction for the lowest-valued action and the prediction for the highest-valued action.

Trajectories containing the states with the highest values according to this procedure are then chosen to be presented to the observer. These trajectories are considered critical moments of the agent and are therefore relevant for analysis. For this family of methods, the *selected trajectories* represent the interpretable information provided to the observer.

Hierarchical Decomposition

Hierarchical agents are composed of a set of sub-policies specialized for specific tasks within subsets of the environment's states. This family of methods distributes the optimization of the reward function across multiple policies. This decomposition of the policy function can also provide explainability in the agent's strategy [74].

If a sub-policy is selected, it indicates that the agent will execute a specialized behavior. This behavioral specialization provides insight into the strategy the agent employs to maximize the reward function. The agent's *trajectory*, together with the *sub-policy* selected for each action, is used as interpretable information.

5.3.5 Trajectory Prediction

Making predictions about the future of a trajectory allows one to form a representation of what the system expects, based on its training experience. These predictions are produced using value functions. Such functions provide insights into the future evolution of the agent's trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent.

Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$. This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. Unlike the other forms of explanation, this form is currently associated with a single family of methods, aimed at improving the interpretability of value-based explanations: *critic enrichment*.

Critic Enrichment

In order to obtain interpretable information from the predictions of the critic, it is possible to decompose the critic into multiple sub-functions [79]. The reward function is thus decomposed into multiple sub-reward functions, each representing a sub-objective of the task to be accomplished. Similarly, for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics.

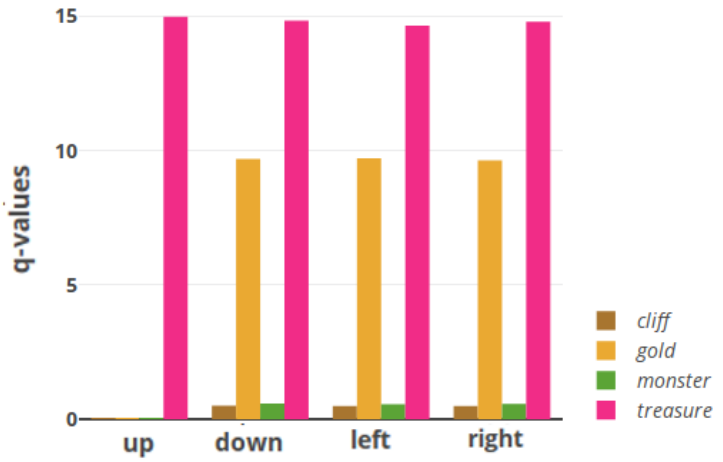


Figure 5.7: Prediction of sub-rewards as a function of the action selected by the agent for a given state [79].

Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize, so that the action chosen by the agent reveals which sub-reward it aims to maximize (see Figure 5.7). This *prediction* is then used as the explanation for the observer.

5.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent's decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent–environment interaction, this type of explanation aims to explain the agent's strategy.

The families of methods used to obtain explanations in the form of agent–environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of random variables connected through a Bayesian network [98]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables.

At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer, as illustrated for the video game StarCraft II in Figure 5.8, where nodes represent random variables and edges represent causal relationships between them. This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

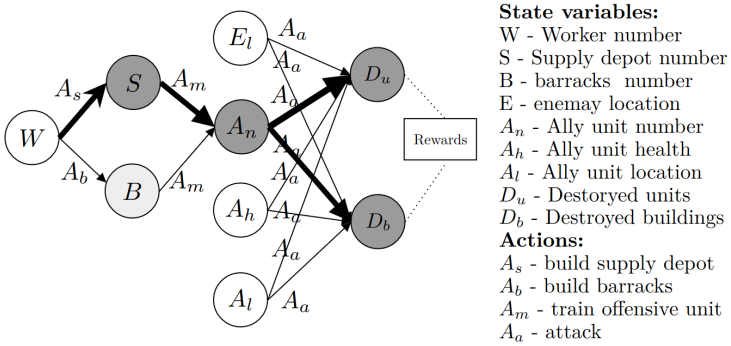


Figure 5.8: Agent–environment interaction model for the video game StarCraft II [98].

Latent Space Usage for Markov Decision Process Generation

As discussed in Chapter 3, each layer of a NN can be viewed as a projection from one space to another [57]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent's perception.

Based on this observation, it is possible to redefine an MDP by leveraging the abstract representations produced by the final layers of a NN. Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 5.9 illustrates this idea using a two-dimensional *t-SNE* projection of the LS of the NN used to represent the policy. Each point is colored according to the value function's prediction for the corresponding state, with warmer colors indicating higher predicted values. An agreement can be observed between the structure of this projected space and the critic's predictions. States that are assigned similar values by the critic also tend to be located close to one another in the projection. This agreement makes it possible to manually identify clusters of states, which likely correspond to states that are perceived as similar by the agent.

5.4 Conclusion

The taxonomy and the state-of-the-art review presented in this chapter make it possible to draw three main observations.

5.4.1 Absence of Explanation of the Learning Process

Regarding the subject of explanation, Table 5.1 shows that none of the reviewed methods addresses the learning process itself. Such an explanation would concern how the policy is progressively updated throughout training, and why it evolves

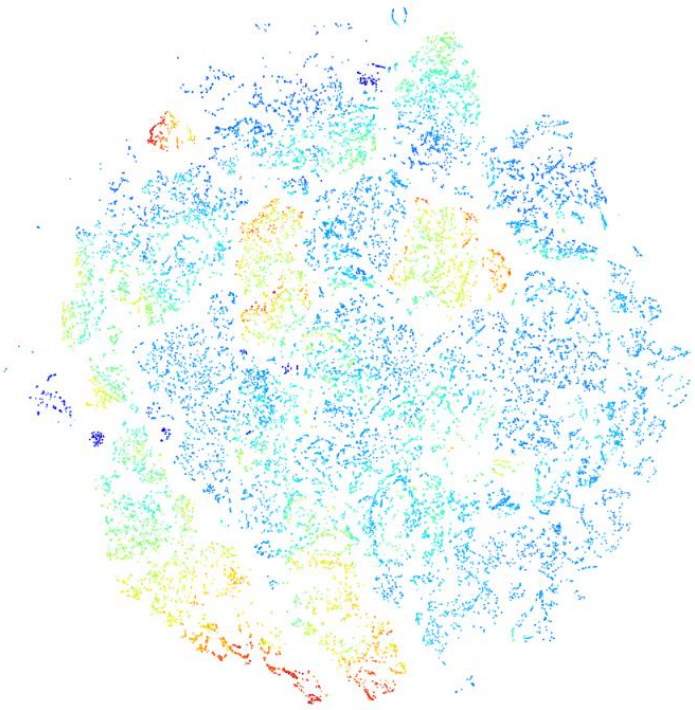


Figure 5.9: *t-SNE* projection of an NN's LS [57].

over successive iterations to converge toward a particular solution. Several reasons explain this absence.

First, among the different machine learning paradigms, RL is arguably the one that would benefit the most from explanations of the learning process. Unlike supervised or unsupervised learning, it relies on an explicit exploration–exploitation trade-off, making the learning trajectory itself an informative subject of explanation. However, most XRL methods are inherited from the XAI literature, where explanations focus exclusively on the behavior of already-trained models. This limitation has naturally carried over to XRL.

Second, explanations of the learning process have limited value from a practical perspective. In most real-world applications, practitioners deploy an already-trained agent. Practitioners are therefore primarily interested in understanding the behavior of the final policy rather than the optimization process that produced it.

Finally, explaining the learning process raises significant methodological challenges. Training in RL can be viewed as the iterative improvement of a policy through repeated interactions with the environment. In this sense, explaining learning would amount to explaining the evolution of an entire sequence of policies rather than a single one. Current XRL methods still struggle to provide comprehensive explanations of an individual policy, so extending them to a full training trajectory appears considerably more difficult.

5.4.2 Lack of Unified Metrics for Explainability

The diversity of explanation forms identified in this review also reveals another limitation shared by nearly all families of XRL methods, which is the absence of reliable metrics for evaluating explainability. In many studies, explanations are not evaluated with human participants. Instead, authors focus on technical properties of the proposed method, such as fidelity, sparsity, or computational efficiency.

When human evaluations are conducted, they usually rely on subjective satisfaction questionnaires. Although useful, self-reported satisfaction remains prone to numerous biases and does not necessarily indicate that the explanation has improved the observer's mental model or understanding of the agent.

This limitation is not specific to XRL, it reflects a broader challenge affecting XAI as a whole. In our view, this limitation comes from how explainability has developed in AI. Most of the proposed approaches have been developed by the computer science community. Explainability has consequently often been studied from an algorithmic or mathematical perspective, with less attention given to how humans actually benefit from explanations.

Our definition of explanation explicitly places the observer at the center of the explanatory process. From this perspective, the value of an explanation should not only depend on its technical properties but also on its ability to improve the observer's understanding. Addressing this challenge will require stronger interdisciplinary collaboration between computer science and disciplines such as cognitive science, psychology, philosophy, and sociology. These fields provide the theoretical and experimental tools needed to study human understanding and to develop evaluation protocols that measure the actual effectiveness of explanations.

5.4.3 Toward Intrinsic Explainability in Reinforcement Learning

Our review also suggests that current XRL research remains largely dominated by substitutive explainability approaches. Replacing a deep model with a simpler one therefore risks losing precisely the representational power that makes deep RL effective. For this reason, we argue that a promising direction for XRL is to develop explanations directly from the NN that forms the agent's policy. Such approaches aim to analyze the representations learned by the network itself.

Among the methods reviewed, *latent space usage for MDP generation* (Section 5.3.6) appears closest to this intrinsic perspective. However, this approach still relies on a largely manual and qualitative analysis of the LS, which limits its reproducibility and its ability to produce systematic explanations.

The next chapter builds upon this idea and proposes a mechanistic XRL approach based on LS analysis.

Chapter 6

Clustering Agent Representations

"Our analysis shows that there are three kinds of people in the world: those who use k-means clustering with $k = 3$, and two other types whose qualitative interpretation is unclear."

Randall Munroe, *xkcd* no. 2731 (2023) [127]

Contents

6.1 Representation Convergence Hypothesis . . . 88
6.2 Latent Clustering of Surrogate Policies . . . 89
6.2.1 Multiple Surrogate Policies . . . 90
6.2.2 Training Surrogate Policies . . . 91
6.2.3 Cluster Extraction . . . 92
6.3 Evaluation Metrics . . . 95
6.3.1 Behavior Reconstruction . . . 95
6.3.2 Adjusted Rand Index . . . 96
6.3.3 Clustering Universality . . . 97
6.3.4 Abstraction Score . . . 98
6.3.5 Adjusted Clustering Universality . . . 99
6.4 Conclusion . . . 99

As discussed in the previous chapter, current explainability methods for Reinforcement Learning (RL) still lack intrinsic approaches capable of explaining the internal mechanisms of deep neural policies. To the best of our knowledge, only Zahavy et al. [57], in their paper "Graying the black box: Understanding DQNs", have proposed exploiting the Latent Space (LS) learned by neural policies in order to identify the concepts used by the agent. Its main idea is to analyze the Latent Representations (LRs) produced by the Neural Network (NN) and organize them into clusters that correspond to concepts.

While the core idea of this paper is closely aligned with our vision of explainability in RL, this approach suffers from two important limitations. First, the clusters are manually constructed. Such a procedure heavily depends on the

user's expertise and neither guarantees reproducibility or scales. Second, no evaluation protocol is proposed to assess the quality of the obtained clusters. Their relevance is essentially determined through visual inspection.

The Clustering Agent Representation (CAR) method proposed in this chapter addresses these two limitations. First, it automates concept discovery by relying on unsupervised clustering algorithms. Second, it introduces an objective evaluation metric based on the stability of the extracted representations across independently trained policies.

This chapter is organized as follows:

- Section 6.1 lays out the theoretical assumptions underlying the proposed CAR method, which we term the *Representation Convergence Hypothesis* (Hypothesis 4).
- Section 6.2 then details the CAR method itself, covering both the extraction of LRs using surrogate policies and the automatic detection of Concept Structures (CSs) through clustering.
- Section 6.3 introduces the metrics we propose for evaluation.
- Section 6.4 situates CAR within the broader landscape of Mechanistic Explainability (ME).

6.1 Representation Convergence Hypothesis

The CAR method relies on an assumption, referred to as the *Representation Convergence Hypothesis* (Hypothesis 4). This hypothesis follows the two assumptions established previously throughout this thesis.

The *Mechanistic Hypothesis* (Hypothesis 3) rejects the notion that deep NNs constitute irreducible "black boxes". Adopting a mechanistic perspective, we consider that every process results from a sequence of causal computations. Although these computations may be complex, they remain explainable through the analysis of the internal mechanisms.

The *Concept Hypothesis* (Hypothesis 1) states that NNs achieve generalization by learning concepts. During optimization, the network progressively organizes its LS into representations that capture the information required to solve the task.

Following these two hypotheses, it becomes possible to explain a NN by studying the organization of its LS. This family of approaches corresponds to ME (Definition 1.2.2). Since the CAR method is grounded in these two hypotheses, it falls within this field of research.

More specifically, CAR focuses on a specific aspect of ME known as universality. Universality refers to the consistency with which a concept is recovered across different models trained on the same task [139]. In the literature, universality is mainly reported as an empirical observation, without an explanation of why it occurs. In this thesis, we make it a central proposition for explaining NNs. If the same concepts emerge in every model trained on a task, these concepts characterize the task itself, and they can be used to explain the behavior of any of these models.

To provide an explanation for this phenomenon, we developed the *Representation Convergence Hypothesis* (Hypothesis 4). This hypothesis states that,

when NNs are trained to solve the same task, a set of concepts should systematically emerge because these concepts provide an effective way of representing the problem. As a result, independently trained models should develop similar concepts in their learned representations.

Hypothesis 4: Representation Convergence Hypothesis

For a given task, independently trained NNs tend to develop similar sets of concepts because these concepts provide an efficient representation for solving the task.

The CAR method is designed to investigate the *Representation Convergence Hypothesis* (Hypothesis 4) by assessing whether it can consistently recover similar concepts across independently trained models. The hypothesis therefore serves both as the foundation of the method and as the phenomenon it aims to investigate.

If CAR consistently recovers the same set of concepts across different training runs, this set is more likely to reflect invariant properties of the task than model-specific artifacts. Conversely, the absence of representation convergence would provide evidence against the hypothesis and would also question the validity of the CAR approach.

It is important to clarify that this hypothesis does not imply convergence at the parameter level. Independently trained models are expected to converge toward different sets of weights. The proposed hypothesis concerns a abstract level of representation. We assume that the conceptual organization induced by their LRs converges toward an equivalent CS, even though the numerical implementation of the networks differ.

The remainder of this chapter develops the methodology used to investigate this hypothesis.

6.2 Latent Clustering of Surrogate Policies

Following Zahavy et al. [57] in "Graying the black box: Understanding DQNs", we analyze the LS through clustering techniques, placing our approach within the broader field of deep clustering. This design choice is motivated by our rejection of the *Superposition Hypothesis* (Hypothesis 2) and, more generally, of any view that assumes a linearly organized CS, as discussed in Section 3.5.

The CAR methodology is composed of two main stages. The first stage trains multiple independent NNs, called surrogate policies, which implement the same decision policy. The second stage clusters their LRs in order to identify the underlying conceptual organization. Once these two stages have been completed, the resulting clustering can be analyzed. The consistency of the resulting clusters across independent training runs can be assessed to determine whether the identified clusters are stable. Figure 6.1 provides a visual overview of the two stages, from the imitation of the initial policy to the clustering of the surrogate representations.

This section is organized into three parts. The first explains why multiple surrogate policies are used to study the stability of the representations. The second describes how these policies are trained to match the initial policy as

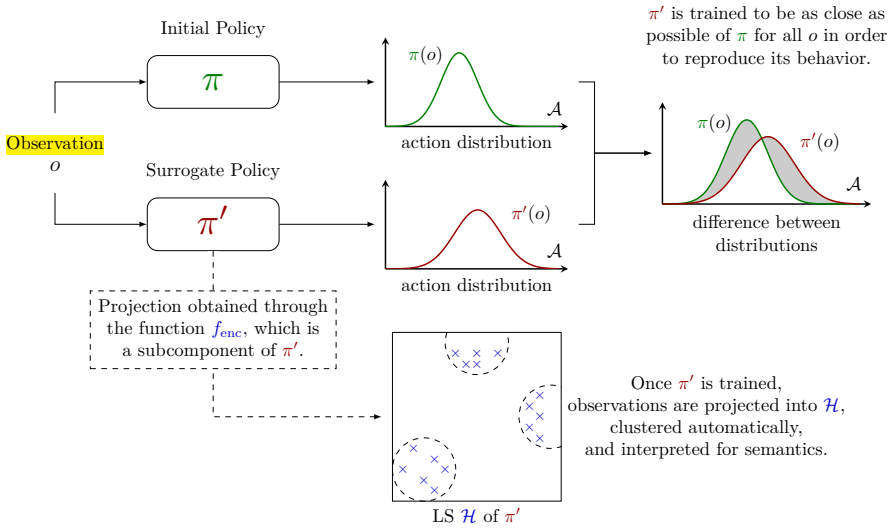


Figure 6.1: Illustration of the Clustering Agent Representation method.

closely as possible. The third presents how their LRs are clustered to identify the CSs.

6.2.1 Multiple Surrogate Policies

The CAR method was developed to test the *Representation Convergence Hypothesis* (Hypothesis 4). This requires training several NNs independently and comparing their LRs. Consistent similarities in the resulting CSs would provide empirical support for this hypothesis.

In supervised learning, this protocol is straightforward since several models can be independently trained on the same target function. RL introduces an additional difficulty. Because of stochastic exploration, independently trained agents are not guaranteed to converge toward the same policy. Even if they achieve comparable performance, they may solve the task using different strategies. This makes it difficult to determine whether differences in the LS result from different representations of the same strategy or from differences in the learned strategies themselves.

To overcome this limitation, we consider an already trained initial policy. This policy is then used as a target for imitation learning [42]. In this process, several surrogate NNs are independently trained in a supervised manner to reproduce the behavior of the same agent. This allows us to investigate whether NNs that learn equivalent policies also develop similar conceptual organizations.

It is important to emphasize that the use of surrogate policies does not make CAR a surrogate explainability method, as defined in Subsection 5.1.2. The surrogate policies remain deep NNs. Their role is not to simplify the decision process but to provide multiple independent realizations of the same policy function, enabling the study of representation convergence.

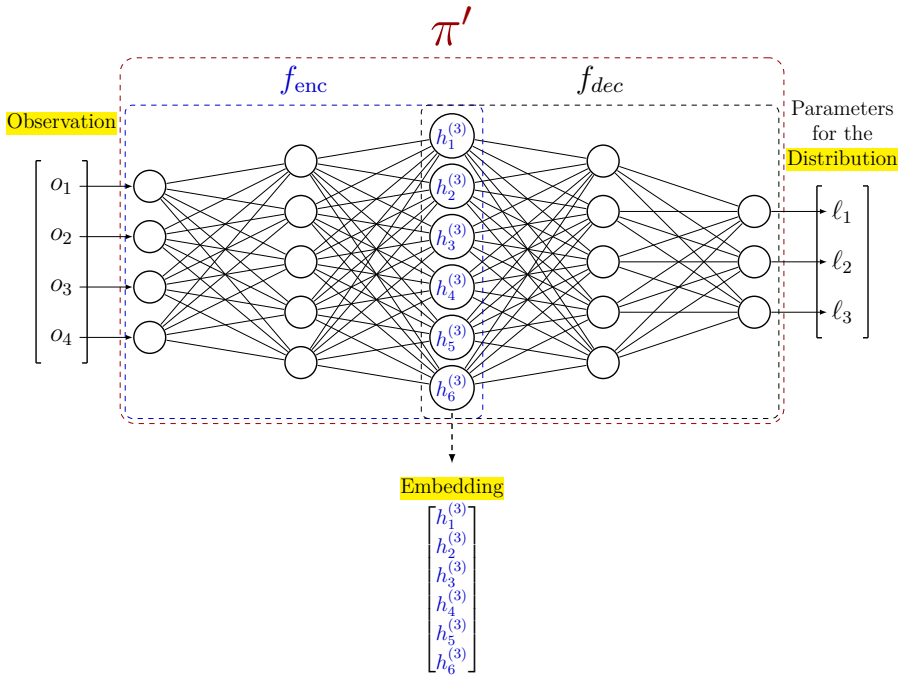


Figure 6.2: Example of a surrogate policy architecture π'.

6.2.2 Training Surrogate Policies

Throughout the remainder of this thesis, the initial policy we aim to explain is denoted as π. This initial policy is obtained without imposing any constraints on its origin. It may result from classical or deep RL [73], alpha-beta algorithm [18], or even from demonstrations provided by humans [39]. The crucial requirement is the availability of a dataset containing a sufficient number of projections from the observation space $\mathcal{O}$ to the action space $\mathcal{A}$ induced by π to train a NN.

The replicated policies are referred to as surrogate policies. To assess the stability of the learned representations, n surrogate policies are trained independently, denoted by $\pi'_1, \ldots, \pi'_n$.

All surrogate policies share the same architecture. It consists of a NN with two components, an encoder f_{enc} and a decoder f_{dec}, highlighted by the blue and black dashed rectangles, respectively, in Figure 6.2. The encoder maps the observation o to the embedding, and the decoder reconstructs the parameters of the action distribution from this embedding. For a given surrogate policy π'_i, parameterized by $\boldsymbol{\theta}_i$, its encoder and decoder are written $f_{\text{enc},i}$ and $f_{\text{dec},i}$. The figure represents the encoder with fully connected layers, but CAR places no constraint on its architecture. The encoder can also rely on convolutional layers or on attention mechanisms, since these architectures produce intermediate representations shaped by the same learning process, as discussed in Section 2.6.

To train each surrogate policy π'_i to accurately replicate the action distribution, we define a loss function $\mathcal{L}$ optimized via gradient descent as introduced in Section 2.4. This loss, defined in Equation (6.1), is computed as the Mean Squared Error (MSE) between the parameters of the action distributions, obtained by

applying the function $\boldsymbol{\ell}$, which maps a distribution to its parameters, namely the logits. In practice, this function $\boldsymbol{\ell}$ does not need to be explicitly defined, as neural policies directly output the parameters of the action distribution. Each surrogate policy π'_i is thus trained by solving:

$$\min_{\boldsymbol{\theta}_i} \sum_{o \in \mathcal{O}} \mathrm{MSE}\left(\boldsymbol{\ell}\big(\pi\,(o)\,\big), \boldsymbol{\ell}\big(\pi'_i\,(o)\,\big)\right). \tag{6.1}$$

6.2.3 Cluster Extraction

Once the surrogate policies are trained, their encoders are used to generate a point cloud from a set of observations. This point cloud provides a sample of the LS to be analyzed. The proposed CAR method adopts the deep clustering approach introduced in Section 3.5. Under this approach, conceptually similar observations are expected to produce nearby LRs. A CS can therefore be viewed as a region of the LS containing related LRs. To identify these regions, a clustering algorithm is applied to the point cloud.

The clustering is performed independently for each surrogate policy. For a surrogate policy π'_i, applying its encoder $f_{\mathrm{enc},i}$ to a subset of observations $\mathcal{O}_{\mathrm{data}} \subseteq \mathcal{O}$ yields the set of embeddings $\mathcal{H}_i$. A clustering function f_{clust} then partitions the point cloud $\mathcal{H}_i$ into groups of nearby LRs, assigning each observation a cluster label and yielding the partition $\mathcal{C}_i$.

The same observation set $\mathcal{O}_{\mathrm{data}}$ is used for every surrogate policy, so that the partitions $\mathcal{C}_1, \ldots, \mathcal{C}_n$ are defined over the same observations and can be compared. This comparison is the basis of the evaluation introduced in the following section.

Algorithm 5 states the complete CAR procedure, whose lines map directly onto the stages described above. Lines 1–2 prepare the imitation of the initial policy. They build the training dataset by pairing each observation with the parameters of the action distribution produced by π, and define the MSE loss of Equation (6.1). Lines 3–5 correspond to the first stage, in which each of the n surrogate policies is initialized randomly and trained independently by Stochastic Gradient Descent (SGD).

Lines 6–8 correspond to the second stage. The encoder of each surrogate policy embeds the shared observation set $\mathcal{O}_{\mathrm{data}}$ (Line 7), and the resulting point cloud is partitioned by f_{clust} (Line 8). Finally, Line 9 returns the surrogate policies together with their partitions, which are the inputs of the evaluation.

Two choices govern this analysis without being part of the clustering algorithm itself. The influence of both choices is studied in Chapter 7.

The first is the choice of layer used to extract the representations. Each hidden layer defines its own LS, reflecting the information represented at that stage of the network. CAR can therefore be applied to the LRs extracted from any layer. This allows the conceptual organization of the policy to be examined at different levels of the network.

The second is the number of clusters. CAR does not assume that a task admits a single correct number of concepts. This choice instead determines the granularity at which the conceptual organization is examined. A small number reveals broad, strategic concepts, while a larger number reveals finer, tactical ones.

Algorithm 5: Clustering Agent Representation (CAR)

Input:
Initial policy π
Observation dataset $\mathcal{O}_{\text{data}}$
Number of surrogate policies n
Surrogate policy architecture π' (encoder f_{enc}, decoder f_{dec})
Learning rate η
Clustering function f_{clust}
Output:
Trained surrogate policies $\pi'_1, \dots, \pi'_n$
Clusterings $\mathcal{C}_1, \dots, \mathcal{C}_n$

Behavioral cloning dataset built from the target policy π
$\mathcal{D} \leftarrow \{(o, \boldsymbol{\ell}(\pi(o))) \mid o \in \mathcal{O}_{\text{data}}\}$;
MSE loss between action-distribution parameters, as defined in Equation (6.1)
$\mathcal{L} \leftarrow \text{MSE}$;
Independently train n surrogate policies by behavioral cloning
for $i = 1, \dots, n$ **do**
  Initialize $\boldsymbol{\theta}_i$ randomly;
  Behavioral cloning of π via SGD, see Algorithm 1
  $\boldsymbol{\theta}_i \leftarrow \texttt{SGD}(\mathcal{D}, \pi', \mathcal{L}, \eta, \boldsymbol{\theta}_i)$;
Cluster the latent representations of each surrogate policy
for $i = 1, \dots, n$ **do**
  Embeddings produced by the encoder of π'_i
  $\mathcal{H}_i \leftarrow \{f_{\text{enc},i}(o) \mid o \in \mathcal{O}_{\text{data}}\}$;
  Partition of the embedded observations into clusters
  $\mathcal{C}_i \leftarrow f_{\text{clust}}(\mathcal{H}_i)$;
Return $(\pi'_1, \dots, \pi'_n), (\mathcal{C}_1, \dots, \mathcal{C}_n)$

Figure 6.3 shows the outcome of this procedure for a surrogate policy trained on Space Invaders, using five clusters. The clusters are computed in the original high-dimensional LS. The point cloud is a two-dimensional projection used only for visualization. Each cluster gathers observations that correspond to a distinct, interpretable stage of a game:

- **Cluster 1:** The start of a game, with few or no invaders neutralized.
- **Cluster 2:** The right-hand group of invaders eliminated.
- **Cluster 3:** The pursuit of the bonus saucer.
- **Cluster 4:** The end of a game, with a few fast invaders close to the ground.
- **Cluster 5:** The middle of a game, with the invaders descending but already thinned out.

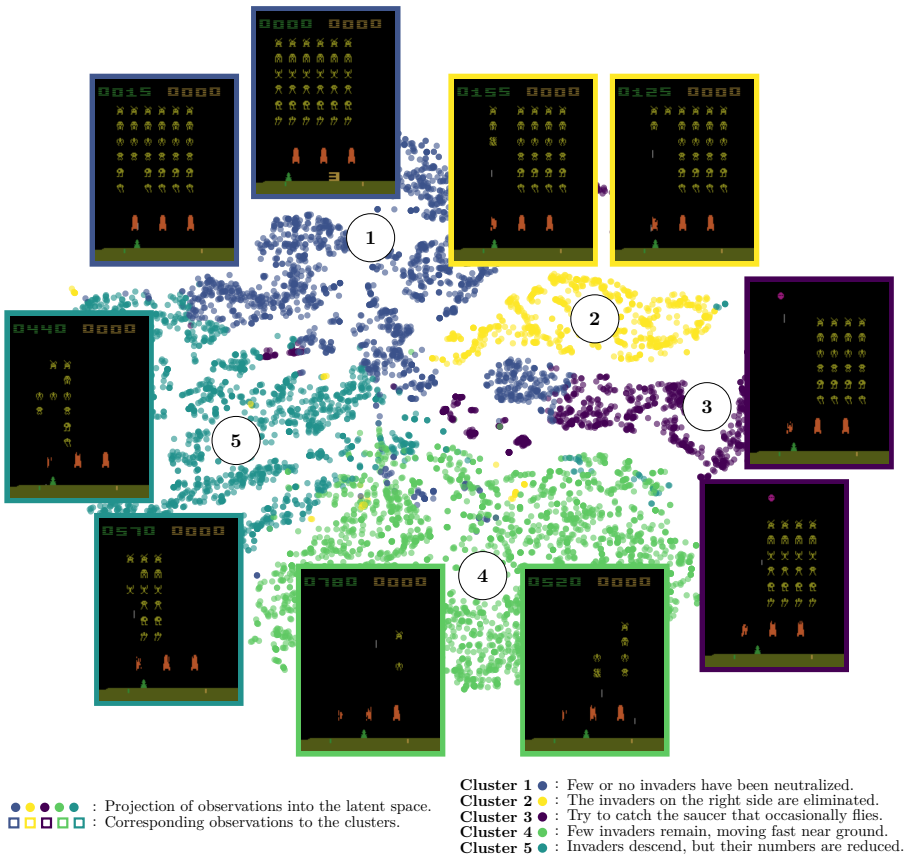


Figure 6.3: Two-dimensional projection of the LS embeddings of a surrogate policy trained on Space Invaders, together with the CAR clusters and their corresponding observations.

The position of an observation in the LS thus reflects the stage of the environment it belongs to, suggesting a correspondence between the geometry of the LS and the semantic properties of environment states. This example illustrates what CAR is expected to produce.

However, the visual inspection of a single surrogate policy does not provide an objective criterion for trusting such an analysis. Three aspects must therefore be evaluated before such an analysis can be trusted. First, the surrogate policy must faithfully reproduce the initial policy so that its explanations remain relevant to the target policy. Second, the recovered conceptual organization must be consistent across independently trained surrogate policies. Finally, the clusters must capture genuine abstractions rather than simply reproducing structure already present in the observations or action distributions. The following section introduces one evaluation metric for each of these aspects and details how it is computed.

6.3 Evaluation Metrics

To evaluate the effectiveness of CAR, four evaluation metrics are introduced:

- ***Behavior Reconstruction*** : it measures how well a surrogate policy faithfully reproduces the behavior of the initial policy by comparing their cumulative rewards.
- ***Clustering Universality* (CU)** : it evaluates the stability of clusterings by measuring the similarity of clusters obtained from independently trained surrogate policies.
- ***Abstraction Score* (AS)** : it quantifies the extent to which LS clusters capture abstractions.
- ***Adjusted Clustering Universality* (ACU)**: it combines the two previous metrics to measure the agreement between surrogate policies beyond what can be explained by the baselines used in AS.

The remainder of this section details the motivation for each metric and describes how they are calculated. In the first subsection, we develop the *Behavior Reconstruction*. We then introduce the Adjusted Rand Index (ARI) measure, which serves as the foundation for the three other metrics introduced in the subsequent subsections, namely CU, AS, and ACU.

6.3.1 Behavior Reconstruction

We introduce the *Behavior Reconstruction* metric to assess whether π' faithfully reproduces the behavior of π. This metric compares the returns obtained by the initial policy π and the surrogate policies π' over a set of evaluation episodes. We use the definition of return introduced in Section 4.3.4, where $G(\tau)$ denotes the discounted sum of rewards collected along a whole trajectory τ. Here, the return is computed over complete episodes with $\gamma = 1$, so $G(\tau^{\text{ep}})$ corresponds to the total reward collected during the episode τ^{ep}.

Each policy is evaluated over M episodes. We denote by $g_{\pi}^{(e)}$ the return obtained by the initial policy in episode e, and by $g_{\pi'_j}^{(e)}$ the return obtained by surrogate policy π'_j in episode e. The *Behavior Reconstruction* is then defined as the mean return of the surrogate policies normalized by the mean return of the initial policy:

$$\text{BR} = \frac{1}{n \sum_{e=1}^{M} g_{\pi}^{(e)}} \sum_{j=1}^{n} \sum_{e=1}^{M} g_{\pi'_j}^{(e)}. \tag{6.2}$$

In imitation learning, it is unlikely to expect the surrogate policy π' to outperform the initial policy π it aims to approximate through supervised learning [39]. Consequently this metric ranges in $[0, 1]$. Values approaching 1 indicate minimal loss in reproducing π, while values close to 0 indicate poor reproduction.

We deliberately adopt a reward-based definition rather than alternatives based on actions or action distributions (e.g., training loss, accuracy, KL divergence [48], or optimal transport [87]). Since surrogate policies are trained in a supervised

manner, they cannot develop strategies beyond those demonstrated by the initial policy. This means that if the rewards achieved are comparable, the surrogate policy is effectively copying the initial policy's behavior.

6.3.2 Adjusted Rand Index

Within CAR, concepts are assumed to correspond to clusters in the LS. Combined with the *Representation Convergence Hypothesis* (Hypothesis 4), this suggests that independently trained surrogate policies should develop similar conceptual organizations. Their LRs should therefore produce similar clusterings when analyzed using the same clustering procedure.

When a particular observation is processed by the network, it is expected to activate the same concept regardless of the specific neural implementation of the policy. If independently trained policies converge toward the same conceptual organization, a given observation should activate the same concept across all models. As a consequence, observations grouped together within a cluster for one policy should also be grouped together for another independently trained policy.

In the clustering literature, the most used measures for comparing two partitions is the ARI [22]. This measure quantifies the agreement between cluster assignments independently of the spatial organization of the clusters. Both metrics introduced later in this chapter, CU and AS, rely on the ARI. Since the ARI is a complex measure, we review its definition and discuss its main properties.

The ARI is derived from the Rand Index (RI) [17], which measures the agreement between two partitions by considering every possible pair of points in the point cloud. For each pair, four situations are possible:

- **Agreement:** The points belong to the same cluster in both partitions.
- **Agreement:** The points belong to different clusters in both partitions.
- **Disagreement:** The points belong to the same cluster in the first partition but to different clusters in the second.
- **Disagreement:** The points belong to different clusters in the first partition but to the same cluster in the second.

The RI measures the proportion of pairs for which the two partitions agree:

$$\mathrm{RI} = \frac{\text{number of agreeing pairs}}{\text{total number of pairs}}.$$

Although the RI is intuitive, it suffers from an important limitation. In most clustering problems, the majority of pairs of points belong to different clusters. Consequently, even two completely independent partitions will agree on a large number of pairs, simply because both assign them to different clusters. This effect becomes more pronounced as the number of points or clusters increases.

The ARI addresses this issue by accounting for the agreement that would be expected purely by chance. It compares the observed agreement RI to the expected agreement under random partitions $\mathbb{E}[\mathrm{RI}]$. The score is then normalized by the maximum agreement achievable beyond this random baseline,

$\max(\text{RI}) - \mathbb{E}[\text{RI}]$. As a result, an ARI of 0 corresponds to the level of agreement expected from random clusterings, while an ARI of 1 indicates perfect agreement between the two partitions:

$$\text{ARI} = \frac{\text{RI} - \mathbb{E}[\text{RI}]}{\max(\text{RI}) - \mathbb{E}[\text{RI}]}.$$

In practice, the ARI is computed directly from the cluster assignments of the two partitions. Let clustering $\mathcal{C}$ be composed of clusters $\{C_i\}$ and clustering $\mathcal{C}'$ of clusters $\{C'_j\}$. Let $a_i = |C_i|$ denote the number of samples in cluster i of clustering $\mathcal{C}$, and $b_j = |C'_j|$ the number of samples in cluster j of clustering $\mathcal{C}'$. Furthermore, let $n_{ij} = |C_i \cap C'_j|$ be the number of samples shared by cluster i of clustering $\mathcal{C}$ and cluster j of clustering $\mathcal{C}'$, and let N be the total number of samples. With these definitions, the ARI can be computed as:

$$\text{ARI} = \frac{\sum_{i,j} \binom{n_{ij}}{2} - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] / \binom{N}{2}}{\frac{1}{2}\left[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\right] - \left[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\right] / \binom{N}{2}}.$$

Because it is bounded by 1, a value such as 0.5 can be misleadingly interpreted as "halfway to a perfect match". This interpretation is incorrect because the ARI is normalized against the agreement expected from two random partitions. As a result, values close to 1 require the two clusterings to be nearly identical, while a value of 0.5 already indicates substantial agreement [91].

6.3.3 Clustering Universality

Having introduced the ARI, we can now define the *Clustering Universality* (CU) metric. While the ARI quantifies the similarity between two partitions, the objective of CAR is to assess the consistency of conceptual organizations across an entire set of independently trained surrogate policies.

To this end, the same set of observations is projected into the LS of each surrogate policy and clustered. Under the *Representation Convergence Hypothesis* (Hypothesis 4), these independently obtained partitions are expected to be similar. CU quantifies this by computing the mean pairwise ARI:

$$\text{CU} = \frac{1}{\binom{n}{2}} \sum_{i=2}^{n} \sum_{j=1}^{i-1} \text{ARI}(\mathcal{C}_i, \mathcal{C}_j). \tag{6.3}$$

This metric takes values in $[-1, 1]$. A value close to 1 indicates that surrogate policies produce highly similar partitions, suggesting that the clusters reflect intrinsic properties of the policy. Conversely, a value near 0 implies that the similarity between partitions is no better than that of two randomly generated clusterings, meaning that the clustering fails to capture any meaningful structure in the policy. A negative value, on the other hand, signals that the agreement between partitions is worse than what would be expected by chance, indicating a systematic disagreement between the clusterings.

Figure 6.4 illustrates CU in the ideal case. It shows three LSs obtained by projecting the same seven observations, labeled 1 to 7, through three independently trained surrogate policies. Although the embeddings occupy different positions, the clusters group exactly the same elements in all three cases. The partitioning is therefore identical, resulting in an ARI of 1.

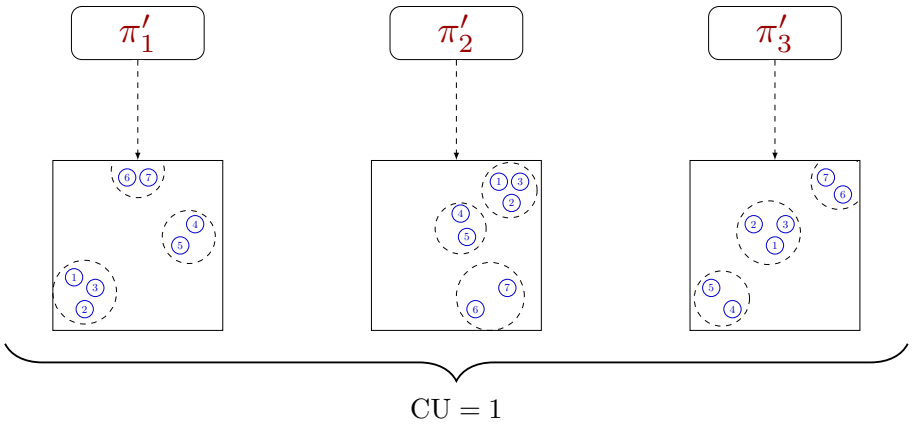


Figure 6.4: Illustration of CU in the ideal case where projections of the same observations are grouped within the same clusters across all surrogate policies.

6.3.4 Abstraction Score

If CU is high, it becomes important to determine what underlies this stability. Two possibilities arise:

(i) the LS captures similarities in decision-making, resulting in behaviorally meaningful clusters;

(ii) the LS provides no abstraction, as defined in Definition 3.1.1, because it either reproduces the observation space or action distribution, or reflects a structure already present in the network architecture before training.

To distinguish among these cases, we introduce the *Abstraction Score* (AS). This metric contextualizes latent clusterings by examining how their patterns align with three reference baselines: the raw observations, the action distribution of the initial policy, and an untrained network of the same architecture. All baseline clusterings use the same algorithm, distance metric, and hyperparameters as those used for $\mathcal{C}_i$ in the LS, so that all clusterings are obtained under the same conditions.

The first baseline addresses the case where the LS merely reproduces the structure of the raw observations. It is the clustering of the observations $\mathcal{O}_{\text{data}}$ themselves, denoted by $\mathcal{C}_{\text{obs}}$.

The second baseline addresses the case where the LS merely reproduces the decisions of the initial policy. It is the clustering, denoted by $\mathcal{C}_{\text{act}}$, of the logits produced by the initial policy for the same observations, which are the targets the surrogate policies are trained to reproduce. These continuous parameters are preferred to the sampled discrete action, whose few distinct values would limit the number of distinguishable clusters.

The third baseline addresses the case where the LS merely reflects an inductive bias of the network architecture rather than something learned. To this end, we introduce U untrained networks sharing the same architecture as the surrogate policies, but with randomly initialized, never-updated weights. Each of these U untrained networks is used to project the same observations into its own LS, which is then clustered in the same way, yielding partitions $\mathcal{C}_{\text{untr},1}, \ldots, \mathcal{C}_{\text{untr},U}$.

This baseline isolates what the architecture already imposes on the representation before any learning takes place, so that a high similarity between $\mathcal{C}_i$ and this baseline indicates that the corresponding structure cannot be attributed to training.

For each latent clustering $\mathcal{C}_i$, we define its similarity to these three baselines using the ARI:

$$\begin{aligned}\rho_i^{\text{obs}} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{\text{obs}}),\\ \rho_i^{\text{act}} &= \text{ARI}(\mathcal{C}_i, \mathcal{C}_{\text{act}}),\\ \rho_i^{\text{untr}} &= \frac{1}{U}\sum_{u=1}^{U} \text{ARI}(\mathcal{C}_i, \mathcal{C}_{\text{untr},u}).\end{aligned}$$

The AS is subsequently calculated as one minus the average of the maximum absolute similarity values across these three reference baselines, where n is the number of surrogate policies:

$$\text{AS} = 1 - \tfrac{1}{n}\textstyle\sum_{i=1}^{n} \max\left(\left|\rho_i^{\text{obs}}\right|, \left|\rho_i^{\text{act}}\right|, \left|\rho_i^{\text{untr}}\right|\right). \tag{6.4}$$

The AS ranges from $[0, 1]$. A score close to 1 indicates that the latent clusters are independent of the observation space, the action distribution, and the network's initialization, capturing abstractions that meaningfully reflect the agent's behavior, case (i). A score around 0 means that the latent clusters are strongly aligned with the observation space, the action distribution, or the untrained network, reflecting no abstraction, case (ii).

6.3.5 Adjusted Clustering Universality

As discussed above, CU alone does not indicate whether the recovered structure is meaningful. We therefore introduce the *Adjusted Clustering Universality* (ACU), which measures how much of the agreement between the different surrogate policies remains after accounting for the three trivial baselines:

$$\text{ACU} = \text{CU} - \tfrac{1}{n}\textstyle\sum_{i=1}^{n} \max\left(\left|\rho_i^{\text{obs}}\right|, \left|\rho_i^{\text{act}}\right|, \left|\rho_i^{\text{untr}}\right|\right). \tag{6.5}$$

This metric ranges in $[-1, 1]$. A value close to 1 indicates that surrogate policies converge toward a partition that is both highly reproducible across seeds and clearly not explained by the observation space, the action distribution, or the network's initialization, providing the strongest evidence for *Representation Convergence Hypothesis* (Hypothesis 4). A value close to 0, or negative, covers two cases. Either the independently trained surrogate policies do not agree with one another, or their agreement is explained by one of the trivial baselines, namely the raw observations, the action distribution of the initial policy, or an untrained network sharing the same architecture.

6.4 Conclusion

The CAR method was developed to investigate how concepts are organized within the representations of RL policies. It is based on three hypotheses introduced throughout this thesis, the *Mechanistic Hypothesis* (Hypothesis 3),

the *Concept Hypothesis* (Hypothesis 1), and the *Representation Convergence Hypothesis* (Hypothesis 4).

CAR translates these assumptions into an empirical analysis. Multiple surrogate policies are trained independently to reproduce the same policy. Their LRs are then clustered and compared to assess whether similar conceptual structures emerge across policies. The CU and AS provide quantitative measures for evaluating the consistency and abstraction of the resulting clusters, and the ACU combines them into a single criterion.

This makes CAR a general procedure for analyzing agent representations. The analysis and evaluation metrics are fully automated and require no manual intervention or environment-specific adjustments. The method can therefore be applied to a wide range of RL settings and provides a basis for systematically studying the internal representations of neural policies.

The next chapter applies CAR to several Atari environments to evaluate its ability to identify meaningful conceptual structures.

Chapter 7

Experiments

"There are three kinds of lies: lies, damned lies, and statistics."
Attributed to Benjamin Disraeli by Mark Twain, *Chapters from My Autobiography* (1907) [5]

Contents

7.1 Software Implementation . . . 102
7.1.1 Initial Policy Training . . . 103
7.1.2 Trajectory Dataset Generation . . . 103
7.1.3 Surrogate Policy Training . . . 104
7.1.4 Surrogate Policy Evaluation . . . 105
7.1.5 Latent Space Analysis . . . 105
7.2 Experimental Configuration . . . 105
7.2.1 Environments . . . 105
7.2.2 Neural Architecture . . . 107
7.2.3 Other Clustering Agent Representation Parameters . . . 108
7.3 Policy Performance and Surrogate Fidelity . . . 108
7.3.1 Initial Policy Training . . . 108
7.3.2 Surrogate Policy Training . . . 109
7.4 Clustering Quality of the Latent Space . . . 110
7.4.1 Analysis Across Layers . . . 112
7.4.2 Analysis of the Selected Layers . . . 113
7.5 Semantic Interpretation of the Clusters . . . 113
7.5.1 Breakout . . . 116
7.5.2 Mario Bros . . . 118
7.5.3 Ms. Pac-Man . . . 120
7.5.4 Pong . . . 122
7.5.5 Space Invaders . . . 124
7.5.6 Surround . . . 126
7.6 Conclusion . . . 128

This chapter presents the experimental study of the Clustering Agent Representation (CAR) method introduced in the previous chapter. It focuses on evaluating the method's ability to identify conceptual structures. The remainder of this chapter is organized as follows:

- Section 7.1 describes the software implementation of the CAR pipeline.
- Section 7.2 details the experimental configuration, covering the environments, the Reinforcement Learning (RL) setup, and the surrogate policy training.
- Section 7.3 presents the training of the initial and surrogate policies and evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy.
- Section 7.4 provides a quantitative analysis of the Latent Representations (LRs) using the *Clustering Universality* (CU), *Abstraction Score* (AS), and *Adjusted Clustering Universality* (ACU) metrics.
- Section 7.5 provides a qualitative analysis of the resulting clusters by manually examining their semantic content.
- Section 7.6 summarizes the findings and discusses their implications.

7.1 Software Implementation

This section details the experimental evaluation of the proposed method. Deep Learning (DL) and RL are strongly empirical fields, where experimental evaluation plays an important role in assessing how methods perform in practice.

Particular attention is therefore given to the implementation of CAR. It relies on established libraries that provide efficient implementations of its main components. The code is made publicly available as open-source to support reproducible replication of the reported results.[1]

The CAR implementation is organized as a pipeline of five stages. Each stage performs a specific operation and produces the artifacts required by the subsequent stages. The stages are implemented as independent executables allowing each stage to be rerun independently.

The whole pipeline is driven by a single configuration file, which is the input of the process and gathers the parameters of every stage. The CAR pipeline is illustrated in Figure 7.1, and Table 7.1 summarizes the libraries used throughout the pipeline. The remainder of this section describes the purpose and implementation of each stage.

[1]The source code of the CAR implementation is available at https://github.com/Maxalaar/Clustering_Agent_Latent_Space.

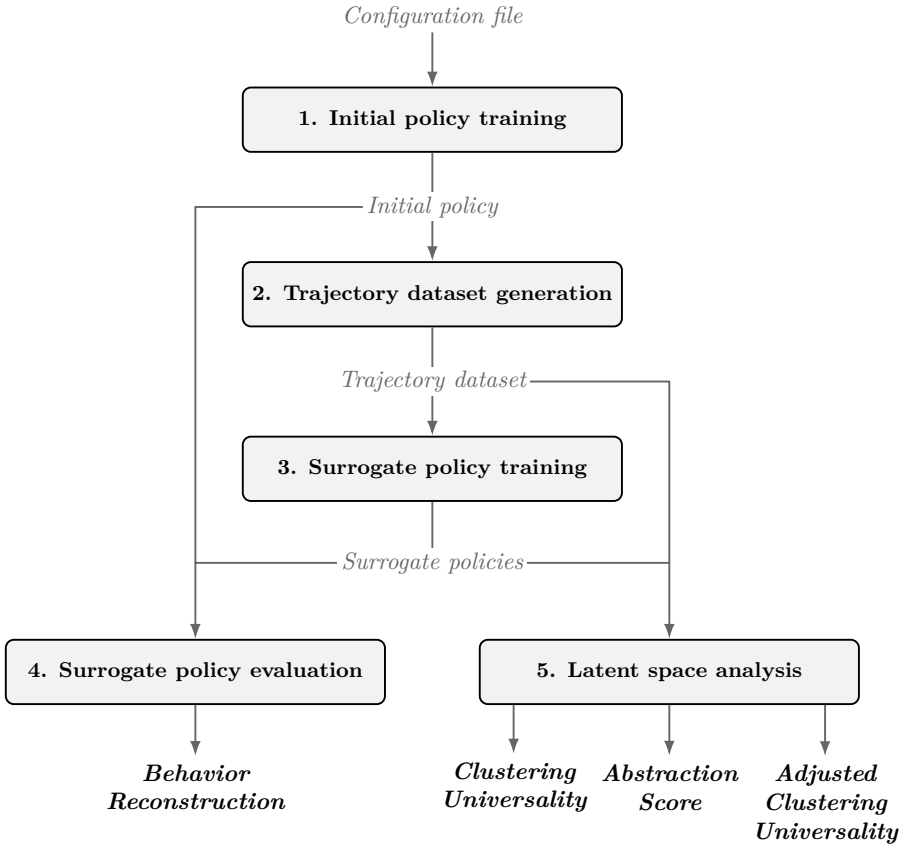


Figure 7.1: The CAR pipeline.

7.1.1 Initial Policy Training

The first stage trains the initial policy using RLlib [69, 71]. RLlib provides scalable implementations of RL algorithms and supports distributed training. The use of RLlib also allows CAR to be applied to environments implemented using the Gymnasium interface [144]. By default, the initial policy is trained using Proximal Policy Optimization (PPO) [62]. Other RLlib algorithms, such as Asynchronous Proximal Policy Optimization (APPO) [64], Deep Q-Network (DQN) [47], and DreamerV3 [141], can also be selected through the same configuration.

The policy and critic networks are implemented using PyTorch [85]. PyTorch is the standard DL framework and is widely used for developing and training Neural Networks (NNs). It provides GPU acceleration and supports a wide range of NN architectures. This makes it well suited for implementing the different policy and critic architectures used in the experiments.

7.1.2 Trajectory Dataset Generation

The second stage builds the dataset used to train the surrogate policies. RLlib is used to collect trajectories in parallel using multiple environment runners. For

Pipeline Stage	Library	Description
Initial policy training	RLlib [69, 71]	Distributed RL library used to train the initial policy.
	PyTorch [85]	Deep learning framework implementing the policy and critic networks.
	Gymnasium [144]	Common interface used to plug in any RL environment.
Trajectory dataset generation	RLlib [69, 71]	Distributed environment runners used to collect trajectories in parallel.
	Apache Arrow [51]	Columnar file format (Parquet) used to store the trajectory datasets.
Surrogate policy training	PyTorch Lightning [76]	Structured training loop used for the behavior cloning of the surrogate policies.
Surrogate policy evaluation	RLlib [69, 71]	Wraps each surrogate policy as an RL policy to evaluate it in the environment.
Latent space analysis	cuML [128]	GPU-accelerated clustering algorithms used to analyze the latent representations.

Table 7.1: Main software libraries used at each stage of the CAR pipeline.

each step, the dataset stores the observation together with the parameters of the action distribution produced by the initial policy.

Trajectories are stored as Parquet files [51]. Its columnar structure allows efficient data access and supports concurrent processing. This is useful here because the datasets contain around one million observation–action pairs and cannot be loaded entirely into memory.

7.1.3 Surrogate Policy Training

The third stage trains the surrogate policies using Behavioral Cloning (BC) on the trajectory dataset. This provides a supervised learning approach for replicating the initial policy. The training procedure is implemented using PyTorch Lightning [76]. PyTorch Lightning provides a simplified interface for training NNs with PyTorch. It handles common training operations such as data loading, optimization, checkpointing, and GPU execution.

7.1.4 Surrogate Policy Evaluation

The fourth stage evaluates how faithfully the surrogate policies reproduce the behavior of the initial policy. This stage therefore requires both the surrogate policies and the initial policy. Each surrogate is wrapped as an RLlib policy and evaluated in the original environment. The initial policy is evaluated under the same conditions to provide the reference return. The mean cumulative reward of each surrogate over multiple episodes is then compared with that of the initial policy. Evaluation episodes are collected using parallel environment runners.

This stage computes the *Behavior Reconstruction*. It therefore verifies that the surrogate policies reproduce the behavior of the initial policy sufficiently well before their LRs are analyzed. If a surrogate does not meet the required level of behavior reconstruction, its training can be repeated.

7.1.5 Latent Space Analysis

The fifth stage performs the main analysis of CAR. The trajectory dataset and trained surrogate policies are used to generate the LRs. These representations form the point clouds that are analyzed using clustering algorithms.

The clustering step relies on the cuML library [128], which provides GPU-accelerated clustering algorithms. This reduces the computational cost and allows point clouds containing tens of thousands of representations to be analyzed.

The resulting partitions are used to compute the CU, the AS, and the ACU. The stage also produces the material required for the qualitative analysis, including the clustered point clouds and the cluster assignment of each observation.

7.2 Experimental Configuration

This section describes the configuration of the experiments reported in this chapter. It covers the environments, the neural architectures used, and the main parameters of the pipeline.

7.2.1 Environments

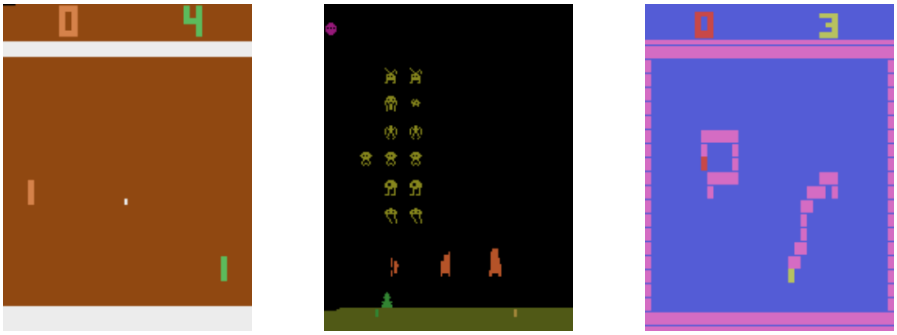
All experiments presented in this thesis use environments from the Atari benchmark [44], accessed through the Arcade Learning Environment. The use of Atari environments offers several advantages:

- **Standard benchmark:** Atari is widely used in the RL community, making the experimental results easier to compare with existing work.
- **Limited environment-specific bias:** The environments were not specifically designed for RL, reducing the risk of introducing biases related to their design.
- **Diversity:** The benchmark provides a diverse set of well-known environments with different types of tasks and levels of complexity, allowing CAR to be evaluated under different conditions.

The configuration of the Atari environments follows common practice in RL research [70]. In each environment, the agent receives visual observations in the



(a) Breakout (b) Mario Bros (c) Ms. Pac-Man



(d) Pong (e) Space Invaders (f) Surround

Figure 7.2: The six Atari environments used to evaluate CAR.

form of raw pixels. Each image is resized to 64×64 pixels and converted to grayscale to reduce memory requirements.

To provide information about the dynamics of the environment, four consecutive observations are stacked along the channel dimension. The resulting observation has a shape of $64 \times 64 \times 4$. This allows the agent to infer the motion of objects from successive observations.

The actions correspond to the different buttons available on the Atari controller. The resulting action space is therefore discrete. Since Atari games are fully deterministic, sticky actions are introduced to add stochasticity to the trajectories. With a probability of 0.05, the environment repeats the action selected at the previous step instead of applying the new action. This prevents the agent from simply memorizing a fixed sequence of actions during training.

To reduce the duration of each episode, frame skipping is used. The agent selects a new action every five frames and repeats it for the following frames. This reduces the number of decisions required during an episode while maintaining sufficient temporal resolution for effective control.

The experiments use the six environments described below:

- Breakout: The agent moves a paddle at the bottom of the screen (Figure 7.2(a)) to bounce a ball into a wall of bricks. Each broken brick scores one point, and the ball speeds up as the wall is cleared.

- Mario Bros: A platform game (Figure 7.2(b)) in which the player clears each phase by hitting enemies from below to flip them, then kicking them off the platforms before they recover. A central POW block can flip all enemies at once but can only be used a few times per phase.
- Ms. Pac-Man: The player moves through a maze (Figure 7.2(c)), eating pellets while avoiding four ghosts. Power pellets temporarily make the ghosts edible.
- Pong: A two-player game (Figure 7.2(d)) in which the agent controls one paddle and must send the ball past the opponent.
- Space Invaders: The agent moves a cannon along the bottom of the screen (Figure 7.2(e)), firing at descending rows of aliens while avoiding their shots and using destructible shields. A bonus saucer occasionally crosses the top of the screen.
- Surround: A two-player game on a grid (Figure 7.2(f)) in which each player leaves a trail behind a moving cursor. The goal is to force the opponent to crash into a trail.

7.2.2 Neural Architecture

The architecture of the surrogate policies was deliberately kept simple. The objective of this chapter is not to search for the best-performing architecture, but to obtain surrogate policies that faithfully reproduce the initial policy while remaining easy to analyze. A standard convolutional architecture was therefore selected because it is sufficient to reach a high *Behavior Reconstruction* on the six environments, as shown in Section 7.3.

The surrogate policies use a Convolutional Neural Network (CNN), as presented in Section 2.6.1, to process the visual observations. The convolutional part consists of four layers with 16, 32, 64, and 128 channels, using 4×4 kernels with a stride of 2. Each convolutional layer is followed by a LeakyReLU activation. The resulting representation is then flattened and passed through a Multilayer Perceptron (MLP) that produces the parameters of the action distribution.

The architecture of the MLP varies across environments. For Pong and Space Invaders, it contains a single hidden layer with 128 units. For the other four environments, it contains three hidden layers with 512, 256, and 128 units. The size of the output layer matches the number of actions available in each environment: 4 for Breakout, 18 for Mario Bros, 9 for Ms. Pac-Man, 6 for Pong, 6 for Space Invaders, and 5 for Surround.

Figure 7.3 illustrates both configurations, together with the shape of the representation produced by each convolutional layer. Throughout the remainder of this chapter, the analyzed representations are numbered from 1 to 5. These representations correspond to the four convolutional layers and the flattened representation, which are common to all six environments. The feature maps produced by the convolutional layers are flattened into vectors before clustering.

The representations produced by the MLP are numbered separately. For Pong and Space Invaders, which use a single hidden layer, the representation is numbered 6a. For the other four environments, which use three hidden layers, the representations are numbered 6b, 7b, and 8b.

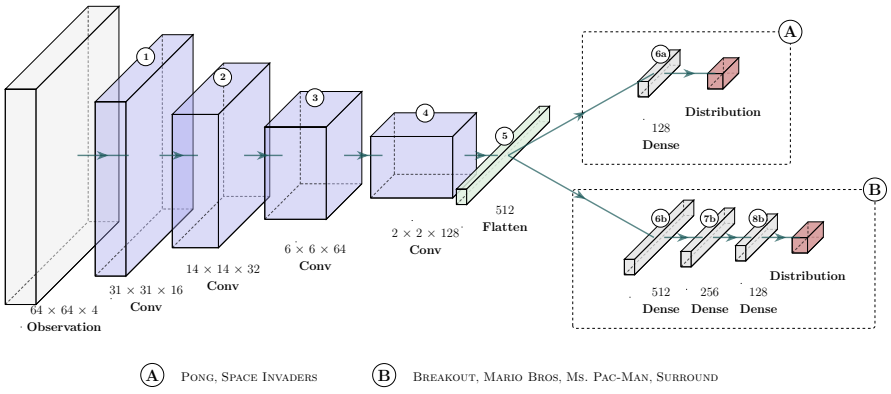


Figure 7.3: Architecture of the surrogate policies.

7.2.3 Other Clustering Agent Representation Parameters

Several other parameters must be defined for the different stages of the CAR pipeline. These parameters are less critical to the overall methodology than those discussed above, but they still need to be specified to reproduce the experiments. Since each stage introduces its own configuration choices, there are too many parameters to discuss them all in detail.

The main parameters used across the experiments are summarized in Table 7.2. All these parameters are provided to the pipeline through the modular configuration file. Each stage reads its own section of this file, so that a parameter can be changed, or a component replaced, without modifying the code of the pipeline.

These settings are specific to the experiments presented in this chapter and do not restrict the CAR methodology itself. The different stages of the pipeline are modular and can be configured or replaced depending on the application. The evaluation metrics provide an objective basis for comparing these alternatives and assessing their effect on the consistency and meaningfulness of the resulting clusters.

7.3 Policy Performance and Surrogate Fidelity

This section reports the results of training the initial and surrogate policies for CAR, applied to the six environments. The first part verifies that the initial policies reach a competent level of play and evaluates how faithfully the surrogate policies reproduce their behavior using the *Behavior Reconstruction*.

7.3.1 Initial Policy Training

Figure 7.4(a) shows how the initial policies improve during PPO training. For each environment, the mean episode return is normalized so that 0% and 100% correspond to the lowest and highest mean return observed during that run.

In every environment the return increases and then reaches a plateau, at which point training is stopped. The number of iterations required to reach this plateau varies from a few hundred, for Pong, to several thousand, for Surround

Environment configuration	
Frame size	64×64
Frames stacked per observation	4
Frame skip	5
Sticky-action probability	0.05
Initial policy training	
Algorithm	PPO
Optimizer	Adam
Learning rate	1.5×10^{-4}
GAE λ	0.95
Policy clipping ε	0.1
Value-function clipping	10
Entropy coefficient	0.01
KL coefficient	0.5
Epochs per update	10
Gradient clipping (global norm)	100
Surrogate policy training	
Surrogate policies per environment	3
Observation–action pairs per dataset	$\geq 10^6$
Batch size	2048
Optimizer	Adam
Learning rate	1×10^{-4}
Latent space analysis	
Embedding per surrogate policy	20 000
Clustering algorithm	k-means
Distance	Euclidean distance
Number of clusters K	2 to 20

Table 7.2: Main experimental parameters.

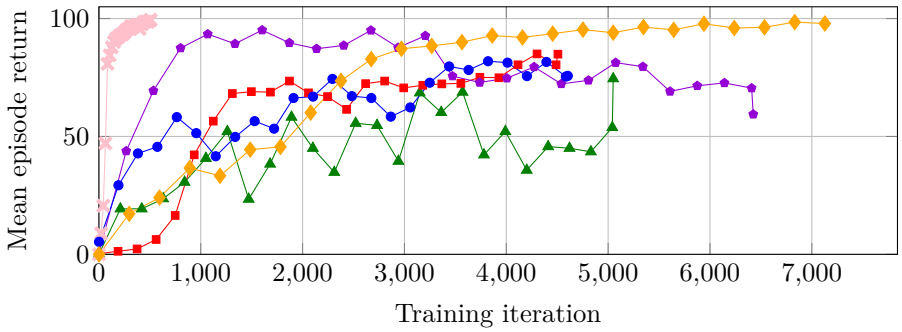
and Ms. Pac-Man. For the subsequent analysis, the checkpoint with the best performance is retained rather than the final checkpoint.

To confirm that the trained policies play competently, their scores are compared with the reference scores reported by [70]. Each initial policy reaches a score of the same order as these references, so it constitutes a reasonable target for imitation. Recordings of the initial policies playing each game are also provided to illustrate their behavior qualitatively.[2]

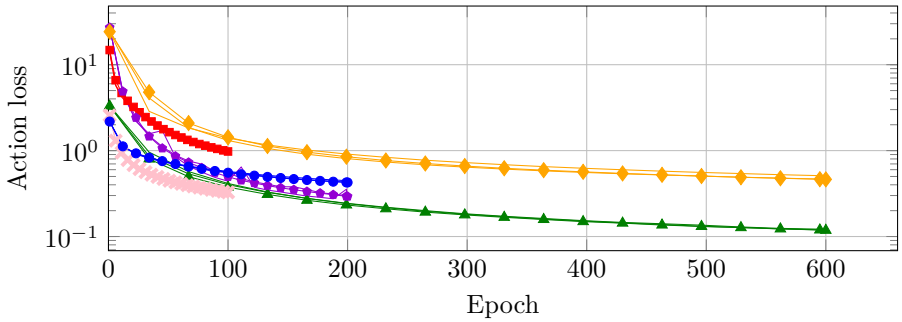
7.3.2 Surrogate Policy Training

Figure 7.4(b) shows, for each environment, the training action loss of the three surrogate policies. In all cases, the loss decreases smoothly, and the three independently trained policies converge to the same value.

[2]The videos have been archived on Zenodo https://zenodo.org/records/23019943.



(a) Training of the initial policies with PPO.



(b) Training action loss of the surrogate policies.



Figure 7.4: Training of the initial and surrogate policies.

We further evaluate the ability of the surrogate policies to replicate the behavior of the initial policies using the *Behavior Reconstruction* (Table 7.3). The surrogate policies reach a *Behavior Reconstruction* of 0.96 on average over 100 evaluation episodes, ranging from 0.83 for Breakout to 1.00 for Ms. Pac-Man and Surround. They therefore reproduce the decisions of the initial policies closely enough for their LRs to be analyzed.

7.4 Clustering Quality of the Latent Space

This section analyzes the Latent Space (LS) of the surrogate policies using CU, AS, and ACU. To do so, a random subset of 20 000 observations is selected for each environment. This subset is then embedded into the LS of each surrogate policy at each analyzed layer. This produces a point cloud for each combination of environment, surrogate policy, and layer. Each point cloud is partitioned using k-means clustering [15] for each number of clusters from 2 to 20.

Environment	*Behavior Reconstruction*
Breakout	0.83 ±0.03
Mario Bros	0.99 ±0.03
Ms. Pac-Man	1.00 ±0.03
Pong	0.95 ±0.01
Space Invaders	0.97 ±0.07
Surround	1.00 ±0.01

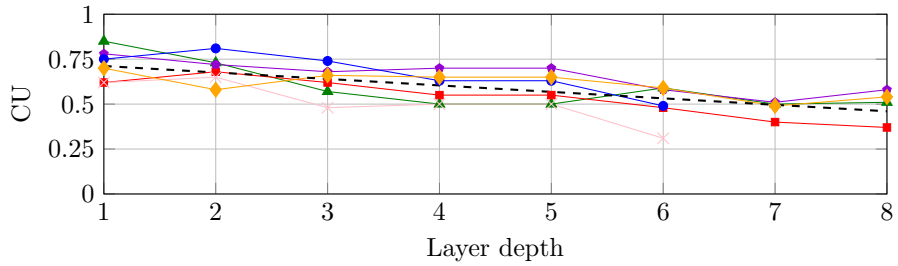
Table 7.3: Evaluation of *Behavior Reconstruction* across the six Atari environments.

The three metrics are then computed from these point clouds and their corresponding clusterings. Their interpretation, detailed in Section 6.3, is briefly recalled here:

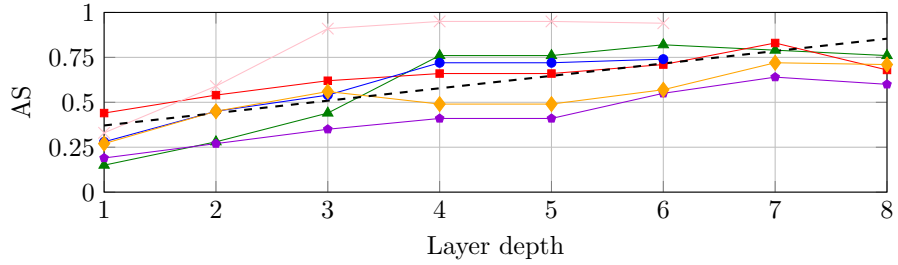
- **CU** measures the agreement between the partitions obtained from independently trained surrogate policies. A value close to 1 indicates that the surrogate policies group the observations in the same way, while a value close to 0 indicates an agreement no better than chance. Since it relies on the Adjusted Rand Index (ARI), a value of 0.5 already corresponds to a substantial agreement.
- **AS** measures how far the latent partitions are from three trivial baselines: the clustering of the raw observations, of the action distribution of the initial policy, and of untrained networks. A value close to 1 indicates that the clusters capture an abstraction learned by the policy, while a value close to 0 indicates that they merely reproduce one of these baselines.
- **ACU** subtracts the agreement explained by these baselines from CU. A value close to 1 indicates that the surrogate policies converge toward a partition that is both reproducible and non-trivial, providing the strongest support for the *Representation Convergence Hypothesis* (Hypothesis 4). A value close to 0, or negative, indicates that the observed agreement is largely explained by the baselines.

A high CU alone is therefore not sufficient, since it can result from a trivial structure shared by all networks. It is the combination of a high CU and a high AS, summarized by ACU, that supports the existence of a stable conceptual organization.

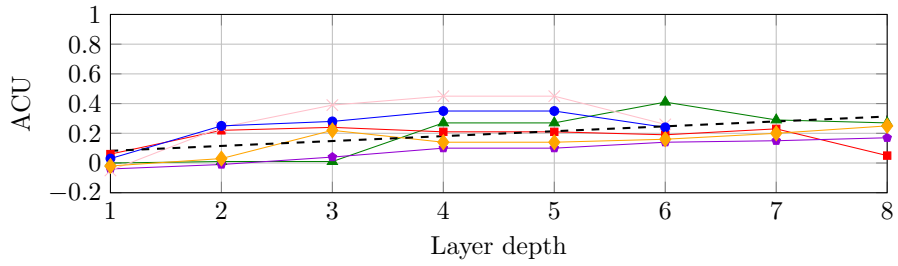
The full results across all environments and analyzed layers are reported in Appendix 1. Two complementary analyses are then conducted on these results. First, the evolution of the three metrics across the different layers is examined to study the effect of representation depth. Second, for each environment, the layer achieving the highest average ACU is selected and analyzed in more detail across the different numbers of clusters.



(a) CU with respect to the layer depth.



(b) AS with respect to the layer depth.



(c) ACU with respect to the layer depth.



Figure 7.5: CU, AS, and ACU averaged over cluster numbers as a function of layer depth.

7.4.1 Analysis Across Layers

The first analysis examines how the three metrics evolve with the depth of the analyzed layer. For each layer, CU, AS, and ACU are averaged over the tested cluster numbers. Figure 7.5 reports these averages as a function of layer depth.

Across environments, CU tends to be highest in the earliest layers and decreases with depth. AS follows the opposite trend and generally increases as the representation becomes less tied to the raw observation, the action distribution, and the network's initialization. This empirical trend is consistent with the intuition, discussed in Section 3.1, that each hidden layer progressively refines the LR of the previous one into a more abstract form [23].

Environment	Layer	CU	AS	ACU
Breakout	6b	0.59 ±0.06	0.82 ±0.05	0.41 ±0.06
Mario Bros	3	0.62 ±0.12	0.62 ±0.15	0.24 ±0.10
Ms. Pac-Man	8b	0.58 ±0.08	0.60 ±0.05	0.17 ±0.06
Pong	4	0.50 ±0.04	0.95 ±0.03	0.45 ±0.06
Space Invaders	4	0.63 ±0.11	0.73 ±0.06	0.35 ±0.09
Surround	8b	0.54 ±0.10	0.71 ±0.05	0.25 ±0.06

Table 7.4: Layer that maximizes the average ACU across $K \in [2, 20]$ for each environment, together with the corresponding average CU, AS, and ACU over K.

This increase is however not monotonic in the deepest layers. For Mario Bros and Ms. Pac-Man, AS decreases at the last layer, from 0.83 to 0.68 and from 0.64 to 0.60 respectively, as the latent clustering becomes closer to the clustering of the action distribution, this layer being directly followed by the output layer. As ACU combines these two effects, its evolution depends on the environment.

The corresponding linear fits show that CU decreases from 0.72 at the first layer to 0.50 at the last, with a slope of approximately -0.04 per layer, while AS increases from 0.28 to 0.69, with a slope of approximately $+0.06$ per layer. Over the same range, ACU increases more moderately from 0.00 to 0.19, with a slope of approximately $+0.03$ per layer. This suggests that, despite the environment-specific variations, deeper representations tend on average to provide a better trade-off between CU and AS.

7.4.2 Analysis of the Selected Layers

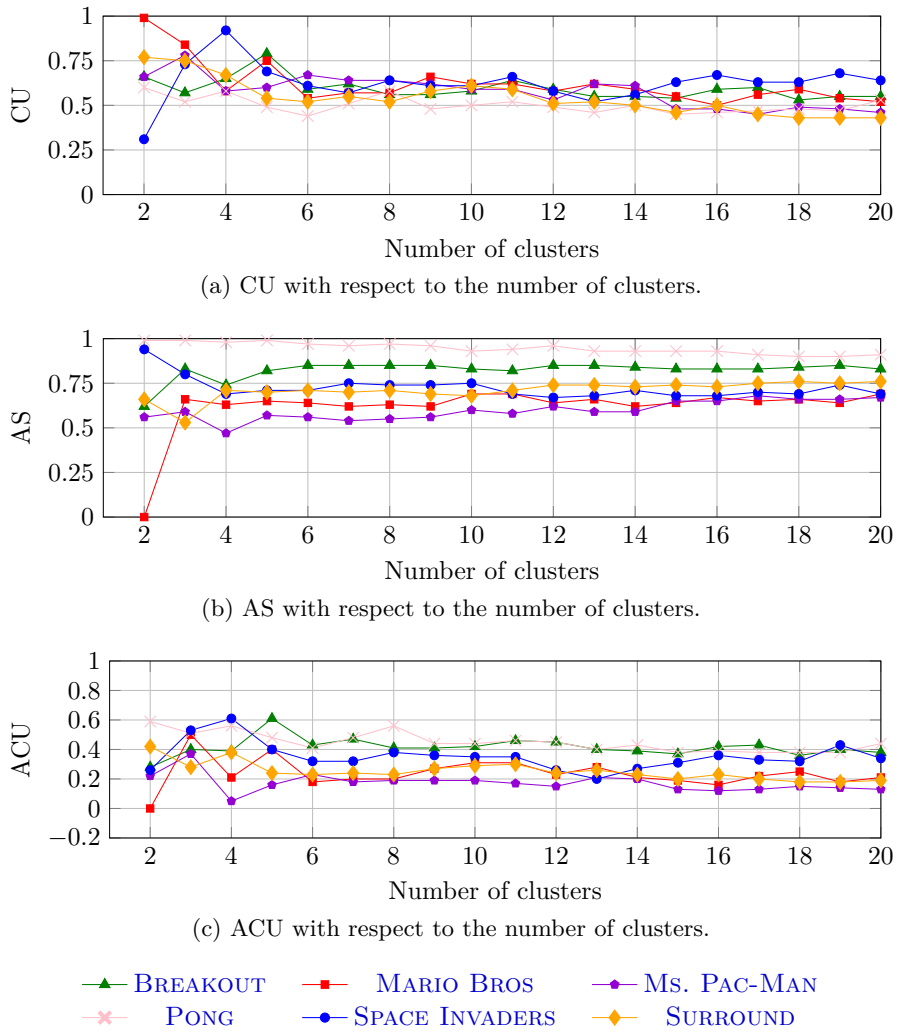
The second analysis examines the effect of the number of clusters on the three metrics. For each environment, the layer with the highest average ACU across the tested cluster numbers is selected. Table 7.4 reports the selected layer, together with the corresponding average CU, AS, and ACU.

Figure 7.6 reports the evolution of CU, AS, and ACU as the number of clusters varies. Pong achieves the highest average ACU among the six environments, with a value of 0.45 ± 0.06. This results from a moderate CU combined with an AS that saturates from the third layer onward. In contrast, Ms. Pac-Man achieves the lowest AS, with 0.60 ± 0.05. The resulting ACU, 0.17 ± 0.06, is the lowest across the six environments.

The quantitative analysis provides a measure of the consistency and abstraction of the identified clusters, but does not directly reveal their semantic content. The following section therefore provides a qualitative analysis of the clusters to examine the concepts they represent.

7.5 Semantic Interpretation of the Clusters

The metrics reported in Section 7.4 evaluate the LS clusterings quantitatively but do not indicate what the resulting clusters actually represent. This section



(a) CU with respect to the number of clusters.

(b) AS with respect to the number of clusters.

(c) ACU with respect to the number of clusters.

Figure 7.6: CU, AS, and ACU with respect to the number of clusters, for each of the six environments, for the layer that maximizes the average ACU in that environment.

complements them with a qualitative, hand-labeled inspection of the clusters obtained for each environment.

For each environment, two clusterings are selected for the qualitative analysis. They are obtained using two different numbers of clusters and may correspond to different layers of the network. The choice of the layer and the number of clusters is based on the following criteria:

- **High ACU:** The selected clustering should achieve a sufficiently high ACU, indicating a good balance between consistency and abstraction.
- **Manual analysis:** The number of clusters must remain small enough to allow the resulting clusters to be examined manually.
- **Limited use of two-cluster partitions:** Clusterings with two clusters often achieve a high ACU, but provide limited information about the organization of the LS. Therefore, at most one of the two selected clusterings uses two clusters, while the other uses a larger number of clusters to provide a more detailed view of the LS.

Only these two clusterings are discussed below, but the clusters obtained for every analyzed layer can also be inspected.[3]

[3]Images of the clusters obtained for every analyzed layer have been archived on Zenodo https://zenodo.org/records/23021063.

7.5.1 Breakout

Two clusterings are examined for Breakout (Table 7.5), obtained at layer 4 with $K = 2$ and at layer 6b with $K = 5$.

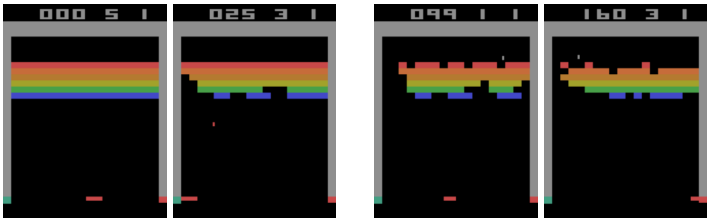
At layer 4 (Figure 7.7), the partition captures the position of the ball relative to the brick wall. The ball remains below the wall in cluster A. The ball has passed behind the wall in cluster B, allowing it to continue clearing bricks without further interaction with the paddle.

At layer 6b (Figure 7.8), the partition again isolates situations in which the ball has passed behind the wall in cluster E. It also identifies the digging of a tunnel through the left side of the wall in cluster C. Finally, it distinguishes different paddle positions at the beginning of the episode in clusters D, F, and G.

The clustering at layer 4 with $K = 2$ is almost perfectly reproducible across surrogate policies, with a CU of 0.99 and an ACU of 0.87. The clustering at layer 6b with $K = 5$ is less consistent, with a CU of 0.79 and an ACU of 0.61, but reveals additional information about the organization of the LS.

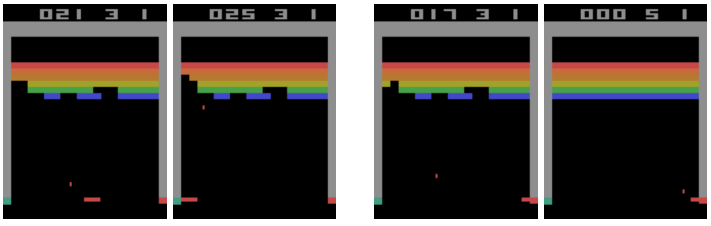
Layer 4	$K = 2$	**CU 0.99**	**AS 0.88**	**ACU 0.87**
A	The ball is below the brick wall.			
B	The ball passes behind the brick wall.			
Layer 6b	$K = 5$	**CU 0.79**	**AS 0.82**	**ACU 0.61**
C	Digging a tunnel through the left side of the wall.			
D	Start of the episode, paddle on the right.			
E	The ball is behind the brick wall.			
F	Start of the episode, paddle in the middle.			
G	Start of the episode, paddle on the left.			

Table 7.5: CU, AS, and ACU of the two clusterings analyzed in the Breakout environment.

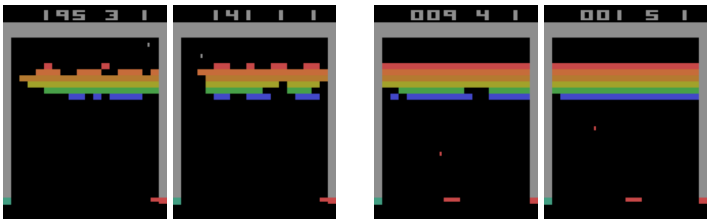


A **B**

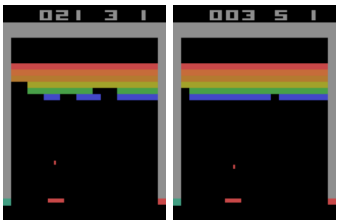
Figure 7.7: Clusters A–B identified by CAR in the BREAKOUT environment (layer 4, $K = 2$).



C **D**



E **F**



G

Figure 7.8: Clusters C–G identified by CAR in the BREAKOUT environment (layer 6b, $K = 5$).

7.5.2 Mario Bros

Two clusterings are examined for Mario Bros (Table 7.6), both obtained at layer 3, with $K = 3$ and $K = 5$.

With $K = 3$ (Figure 7.9), the partition is organized around the location of the action. Mario fights enemies on the left side of the screen in cluster A. He fights enemies on the right side of the screen in cluster C. The POW block is present in cluster B, isolating a fixed and recognizable element of the level.

With $K = 5$ (Figure 7.10), the partition provides a more detailed description of the situations occurring in the level. Mario is on the top-left platform in cluster D. Mario attacks enemies at the top left in cluster F. Mario hits enemies from below in cluster H. Mario simply moves through the level in cluster G. The POW block remains isolated in cluster E.

Increasing K from 3 to 5 lowers CU from 0.84 to 0.75. The AS remains nearly unchanged, with values of 0.66 and 0.65. This results in a decrease in ACU from 0.50 to 0.40.

Layer 3	$K = 3$ **CU 0.84**	**AS 0.66**	**ACU 0.50**
A	Mario fights enemies on the left side of the screen.		
B	The POW block is present.		
C	Mario fights enemies on the right side of the screen.		
Layer 3	$K = 5$ **CU 0.75**	**AS 0.65**	**ACU 0.40**
D	Mario is on the top-left platform.		
E	The POW block is present.		
F	Mario attacks the enemies at the top right.		
G	Mario moves through the level.		
H	Mario hits enemies from below on the left side.		

Table 7.6: CU, AS, and ACU of the two clusterings analyzed in the Mario Bros environment.

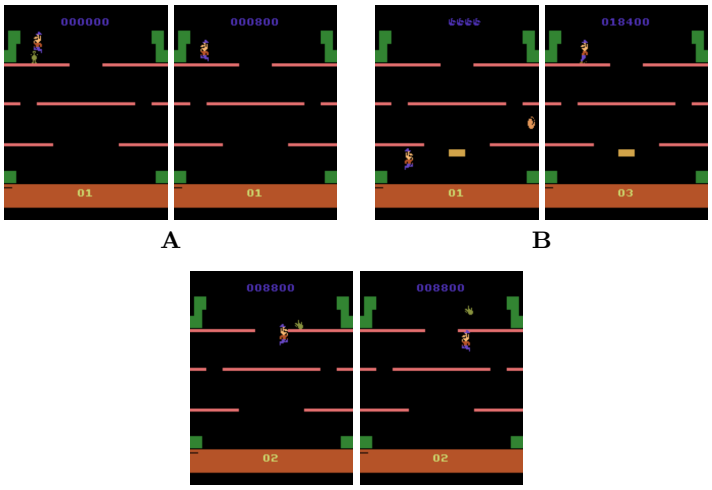


Figure 7.9: Clusters A–C identified by CAR in the Mario Bros environment (layer 3, $K = 3$).

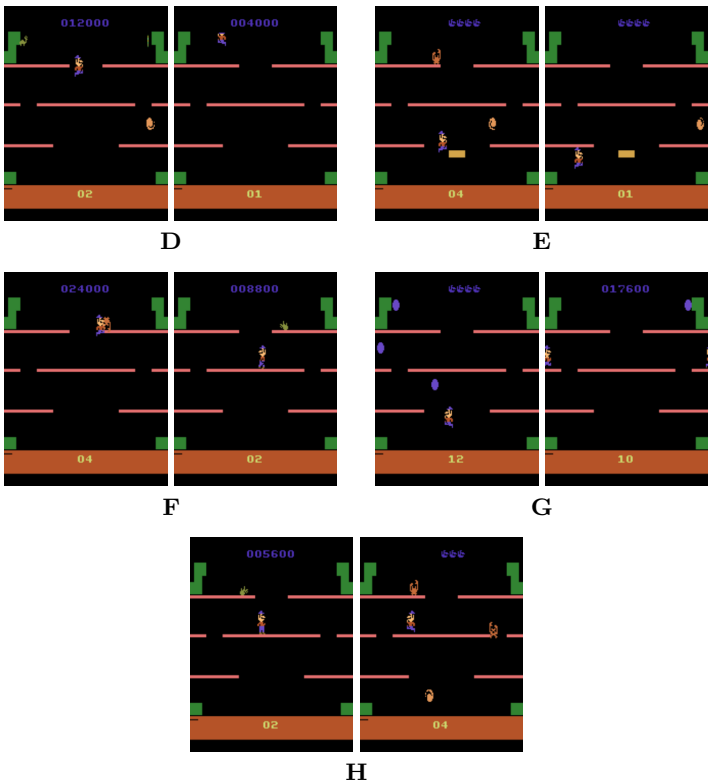


Figure 7.10: Clusters D–H identified by CAR in the Mario Bros environment (layer 3, $K = 5$).

7.5.3 Ms. Pac-Man

Two clusterings are examined for Ms. Pac-Man (Table 7.7), obtained at layer 6b with $K = 3$ and at layer 4 with $K = 5$.

With $K = 3$ (Figure 7.11), the partition captures the progress and state of the game rather than a specific spatial feature of the observation. The start of the episode is captured in cluster B. The middle of the episode is captured in cluster A. The end of the episode is captured in cluster C.

With $K = 5$ (Figure 7.12), the partition provides a more detailed description of the episode. The initial animation, during which the character cannot move, is isolated in cluster H. The beginning of actual play is captured in cluster F. A situation specific to the game mechanics, in which Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts, is isolated in cluster D. The end of the episode, when at most one power pellet remains, is separated into two similar clusters, E and G.

The two clusterings have the same CU of 0.88. The clustering at layer 4 has a lower AS of 0.47 compared with 0.52 at layer 6b. This results in a lower ACU of 0.35 compared with 0.40.

Layer 6b	$K = 3$ **CU 0.88**	**AS 0.52**	**ACU 0.40**
A	Middle of the episode.		
B	Start of the episode.		
C	End of the episode.		
Layer 4	$K = 5$ **CU 0.88**	**AS 0.47**	**ACU 0.35**
D	Pac-Man is in booster mode and waits in front of the ghost spawn to eat the ghosts.		
E	End of the episode, at most one power pellet remains (similar to cluster G).		
F	Start of the episode.		
G	End of the episode, at most one power pellet remains (similar to cluster E).		
H	Start-of-episode animation, the character cannot move.		

Table 7.7: CU, AS, and ACU of the two clusterings analyzed in the Ms. Pac-Man environment.

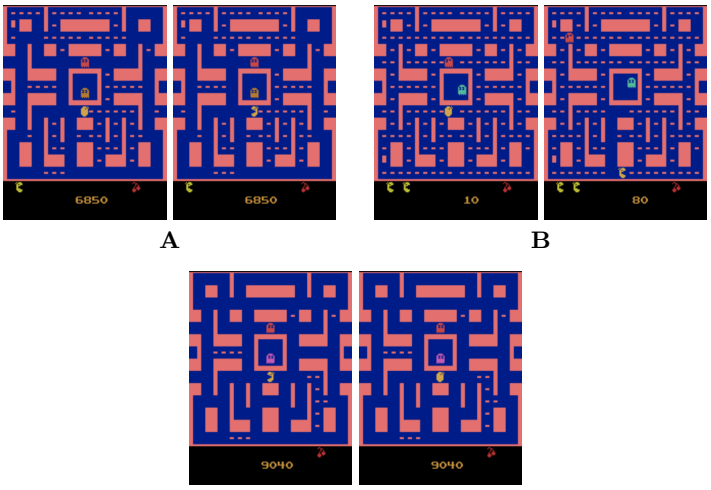


Figure 7.11: Clusters A–C identified by CAR in the Ms. Pac-Man environment (layer 6b, $K = 3$).

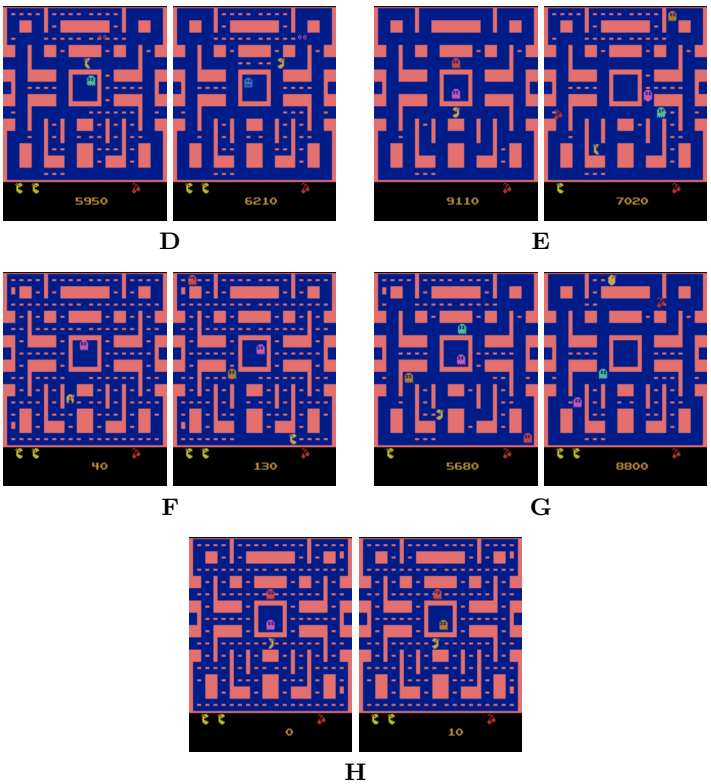


Figure 7.12: Clusters D–H identified by CAR in the Ms. Pac-Man environment (layer 4, $K = 5$).

7.5.4 Pong

Two clusterings are examined for Pong (Table 7.8), obtained at layer 2 with $K = 2$ and at layer 3 with $K = 3$.

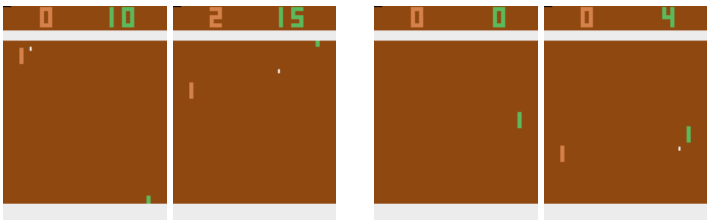
At layer 2 (Figure 7.13), the partition separates the observations according to the number of digits in the player's score. A one-digit score corresponds to cluster A. A two-digit score corresponds to cluster B. This reflects the progress of the game, but only through its visual display at the top of the screen. The partition therefore captures a feature of the observation that provides no information about how the agent plays.

At layer 3 (Figure 7.14), the partition instead follows the vertical position of the ball. The ball is at the top of the field in cluster C. It is in the middle of the field in cluster D. It is at the bottom of the field in cluster E. This feature directly determines where the agent needs to place its paddle.

The clustering at layer 2 is perfectly reproducible and perfectly abstract, with CU, AS, and ACU all equal to 1.00. The clustering at layer 3 is less consistent, with a CU of 0.67, and reaches a lower ACU of 0.65.

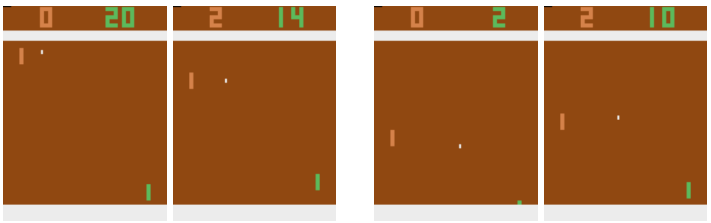
Layer 2	$K = 2$	**CU 1.00**	**AS 1.00**	**ACU 1.00**
A	The player's score has two digits.			
B	The player's score has one digit.			
Layer 3	$K = 3$	**CU 0.67**	**AS 0.98**	**ACU 0.65**
C	The ball is at the top.			
D	The ball is in the middle.			
E	The ball is at the bottom.			

Table 7.8: CU, AS, and ACU of the two clusterings analyzed in the Pong environment.

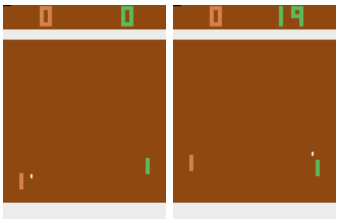


A **B**

Figure 7.13: Clusters A–B identified by CAR in the PONG environment (layer 2, $K = 2$).



C **D**



E

Figure 7.14: Clusters C–E identified by CAR in the PONG environment (layer 3, $K = 3$).

7.5.5 Space Invaders

Two clusterings are examined for Space Invaders (Table 7.9), obtained at layer 4 with $K = 4$ and at layer 2 with $K = 5$.

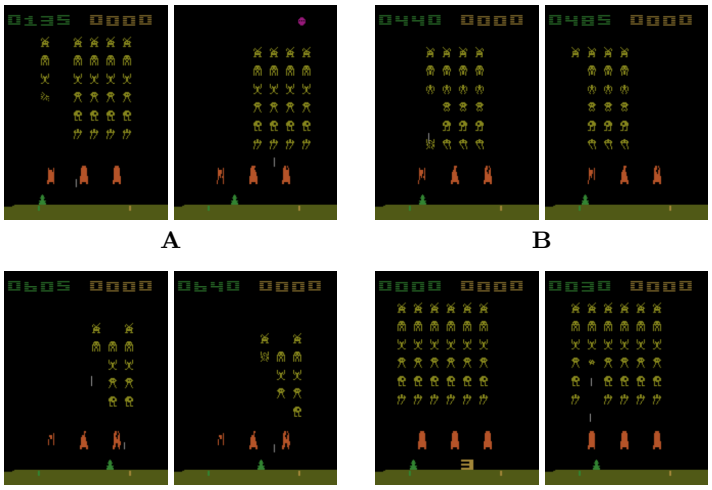
At layer 4 (Figure 7.15), the partition tracks the progression of the episode. The start of the episode is captured in cluster D. The middle of the episode is captured in cluster B. The end of the episode is captured in cluster C. A specific behavior in which the agent has cleared the leftmost column and is positioned to catch the flying saucer that occasionally crosses the top of the screen is captured in cluster A.

At layer 2 (Figure 7.16), the partition again tracks the progression of the episode. The start of the episode is captured in cluster G. The middle of the episode is captured in cluster F. The end of the episode is captured in cluster I. Attacking the first two columns of the formation is captured in cluster E. Trying to catch the flying saucer is captured in cluster H.

The clustering at layer 4 has a CU of 0.92 and a AS of 0.69, resulting in a ACU of 0.61. The clustering at layer 2 has the same CU of 0.92, but a lower AS of 0.49, resulting in a lower ACU of 0.41.

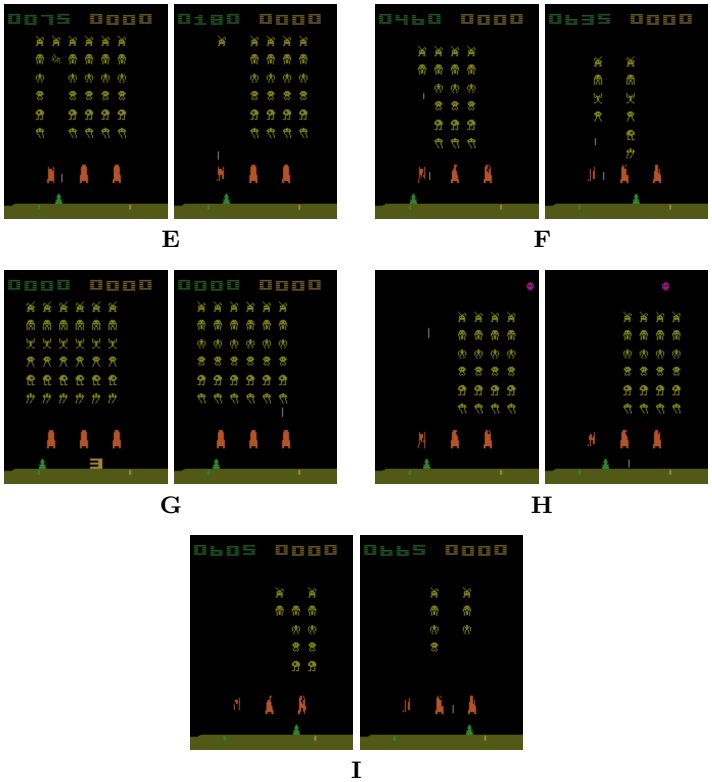
Layer 4	$K = 4$ **CU 0.92**	**AS 0.69**	**ACU 0.61**
A	The left column has been cleared; the flying saucer occasionally appears.		
B	Middle of the episode.		
C	End of the episode.		
D	Start of the episode.		
Layer 2	$K = 5$ **CU 0.92**	**AS 0.49**	**ACU 0.41**
E	Attacks the first two columns of the formation.		
F	Middle of the episode.		
G	Start of the episode.		
H	Tries to catch the flying saucer.		
I	End of the episode.		

Table 7.9: CU, AS, and ACU of the two clusterings analyzed in the Space Invaders environment.



A B C D

Figure 7.15: Clusters A–D identified by CAR in the Space Invaders environment (layer 4, $K = 4$).



E F G H I

Figure 7.16: Clusters E–I identified by CAR in the Space Invaders environment (layer 2, $K = 5$).

7.5.6 Surround

Two clusterings are examined for Surround (Table 7.10), obtained at layer 8b with $K = 4$ and at layer 7b with $K = 6$.

At layer 8b (Figure 7.17), the partition captures the start of the episode in cluster B. The outcome of the episode, when the opponent loses, is captured in cluster D. The player's trail is confined to the bottom-left area in cluster A. The player reaches the top of the level from the left and then moves along the top in cluster C.

At layer 7b (Figure 7.18), the partition again captures the start of the episode in cluster J. The outcome of the episode, when the opponent loses, is captured in cluster F. The player moves in the upper-left corner in cluster E. The player moves in the lower-left corner in cluster G. The end of the game is played in the lower-left corner in cluster H. The player crosses the level from the top left to the bottom right in cluster I.

The clustering at layer 8b has a CU of 0.67, a AS of 0.71, and a ACU of 0.38. The clustering at layer 7b reaches the same values, with a CU of 0.67, a AS of 0.71, and a ACU of 0.38.

Layer 8b	$K = 4$ **CU 0.67**	**AS 0.71**	**ACU 0.38**
A	The agent's trail remains confined to the bottom-left area.		
B	Start of the episode.		
C	The agent moves along the top of the level, arriving from the left.		
D	The opponent loses by running into a trail.		
Layer 7b	$K = 6$ **CU 0.67**	**AS 0.71**	**ACU 0.38**
E	The player moves in the upper-left corner.		
F	The opponent loses the confrontation.		
G	The player moves in the lower-left corner.		
H	The end of the game is played in the lower-left corner.		
I	The player starts at the top left and ends at the bottom right.		
J	Start of the episode.		

Table 7.10: CU, AS, and ACU of the two clusterings analyzed in the Surround environment.

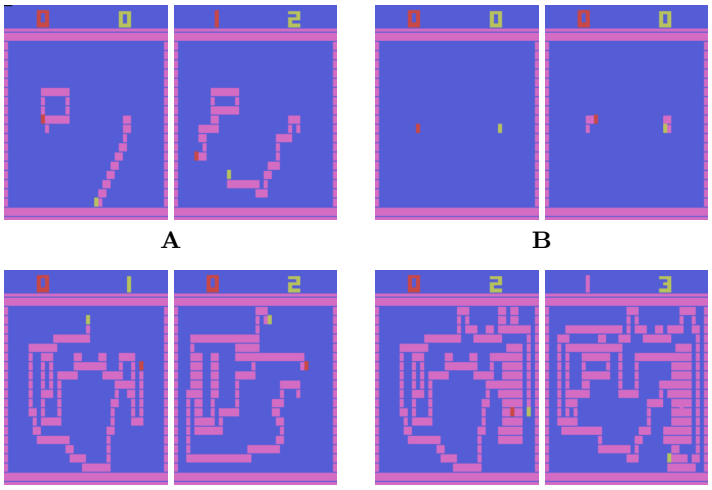


Figure 7.17: Clusters A–D identified by CAR in the SURROUND environment (layer 8b, $K = 4$).

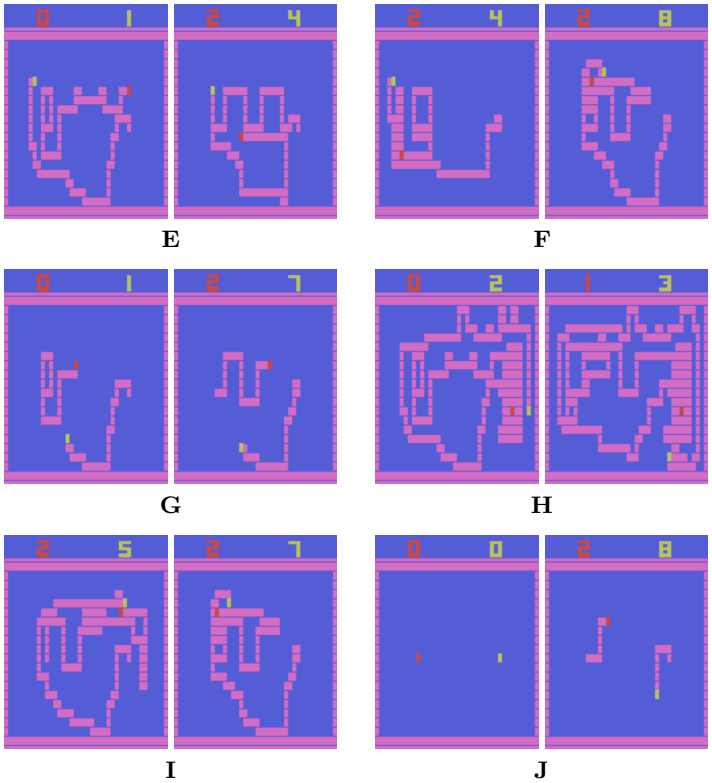


Figure 7.18: Clusters E–J identified by CAR in the SURROUND environment (layer 7b, $K = 6$).

7.6 Conclusion

The quantitative results show that CAR consistently recovers similar clustering structures across independently trained surrogate policies. Across the six environments, the CU frequently exceeds 0.5, indicating that independently trained policies can develop similarly structured LRs [91]. The CU also tends to decrease as the network gets deeper, with the highest values generally observed in the earliest layers. Considering consistency alone would therefore favor the earliest layers of the network.

To complement this analysis, the level of abstraction of these representations must also be considered. For this purpose, AS is analyzed and shows that the earliest layers tend to exhibit lower levels of abstraction. Their representations remain strongly related to the observations and the initial structure of the network. In the deepest layers, the abstraction can decrease again in some environments, as the latent clustering becomes closer to the clustering of the action distribution. Both trends are consistent with the literature, in which each layer progressively reshapes the LR into a more abstract form that becomes increasingly aligned with the output space [23].

This highlights the importance of considering both consistency and abstraction when identifying meaningful LRs. The ACU addresses this limitation by combining these two properties into a single measure, providing a practical criterion for identifying layers that achieve a suitable balance between representational consistency and abstraction.

These quantitative analyses are complemented by the qualitative analysis, which shows that the clusters identified by CAR correspond to interpretable properties of the environment and the agent's behavior. Across the different environments, the clusters capture recurring aspects such as episode progression, specific game situations, and behavioral patterns. These observations provide qualitative evidence that the structures recovered by CAR can reflect semantically aspects of the underlying task.

The qualitative analysis also reveals a trade-off associated with the number of clusters. Increasing the number of clusters allows more specific situations to be distinguished, but tends to reduce the stability of the resulting partitions across surrogate policies. Finer clusters have less clear semantic boundaries, reducing agreement across independently trained policies. The choice of the number of clusters therefore involves a trade-off between recovering stable, shared concepts and capturing finer distinctions between situations.

Overall, these results provide empirical support for the assumptions underlying CAR, and in particular for the *Representation Convergence Hypothesis* (Hypothesis 4). Independently trained surrogate policies exhibit a degree of convergence in the organization of their LRs that cannot be explained by the considered baselines. The qualitative analysis also shows that these clusters can correspond to meaningful properties of the environment and the agent's behavior. Taken together, these findings suggest that the concepts recovered by CAR reflect properties of the task rather than artifacts of a particular network.

Chapter 8

Conclusion

"Look on my Works, ye Mighty, and despair!"
Percy Bysshe Shelley, *Ozymandias* (1818) [2]

Contents

8.1 Summary of Contributions 129
8.1.1 Survey of Latent Space Analysis Methods 130
8.1.2 Taxonomy of Explainable Reinforcement Learning Methods 130
8.1.3 The CAR Method and Its Evaluation 130
8.1.4 A Coherent Chain of Hypotheses 131
8.2 Limitations 131
8.3 Perspectives 132
8.3.1 Automatic Textual Description of Clusters 132
8.3.2 From the Latent Space to the Strategy 132
8.3.3 Hypotheses on the Structure of the Latent Space 133
8.3.4 Human Evaluation of Explanations 133
8.4 Final Remarks 133

This final chapter reviews the work presented in this thesis. It begins by revisiting each research question in light of our contributions. It then discusses the limitations of these contributions. Next, it presents the directions for future research that these limitations open. Finally, it closes with a broader reflection on this work.

8.1 Summary of Contributions

This section revisits the four research questions introduced in Section 1.3. It is organized around the three contributions of this thesis, each of which is presented in a dedicated subsection. A final subsection then brings together the hypotheses formulated throughout this manuscript and shows how they form a coherent chain.

8.1.1 Survey of Latent Space Analysis Methods

Research Question 1 concerned the approaches proposed in the literature to analyze the internal workings of Neural Networks (NNs). To answer it, we first made explicit the working hypothesis shared by most of this literature, the *Concept Hypothesis* (Hypothesis 1). According to this hypothesis, NNs generalize by organizing their Latent Spaces (LSs) around concepts. We then organized existing methods into four families, namely probing, interpretable directions, Sparse Autoencoders (SAEs), and clustering.

This review shows that the first three families assume that concepts can be recovered from Latent Representations (LRs) through a linear decomposition. Clustering is the only family that does not rely on this assumption, which makes it the basis of the method proposed in this thesis. The review also reveals that there is no consensus on how to evaluate whether these methods truly capture the concepts of a model.

8.1.2 Taxonomy of Explainable Reinforcement Learning Methods

Research Question 2 concerned the organization of existing Explainable Reinforcement Learning (XRL) methods [129, 145]. Our starting point was to clarify what an explanation means in the context of Reinforcement Learning (RL). Building on this framework, we characterized each explanation by its subject, its source, and its form. The subject distinguishes the explanation of a single decision from that of the agent's strategy, which is specific to the temporal nature of RL. From these elements, we proposed a taxonomy that accounts for the specificities of RL and used it to review the state of the art.

This review led to three main observations. None of the reviewed methods explains the learning process itself. The field lacks unified metrics for evaluating explanations. Finally, current research remains dominated by substitutive approaches, which replace the deep policy with a simpler model. These observations point to the need for an intrinsic and objectively evaluated method.

8.1.3 The CAR Method and Its Evaluation

Research Questions 3 and 4 concerned the use of LRs to explain RL agents and the objective evaluation of such explanations. To address them, we introduced Clustering Agent Representation (CAR), an XRL method based on the principles of Mechanistic Explainability (ME) [146]. CAR independently trains several surrogate policies to reproduce the same behavior, then clusters their LRs and compares the resulting partitions. This procedure relies on the *Representation Convergence Hypothesis* (Hypothesis 4), according to which networks trained on the same task develop similar concepts. To evaluate it, we introduced three metrics. The *Clustering Universality* (CU) measures the consistency of the clusters across surrogate policies, the *Abstraction Score* (AS) measures how much they differ from trivial baselines, and the *Adjusted Clustering Universality* (ACU) combines both properties.

Applied to six Atari environments, CAR recovers similar clustering structures across independently trained surrogate policies, with a CU that frequently exceeds 0.5. This convergence reflects concepts that emerge during learning,

since the AS verifies that the clusters are abstract and do not merely reproduce the structure of the observations, the actions, or an untrained network. This interpretation is confirmed by the qualitative analysis, which shows that the clusters capture interpretable aspects of the task, such as the progression of the episode or specific game situations. Overall, these results provide empirical support for the *Representation Convergence Hypothesis* (Hypothesis 4).

8.1.4 A Coherent Chain of Hypotheses

Throughout these contributions, we developed three hypotheses that form a coherent chain, each one building on the previous one. The *Mechanistic Hypothesis* (Hypothesis 3) states that NNs can be understood through their internal mechanisms. It justifies analyzing the network itself rather than replacing it with a simpler model. The *Concept Hypothesis* (Hypothesis 1) states that these mechanisms are organized around concepts. It specifies what this analysis should look for in the LS. The *Representation Convergence Hypothesis* (Hypothesis 4) states that these concepts reflect the task. It provides a criterion to determine whether the identified concepts are reliable, since concepts shared by independently trained networks.

These three hypotheses do not have the same status. The first two are working hypotheses widely adopted in the literature. The third one is directly put to the test by CAR, and is therefore the object of its investigation. The experimental results presented in this thesis provide empirical support for it.

8.2 Limitations

This section discusses the four main limitations identified at this stage.

First, CAR describes how the LS of an agent is organized, but not what each cluster means. The CU, AS, and ACU metrics quantify the consistency and abstraction of the clusters, but they say nothing about their content. In this thesis, the meaning of each cluster was assigned through manual inspection of the corresponding observations. This step is time-consuming, subjective, and difficult to scale to a large number of clusters or environments.

Second, the clusters identified by CAR are not yet directly related to the agent's decisions. We show that these representations are stable across independent training runs and carry meaningful semantic information. However, CAR does not show how they influence the agent's decisions. The interpretation of the identified representations therefore remains partly subjective.

Third, the experiments were conducted on six Atari environments. Whether CAR can scale to more complex environments and larger networks therefore remains an open question. In addition, only one clustering method was evaluated, based on k-means with Euclidean distance. Other distances and other clustering methods could reveal different structures in the LS. Since the pipeline of CAR is modular, these alternatives can be tested and compared using the proposed metrics.

Finally, these metrics have only been applied to deep clustering. The other families of methods reviewed in Chapter 3 have not yet been evaluated within this framework. This is particularly the case for methods based on the *Superposition Hypothesis* (Hypothesis 2). These methods are widely used, but

their assumptions remain debated. Our metrics could be used to test these assumptions quantitatively, and to compare these methods with CAR on the same tasks.

8.3 Perspectives

The limitations discussed above suggest several directions for future research. This section presents four of them, ordered from short-term to long-term. For each of them, we first identify the main scientific challenge it raises, then describe the approach we propose and how it could be validated.

8.3.1 Automatic Textual Description of Clusters

The first perspective addresses the manual interpretation of the clusters, since the meaning of each cluster is currently assigned by hand. The main challenge is to obtain a reliable textual description of each cluster automatically.

One possible approach is to first extract, for each cluster, the video segments formed by consecutive observations belonging to it. Each segment is then described in natural language by a vision-language model. The descriptions of all segments of a cluster are synthesized into a single description of the cluster.

The reliability of these descriptions can be assessed through a classification test. Given a new video segment, not used to produce the descriptions, a vision-language model must identify the cluster to which it belongs, using only the textual descriptions of the clusters. The predicted cluster is then compared with the one assigned by CAR. If the classification accuracy is well above chance level, the textual description faithfully captures what distinguishes the cluster. This approach would thus fully automate the CAR method, from the identification of the clusters to their interpretation.

8.3.2 From the Latent Space to the Strategy

The second perspective addresses the lack of a direct link between the clusters and the agent's decisions. Its goal is to derive an explanation of the agent's strategy from the similarity of its LRs. The difficulty is that the clusters group states that the agent perceives as similar, but provide no information about the dynamics of its interaction with the environment. It is therefore necessary to move from a static view of the LS to a dynamic view of the agent's behavior.

Each episode can first be converted into a sequence of clusters, by assigning each observation to its cluster and merging consecutive repetitions. Each cluster then becomes an abstract state. The transitions observed between clusters define a graph that represents the dynamics of the interaction between the agent and its environment. Its nodes are the concepts identified by CAR, and its edges describe how the agent moves between them. Such a graph thus summarizes the agent's strategy.

The graph can be validated by checking that it correctly predicts the transitions observed in episodes not used to build it.

8.3.3 Hypotheses on the Structure of the Latent Space

The third perspective addresses the comparison of CAR with the other families of methods, and in particular with those based on superposition. CAR assumes that concepts are organized as clusters, whereas the *Superposition Hypothesis* (Hypothesis 2) assumes that they are encoded as directions. The objective is to determine which of these hypotheses best describes the LS. The main challenge is that the two families of methods do not produce the same kind of object, a partition in one case and a set of directions in the other.

The metrics proposed in this thesis, namely CU, AS, and ACU, can serve as a common framework. They can be applied to the concepts extracted by methods based on superposition, provided that these concepts are expressed as a partition of the observations. For instance, each observation can be assigned to the direction along which it is most strongly activated. To ensure a fair comparison, both families of methods should produce the same number of concepts. Both hypotheses can then be evaluated with these metrics on the same agents and environments. If one hypothesis yields more universal and abstract concepts, this would provide evidence that it better describes the structure of the LS.

8.3.4 Human Evaluation of Explanations

The last perspective concerns the observers to whom the explanations are addressed. Its objective is to verify that the explanations produced by CAR actually improve the observer's mental model of the agent. Measuring understanding is difficult, since it has no consensual operational definition. Such an evaluation requires an experimental protocol involving human observers. This protocol should be defined in collaboration with researchers in cognitive science and psychology.

Building on the first two perspectives, the protocol can assess understanding at two levels of increasing difficulty. At the level of concepts, observers receive the textual descriptions of the clusters and must assign new video segments to the corresponding cluster. Their accuracy would show whether the descriptions, in addition to being meaningful for a vision-language model, are also meaningful for humans. At the level of strategy, observers first study the graph summarizing the agent's strategy. They are then shown the beginning of a new episode and must predict which clusters the agent will go through next, and in which order. Since this prediction requires anticipating the agent's behavior in a situation that has not been observed before, its accuracy would show whether the graph has actually conveyed the strategy to the observer.

8.4 Final Remarks

To conclude, let us step back and consider this work as a whole. This thesis has proposed explainability methods for RL agents that rely on the LRs learned by NNs. In line with intrinsic explainability, understanding is not achieved despite the use of NNs, but through them.

Because these representations are what the agent actually uses to act, explanations derived from them remain grounded in the agent's own decision process. These representations can be seen as a language that, once deciphered, reveals how an agent perceives its environment and makes decisions. LRs offer a way to

give access to what the agent has learned rather than only to what it does. As RL agents are increasingly deployed in open-ended settings, such understanding becomes a prerequisite for ensuring their alignment (Definition 1.2.1).

This effort may also extend beyond artificial systems. Since NNs are partly inspired by biological intelligence, understanding how they represent the world may shed light on how we do so ourselves. Unlike the brain, an artificial network can be observed in its entirety, probed at will, and retrained under controlled conditions, which makes it a unique experimental object for studying representation and decision-making.

The study of LSs remains a largely open field, one whose exploration has only just begun.

Appendix 1

Additional Results

This appendix reports the complete results underlying the analysis of Section 7.4. For each of the six Atari environments studied in Chapter 7, it gives the evolution of CU, AS, and ACU for every analyzed layer and for a number of clusters $K \in [2, 20]$. It also gives the three baseline similarities entering the AS. These are the similarity to the clustering of the observations (ρ^{obs}), to the latent clustering of an untrained network (ρ^{untr}), and to the clustering of the action distribution, computed on the policy logits before the softmax (ρ^{act}). All these quantities are computed with the Adjusted Rand Index (ARI), consistent with the rest of this thesis. Images of the clusters obtained for every analyzed layer, which extend the qualitative analysis of Section 7.5, have been archived on Zenodo `https://zenodo.org/records/23021063`.

For each environment, one table is reported per metric and per baseline. The rows correspond to the analyzed layers and the columns to the values of K. The background shade of a cell is proportional to its value, so that the closer the value is to 1, the darker the cell. The tables of CU are shown in green, those of AS and of its baselines in blue, and those of ACU in red.

A value in **bold** is the highest of its column, which identifies the best layer for this number of clusters. An underlined value is the highest of its row, which identifies the best number of clusters for this layer. A value both **bold and underlined** is the best of its row and of its column. The last column reports the mean ± standard deviation across K for each layer, and the last row the mean ± standard deviation across layers for each K. In both, the best mean is shown in bold. The bottom-right cell reports the mean ± standard deviation over the whole table.

The baseline tables must be read the other way around. A high value, shown as a dark cell, indicates that the latent clustering is strongly aligned with the baseline. It therefore signals an absence of abstraction and lowers the AS.

1.1 Breakout

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.97	**0.97**	**0.97**	0.62	**0.84**	**0.81**	**0.86**	**0.84**	**0.81**	**0.85**	**0.85**	**0.87**	**0.88**	**0.84**	**0.86**	**0.85**	**0.85**	**0.78**	**0.87**	***0.85*** *± 0.07*
2	0.40	0.91	0.84	0.62	0.79	0.80	0.76	0.76	0.73	0.75	0.67	0.73	0.74	0.76	0.72	0.79	0.66	0.73	0.73	*0.73 ± 0.10*
3	**1.00**	0.85	0.86	0.61	0.46	0.55	0.73	0.52	0.50	0.63	0.49	0.57	0.44	0.38	0.44	0.40	0.40	0.50	0.59	*0.57 ± 0.17*
4	0.99	0.85	0.26	0.42	0.38	0.52	0.38	0.49	0.50	0.51	0.53	0.47	0.39	0.47	0.46	0.54	0.47	0.54	0.41	*0.50 ± 0.16*
5	0.99	0.85	0.26	0.42	0.38	0.52	0.38	0.49	0.50	0.51	0.53	0.47	0.39	0.47	0.46	0.54	0.47	0.54	0.41	*0.50 ± 0.16*
6b	0.66	0.57	0.65	**0.79**	0.59	0.62	0.56	0.56	0.58	0.64	0.59	0.55	0.55	0.54	0.59	0.60	0.53	0.55	0.55	*0.59 ± 0.06*
7b	0.78	0.41	0.46	0.57	0.52	0.56	0.55	0.41	0.47	0.47	0.47	0.43	0.45	0.47	0.50	0.52	0.48	0.50	0.46	*0.50 ± 0.08*
8b	0.68	0.50	0.50	0.50	0.50	0.52	0.50	0.44	0.55	0.54	0.55	0.48	0.48	0.49	0.51	0.50	0.51	0.50	0.45	*0.51 ± 0.05*
Mean ± Std	***0.81*** *± 0.20*	*0.74 ± 0.20*	*0.60 ± 0.26*	*0.57 ± 0.11*	*0.56 ± 0.16*	*0.61 ± 0.12*	*0.59 ± 0.17*	*0.56 ± 0.14*	*0.58 ± 0.12*	*0.61 ± 0.12*	*0.58 ± 0.12*	*0.57 ± 0.14*	*0.54 ± 0.17*	*0.55 ± 0.15*	*0.57 ± 0.14*	*0.59 ± 0.14*	*0.55 ± 0.13*	*0.58 ± 0.10*	*0.56 ± 0.16*	*0.60 ± 0.17*

Table 1.1: CU — Breakout.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.05	0.06	0.03	0.16	0.16	0.16	0.16	0.18	0.15	0.15	0.18	0.14	0.13	0.20	0.16	0.16	0.20	0.23	0.15	*0.15 ± 0.05*
2	0.60	0.09	0.20	0.29	0.27	0.28	0.24	0.23	0.25	0.28	0.31	0.33	0.28	0.28	0.29	0.29	0.31	0.27	0.27	*0.28 ± 0.09*
3	0.85	0.14	0.20	0.36	0.43	0.41	0.29	0.41	0.41	0.44	0.47	0.43	0.50	0.50	0.50	0.47	0.47	0.46	0.57	*0.44 ± 0.14*
4	**0.88**	0.28	0.64	0.72	0.75	0.80	0.77	0.81	0.80	0.81	0.81	0.82	0.80	0.81	0.80	0.79	0.79	0.82	0.80	*0.76 ± 0.12*
5	**0.88**	0.28	0.64	0.72	0.75	0.80	0.77	0.81	0.80	0.81	0.81	0.82	0.80	0.81	0.80	0.79	0.79	0.82	0.80	*0.76 ± 0.12*
6b	0.62	**0.83**	**0.74**	**0.82**	**0.85**	**0.85**	**0.85**	**0.85**	**0.83**	**0.82**	**0.85**	**0.85**	**0.84**	**0.83**	**0.83**	**0.83**	**0.84**	**0.85**	**0.83**	***0.82*** *± 0.05*
7b	0.64	0.75	0.69	0.67	0.74	0.79	0.79	0.80	0.80	**0.82**	0.82	0.83	0.83	0.82	**0.83**	**0.83**	0.83	**0.85**	**0.83**	*0.79 ± 0.06*
8b	0.71	0.67	0.66	0.64	0.78	0.77	0.75	0.80	0.76	0.78	0.77	0.79	0.77	0.78	0.78	0.78	0.78	0.80	0.78	*0.76 ± 0.05*
Mean ± Std	***0.65*** *± 0.25*	*0.39 ± 0.29*	*0.47 ± 0.26*	*0.55 ± 0.23*	*0.59 ± 0.25*	*0.61 ± 0.26*	*0.58 ± 0.27*	*0.61 ± 0.27*	*0.60 ± 0.26*	*0.61 ± 0.26*	*0.63 ± 0.25*	*0.63 ± 0.26*	*0.62 ± 0.26*	*0.63 ± 0.25*	*0.62 ± 0.25*	*0.62 ± 0.25*	*0.63 ± 0.24*	*0.64 ± 0.25*	*0.63 ± 0.26*	*0.59 ± 0.27*

Table 1.2: AS — Breakout.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.01	0.03	0.00	-0.22	-0.01	-0.03	0.03	0.01	-0.04	0.00	0.02	0.01	0.02	0.04	0.02	0.01	0.05	0.01	0.02	*0.00 ± 0.06*
2	0.01	0.00	0.04	-0.09	0.05	0.08	0.00	-0.01	-0.02	0.03	-0.02	0.06	0.02	0.03	0.01	0.08	-0.03	0.00	0.00	*0.01 ± 0.04*
3	0.85	-0.02	0.06	-0.03	-0.10	-0.05	0.02	-0.07	-0.09	0.07	-0.04	0.00	-0.06	-0.12	-0.05	-0.13	-0.12	-0.05	0.16	*0.01 ± 0.21*
4	**0.87**	0.13	-0.10	0.14	0.13	0.31	0.15	0.30	0.30	0.31	0.34	0.28	0.19	0.28	0.26	0.33	0.26	0.36	0.21	*0.27 ± 0.18*
5	**0.87**	0.13	-0.10	0.14	0.13	0.31	0.15	0.30	0.30	0.31	0.34	0.28	0.19	0.28	0.26	0.33	0.26	0.36	0.21	*0.27 ± 0.18*
6b	0.28	**0.40**	**0.39**	**0.61**	**0.43**	**0.47**	**0.41**	**0.41**	**0.42**	**0.46**	**0.45**	**0.40**	**0.39**	**0.37**	**0.42**	**0.43**	**0.36**	**0.40**	**0.38**	***0.41*** *± 0.06*
7b	0.42	0.16	0.16	0.24	0.26	0.35	0.35	0.21	0.27	0.29	0.29	0.26	0.28	0.30	0.33	0.35	0.31	0.34	0.29	*0.29 ± 0.06*
8b	0.40	0.17	0.16	0.14	0.28	0.28	0.25	0.24	0.31	0.32	0.32	0.27	0.26	0.27	0.29	0.28	0.29	0.30	0.23	*0.27 ± 0.06*
Mean ± Std	***0.46*** *± 0.34*	*0.12 ± 0.12*	*0.08 ± 0.15*	*0.12 ± 0.23*	*0.15 ± 0.16*	*0.21 ± 0.18*	*0.17 ± 0.14*	*0.17 ± 0.16*	*0.18 ± 0.18*	*0.22 ± 0.16*	*0.21 ± 0.18*	*0.20 ± 0.14*	*0.16 ± 0.14*	*0.18 ± 0.16*	*0.19 ± 0.16*	*0.21 ± 0.19*	*0.17 ± 0.17*	*0.21 ± 0.18*	*0.19 ± 0.12*	*0.19 ± 0.19*

Table 1.3: ACU — Breakout.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.66**	0.82	**0.73**	0.55	**0.74**	**0.80**	**0.80**	**0.79**	**0.79**	**0.73**	**0.74**	**0.77**	**0.74**	**0.78**	**0.73**	**0.76**	**0.77**	0.61	**0.74**	***0.74*** *± 0.07*
2	0.32	**0.91**	0.63	**0.61**	0.73	0.72	0.76	0.76	0.75	0.65	0.69	0.67	0.71	0.70	0.64	0.71	0.63	**0.63**	0.67	*0.68 ± 0.11*
3	0.09	0.86	0.67	0.59	0.47	0.58	0.71	0.58	0.59	0.56	0.53	0.57	0.50	0.50	0.50	0.48	0.49	0.51	0.42	*0.54 ± 0.14*
4	0.09	0.72	0.36	0.28	0.21	0.18	0.21	0.18	0.19	0.19	0.18	0.18	0.20	0.19	0.20	0.21	0.21	0.18	0.20	*0.23 ± 0.13*
5	0.09	0.72	0.36	0.28	0.21	0.18	0.21	0.18	0.19	0.19	0.18	0.18	0.20	0.19	0.20	0.21	0.21	0.18	0.20	*0.23 ± 0.13*
6b	0.38	0.16	0.26	0.18	0.15	0.14	0.15	0.15	0.17	0.18	0.15	0.15	0.16	0.17	0.17	0.17	0.16	0.15	0.17	*0.18 ± 0.05*
7b	0.36	0.25	0.31	0.33	0.23	0.21	0.21	0.20	0.20	0.18	0.18	0.17	0.17	0.18	0.17	0.17	0.17	0.15	0.17	*0.21 ± 0.06*
8b	0.29	0.33	0.34	0.36	0.22	0.23	0.23	0.20	0.24	0.22	0.23	0.21	0.21	0.22	0.22	0.22	0.22	0.18	0.22	*0.24 ± 0.05*
Mean ± Std	*0.28 ± 0.18*	***0.60*** *± 0.28*	*0.46 ± 0.17*	*0.40 ± 0.15*	*0.37 ± 0.23*	*0.38 ± 0.26*	*0.41 ± 0.27*	*0.38 ± 0.26*	*0.39 ± 0.25*	*0.36 ± 0.22*	*0.36 ± 0.23*	*0.36 ± 0.24*	*0.36 ± 0.23*	*0.37 ± 0.24*	*0.35 ± 0.22*	*0.37 ± 0.23*	*0.36 ± 0.22*	*0.32 ± 0.20*	*0.35 ± 0.22*	*0.38 ± 0.24*

Table 1.4: Observation baseline (ρ^{obs}) — BREAKOUT.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.95**	**0.94**	**0.97**	**0.76**	**0.84**	**0.83**	**0.84**	**0.82**	**0.85**	**0.85**	**0.82**	**0.86**	**0.87**	**0.77**	**0.84**	**0.84**	**0.80**	**0.77**	**0.85**	***0.85*** *± 0.06*
2	0.38	0.63	0.80	0.63	0.66	0.67	0.68	0.76	0.67	0.72	0.65	0.66	0.69	0.72	0.71	0.69	0.69	0.73	0.73	*0.68 ± 0.08*
3	0.15	0.68	0.80	0.64	0.54	0.47	0.53	0.48	0.45	0.45	0.46	0.50	0.46	0.44	0.46	0.51	0.48	0.54	0.42	*0.50 ± 0.12*
4	0.12	0.48	0.32	0.27	0.25	0.20	0.23	0.19	0.20	0.17	0.18	0.18	0.18	0.18	0.18	0.17	0.17	0.18	0.18	*0.21 ± 0.08*
5	0.12	0.48	0.32	0.27	0.25	0.20	0.23	0.19	0.20	0.17	0.18	0.18	0.18	0.18	0.18	0.17	0.17	0.18	0.18	*0.21 ± 0.08*
6b	0.32	0.16	0.22	0.16	0.15	0.15	0.13	0.14	0.14	0.16	0.13	0.13	0.14	0.14	0.14	0.14	0.13	0.13	0.14	*0.16 ± 0.04*
7b	0.31	0.22	0.26	0.23	0.26	0.16	0.16	0.17	0.17	0.14	0.12	0.14	0.13	0.11	0.12	0.11	0.13	0.11	0.12	*0.17 ± 0.06*
8b	0.18	0.27	0.17	0.19	0.20	0.16	0.22	0.16	0.17	0.15	0.13	0.11	0.13	0.14	0.14	0.13	0.14	0.13	0.10	*0.16 ± 0.04*
Mean ± Std	*0.32 ± 0.26*	*0.48 ± 0.25*	***0.48*** *± 0.30*	*0.39 ± 0.22*	*0.39 ± 0.24*	*0.35 ± 0.25*	*0.38 ± 0.25*	*0.36 ± 0.27*	*0.36 ± 0.25*	*0.35 ± 0.27*	*0.33 ± 0.26*	*0.35 ± 0.27*	*0.35 ± 0.27*	*0.34 ± 0.26*	*0.35 ± 0.27*	*0.34 ± 0.27*	*0.34 ± 0.26*	*0.35 ± 0.27*	*0.34 ± 0.28*	*0.37 ± 0.27*

Table 1.5: Untrained-network baseline (ρ^{untr}) — BREAKOUT.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.02	0.02	0.04	0.03	0.03	0.05	0.05	0.05	0.03	0.06	0.07	0.08	0.10	0.09	0.09	0.08	0.06	0.09	0.11	*0.06 ± 0.03*
2	0.03	0.01	0.03	0.04	0.02	0.04	0.05	0.05	0.03	0.06	0.05	0.06	0.08	0.09	0.08	0.07	0.05	0.09	0.09	*0.05 ± 0.02*
3	0.05	0.02	0.04	0.04	0.03	0.05	0.06	0.05	0.04	0.06	0.07	0.06	0.08	0.09	0.07	0.06	0.05	0.08	0.06	*0.06 ± 0.02*
4	0.05	0.02	0.02	0.02	0.02	0.03	0.04	0.03	0.02	0.03	0.04	0.03	0.04	0.05	0.04	0.04	0.03	0.04	0.04	*0.03 ± 0.01*
5	0.05	0.02	0.02	0.02	0.02	0.03	0.04	0.03	0.02	0.03	0.04	0.03	0.04	0.05	0.04	0.04	0.03	0.04	0.04	*0.03 ± 0.01*
6b	0.05	0.02	0.03	0.03	0.02	0.03	0.03	0.04	0.03	0.03	0.04	0.04	0.05	0.04	0.05	0.05	0.04	0.05	0.06	*0.04 ± 0.01*
7b	0.05	0.02	0.04	0.05	0.04	0.06	0.07	0.06	0.05	0.05	0.05	0.06	0.08	0.08	0.07	0.07	0.05	0.08	0.08	*0.06 ± 0.02*
8b	**0.11**	**0.10**	**0.13**	**0.16**	**0.17**	**0.21**	**0.23**	**0.16**	**0.13**	**0.17**	**0.19**	**0.18**	**0.20**	**0.19**	**0.18**	**0.16**	**0.13**	**0.18**	**0.18**	***0.17*** *± 0.03*
Mean ± Std	*0.05 ± 0.02*	*0.03 ± 0.03*	*0.04 ± 0.03*	*0.05 ± 0.04*	*0.04 ± 0.05*	*0.06 ± 0.06*	*0.07 ± 0.06*	*0.06 ± 0.04*	*0.04 ± 0.03*	*0.06 ± 0.04*	*0.07 ± 0.05*	*0.07 ± 0.05*	*0.08 ± 0.05*	***0.09*** *± 0.04*	*0.08 ± 0.04*	*0.07 ± 0.04*	*0.06 ± 0.03*	*0.08 ± 0.04*	*0.08 ± 0.04*	*0.06 ± 0.05*

Table 1.6: Action baseline (ρ^{act}) — BREAKOUT.

1.2 Mario Bros

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**1.00**	**0.93**	0.82	0.65	0.51	0.53	**0.67**	0.45	0.58	0.54	0.52	0.54	0.62	0.60	0.54	0.49	0.62	0.55	**0.58**	*0.62 ± 0.14*
2	**1.00**	0.51	**0.91**	**0.77**	**0.72**	**0.70**	0.65	**0.74**	**0.79**	**0.65**	**0.66**	**0.62**	**0.68**	**0.62**	**0.57**	**0.59**	**0.65**	**0.59**	0.57	***0.68*** *± 0.12*
3	0.99	0.84	0.58	0.75	0.54	0.57	0.57	0.66	0.62	0.62	0.58	**0.62**	0.59	0.55	0.50	0.56	0.59	0.54	0.52	*0.62 ± 0.12*
4	0.97	0.81	0.50	0.53	0.63	0.63	0.51	0.47	0.48	0.46	0.50	0.47	0.44	0.50	0.48	0.50	0.49	0.49	0.53	*0.55 ± 0.13*
5	0.97	0.81	0.50	0.53	0.63	0.63	0.51	0.47	0.48	0.46	0.50	0.47	0.44	0.50	0.48	0.50	0.49	0.49	0.53	*0.55 ± 0.13*
6b	0.94	0.47	0.39	0.42	0.54	0.52	0.50	0.47	0.47	0.47	0.48	0.44	0.41	0.47	0.43	0.43	0.40	0.42	0.42	*0.48 ± 0.12*
7b	0.53	0.41	0.34	0.42	0.45	0.38	0.39	0.38	0.35	0.40	0.42	0.40	0.37	0.40	0.40	0.37	0.36	0.38	0.39	*0.40 ± 0.04*
8b	0.68	0.38	0.49	0.36	0.35	0.33	0.32	0.36	0.36	0.35	0.36	0.33	0.35	0.33	0.36	0.33	0.33	0.33	0.34	*0.37 ± 0.08*
Mean ± Std	***0.89*** *± 0.17*	*0.65 ± 0.21*	*0.57 ± 0.19*	*0.55 ± 0.15*	*0.55 ± 0.11*	*0.54 ± 0.12*	*0.52 ± 0.11*	*0.50 ± 0.12*	*0.52 ± 0.14*	*0.49 ± 0.10*	*0.50 ± 0.09*	*0.49 ± 0.10*	*0.49 ± 0.12*	*0.50 ± 0.09*	*0.47 ± 0.07*	*0.47 ± 0.08*	*0.49 ± 0.11*	*0.47 ± 0.08*	*0.48 ± 0.08*	*0.53 ± 0.15*

Table 1.7: CU — Mario Bros.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.01	0.11	0.23	0.50	0.46	0.44	0.49	0.48	0.58	0.51	0.49	0.50	0.44	0.46	0.58	0.54	0.52	0.50	0.48	*0.44 ± 0.15*
2	0.00	0.66	0.47	0.55	0.49	0.58	0.53	0.48	0.62	0.62	0.54	0.62	0.56	0.57	0.56	0.59	0.61	0.56	0.57	*0.54 ± 0.14*
3	0.00	0.66	0.63	0.65	0.64	0.62	0.63	0.62	0.69	0.69	0.64	0.66	0.62	0.64	0.67	0.65	0.66	0.64	0.69	*0.62 ± 0.15*
4	0.02	0.53	0.67	0.71	0.64	0.69	0.65	0.65	0.71	0.72	0.71	0.76	0.72	0.74	0.73	0.76	0.75	0.72	0.75	*0.66 ± 0.16*
5	0.02	0.53	0.67	0.71	0.64	0.69	0.65	0.65	0.71	0.72	0.71	0.76	0.72	0.74	0.73	0.76	0.75	0.72	0.75	*0.66 ± 0.16*
6b	0.05	0.66	0.68	0.72	0.71	0.76	0.72	0.72	0.74	0.76	0.76	0.79	0.76	0.78	0.78	0.79	0.79	0.75	0.78	*0.71 ± 0.16*
7b	**0.76**	**0.85**	**0.86**	**0.83**	**0.83**	**0.85**	**0.83**	**0.83**	**0.83**	**0.82**	**0.82**	**0.83**	**0.82**	**0.84**	**0.84**	**0.86**	**0.84**	**0.84**	**0.84**	***0.83*** *± 0.02*
8b	0.39	0.55	0.64	0.68	0.67	0.68	0.69	0.69	0.68	0.71	0.68	0.74	0.74	0.74	0.73	0.74	0.75	0.75	0.76	*0.68 ± 0.08*
Mean ± Std	*0.16 ± 0.26*	*0.57 ± 0.20*	*0.61 ± 0.17*	*0.67 ± 0.10*	*0.64 ± 0.11*	*0.66 ± 0.11*	*0.65 ± 0.10*	*0.64 ± 0.11*	*0.69 ± 0.07*	*0.69 ± 0.09*	*0.67 ± 0.10*	*0.71 ± 0.10*	*0.67 ± 0.12*	*0.69 ± 0.12*	*0.70 ± 0.09*	***0.71*** *± 0.10*	*0.71 ± 0.10*	*0.69 ± 0.10*	*0.70 ± 0.11*	*0.64 ± 0.17*

Table 1.8: AS — Mario Bros.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.00	0.04	0.05	0.16	-0.03	-0.03	0.16	-0.07	0.16	0.05	0.01	0.04	0.06	0.06	0.12	0.03	0.14	0.05	0.06	*0.06 ± 0.07*
2	0.00	0.16	**0.38**	0.32	0.21	0.29	0.18	0.22	**0.41**	0.28	0.20	0.24	**0.24**	0.19	0.13	0.18	**0.26**	0.16	0.15	*0.22 ± 0.09*
3	0.00	**0.50**	0.21	**0.40**	0.18	0.20	0.20	**0.27**	0.31	**0.31**	0.23	**0.28**	0.21	0.19	0.16	0.22	0.25	0.18	0.21	***0.24*** *± 0.10*
4	-0.01	0.34	0.17	0.24	0.27	**0.32**	0.16	0.12	0.19	0.18	0.20	0.23	0.17	0.23	0.21	**0.26**	0.24	0.21	**0.28**	*0.21 ± 0.07*
5	-0.01	0.34	0.17	0.24	0.27	**0.32**	0.16	0.12	0.19	0.18	0.20	0.23	0.17	0.23	0.21	**0.26**	0.24	0.21	**0.28**	*0.21 ± 0.07*
6b	-0.02	0.13	0.07	0.14	0.25	0.28	**0.22**	0.19	0.21	0.23	**0.25**	0.23	0.18	**0.25**	0.21	0.22	0.20	0.17	0.20	*0.19 ± 0.07*
7b	**0.29**	0.26	0.19	0.25	**0.29**	0.22	**0.22**	0.21	0.17	0.22	0.24	0.23	0.19	0.24	**0.25**	0.23	0.20	**0.22**	0.23	*0.23 ± 0.03*
8b	0.07	-0.07	0.13	0.04	0.02	0.01	0.01	0.05	0.04	0.06	0.04	0.07	0.08	0.07	0.09	0.07	0.08	0.08	0.10	*0.05 ± 0.04*
Mean ± Std	*0.04 ± 0.10*	*0.21 ± 0.17*	*0.17 ± 0.09*	***0.22*** *± 0.10*	*0.18 ± 0.11*	*0.20 ± 0.13*	*0.16 ± 0.06*	*0.14 ± 0.10*	*0.21 ± 0.10*	*0.19 ± 0.09*	*0.17 ± 0.09*	*0.19 ± 0.08*	*0.16 ± 0.06*	*0.18 ± 0.07*	*0.17 ± 0.05*	*0.18 ± 0.08*	*0.20 ± 0.06*	*0.16 ± 0.06*	*0.19 ± 0.07*	*0.18 ± 0.10*

Table 1.9: ACU — Mario Bros.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.99	**0.89**	**0.77**	**0.37**	0.49	**0.54**	**0.47**	**0.52**	**0.39**	**0.47**	**0.47**	**0.42**	**0.56**	**0.49**	0.41	**0.39**	**0.48**	**0.49**	**0.52**	***0.53*** *± 0.16*
2	**1.00**	0.34	0.53	0.33	**0.51**	0.42	**0.47**	**0.52**	0.38	0.37	0.40	0.37	0.42	0.39	**0.43**	0.32	0.38	0.44	0.38	*0.44 ± 0.14*
3	0.99	0.24	0.35	0.29	0.36	0.32	0.37	0.37	0.31	0.28	0.34	0.27	0.31	0.31	0.30	0.26	0.28	0.33	0.28	*0.35 ± 0.16*
4	0.98	0.24	0.31	0.25	0.36	0.31	0.35	0.35	0.29	0.28	0.29	0.24	0.28	0.26	0.27	0.21	0.25	0.28	0.25	*0.32 ± 0.16*
5	0.98	0.24	0.31	0.25	0.36	0.31	0.35	0.35	0.29	0.28	0.29	0.24	0.28	0.26	0.27	0.21	0.25	0.28	0.25	*0.32 ± 0.16*
6b	0.95	0.25	0.29	0.22	0.27	0.24	0.28	0.28	0.26	0.24	0.24	0.21	0.24	0.22	0.22	0.21	0.21	0.25	0.22	*0.28 ± 0.16*
7b	-0.06	-0.00	0.05	0.07	0.11	0.12	0.14	0.15	0.15	0.18	0.15	0.16	0.17	0.15	0.15	0.14	0.16	0.16	0.16	*0.12 ± 0.06*
8b	-0.09	-0.04	0.02	0.02	0.03	0.06	0.07	0.07	0.06	0.07	0.05	0.06	0.09	0.06	0.07	0.06	0.08	0.08	0.07	*0.05 ± 0.04*
Mean ± Std	***0.72*** *± 0.46*	*0.27 ± 0.26*	*0.33 ± 0.23*	*0.23 ± 0.11*	*0.31 ± 0.16*	*0.29 ± 0.14*	*0.31 ± 0.13*	*0.33 ± 0.15*	*0.27 ± 0.10*	*0.27 ± 0.11*	*0.28 ± 0.13*	*0.25 ± 0.11*	*0.29 ± 0.14*	*0.27 ± 0.13*	*0.27 ± 0.11*	*0.23 ± 0.10*	*0.26 ± 0.12*	*0.29 ± 0.13*	*0.27 ± 0.13*	*0.30 ± 0.20*

Table 1.10: Observation baseline (ρ^{obs}) — MARIO BROS.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.99	**0.70**	**0.51**	**0.50**	**0.45**	**0.49**	**0.48**	**0.46**	**0.42**	**0.49**	**0.50**	**0.50**	0.41	**0.54**	0.40	**0.46**	**0.46**	**0.47**	**0.47**	***0.51*** *± 0.13*
2	**1.00**	0.32	0.42	0.45	**0.45**	0.35	0.43	0.42	0.35	0.37	0.46	0.38	**0.44**	0.43	**0.41**	0.41	0.38	0.40	0.43	*0.44 ± 0.14*
3	**1.00**	0.34	0.37	0.35	0.30	0.38	0.34	0.37	0.31	0.31	0.35	0.34	0.38	0.36	0.33	0.35	0.34	0.35	0.31	*0.38 ± 0.15*
4	0.98	0.47	0.32	0.29	0.26	0.29	0.29	0.23	0.25	0.23	0.23	0.23	0.23	0.22	0.22	0.24	0.23	0.23	0.23	*0.30 ± 0.17*
5	0.98	0.47	0.32	0.29	0.26	0.29	0.29	0.23	0.25	0.23	0.23	0.22	0.23	0.22	0.22	0.24	0.23	0.24	0.23	*0.30 ± 0.17*
6b	0.95	0.34	0.28	0.27	0.27	0.18	0.21	0.21	0.17	0.19	0.21	0.16	0.19	0.18	0.19	0.17	0.17	0.19	0.19	*0.25 ± 0.17*
7b	-0.06	0.02	0.06	0.06	0.07	0.09	0.09	0.09	0.11	0.13	0.12	0.12	0.10	0.13	0.13	0.12	0.13	0.13	0.12	*0.09 ± 0.05*
8b	-0.09	-0.02	0.01	0.02	0.00	0.02	0.02	0.02	0.03	0.04	0.04	0.04	0.05	0.03	0.04	0.03	0.04	0.04	0.05	*0.02 ± 0.03*
Mean ± Std	***0.72*** *± 0.46*	*0.33 ± 0.22*	*0.29 ± 0.16*	*0.28 ± 0.16*	*0.26 ± 0.15*	*0.26 ± 0.15*	*0.27 ± 0.15*	*0.25 ± 0.14*	*0.24 ± 0.12*	*0.25 ± 0.13*	*0.27 ± 0.15*	*0.25 ± 0.14*	*0.25 ± 0.13*	*0.26 ± 0.16*	*0.24 ± 0.12*	*0.25 ± 0.14*	*0.25 ± 0.13*	*0.26 ± 0.13*	*0.25 ± 0.13*	*0.29 ± 0.21*

Table 1.11: Untrained-network baseline (ρ^{untr}) — MARIO BROS.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.09	-0.07	-0.01	0.02	0.03	0.02	0.04	0.03	0.03	0.03	0.03	0.04	0.04	0.03	0.03	0.03	0.04	0.03	0.03	*0.02 ± 0.04*
2	-0.09	-0.02	-0.02	0.01	0.01	0.01	0.01	0.01	0.02	0.02	0.01	0.02	0.01	0.02	0.03	0.02	0.02	0.02	0.01	*0.01 ± 0.03*
3	-0.09	-0.02	-0.01	-0.00	0.01	0.02	0.01	0.01	0.01	0.01	0.01	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	*0.01 ± 0.03*
4	-0.08	0.01	0.01	0.01	0.03	0.03	0.03	0.03	0.02	0.03	0.04	0.04	0.03	0.03	0.03	0.03	0.03	0.03	0.03	*0.02 ± 0.03*
5	-0.08	0.01	0.01	0.01	0.03	0.03	0.03	0.03	0.02	0.03	0.04	0.04	0.03	0.03	0.03	0.03	0.03	0.03	0.03	*0.02 ± 0.03*
6b	-0.08	0.08	0.08	0.06	0.07	0.08	0.07	0.08	0.08	0.07	0.07	0.07	0.07	0.06	0.06	0.06	0.05	0.05	0.05	*0.06 ± 0.03*
7b	0.24	0.15	0.14	0.16	0.15	0.14	0.15	0.15	0.14	0.14	0.16	0.16	0.16	0.15	0.14	0.13	0.13	0.12	0.12	*0.15 ± 0.02*
8b	**0.61**	**0.45**	**0.36**	**0.32**	**0.33**	**0.32**	**0.31**	**0.31**	**0.32**	**0.29**	**0.32**	**0.26**	**0.26**	**0.26**	**0.27**	**0.26**	**0.25**	**0.25**	**0.24**	***0.32*** *± 0.08*
Mean ± Std	*0.04 ± 0.24*	*0.07 ± 0.16*	*0.07 ± 0.12*	*0.07 ± 0.11*	*0.08 ± 0.10*	*0.08 ± 0.10*	*0.08 ± 0.10*	*0.08 ± 0.10*	*0.08 ± 0.10*	*0.08 ± 0.09*	***0.09*** *± 0.10*	*0.08 ± 0.08*	*0.08 ± 0.08*	*0.07 ± 0.08*	*0.08 ± 0.08*	*0.07 ± 0.08*	*0.07 ± 0.08*	*0.07 ± 0.08*	*0.07 ± 0.07*	*0.07 ± 0.11*

Table 1.12: Action baseline (ρ^{act}) — MARIO BROS.

1.3 Ms. Pac-Man

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.95**	0.77	**0.90**	**0.90**	**0.74**	**0.87**	0.81	0.79	**0.80**	**0.81**	0.75	**0.74**	**0.75**	**0.75**	**0.68**	0.69	**0.77**	**0.66**	0.62	***0.78*** *± 0.08*
2	0.74	0.80	**0.90**	0.78	0.66	0.75	**0.82**	**0.80**	0.68	0.65	**0.76**	0.66	0.72	0.68	0.67	**0.72**	0.63	0.63	**0.69**	*0.72 ± 0.07*
3	0.92	0.58	0.83	0.77	**0.74**	0.73	0.74	0.74	0.63	0.66	0.62	0.64	0.62	0.64	0.65	0.62	0.63	0.63	0.58	*0.68 ± 0.09*
4	0.88	0.76	0.73	0.88	0.60	0.81	0.76	0.74	0.72	0.67	0.66	0.61	0.60	0.61	0.60	0.68	0.67	0.59	0.64	*0.70 ± 0.09*
5	0.88	0.76	0.73	0.88	0.60	0.81	0.76	0.74	0.72	0.67	0.66	0.61	0.60	0.61	0.60	0.68	0.67	0.59	0.64	*0.70 ± 0.09*
6b	0.78	**0.88**	0.80	0.59	0.61	0.68	0.58	0.52	0.51	0.55	0.51	0.48	0.51	0.48	0.52	0.51	0.53	0.48	0.54	*0.58 ± 0.11*
7b	0.51	0.78	0.65	0.59	0.56	0.59	0.54	0.54	0.56	0.51	0.52	0.47	0.42	0.43	0.44	0.43	0.41	0.40	0.39	*0.51 ± 0.10*
8b	0.66	0.78	0.58	0.60	0.67	0.64	0.64	0.62	0.59	0.59	0.53	0.62	0.61	0.48	0.48	0.45	0.49	0.48	0.46	*0.58 ± 0.09*
Mean ± Std	***0.79*** *± 0.14*	*0.76 ± 0.08*	*0.77 ± 0.11*	*0.75 ± 0.13*	*0.65 ± 0.06*	*0.73 ± 0.09*	*0.71 ± 0.10*	*0.69 ± 0.10*	*0.65 ± 0.09*	*0.64 ± 0.09*	*0.63 ± 0.09*	*0.60 ± 0.08*	*0.60 ± 0.10*	*0.58 ± 0.10*	*0.58 ± 0.08*	*0.60 ± 0.11*	*0.60 ± 0.11*	*0.56 ± 0.09*	*0.57 ± 0.09*	*0.66 ± 0.12*

Table 1.13: CU — Ms. Pac-Man.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.04	0.14	0.08	0.17	0.20	0.10	0.15	0.12	0.12	0.18	0.18	0.21	0.26	0.29	0.22	0.29	0.23	0.28	0.28	*0.19 ± 0.07*
2	0.28	0.25	0.13	0.31	0.34	0.21	0.16	0.16	0.24	0.27	0.24	0.27	0.29	0.28	0.32	0.31	0.33	0.32	0.34	*0.27 ± 0.06*
3	0.18	0.34	0.24	0.33	0.32	0.31	0.31	0.31	0.37	0.37	0.39	0.39	0.40	0.41	0.42	0.42	0.38	0.39	0.42	*0.35 ± 0.06*
4	0.12	0.52	0.51	0.47	0.47	0.38	0.32	0.35	0.41	0.38	0.38	0.41	0.41	0.40	0.44	0.45	0.43	0.49	0.42	*0.41 ± 0.08*
5	0.12	0.52	0.51	0.47	0.47	0.38	0.32	0.35	0.41	0.38	0.38	0.41	0.41	0.40	0.44	0.45	0.43	0.49	0.42	*0.41 ± 0.08*
6b	0.19	0.52	0.56	**0.60**	0.59	0.55	0.55	0.58	0.62	0.58	0.57	0.56	0.60	0.59	0.55	0.57	0.55	0.59	0.58	*0.55 ± 0.09*
7b	0.42	0.54	**0.61**	0.59	**0.67**	**0.63**	**0.65**	**0.65**	**0.66**	**0.65**	**0.65**	**0.63**	**0.69**	**0.67**	**0.68**	0.67	**0.67**	**0.68**	**0.70**	***0.64*** *± 0.06*
8b	**0.56**	**0.59**	0.47	0.57	0.56	0.54	0.55	0.56	0.60	0.58	0.62	0.59	0.59	0.65	0.65	**0.68**	0.66	0.66	0.67	*0.60 ± 0.05*
Mean ± Std	*0.24 ± 0.16*	*0.43 ± 0.15*	*0.39 ± 0.19*	*0.44 ± 0.15*	*0.45 ± 0.15*	*0.39 ± 0.17*	*0.38 ± 0.17*	*0.39 ± 0.18*	*0.43 ± 0.18*	*0.42 ± 0.15*	*0.43 ± 0.16*	*0.43 ± 0.14*	*0.46 ± 0.14*	*0.46 ± 0.14*	*0.47 ± 0.15*	*0.48 ± 0.14*	*0.46 ± 0.15*	***0.49*** *± 0.14*	*0.48 ± 0.14*	*0.43 ± 0.17*

Table 1.14: AS — Ms. Pac-Man.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.01	-0.10	-0.02	0.08	-0.06	-0.03	-0.04	-0.09	-0.08	-0.01	-0.07	-0.05	0.01	0.03	-0.10	-0.02	0.00	-0.06	-0.10	*-0.04 ± 0.05*
2	0.03	0.05	0.03	0.09	0.01	-0.04	-0.02	-0.04	-0.09	-0.09	0.01	-0.07	0.01	-0.04	-0.01	0.04	-0.05	-0.06	0.03	*-0.01 ± 0.05*
3	0.10	-0.08	0.07	0.10	0.05	0.05	0.05	0.05	0.00	0.03	0.01	0.03	0.02	0.05	0.07	0.04	0.01	0.03	0.00	*0.04 ± 0.04*
4	0.00	0.28	0.25	**0.35**	0.07	0.19	0.08	0.10	0.13	0.06	0.04	0.01	0.01	0.01	0.05	0.12	0.09	0.08	0.06	*0.10 ± 0.10*
5	0.00	0.28	0.25	**0.35**	0.07	0.19	0.08	0.10	0.13	0.06	0.04	0.01	0.01	0.01	0.05	0.12	0.09	0.08	0.06	*0.10 ± 0.10*
6b	-0.03	**0.40**	**0.35**	0.20	0.20	**0.23**	0.13	0.11	0.14	0.13	0.08	0.04	0.10	0.07	0.07	0.08	0.08	0.07	0.12	*0.14 ± 0.10*
7b	-0.07	0.32	0.25	0.18	**0.23**	0.22	0.18	**0.19**	**0.22**	**0.17**	**0.17**	0.10	0.11	0.10	0.11	0.10	0.08	0.09	0.09	*0.15 ± 0.08*
8b	**0.22**	0.37	0.05	0.16	**0.23**	0.18	**0.19**	**0.19**	0.19	**0.17**	0.15	**0.21**	**0.20**	**0.13**	**0.12**	**0.13**	**0.15**	**0.14**	**0.13**	***0.17*** *± 0.06*
Mean ± Std	*0.03 ± 0.09*	***0.19*** *± 0.19*	*0.15 ± 0.13*	*0.19 ± 0.10*	*0.10 ± 0.10*	*0.12 ± 0.11*	*0.08 ± 0.08*	*0.08 ± 0.09*	*0.08 ± 0.11*	*0.07 ± 0.08*	*0.05 ± 0.07*	*0.03 ± 0.08*	*0.06 ± 0.07*	*0.05 ± 0.05*	*0.04 ± 0.07*	*0.08 ± 0.05*	*0.06 ± 0.06*	*0.05 ± 0.07*	*0.05 ± 0.07*	*0.08 ± 0.10*

Table 1.15: ACU — Ms. Pac-Man.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.55	**0.84**	0.48	**0.65**	**0.75**	**0.73**	**0.85**	**0.86**	**0.86**	**0.81**	**0.82**	**0.75**	0.63	0.64	**0.73**	**0.70**	**0.77**	**0.72**	**0.67**	***0.73*** *± 0.10*
2	0.61	0.68	0.50	0.60	0.55	0.70	0.83	0.84	0.73	0.73	0.71	0.69	**0.66**	**0.66**	0.63	0.68	0.67	0.65	0.66	*0.67 ± 0.08*
3	0.82	0.53	**0.55**	0.54	0.54	0.60	0.65	0.65	0.55	0.60	0.58	0.59	0.57	0.56	0.57	0.55	0.62	0.57	0.56	*0.59 ± 0.06*
4	**0.88**	0.42	0.39	0.45	0.50	0.62	0.68	0.63	0.56	0.60	0.59	0.57	0.59	0.54	0.54	0.55	0.57	0.50	0.58	*0.57 ± 0.10*
5	**0.88**	0.42	0.39	0.45	0.50	0.62	0.68	0.63	0.56	0.60	0.59	0.57	0.59	0.54	0.54	0.55	0.57	0.50	0.58	*0.57 ± 0.10*
6b	0.81	0.40	0.34	0.38	0.37	0.43	0.45	0.42	0.37	0.42	0.43	0.44	0.40	0.41	0.45	0.43	0.45	0.41	0.42	*0.43 ± 0.09*
7b	0.58	0.39	0.30	0.36	0.31	0.37	0.35	0.35	0.34	0.35	0.35	0.37	0.31	0.33	0.32	0.33	0.33	0.32	0.30	*0.35 ± 0.06*
8b	0.38	0.14	0.23	0.31	0.27	0.30	0.23	0.22	0.25	0.25	0.26	0.24	0.21	0.23	0.19	0.25	0.24	0.19	0.19	*0.24 ± 0.05*
Mean ± Std	***0.69*** *± 0.17*	*0.48 ± 0.20*	*0.40 ± 0.10*	*0.47 ± 0.11*	*0.47 ± 0.14*	*0.55 ± 0.15*	*0.59 ± 0.21*	*0.57 ± 0.21*	*0.53 ± 0.19*	*0.55 ± 0.18*	*0.54 ± 0.17*	*0.53 ± 0.16*	*0.49 ± 0.16*	*0.49 ± 0.14*	*0.50 ± 0.16*	*0.51 ± 0.15*	*0.53 ± 0.17*	*0.48 ± 0.16*	*0.49 ± 0.16*	*0.52 ± 0.18*

Table 1.16: Observation baseline (ρ^{obs}) — Ms. Pac-Man.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.96**	**0.86**	**0.92**	**0.83**	**0.77**	**0.90**	0.80	**0.78**	**0.79**	**0.78**	0.74	**0.78**	**0.74**	**0.71**	**0.73**	**0.71**	**0.67**	**0.66**	**0.68**	***0.78*** *± 0.08*
2	0.72	0.75	0.87	0.69	0.66	0.79	**0.81**	0.77	0.76	0.69	**0.76**	0.70	0.71	0.70	0.68	0.66	0.63	0.65	0.65	*0.72 ± 0.06*
3	0.65	0.65	0.76	0.67	0.68	0.69	0.69	0.69	0.63	0.61	0.61	0.60	0.60	0.59	0.55	0.58	0.56	0.60	0.54	*0.63 ± 0.06*
4	0.64	0.48	0.49	0.53	0.52	0.56	0.64	0.64	0.59	0.62	0.60	0.57	0.56	0.57	0.55	0.50	0.53	0.49	0.49	*0.56 ± 0.05*
5	0.64	0.48	0.49	0.53	0.52	0.56	0.64	0.64	0.59	0.62	0.60	0.57	0.56	0.57	0.55	0.50	0.53	0.51	0.49	*0.56 ± 0.05*
6b	0.51	0.48	0.44	0.37	0.41	0.44	0.42	0.38	0.37	0.40	0.42	0.40	0.37	0.39	0.40	0.38	0.39	0.37	0.39	*0.41 ± 0.04*
7b	0.42	0.46	0.37	0.39	0.31	0.36	0.29	0.29	0.29	0.30	0.29	0.29	0.27	0.26	0.27	0.27	0.26	0.26	0.24	*0.31 ± 0.06*
8b	0.39	0.27	0.29	0.27	0.31	0.22	0.19	0.20	0.19	0.16	0.16	0.18	0.15	0.18	0.17	0.16	0.17	0.20	0.18	*0.21 ± 0.06*
Mean ± Std	***0.62*** *± 0.17*	*0.55 ± 0.18*	*0.58 ± 0.22*	*0.54 ± 0.18*	*0.52 ± 0.16*	*0.57 ± 0.21*	*0.56 ± 0.22*	*0.55 ± 0.21*	*0.53 ± 0.21*	*0.52 ± 0.20*	*0.52 ± 0.20*	*0.51 ± 0.19*	*0.49 ± 0.20*	*0.50 ± 0.19*	*0.49 ± 0.18*	*0.47 ± 0.18*	*0.47 ± 0.17*	*0.47 ± 0.16*	*0.46 ± 0.17*	*0.52 ± 0.19*

Table 1.17: Untrained-network baseline (ρ^{untr}) — Ms. Pac-Man.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.13	0.07	0.21	0.25	0.12	0.12	0.09	0.09	0.09	0.10	0.09	0.08	0.10	0.07	0.08	0.11	0.10	0.10	0.10	*0.11 ± 0.04*
2	0.10	0.11	0.22	0.24	0.16	0.13	0.08	0.07	0.09	0.11	0.08	0.06	0.09	0.11	0.09	0.09	0.10	0.11	0.09	*0.11 ± 0.05*
3	0.05	0.12	0.20	0.21	0.18	0.15	0.18	0.12	0.12	0.11	0.11	0.10	0.09	0.10	0.10	0.10	0.10	0.10	0.12	*0.12 ± 0.04*
4	-0.00	-0.01	0.12	0.16	0.10	0.14	0.10	0.11	0.06	0.06	0.06	0.06	0.04	0.06	0.06	0.08	0.08	0.09	0.09	*0.08 ± 0.04*
5	-0.00	-0.01	0.12	0.16	0.10	0.14	0.10	0.11	0.06	0.06	0.06	0.06	0.04	0.06	0.06	0.08	0.08	0.09	0.09	*0.08 ± 0.04*
6b	-0.02	0.05	0.19	0.16	0.19	0.17	0.15	0.16	0.10	0.14	0.11	0.10	0.11	0.12	0.11	0.12	0.12	0.12	0.13	*0.12 ± 0.05*
7b	0.13	0.08	0.31	0.32	0.28	0.25	0.22	0.23	0.19	0.19	0.19	0.16	0.14	0.16	0.16	0.19	0.16	0.18	0.18	*0.20 ± 0.06*
8b	**0.26**	**0.41**	**0.53**	**0.43**	**0.44**	**0.46**	**0.45**	**0.44**	**0.40**	**0.42**	**0.38**	**0.41**	**0.41**	**0.35**	**0.35**	**0.32**	**0.34**	**0.34**	**0.33**	***0.39*** *± 0.06*
Mean ± Std	*0.08 ± 0.09*	*0.10 ± 0.12*	*0.24 ± 0.12*	***0.24*** *± 0.09*	*0.20 ± 0.11*	*0.20 ± 0.11*	*0.17 ± 0.11*	*0.17 ± 0.11*	*0.14 ± 0.11*	*0.15 ± 0.11*	*0.14 ± 0.10*	*0.13 ± 0.11*	*0.13 ± 0.11*	*0.13 ± 0.09*	*0.13 ± 0.09*	*0.14 ± 0.08*	*0.14 ± 0.08*	*0.14 ± 0.08*	*0.14 ± 0.08*	*0.15 ± 0.11*

Table 1.18: Action baseline (ρ^{act}) — Ms. Pac-Man.

1.4 Pong

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.33	0.01	**0.91**	**0.80**	0.64	**0.69**	0.63	0.56	**0.66**	**0.68**	0.57	**0.76**	**0.70**	**0.70**	**0.60**	**0.76**	0.61	**0.56**	**0.64**	*0.62 ± 0.18*
2	**1.00**	**0.91**	0.74	0.63	0.67	0.59	**0.65**	0.51	**0.66**	0.65	**0.59**	0.56	0.66	0.59	**0.60**	0.61	**0.63**	0.51	0.55	***0.65*** *± 0.12*
3	0.31	0.67	0.66	0.55	**0.73**	0.64	0.63	**0.62**	0.46	0.47	0.39	0.38	0.39	0.40	0.37	0.35	0.37	0.39	0.38	*0.48 ± 0.13*
4	0.60	0.52	0.58	0.49	0.44	0.51	0.58	0.48	0.50	0.52	0.49	0.46	0.50	0.45	0.46	0.47	0.48	0.47	0.53	*0.50 ± 0.04*
5	0.60	0.52	0.58	0.49	0.44	0.51	0.58	0.48	0.50	0.52	0.49	0.46	0.50	0.45	0.46	0.47	0.48	0.47	0.53	*0.50 ± 0.04*
6a	0.37	0.32	0.29	0.30	0.28	0.28	0.29	0.31	0.32	0.33	0.32	0.31	0.33	0.32	0.32	0.32	0.32	0.31	0.30	*0.31 ± 0.02*
Mean ± Std	*0.54 ± 0.24*	*0.49 ± 0.28*	***0.63*** *± 0.19*	*0.54 ± 0.15*	*0.53 ± 0.16*	*0.54 ± 0.13*	*0.56 ± 0.12*	*0.49 ± 0.10*	*0.52 ± 0.12*	*0.53 ± 0.12*	*0.47 ± 0.09*	*0.49 ± 0.14*	*0.51 ± 0.13*	*0.48 ± 0.13*	*0.47 ± 0.11*	*0.50 ± 0.15*	*0.48 ± 0.11*	*0.45 ± 0.08*	*0.49 ± 0.11*	*0.51 ± 0.15*

Table 1.19: CU — Pong.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.33	0.58	0.26	0.17	0.17	0.28	0.31	0.37	0.38	0.36	0.36	0.24	0.19	0.25	0.29	0.35	0.40	0.46	0.43	*0.33 ± 0.10*
2	**1.00**	0.64	0.44	0.27	0.38	0.49	0.51	0.53	0.61	0.63	0.62	0.64	0.63	0.64	0.61	0.67	0.63	0.68	0.66	*0.59 ± 0.14*
3	0.99	0.98	**0.99**	0.94	**0.98**	**0.96**	0.93	0.93	0.92	0.89	0.89	0.87	0.89	0.87	0.85	0.84	0.87	0.87	0.83	*0.91 ± 0.05*
4	0.99	**0.99**	0.98	**0.99**	0.97	**0.96**	**0.97**	**0.96**	0.93	**0.94**	**0.96**	0.93	0.93	0.93	0.93	0.91	0.90	0.90	0.91	***0.95*** *± 0.03*
5	0.99	**0.99**	0.98	**0.99**	0.97	**0.96**	**0.97**	**0.96**	0.93	**0.94**	**0.96**	0.92	0.93	0.93	0.93	0.91	0.90	0.90	0.91	*0.95 ± 0.03*
6a	0.97	0.94	0.95	0.95	0.93	0.95	0.95	0.95	**0.94**	**0.94**	0.94	**0.94**	**0.95**	**0.94**	**0.95**	**0.94**	**0.94**	**0.94**	**0.93**	*0.94 ± 0.01*
Mean ± Std	***0.88*** *± 0.25*	*0.85 ± 0.17*	*0.77 ± 0.30*	*0.72 ± 0.35*	*0.73 ± 0.33*	*0.77 ± 0.28*	*0.77 ± 0.26*	*0.78 ± 0.24*	*0.79 ± 0.22*	*0.78 ± 0.22*	*0.79 ± 0.23*	*0.76 ± 0.25*	*0.75 ± 0.27*	*0.76 ± 0.25*	*0.76 ± 0.24*	*0.77 ± 0.21*	*0.77 ± 0.20*	*0.79 ± 0.17*	*0.78 ± 0.18*	*0.78 ± 0.25*

Table 1.20: AS — Pong.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.34	-0.41	0.18	-0.04	-0.19	-0.03	-0.06	-0.07	0.03	0.04	-0.07	0.00	-0.10	-0.06	-0.11	0.11	0.01	0.03	0.07	*-0.05 ± 0.14*
2	**1.00**	0.55	0.18	-0.09	0.05	0.08	0.16	0.04	0.27	0.28	0.21	0.20	0.29	0.23	0.20	0.27	0.26	0.19	0.21	*0.24 ± 0.22*
3	0.30	**0.65**	**0.64**	**0.49**	**0.71**	**0.60**	**0.56**	**0.55**	0.38	0.36	0.29	0.26	0.28	0.27	0.22	0.19	0.23	0.26	0.20	*0.39 ± 0.17*
4	0.59	0.51	0.56	0.48	0.41	0.48	**0.56**	0.44	**0.44**	**0.46**	**0.45**	**0.40**	**0.43**	**0.38**	**0.39**	**0.38**	**0.38**	**0.38**	**0.44**	***0.45*** *± 0.06*
5	0.59	0.51	0.56	0.48	0.41	0.48	**0.56**	0.44	**0.44**	**0.46**	**0.45**	0.38	**0.43**	**0.38**	**0.39**	**0.38**	**0.38**	**0.38**	**0.44**	*0.45 ± 0.06*
6a	0.33	0.27	0.24	0.24	0.21	0.23	0.24	0.26	0.26	0.27	0.27	0.26	0.27	0.26	0.26	0.26	0.26	0.25	0.23	*0.26 ± 0.02*
Mean ± Std	***0.41*** *± 0.41*	*0.35 ± 0.36*	*0.39 ± 0.20*	*0.26 ± 0.25*	*0.27 ± 0.29*	*0.31 ± 0.23*	*0.34 ± 0.24*	*0.28 ± 0.23*	*0.30 ± 0.14*	*0.31 ± 0.14*	*0.27 ± 0.18*	*0.25 ± 0.13*	*0.27 ± 0.18*	*0.24 ± 0.15*	*0.23 ± 0.17*	*0.27 ± 0.10*	*0.25 ± 0.12*	*0.25 ± 0.12*	*0.27 ± 0.13*	*0.29 ± 0.22*

Table 1.21: ACU — Pong.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.67**	**0.30**	**0.74**	**0.83**	**0.81**	**0.72**	**0.69**	**0.62**	**0.60**	**0.61**	**0.62**	**0.74**	**0.81**	**0.75**	**0.71**	**0.62**	**0.58**	**0.54**	**0.54**	***0.66*** *± 0.12*
2	-0.00	0.00	0.56	0.73	0.60	0.50	0.48	0.47	0.38	0.34	0.32	0.33	0.36	0.33	0.34	0.29	0.26	0.25	0.24	*0.36 ± 0.18*
3	-0.00	0.00	0.00	0.05	0.00	0.02	0.06	0.05	0.04	0.06	0.06	0.07	0.05	0.07	0.08	0.07	0.05	0.05	0.06	*0.04 ± 0.03*
4	0.00	-0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.02	0.00	0.01	0.01	0.00	*0.00 ± 0.01*
5	0.00	-0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.00	0.00	0.02	0.00	0.01	0.01	0.00	*0.00 ± 0.01*
6a	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.01	0.01	0.01	0.01	0.01	0.01	0.03	0.02	0.02	0.02	0.02	*0.01 ± 0.01*
Mean ± Std	*0.11 ± 0.25*	*0.05 ± 0.11*	*0.22 ± 0.31*	***0.27*** *± 0.36*	*0.24 ± 0.34*	*0.21 ± 0.29*	*0.20 ± 0.28*	*0.19 ± 0.26*	*0.17 ± 0.23*	*0.17 ± 0.23*	*0.17 ± 0.23*	*0.20 ± 0.27*	*0.21 ± 0.30*	*0.19 ± 0.27*	*0.20 ± 0.25*	*0.17 ± 0.23*	*0.15 ± 0.21*	*0.15 ± 0.19*	*0.14 ± 0.20*	*0.18 ± 0.26*

Table 1.22: Observation baseline (ρ^{obs}) — PONG.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.45**	0.23	**0.53**	**0.60**	**0.79**	**0.63**	**0.61**	**0.54**	**0.62**	**0.63**	**0.59**	**0.72**	**0.67**	**0.67**	**0.65**	**0.64**	**0.59**	**0.50**	**0.57**	***0.59*** *± 0.11*
2	-0.00	**0.36**	0.43	0.52	0.61	0.47	0.46	0.41	0.38	0.37	0.38	0.34	0.37	0.36	0.39	0.33	0.37	0.32	0.34	*0.38 ± 0.11*
3	-0.00	0.02	0.00	0.06	0.02	0.04	0.07	0.07	0.08	0.11	0.11	0.13	0.11	0.13	0.15	0.16	0.13	0.13	0.17	*0.09 ± 0.05*
4	0.00	-0.00	0.02	0.01	0.03	0.04	0.03	0.04	0.07	0.06	0.04	0.07	0.07	0.07	0.07	0.09	0.10	0.10	0.09	*0.05 ± 0.03*
5	0.00	-0.00	0.02	0.01	0.03	0.04	0.03	0.04	0.07	0.06	0.04	0.08	0.07	0.07	0.07	0.09	0.10	0.10	0.09	*0.05 ± 0.03*
6a	0.01	0.02	0.02	0.01	0.05	0.03	0.03	0.04	0.06	0.06	0.06	0.06	0.05	0.06	0.05	0.06	0.06	0.06	0.07	*0.05 ± 0.02*
Mean ± Std	*0.08 ± 0.17*	*0.10 ± 0.14*	*0.17 ± 0.22*	*0.20 ± 0.26*	***0.26*** *± 0.32*	*0.21 ± 0.25*	*0.21 ± 0.24*	*0.19 ± 0.21*	*0.21 ± 0.21*	*0.21 ± 0.22*	*0.20 ± 0.21*	*0.23 ± 0.24*	*0.22 ± 0.23*	*0.23 ± 0.22*	*0.23 ± 0.22*	*0.23 ± 0.20*	*0.23 ± 0.19*	*0.20 ± 0.16*	*0.22 ± 0.18*	*0.20 ± 0.22*

Table 1.23: Untrained-network baseline (ρ^{untr}) — PONG.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.00	-0.00	-0.01	0.00	0.00	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	*0.01 ± 0.01*
2	-0.00	-0.00	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	*0.01 ± 0.00*
3	0.01	0.01	0.00	0.01	0.01	0.01	0.02	0.01	0.02	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	0.01	*0.01 ± 0.00*
4	-0.01	0.00	0.00	0.01	0.02	0.01	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	*0.02 ± 0.01*
5	-0.01	0.00	0.00	0.01	0.02	0.01	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	0.02	*0.02 ± 0.01*
6a	**0.03**	**0.06**	**0.04**	**0.05**	**0.05**	**0.05**	**0.05**	**0.04**	**0.04**	**0.04**	**0.04**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	***0.04*** *± 0.01*
Mean ± Std	*0.00 ± 0.01*	*0.01 ± 0.02*	*0.01 ± 0.02*	*0.02 ± 0.02*	*0.02 ± 0.02*	*0.02 ± 0.01*	***0.02*** *± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*	*0.02 ± 0.01*

Table 1.24: Action baseline (ρ^{act}) — PONG.

1.5 Space Invaders

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.43	**1.00**	0.96	**0.94**	0.77	0.71	0.80	0.67	0.65	0.60	0.63	**0.71**	**0.83**	0.76	0.64	0.67	**0.83**	**0.82**	**0.83**	*0.75 ± 0.14*
2	**0.99**	0.99	**0.98**	0.92	**0.81**	**0.85**	**0.95**	**0.85**	**0.67**	**0.74**	**0.75**	**0.71**	0.63	**0.82**	**0.73**	**0.71**	0.67	0.78	0.82	***0.81*** *± 0.11*
3	**0.99**	0.98	0.93	0.78	0.74	0.71	0.69	0.67	0.63	0.60	0.62	0.60	0.74	0.72	0.71	0.69	0.77	0.72	0.76	*0.74 ± 0.11*
4	0.31	0.73	0.92	0.69	0.61	0.57	0.64	0.61	0.61	0.66	0.58	0.52	0.56	0.63	0.67	0.63	0.63	0.68	0.64	*0.63 ± 0.11*
5	0.31	0.73	0.92	0.69	0.61	0.57	0.64	0.61	0.61	0.66	0.58	0.52	0.56	0.63	0.67	0.63	0.63	0.68	0.64	*0.63 ± 0.11*
6a	0.11	0.57	0.51	0.53	0.49	0.53	0.42	0.51	0.52	0.50	0.48	0.51	0.52	0.53	0.51	0.55	0.51	0.54	0.54	*0.49 ± 0.10*
Mean ± Std	*0.52 ± 0.34*	*0.83 ± 0.17*	***0.87*** *± 0.16*	*0.76 ± 0.14*	*0.67 ± 0.11*	*0.66 ± 0.11*	*0.69 ± 0.16*	*0.65 ± 0.10*	*0.61 ± 0.05*	*0.63 ± 0.07*	*0.61 ± 0.08*	*0.59 ± 0.09*	*0.64 ± 0.11*	*0.68 ± 0.10*	*0.66 ± 0.07*	*0.65 ± 0.05*	*0.67 ± 0.10*	*0.70 ± 0.09*	*0.70 ± 0.11*	*0.67 ± 0.15*

Table 1.25: CU — Space Invaders.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.40	0.14	0.08	0.13	0.14	0.22	0.41	0.36	0.38	0.37	0.27	0.35	0.29	0.30	0.35	0.26	0.32	0.28	0.29	*0.28 ± 0.10*
2	0.83	0.20	0.22	0.49	0.50	0.37	0.42	0.39	0.54	0.43	0.46	0.34	0.43	0.49	0.48	0.47	0.47	0.48	0.47	*0.45 ± 0.13*
3	0.83	0.27	0.52	0.57	0.48	0.55	0.61	0.51	0.60	0.55	0.52	0.47	0.59	0.55	0.51	0.53	0.56	0.57	0.44	*0.54 ± 0.10*
4	0.94	0.80	0.69	0.71	0.71	**0.75**	0.74	**0.74**	**0.75**	0.69	0.67	0.68	0.71	0.68	0.68	0.70	0.69	**0.74**	0.69	*0.72 ± 0.06*
5	0.94	0.80	0.69	0.71	0.71	**0.75**	0.74	**0.74**	**0.75**	0.69	0.67	0.68	0.71	0.68	0.68	0.70	0.69	**0.74**	0.69	*0.72 ± 0.06*
6a	**0.96**	**0.83**	**0.77**	**0.72**	**0.73**	0.73	**0.77**	0.72	0.74	**0.73**	**0.69**	**0.72**	**0.76**	**0.72**	**0.70**	**0.71**	**0.70**	0.73	**0.72**	***0.74*** *± 0.06*
Mean ± Std	***0.82*** *± 0.19*	*0.51 ± 0.31*	*0.49 ± 0.26*	*0.55 ± 0.21*	*0.54 ± 0.21*	*0.56 ± 0.21*	*0.61 ± 0.15*	*0.58 ± 0.16*	*0.63 ± 0.14*	*0.58 ± 0.14*	*0.55 ± 0.15*	*0.54 ± 0.16*	*0.58 ± 0.17*	*0.57 ± 0.15*	*0.57 ± 0.13*	*0.56 ± 0.16*	*0.57 ± 0.14*	*0.59 ± 0.17*	*0.55 ± 0.16*	*0.58 ± 0.19*

Table 1.26: AS — Space Invaders.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.16	0.14	0.04	0.08	-0.09	-0.07	0.21	0.03	0.03	-0.03	-0.11	0.05	0.12	0.06	-0.01	-0.07	0.15	0.10	0.12	*0.03 ± 0.10*
2	**0.83**	0.19	0.20	**0.41**	0.31	0.22	0.37	0.24	0.21	0.17	0.20	0.05	0.06	**0.31**	0.20	0.18	0.14	0.26	0.28	*0.25 ± 0.16*
3	0.82	0.26	0.44	0.35	0.22	0.27	0.29	0.18	0.23	0.15	0.14	0.08	**0.33**	0.27	0.22	0.23	**0.33**	0.28	0.20	*0.28 ± 0.15*
4	0.26	**0.53**	**0.61**	0.40	**0.32**	**0.32**	**0.38**	**0.36**	**0.35**	**0.35**	**0.26**	0.20	0.27	**0.31**	**0.36**	**0.33**	0.32	**0.43**	**0.34**	***0.35*** *± 0.09*
5	0.26	**0.53**	**0.61**	0.40	**0.32**	**0.32**	**0.38**	**0.36**	**0.35**	**0.35**	**0.26**	0.20	0.27	**0.31**	**0.36**	**0.33**	0.32	**0.43**	**0.34**	***0.35*** *± 0.09*
6a	0.07	0.39	0.28	0.24	0.22	0.26	0.19	0.23	0.26	0.23	0.17	**0.24**	0.28	0.25	0.21	0.26	0.22	0.27	0.26	*0.24 ± 0.06*
Mean ± Std	*0.35 ± 0.37*	*0.34 ± 0.15*	***0.36*** *± 0.21*	*0.31 ± 0.12*	*0.22 ± 0.14*	*0.22 ± 0.13*	*0.30 ± 0.08*	*0.23 ± 0.11*	*0.24 ± 0.11*	*0.20 ± 0.13*	*0.15 ± 0.13*	*0.14 ± 0.08*	*0.22 ± 0.10*	*0.25 ± 0.09*	*0.22 ± 0.12*	*0.21 ± 0.14*	*0.25 ± 0.08*	*0.29 ± 0.11*	*0.26 ± 0.08*	*0.25 ± 0.16*

Table 1.27: ACU — Space Invaders.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.43**	**0.82**	**0.92**	**0.87**	**0.84**	**0.76**	**0.53**	**0.56**	**0.60**	**0.58**	**0.70**	0.59	**0.71**	**0.70**	**0.58**	**0.63**	**0.58**	**0.57**	**0.57**	***0.66*** *± 0.13*
2	0.17	0.80	0.78	0.46	0.41	0.43	0.41	0.45	0.41	0.57	0.54	**0.66**	0.47	0.51	0.46	0.49	0.48	0.45	0.45	*0.49 ± 0.14*
3	0.17	0.73	0.48	0.43	0.42	0.39	0.33	0.35	0.34	0.37	0.46	0.50	0.37	0.41	0.36	0.37	0.41	0.37	0.41	*0.40 ± 0.10*
4	0.06	0.20	0.31	0.29	0.28	0.23	0.21	0.24	0.25	0.30	0.33	0.32	0.28	0.30	0.32	0.28	0.31	0.26	0.30	*0.27 ± 0.06*
5	0.06	0.20	0.31	0.29	0.28	0.23	0.21	0.24	0.25	0.30	0.33	0.32	0.28	0.30	0.32	0.28	0.31	0.26	0.30	*0.27 ± 0.06*
6a	0.01	0.17	0.23	0.28	0.25	0.27	0.20	0.25	0.25	0.26	0.31	0.28	0.24	0.28	0.30	0.29	0.30	0.27	0.28	*0.25 ± 0.07*
Mean ± Std	*0.15 ± 0.14*	*0.49 ± 0.30*	***0.51*** *± 0.26*	*0.44 ± 0.21*	*0.41 ± 0.20*	*0.39 ± 0.18*	*0.32 ± 0.12*	*0.35 ± 0.12*	*0.35 ± 0.13*	*0.40 ± 0.13*	*0.45 ± 0.14*	*0.45 ± 0.15*	*0.39 ± 0.16*	*0.42 ± 0.15*	*0.39 ± 0.10*	*0.39 ± 0.13*	*0.40 ± 0.10*	*0.36 ± 0.12*	*0.39 ± 0.10*	*0.39 ± 0.18*

Table 1.28: Observation baseline (ρ^{obs}) — SPACE INVADERS.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.52**	**0.86**	**0.89**	**0.69**	**0.80**	**0.74**	**0.59**	**0.63**	**0.61**	**0.60**	**0.72**	**0.64**	**0.69**	**0.66**	**0.65**	**0.70**	**0.68**	**0.72**	**0.71**	***0.69*** *± 0.09*
2	0.15	0.76	0.74	0.51	0.50	0.63	0.58	0.61	0.46	0.54	0.52	0.49	0.55	0.47	0.51	0.51	0.52	0.52	0.53	*0.53 ± 0.12*
3	0.17	0.70	0.42	0.43	0.52	0.45	0.39	0.49	0.40	0.45	0.41	0.49	0.41	0.45	0.49	0.47	0.44	0.43	0.56	*0.45 ± 0.10*
4	0.05	0.16	0.26	0.24	0.29	0.25	0.26	0.25	0.25	0.31	0.28	0.28	0.28	0.29	0.31	0.30	0.30	0.25	0.30	*0.26 ± 0.06*
5	0.05	0.16	0.26	0.24	0.29	0.25	0.26	0.25	0.25	0.31	0.28	0.28	0.28	0.29	0.31	0.30	0.30	0.25	0.30	*0.26 ± 0.06*
6a	0.03	0.12	0.16	0.27	0.27	0.26	0.23	0.28	0.26	0.27	0.25	0.21	0.21	0.23	0.26	0.24	0.22	0.23	0.23	*0.22 ± 0.06*
Mean ± Std	*0.16 ± 0.17*	***0.46*** *± 0.32*	*0.46 ± 0.27*	*0.40 ± 0.17*	*0.45 ± 0.19*	*0.43 ± 0.20*	*0.39 ± 0.15*	*0.42 ± 0.16*	*0.37 ± 0.13*	*0.41 ± 0.13*	*0.41 ± 0.17*	*0.40 ± 0.15*	*0.40 ± 0.17*	*0.40 ± 0.15*	*0.42 ± 0.14*	*0.42 ± 0.16*	*0.41 ± 0.16*	*0.40 ± 0.18*	*0.44 ± 0.17*	*0.40 ± 0.19*

Table 1.29: Untrained-network baseline (ρ^{untr}) — SPACE INVADERS.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.00**	0.00	**0.01**	**0.01**	**0.01**	0.01	0.01	0.01	**0.02**	0.01	**0.02**	0.01	0.02	0.02	0.02	0.02	0.02	0.02	0.02	*0.01 ± 0.01*
2	**0.00**	0.00	**0.01**	**0.01**	**0.01**	0.01	0.01	0.01	**0.02**	**0.02**	**0.02**	0.02	0.02	0.02	0.02	**0.03**	**0.03**	**0.03**	**0.03**	*0.02 ± 0.01*
3	**0.00**	0.00	**0.01**	**0.01**	**0.01**	0.01	0.01	0.01	**0.02**	**0.02**	**0.02**	0.02	0.02	0.02	0.02	**0.03**	**0.03**	**0.03**	**0.03**	*0.02 ± 0.01*
4	**0.00**	0.00	**0.01**	**0.01**	**0.01**	**0.02**	0.01	0.01	**0.02**	**0.02**	**0.02**	0.02	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	*0.02 ± 0.01*
5	**0.00**	0.00	**0.01**	**0.01**	**0.01**	**0.02**	0.01	0.01	**0.02**	**0.02**	**0.02**	0.02	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	*0.02 ± 0.01*
6a	**-0.00**	**0.01**	**0.01**	**0.01**	**0.01**	**0.02**	**0.02**	**0.02**	**0.02**	**0.02**	**0.02**	**0.03**	**0.03**	0.02	**0.03**	**0.03**	**0.03**	**0.03**	**0.03**	***0.02*** *± 0.01*
Mean ± Std	*0.00 ± 0.00*	*0.00 ± 0.00*	*0.01 ± 0.00*	*0.01 ± 0.00*	*0.01 ± 0.00*	*0.01 ± 0.01*	*0.01 ± 0.00*	*0.01 ± 0.00*	*0.02 ± 0.00*	*0.02 ± 0.00*	*0.02 ± 0.00*	*0.02 ± 0.01*	*0.03 ± 0.00*	*0.02 ± 0.00*	*0.03 ± 0.00*	***0.03*** *± 0.00*	***0.03*** *± 0.00*	***0.03*** *± 0.00*	***0.03*** *± 0.00*	*0.02 ± 0.01*

Table 1.30: Action baseline (ρ^{act}) — SPACE INVADERS.

1.6 Surround

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.91**	0.41	0.67	0.84	**0.87**	**0.78**	**0.83**	**0.73**	**0.76**	**0.79**	**0.69**	**0.76**	**0.66**	0.53	**0.63**	0.56	0.56	**0.68**	**0.73**	***0.70*** *± 0.12*
2	0.57	0.63	**0.86**	0.66	0.65	0.51	0.48	0.49	0.54	0.59	0.53	0.53	0.46	0.59	0.59	0.58	0.48	0.63	0.62	*0.58 ± 0.09*
3	0.82	0.61	0.84	**0.85**	0.68	0.72	0.71	0.59	0.68	0.68	0.59	0.61	0.65	0.56	0.60	0.56	**0.63**	0.57	0.58	*0.66 ± 0.09*
4	0.85	**0.77**	0.80	0.60	0.69	0.67	0.75	0.65	0.73	0.67	0.63	0.59	0.62	**0.62**	0.57	**0.60**	0.53	0.52	0.53	*0.65 ± 0.09*
5	0.85	**0.77**	0.80	0.60	0.69	0.67	0.75	0.65	0.73	0.67	0.63	0.59	0.62	**0.62**	0.57	**0.60**	0.53	0.52	0.53	*0.65 ± 0.09*
6b	0.83	0.70	0.74	0.58	0.60	0.62	0.49	0.50	0.61	0.66	0.57	0.51	0.50	0.58	0.57	0.52	0.56	0.53	0.53	*0.59 ± 0.09*
7b	0.73	0.63	0.55	0.57	0.67	0.53	0.48	0.46	0.48	0.42	0.44	0.45	0.42	0.43	0.41	0.41	0.40	0.39	0.36	*0.49 ± 0.10*
8b	0.77	0.75	0.67	0.54	0.52	0.55	0.52	0.58	0.61	0.59	0.51	0.52	0.50	0.46	0.50	0.45	0.43	0.43	0.43	*0.54 ± 0.10*
Mean ± Std	***0.79*** *± 0.10*	*0.66 ± 0.11*	*0.74 ± 0.10*	*0.66 ± 0.11*	*0.67 ± 0.09*	*0.63 ± 0.09*	*0.63 ± 0.14*	*0.58 ± 0.09*	*0.64 ± 0.09*	*0.63 ± 0.10*	*0.57 ± 0.07*	*0.57 ± 0.09*	*0.55 ± 0.09*	*0.55 ± 0.07*	*0.55 ± 0.06*	*0.54 ± 0.07*	*0.52 ± 0.07*	*0.53 ± 0.09*	*0.54 ± 0.11*	*0.61 ± 0.12*

Table 1.31: CU — Surround.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.10	0.34	0.27	0.15	0.15	0.21	0.20	0.23	0.26	0.22	0.34	0.28	0.26	0.33	0.30	0.41	0.35	0.44	0.34	*0.27 ± 0.09*
2	0.45	0.29	0.21	0.32	0.29	0.46	0.41	0.49	0.47	0.56	0.52	0.57	0.50	0.52	0.54	0.56	0.48	0.48	0.48	*0.45 ± 0.10*
3	**0.74**	0.40	0.31	0.42	0.47	0.48	0.57	0.59	0.60	0.58	0.61	0.62	0.64	0.60	0.61	0.63	0.56	0.61	0.62	*0.56 ± 0.10*
4	0.41	0.36	0.34	0.46	0.48	0.50	0.47	0.46	0.48	0.50	0.49	0.53	0.46	0.47	0.53	0.58	0.54	0.57	0.60	*0.49 ± 0.07*
5	0.41	0.36	0.34	0.46	0.48	0.50	0.47	0.46	0.48	0.50	0.49	0.53	0.46	0.47	0.53	0.58	0.54	0.57	0.60	*0.49 ± 0.07*
6b	0.63	0.49	0.44	0.60	0.62	0.61	0.56	0.60	0.54	0.54	0.57	0.58	0.57	0.55	0.63	0.59	0.62	0.53	0.62	*0.57 ± 0.05*
7b	0.66	**0.53**	0.70	**0.70**	**0.71**	**0.72**	**0.71**	**0.74**	**0.71**	**0.73**	**0.75**	**0.76**	**0.75**	**0.74**	**0.76**	**0.75**	0.75	0.73	**0.76**	***0.72*** *± 0.05*
8b	0.66	**0.53**	**0.71**	**0.70**	**0.71**	0.70	**0.71**	0.69	0.68	0.71	0.74	0.74	0.73	**0.74**	0.73	**0.75**	**0.76**	**0.75**	**0.76**	*0.71 ± 0.05*
Mean ± Std	*0.51 ± 0.19*	*0.41 ± 0.09*	*0.41 ± 0.18*	*0.48 ± 0.18*	*0.49 ± 0.18*	*0.52 ± 0.15*	*0.51 ± 0.16*	*0.53 ± 0.15*	*0.53 ± 0.13*	*0.54 ± 0.15*	*0.56 ± 0.13*	*0.58 ± 0.14*	*0.55 ± 0.15*	*0.55 ± 0.13*	*0.58 ± 0.13*	***0.61*** *± 0.10*	*0.57 ± 0.13*	*0.58 ± 0.10*	*0.60 ± 0.13*	*0.53 ± 0.15*

Table 1.32: AS — Surround.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.01	-0.25	-0.06	-0.01	0.02	-0.01	0.02	-0.04	0.01	0.01	0.03	0.04	-0.08	-0.14	-0.08	-0.03	-0.09	0.12	0.07	*-0.02 ± 0.08*
2	0.02	-0.08	0.07	-0.02	-0.05	-0.03	-0.11	-0.02	0.01	0.15	0.05	0.10	-0.05	0.11	0.13	0.14	-0.04	0.12	0.10	*0.03 ± 0.08*
3	**0.55**	0.01	0.15	**0.27**	0.15	0.19	**0.29**	0.17	0.28	0.26	0.20	0.22	**0.29**	0.17	0.21	0.19	**0.19**	**0.18**	**0.20**	*0.22 ± 0.10*
4	0.26	0.12	0.15	0.05	0.18	0.17	0.22	0.11	0.20	0.17	0.12	0.12	0.07	0.09	0.10	0.18	0.07	0.09	0.13	*0.14 ± 0.05*
5	0.26	0.12	0.15	0.05	0.18	0.17	0.22	0.11	0.20	0.17	0.12	0.12	0.07	0.09	0.10	0.18	0.07	0.09	0.13	*0.14 ± 0.05*
6b	0.46	0.19	0.18	0.18	0.22	0.22	0.05	0.10	0.15	0.19	0.14	0.10	0.07	0.13	0.19	0.11	0.18	0.06	0.15	*0.16 ± 0.09*
7b	0.39	0.16	0.25	**0.27**	**0.38**	**0.25**	0.19	0.19	0.19	0.14	0.19	0.20	0.17	0.16	0.17	0.15	0.15	0.12	0.11	*0.20 ± 0.07*
8b	0.42	**0.28**	**0.38**	0.24	0.23	0.24	0.23	**0.27**	**0.29**	**0.30**	**0.24**	**0.26**	0.23	**0.20**	**0.23**	**0.20**	0.18	**0.18**	0.19	*0.25 ± 0.06*
Mean ± Std	***0.30*** *± 0.19*	*0.07 ± 0.16*	*0.16 ± 0.12*	*0.13 ± 0.12*	*0.16 ± 0.12*	*0.15 ± 0.10*	*0.14 ± 0.13*	*0.11 ± 0.10*	*0.17 ± 0.10*	*0.17 ± 0.08*	*0.14 ± 0.07*	*0.14 ± 0.07*	*0.10 ± 0.12*	*0.10 ± 0.10*	*0.13 ± 0.09*	*0.14 ± 0.07*	*0.09 ± 0.10*	*0.12 ± 0.04*	*0.14 ± 0.04*	*0.14 ± 0.12*

Table 1.33: ACU — Surround.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	0.05	0.33	0.73	**0.85**	**0.70**	**0.78**	**0.58**	**0.58**	**0.74**	**0.59**	**0.55**	**0.61**	**0.70**	0.48	**0.64**	**0.56**	**0.54**	**0.52**	**0.49**	***0.58 ± 0.17***
2	0.20	**0.65**	**0.76**	0.66	**0.70**	0.54	0.52	0.51	0.51	0.44	0.43	0.43	0.47	0.48	0.40	0.42	0.45	0.49	0.46	*0.50 ± 0.12*
3	0.26	0.60	0.69	0.58	0.53	0.52	0.41	0.39	0.39	0.36	0.35	0.36	0.36	0.39	0.37	0.34	0.36	0.38	0.35	*0.42 ± 0.11*
4	**0.59**	0.64	0.66	0.54	0.52	0.50	0.53	0.49	0.52	0.47	0.45	0.43	0.49	**0.53**	0.45	0.42	0.45	0.43	0.37	*0.50 ± 0.07*
5	**0.59**	0.64	0.66	0.54	0.52	0.50	0.53	0.49	0.52	0.47	0.45	0.43	0.49	**0.53**	0.45	0.42	0.45	0.43	0.37	*0.50 ± 0.07*
6b	0.37	0.51	0.56	0.40	0.38	0.39	0.44	0.40	0.42	0.46	0.41	0.39	0.43	0.45	0.35	0.41	0.37	0.45	0.37	*0.42 ± 0.05*
7b	0.34	0.47	0.30	0.30	0.29	0.28	0.29	0.24	0.24	0.27	0.24	0.24	0.25	0.26	0.23	0.25	0.25	0.27	0.24	*0.28 ± 0.05*
8b	0.34	0.47	0.29	0.25	0.24	0.25	0.23	0.20	0.20	0.21	0.18	0.19	0.20	0.19	0.19	0.17	0.18	0.22	0.19	*0.23 ± 0.07*
Mean ± Std	*0.34 ± 0.17*	*0.54 ± 0.11*	***0.58** ± 0.17*	*0.52 ± 0.18*	*0.48 ± 0.16*	*0.47 ± 0.16*	*0.44 ± 0.12*	*0.41 ± 0.13*	*0.44 ± 0.16*	*0.41 ± 0.11*	*0.38 ± 0.11*	*0.39 ± 0.12*	*0.42 ± 0.15*	*0.41 ± 0.12*	*0.39 ± 0.13*	*0.37 ± 0.11*	*0.38 ± 0.11*	*0.40 ± 0.10*	*0.35 ± 0.09*	*0.43 ± 0.15*

Table 1.34: Observation baseline (ρ^{obs}) — SURROUND.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	**0.90**	**0.66**	0.70	**0.73**	**0.85**	**0.79**	**0.80**	**0.77**	**0.70**	**0.78**	**0.66**	**0.72**	**0.60**	**0.67**	**0.70**	**0.59**	**0.65**	**0.54**	**0.66**	***0.71** ± 0.09*
2	0.55	0.46	**0.79**	0.68	0.70	0.51	0.55	0.46	0.53	0.40	0.47	0.41	0.50	0.44	0.46	0.44	0.52	0.52	0.52	*0.52 ± 0.10*
3	0.03	0.43	0.68	0.58	0.51	0.50	0.43	0.41	0.39	0.42	0.39	0.38	0.36	0.38	0.39	0.37	0.44	0.38	0.38	*0.41 ± 0.12*
4	0.43	0.38	0.54	0.48	0.48	0.45	0.52	0.53	0.52	0.50	0.51	0.45	0.53	0.52	0.46	0.41	0.46	0.38	0.40	*0.47 ± 0.05*
5	0.43	0.38	0.54	0.48	0.48	0.45	0.52	0.53	0.52	0.50	0.51	0.45	0.53	0.52	0.46	0.41	0.46	0.38	0.40	*0.47 ± 0.05*
6b	0.30	0.39	0.34	0.29	0.32	0.35	0.36	0.38	0.46	0.45	0.43	0.42	0.42	0.41	0.37	0.39	0.37	0.44	0.38	*0.38 ± 0.05*
7b	0.27	0.35	0.21	0.22	0.28	0.26	0.24	0.22	0.22	0.22	0.20	0.24	0.18	0.20	0.23	0.22	0.22	0.18	0.18	*0.23 ± 0.04*
8b	0.21	0.25	0.18	0.19	0.18	0.18	0.16	0.16	0.15	0.17	0.15	0.16	0.17	0.14	0.18	0.14	0.15	0.19	0.17	*0.17 ± 0.03*
Mean ± Std	*0.39 ± 0.24*	*0.41 ± 0.11*	***0.50** ± 0.22*	*0.46 ± 0.19*	*0.47 ± 0.21*	*0.44 ± 0.17*	*0.45 ± 0.19*	*0.43 ± 0.18*	*0.44 ± 0.17*	*0.43 ± 0.18*	*0.41 ± 0.16*	*0.40 ± 0.15*	*0.41 ± 0.15*	*0.41 ± 0.16*	*0.41 ± 0.15*	*0.37 ± 0.13*	*0.41 ± 0.15*	*0.38 ± 0.12*	*0.39 ± 0.15*	*0.42 ± 0.17*

Table 1.35: Untrained-network baseline (ρ^{untr}) — SURROUND.

Layer \ K	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	*Mean ± Std*
1	-0.01	-0.05	0.00	0.01	0.02	0.01	0.02	0.02	0.03	0.03	0.03	0.02	0.03	0.03	0.04	0.03	0.04	0.03	0.04	*0.02 ± 0.02*
2	0.09	-0.02	0.00	0.02	0.03	0.05	0.04	0.06	0.04	0.06	0.05	0.05	0.05	0.05	0.04	0.05	0.05	0.05	0.05	*0.04 ± 0.02*
3	0.12	**0.02**	0.00	0.07	0.07	0.06	0.06	0.07	0.07	0.07	0.05	0.05	0.06	0.06	0.05	0.05	0.06	0.05	0.05	*0.06 ± 0.02*
4	0.04	-0.00	0.00	0.04	0.08	0.09	0.09	0.08	0.08	0.08	0.07	0.06	0.06	0.06	0.06	0.06	0.07	0.05	0.06	*0.06 ± 0.02*
5	0.04	-0.00	0.00	0.04	0.08	0.09	0.09	0.08	0.08	0.08	0.07	0.06	0.06	0.06	0.06	0.06	0.07	0.05	0.06	*0.06 ± 0.02*
6b	0.14	-0.00	0.01	0.13	0.12	0.10	0.11	0.12	0.12	0.12	0.10	0.11	0.09	0.11	0.11	0.11	0.11	0.10	0.10	*0.10 ± 0.03*
7b	0.11	-0.00	0.16	0.16	0.16	0.16	0.17	0.20	0.21	0.18	0.17	0.17	0.17	0.17	0.17	0.16	0.16	0.16	0.14	*0.16 ± 0.04*
8b	**0.15**	-0.00	**0.24**	**0.28**	**0.26**	**0.26**	**0.28**	**0.31**	**0.32**	**0.29**	**0.26**	**0.26**	**0.27**	**0.26**	**0.27**	**0.25**	**0.23**	**0.23**	**0.23**	***0.24** ± 0.07*
Mean ± Std	*0.09 ± 0.05*	*-0.01 ± 0.02*	*0.05 ± 0.09*	*0.09 ± 0.09*	*0.10 ± 0.07*	*0.10 ± 0.07*	*0.11 ± 0.08*	*0.12 ± 0.09*	***0.12** ± 0.09*	*0.11 ± 0.08*	*0.10 ± 0.07*	*0.10 ± 0.08*	*0.10 ± 0.08*	*0.10 ± 0.07*	*0.10 ± 0.08*	*0.10 ± 0.07*	*0.10 ± 0.06*	*0.09 ± 0.07*	*0.09 ± 0.06*	*0.09 ± 0.08*

Table 1.36: Action baseline (ρ^{act}) — SURROUND.

Acronyms

ACU ***Adjusted Clustering Universality***
The Clustering Universality minus the similarity of the latent clusterings to the three trivial baselines of the Abstraction Score (see Section 6.3.5).

AI **Artificial Intelligence**
The field of computer science concerned with building systems that perform tasks usually associated with human intelligence, such as perception, reasoning, learning and decision-making.

APPO **Asynchronous Proximal Policy Optimization**
A variant of Proximal Policy Optimization in which data collection by the workers and policy updates run asynchronously, to scale training.

ARI **Adjusted Rand Index**
The Rand Index corrected for chance, equal to 0 for random partitions and to 1 for identical partitions (see Section 6.3.2).

AS ***Abstraction Score***
A metric measuring how far latent clusterings differ from those of the observations, of the action distribution and of untrained networks (see Section 6.3.4).

BC **Behavioral Cloning**
The training of a policy by supervised learning to reproduce the actions of another policy on a dataset of its interactions.

CAR **Clustering Agent Representation**
The method proposed in this thesis, which trains several surrogate policies of an agent and clusters their latent representations to detect concept structures (see Chapter 6).

CNN **Convolutional Neural Network**
A neural network whose layers apply learned convolution filters, suited to data with a spatial structure such as images.

CS **Concept Structure**
The specific form in which a concept is instantiated within the latent space, for instance a direction or a cluster (see Definition 3.1.5).

CU ***Clustering Universality***
The mean pairwise Adjusted Rand Index between the latent clusterings of independently trained surrogate policies (see Section 6.3.3).

DDPG **Deep Deterministic Policy Gradient**
An actor–critic reinforcement learning algorithm for continuous action spaces, which learns a deterministic policy.

DL **Deep Learning**
The family of machine learning methods based on neural networks with many successive layers, trained from data by gradient descent (see Chapter 2).

DQN **Deep Q-Network**
A reinforcement learning algorithm that approximates the action-value function with a deep neural network.

GPU **Graphics Processing Unit**
A processor specialized in massively parallel computation, used to accelerate the training of neural networks.

LLM **Large Language Model**
A neural network trained on large text corpora to predict the next token of a sequence, used for language understanding and generation.

LR **Latent Representation**
The internal encoding of an input given by an intermediate layer of a trained neural network (see Definition 3.1.2).

LS **Latent Space**
The set of all latent representations produced by a layer over a dataset (see Definition 3.1.3).

LSTM **Long Short-Term Memory**
A recurrent neural network architecture whose gating mechanism allows information to be retained over long sequences.

MDP **Markov Decision Process**
The mathematical framework modeling sequential decision-making as an agent interacting with an environment through states, actions, rewards and transitions (see Section 4.3.1).

ME **Mechanistic Explainability**
The explainability approach that seeks to explain the behavior of a neural network by analyzing its internal mechanisms (see Definition 1.2.2).

MLP **Multilayer Perceptron**
A neural network made of a sequence of fully connected layers.

MSE **Mean Squared Error**
A loss function equal to the average of the squared differences between predictions and targets.

NN **Neural Network**
A parameterized function made of successive layers of artificial neurons, whose parameters are learned from data.

PPO **Proximal Policy Optimization**
A policy-gradient reinforcement learning algorithm that stabilizes training by limiting the size of each policy update through a clipped objective.

RI **Rand Index**
A measure of the agreement between two partitions of the same set of points, computed over all pairs of points (see Section 6.3.2).

RL **Reinforcement Learning**
The learning paradigm in which an agent learns a policy by interacting with an environment so as to maximize the rewards it collects (see Chapter 4).

SAE **Sparse Autoencoder**
An autoencoder trained with a sparsity constraint, used to decompose latent representations into a larger set of more interpretable directions (see Section 3.4).

SGD **Stochastic Gradient Descent**
An optimization algorithm that updates the parameters of a model in the direction opposite to the gradient of the loss, estimated on a random subset of the training data (see Section 2.4).

XAI **Explainable Artificial Intelligence**
The subfield of artificial intelligence that develops methods to make the decisions of a model understandable to a human observer (see Section 5.1.2).

XRL **Explainable Reinforcement Learning**
The branch of explainability dedicated to reinforcement learning agents, whose decisions are sequential and depend on the interaction with an environment (see Chapter 5).

Bibliography

[1] Leonhard Euler. *Methodus inveniendi lineas curvas maximi minimive proprietate gaudentes*. English translation of the quoted passage from W. A. Oldfather, C. A. Ellis and D. M. Brown, Leonhard Euler's elastic curves, *Isis*, 20(1), 1933. Lausanne and Geneva: Marcum-Michaelem Bousquet, 1744. URL: link.

[2] Percy Bysshe Shelley. "Ozymandias". In: *The Examiner* (1818). URL: link.

[3] Friedrich Engels. *Herrn Eugen Dühring's Umwälzung der Wissenschaft*. Part I, Chapter XI. English translation of the quoted passage by E. Burns, *Anti-Dühring: Herr Eugen Dühring's Revolution in Science*. Leipzig, 1878. URL: link.

[4] Charles Sanders Peirce. "How to Make Our Ideas Clear". In: *Popular Science Monthly* 12 (1878), pp. 286–302. URL: link.

[5] Mark Twain. "Chapters from My Autobiography". In: *The North American Review* (1907). URL: link.

[6] Mikoláš Aleš and Adolf Wenig. *Věnec pražských pověstí*. Illustration of the Golem (1899), p. 39. Czech Digital Library. Prague: F. Topič, 1908. URL: link.

[7] Edward L. Thorndike. *Animal Intelligence: Experimental Studies*. New York: Macmillan, 1911. DOI: link. PDF: link.

[8] Karel Čapek. *R.U.R. (Rossum's Universal Robots)*. Prague: Aventinum, 1920. URL: link.

[9] Burrhus Frederic Skinner. *The behavior of organisms: An experimental analysis*. Appleton-Century, 1938. URL: link.

[10] Warren S McCulloch and Walter Pitts. "A logical calculus of the ideas immanent in nervous activity". In: *The Bulletin of Mathematical Biophysics* 5.4 (1943), pp. 115–133. DOI: link.

[11] Alan Turing. *Intelligent Machinery*. Report. National Physical Laboratory, 1948. URL: link.

[12] Richard Ernest Bellman. *Dynamic Programming*. Princeton, NJ: Princeton University Press, 1957.

[13] Frank Rosenblatt. “The perceptron: a probabilistic model for information storage and organization in the brain.” In: *Psychological Review* 65.6 (1958), pp. 386–408. DOI: link.

[14] Ronald A. Howard. *Dynamic Programming and Markov Processes*. Cambridge, MA: MIT Press, 1960.

[15] J MacQueen. “Some methods for classification and analysis of multivariate observations”. In: *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. Vol. 1. 1967, pp. 281–297. URL: link. PDF: link.

[16] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.

[17] William M Rand. “Objective criteria for the evaluation of clustering methods”. In: *Journal of the American Statistical Association* 66.336 (1971), pp. 846–850. DOI: link.

[18] Donald E Knuth and Ronald W Moore. “An analysis of alpha-beta pruning”. In: *Artificial Intelligence* 6.4 (1975), pp. 293–326. DOI: link.

[19] Douglas Adams. *The Hitchhiker’s Guide to the Galaxy*. London: Pan Books, 1979.

[20] Allen Newell. “Physical Symbol Systems”. In: *Cognitive Science* 4.2 (1980), pp. 135–183. DOI: link.

[21] Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson. “Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems”. In: *IEEE Transactions on Systems, Man, and Cybernetics* 13.5 (Sept. 1983), pp. 834–846. DOI: link. PDF: link.

[22] Lawrence Hubert and Phipps Arabie. “Comparing partitions”. In: *Journal of Classification* 2.1 (1985), pp. 193–218. DOI: link.

[23] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *Nature* 323.6088 (1986), pp. 533–536. DOI: link.

[24] Richard S. Sutton. “Learning to Predict by the Methods of Temporal Differences”. In: *Machine Learning* 3.1 (1988), pp. 9–44. DOI: link. PDF: link.

[25] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. “Multilayer feedforward networks are universal approximators”. In: *Neural Networks* 2.5 (1989), pp. 359–366. DOI: link.

[26] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. “Backpropagation applied to handwritten zip code recognition”. In: *Neural Computation* 1.4 (1989), pp. 541–551. DOI: link. PDF: link.

[27] Christopher J. C. H. Watkins. “Learning from Delayed Rewards”. PhD thesis. University of Cambridge, 1989. URL: link.

[28] Christopher J. C. H. Watkins and Peter Dayan. "Q-learning". In: *Machine Learning* 8.3–4 (1992), pp. 279–292. DOI: link.

[29] Norma Comrada. "Golem and robot: a search for connections". In: *Journal of the Fantastic in the Arts* 7.2/3 (1995), pp. 244–254. URL: link.

[30] Mamoru Oshii. *Ghost in the Shell*. Film, Production I.G. English translation of the quoted line from the official *Ghost in the Shell* account. 1995.

[31] Herbert A. Simon. *The Sciences of the Artificial*. 3rd ed. MIT Press, 1996.

[32] Sepp Hochreiter and Jürgen Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: link.

[33] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997. URL: link.

[34] John Tsitsiklis and Benjamin Van Roy. "Analysis of temporal-difference learning with function approximation". In: *IEEE Transactions on Automatic Control* 42.5 (1997), pp. 674–690. DOI: link. PDF: link.

[35] Lisa Nocks. "The Golem: between the technological and the divine". In: *Journal of Social and Evolutionary Systems* 21.3 (1998), pp. 281–303. DOI: link.

[36] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 1st. MIT Press, 1998.

[37] Freeman Dyson. "A meeting with Enrico Fermi". In: *Nature* 427.6972 (2004), p. 297. DOI: link.

[38] Rajat Raina, Anand Madhavan, and Andrew Y Ng. "Large-scale deep unsupervised learning using graphics processors". In: *Proceedings of the 26th International Conference on Machine Learning*. 2009, pp. 873–880. DOI: link. PDF: link.

[39] Stephane Ross and Drew Bagnell. "Efficient Reductions for Imitation Learning". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Yee Whye Teh and Mike Titterington. Vol. 9. Proceedings of Machine Learning Research. Chia Laguna Resort, Sardinia, Italy: PMLR, May 2010, pp. 661–668. URL: link.

[40] Adam Coates, Andrew Y. Ng, and Honglak Lee. "An Analysis of Single-Layer Networks in Unsupervised Feature Learning". In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*. 2011, pp. 215–223. URL: link.

[41] Martin Hilbert and Priscila López. "The world's technological capacity to store, communicate, and compute information". In: *Science* 332.6025 (2011), pp. 60–65. DOI: link.

[42] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning".

In: *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*. Ed. by Geoffrey Gordon, David Dunson, and Miroslav Dudík. Vol. 15. Proceedings of Machine Learning Research. Fort Lauderdale, FL, USA: PMLR, Apr. 2011, pp. 627–635. ArXiv: link. URL: link.

[43] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet classification with deep convolutional neural networks". In: *Advances in Neural Information Processing Systems*. Vol. 25. 2012. URL: link.

[44] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. "The Arcade Learning Environment: An Evaluation Platform for General Agents". In: *Journal of Artificial Intelligence Research* 47 (June 2013), pp. 253–279. DOI: link. ArXiv: link.

[45] Yoshua Bengio, Aaron Courville, and Pascal Vincent. "Representation Learning: A Review and New Perspectives". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8 (2013), pp. 1798–1828. DOI: link. ArXiv: link.

[46] Cheryl Misak. *The American Pragmatists*. Oxford University Press, 2013.

[47] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. *Playing Atari with Deep Reinforcement Learning*. 2013. ArXiv: link.

[48] Jonathon Shlens. *Notes on Kullback-Leibler Divergence and Likelihood*. 2014. ArXiv: link.

[49] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. 2014. ArXiv: link.

[50] Matthew Hausknecht and Peter Stone. "Deep Recurrent Q-Learning for Partially Observable MDPs". In: *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*. 2015. ArXiv: link.

[51] Apache Arrow. *Apache Arrow and Parquet*. 2016. URL: link.

[52] Paul Covington, Jay Adams, and Emre Sargin. "Deep neural networks for YouTube recommendations". In: *Proceedings of the 10th ACM Conference on Recommender Systems*. 2016, pp. 191–198. DOI: link.

[53] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. Cambridge, MA: MIT Press, 2016. URL: link.

[54] Timothy P. Lillicrap, Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. "Continuous control with deep reinforcement learning". In: *International Conference on Learning Representations*. 2016. ArXiv: link.

[55] Matus Telgarsky. "Benefits of depth in neural networks". In: *Proceedings of the 29th Conference on Learning Theory*. Vol. 49. Proceedings of Machine Learning Research. 2016. ArXiv: link. URL: link.

[56] Junyuan Xie, Ross Girshick, and Ali Farhadi. "Unsupervised Deep Embedding for Clustering Analysis". In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. 2016. ArXiv: link. URL: link.

[57] Tom Zahavy, Nir Ben Zrihem, and Shie Mannor. "Graying the black box: Understanding DQNs". In: *Proceedings of the 33rd International Conference on Machine Learning*. Vol. 48. Proceedings of Machine Learning Research. 2016. ArXiv: link. URL: link.

[58] Kai Arulkumaran, Marc Peter Deisenroth, Miles Brundage, and Anil Anthony Bharath. "Deep Reinforcement Learning: A Brief Survey". In: *IEEE Signal Processing Magazine* 34.6 (2017), pp. 26–38. DOI: link. ArXiv: link.

[59] Finale Doshi-Velez and Been Kim. *Towards A Rigorous Science of Interpretable Machine Learning*. 2017. ArXiv: link.

[60] Xifeng Guo, Long Gao, Xinwang Liu, and Jianping Yin. "Improved Deep Embedded Clustering with Local Structure Preservation". In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence*. International Joint Conferences on Artificial Intelligence Organization, 2017, pp. 1753–1759. DOI: link.

[61] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. "Neural Collaborative Filtering". In: *Proceedings of the 26th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 2017, pp. 173–182. DOI: link. ArXiv: link.

[62] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. 2017. ArXiv: link.

[63] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. "Attention is all you need". In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017. ArXiv: link. URL: link.

[64] Lasse Espeholt et al. "IMPALA: Scalable Distributed Deep-RL with Importance Weighted Actor-Learner Architectures". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. 2018. ArXiv: link. URL: link.

[65] Sam Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. "Visualizing and Understanding Atari Agents". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. 2018. ArXiv: link. URL: link. Code: link.

[66] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti, and Dino Pedreschi. "A Survey of Methods for Explaining Black Box Models". In: *ACM Computing Surveys* 51.5 (2018), pp. 1–42. DOI: link. ArXiv: link.

[67] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. "Rainbow: Combining Improvements in Deep Reinforcement Learning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 32. 1. 2018. DOI: link. ArXiv: link.

[68] Arthur Jacot, Franck Gabriel, and Clément Hongler. "Neural Tangent Kernel: Convergence and Generalization in Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018. ArXiv: link. URL: link.

[69] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph E. Gonzalez, Michael I. Jordan, and Ion Stoica. "RLlib: Abstractions for Distributed Reinforcement Learning". In: *Proceedings of the 35th International Conference on Machine Learning*. Vol. 80. Proceedings of Machine Learning Research. 2018. ArXiv: link. URL: link.

[70] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. "Revisiting the Arcade Learning Environment: Evaluation Protocols and Open Problems for General Agents". In: *Journal of Artificial Intelligence Research* 61 (2018), pp. 523–562. DOI: link. ArXiv: link.

[71] Philipp Moritz et al. "Ray: A Distributed Framework for Emerging AI Applications". In: *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 2018, pp. 561–577. ArXiv: link. URL: link.

[72] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. "A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play". In: *Science* 362.6419 (2018), pp. 1140–1144. DOI: link. ArXiv: link.

[73] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2nd. MIT Press, 2018. URL: link.

[74] Benjamin Beyret, Ali Shafti, and A. Aldo Faisal. "Dot-to-Dot: Explainable Hierarchical Reinforcement Learning for Robotic Manipulation". In: *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Nov. 2019, pp. 5014–5019. DOI: link. ArXiv: link.

[75] Yuri Burda, Harri Edwards, Deepak Pathak, Amos Storkey, Trevor Darrell, and Alexei A. Efros. "Large-Scale Study of Curiosity-Driven Learning". In: *International Conference on Learning Representations*. 2019. ArXiv: link. URL: link.

[76] William Falcon. *PyTorch Lightning*. 2019. DOI: link. URL: link.

[77] David Gunning and David Aha. "DARPA's Explainable Artificial Intelligence (XAI) Program". In: *AI Magazine* 40.2 (2019), pp. 44–58. DOI: link.

[78] Tobias Huber, Dominik Schiller, and Elisabeth André. "Enhancing explainability of deep reinforcement learning through selective layer-wise

relevance propagation". In: *KI 2019: Advances in Artificial Intelligence. 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings*. Springer. 2019, pp. 188–202. DOI: link.

[79] Zoe Juozapaitis, Anurag Koul, Alan Fern, Martin Erwig, and Finale Doshi-Velez. "Explainable reinforcement learning via reward decomposition". In: *IJCAI/ECAI Workshop on explainable artificial intelligence*. 2019. URL: link.

[80] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. "Recurrent Experience Replay in Distributed Reinforcement Learning". In: *International Conference on Learning Representations*. 2019. URL: link.

[81] Guo Lu, Wanli Ouyang, Dong Xu, Xiaoyun Zhang, Chunlei Cai, and Zhiyong Gao. "DVC: An End-to-end Deep Video Compression Framework". In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2019, pp. 10998–11007. DOI: link. ArXiv: link.

[82] Tim Miller. "Explanation in Artificial Intelligence: Insights from the Social Sciences". In: *Artificial Intelligence* 267 (2019), pp. 1–38. DOI: link. ArXiv: link.

[83] Alexander Mott, Daniel Zoran, Mike Chrzanowski, Daan Wierstra, and Danilo Jimenez Rezende. "Towards interpretable reinforcement learning using attention augmented agents". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. ArXiv: link. URL: link.

[84] OpenAI et al. *Dota 2 with Large Scale Deep Reinforcement Learning*. 2019. ArXiv: link.

[85] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019. ArXiv: link. URL: link.

[86] Ali Payani and Faramarz Fekri. *Inductive Logic Programming via Differentiable Deep Neural Logic Networks*. 2019. ArXiv: link. Code: link.

[87] Gabriel Peyré and Marco Cuturi. "Computational Optimal Transport: With Applications to Data Science". In: *Foundations and Trends in Machine Learning* 11.5–6 (2019), pp. 355–607. DOI: link. ArXiv: link.

[88] Cynthia Rudin. "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead". In: *Nature Machine Intelligence* 1.5 (2019), pp. 206–215. DOI: link. ArXiv: link.

[89] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. "The StarCraft Multi-Agent Challenge". In: *Proceedings of the 18th International Conference on Autonomous Agents and MultiAgent Systems*. 2019, pp. 2186–2188. DOI: link. ArXiv: link.

[90] Laurens Weitkamp, Elise van der Pol, and Zeynep Akata. "Visual Rationalizations in Deep Reinforcement Learning for Atari Games". In: *Artificial Intelligence*. Communications in Computer and Information Science. Springer International Publishing, 2019, pp. 151–165. DOI: link. ArXiv: link.

[91] Robert A Amezquita, Aaron TL Lun, Etienne Becht, Vince J Carey, Lindsay N Carpp, Ludwig Geistlinger, Federico Marini, Kevin Rue-Albrecht, Davide Risso, Charlotte Soneson, et al. "Orchestrating single-cell analysis with Bioconductor". In: *Nature Methods* 17.2 (2020), pp. 137–145. DOI: link.

[92] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. "Emergent Tool Use From Multi-Agent Autocurricula". In: *International Conference on Learning Representations*. 2020. ArXiv: link. URL: link. Video: link.

[93] Alejandro Barredo Arrieta et al. "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". In: *Information Fusion* 58 (2020), pp. 82–115. DOI: link. ArXiv: link.

[94] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. "Language models are few-shot learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. ArXiv: link. URL: link.

[95] Damien Garreau and Ulrike von Luxburg. "Explaining the Explainer: A First Theoretical Analysis of LIME". In: *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*. Ed. by Silvia Chiappa and Roberto Calandra. Vol. 108. Proceedings of Machine Learning Research. PMLR, Aug. 2020, pp. 1287–1296. ArXiv: link. URL: link.

[96] Victoria Krakovna, Jonathan Uesato, Vladimir Mikulik, Matthew Rahtz, Tom Everitt, Ramana Kumar, et al. *Specification Gaming: The Flip Side of AI Ingenuity*. 2020. URL: link.

[97] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. ArXiv: link. URL: link.

[98] Prashan Madumal, Tim Miller, Liz Sonenberg, and Frank Vetere. "Explainable reinforcement learning through a causal lens". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 34. 03. 2020, pp. 2493–2500. DOI: link. ArXiv: link.

[99] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. "Zoom In: An Introduction to Circuits". In: *Distill* 5.3 (2020). DOI: link.

[100] Emilio Parisotto et al. "Stabilizing Transformers for Reinforcement Learning". In: *Proceedings of the 37th International Conference on Machine*

Learning. Vol. 119. Proceedings of Machine Learning Research. 2020, pp. 7487–7498. ArXiv: link. URL: link.

[101] Erika Puiutta and Eric M. S. P. Veith. "Explainable Reinforcement Learning: A Survey". In: *International Cross-Domain Conference for Machine Learning and Knowledge Extraction (CD-MAKE)*. Springer. 2020, pp. 77–95. DOI: link. ArXiv: link.

[102] Pedro Sequeira and Melinda Gervasio. "Interestingness elements for explainable reinforcement learning: Understanding agents' capabilities and limitations". In: *Artificial Intelligence* 288 (Nov. 2020), p. 103367. DOI: link. ArXiv: link.

[103] Andrey Voynov and Artem Babenko. "Unsupervised Discovery of Interpretable Directions in the GAN Latent Space". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. 2020. ArXiv: link. URL: link.

[104] Matthieu Bellucci, Nicolas Delestre, Nicolas Malandain, and Cecilia Zanni-Merk. "Towards a terminology for a fully contextualized XAI". In: *Procedia Computer Science* 192 (2021), pp. 241–250. DOI: link.

[105] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. *Evaluating Large Language Models Trained on Code*. 2021. ArXiv: link.

[106] Hidenori Itaya, Tsubasa Hirakawa, Takayoshi Yamashita, Hironobu Fujiyoshi, and Komei Sugiura. "Visual Explanation using Attention Mechanism in Actor-Critic-based Deep Reinforcement Learning". In: *2021 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 2021, pp. 1–10. DOI: link. ArXiv: link.

[107] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, et al. "Highly accurate protein structure prediction with AlphaFold". In: *Nature* 596.7873 (2021), pp. 583–589. DOI: link.

[108] Matthew L Olson, Roli Khanna, Lawrence Neal, Fuxin Li, and Weng-Keen Wong. "Counterfactual state explanations for reinforcement learning agents via generative deep learning". In: *Artificial Intelligence* 295 (2021), p. 103455. DOI: link. ArXiv: link. Code: link.

[109] Alec Radford et al. "Learning Transferable Visual Models From Natural Language Supervision". In: *Proceedings of the 38th International Conference on Machine Learning*. Vol. 139. Proceedings of Machine Learning Research. 2021. ArXiv: link. URL: link.

[110] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th. Pearson, 2021.

[111] Alexander Sieusahai and Matthew Guzdial. "Explaining Deep Reinforcement Learning Agents In The Atari Domain through a Surrogate Model". In: *Proceedings of the AAAI Conference on Artificial Intelligence and In-*

teractive Digital Entertainment 17.1 (2021), pp. 82–90. DOI: link. ArXiv: link.

[112] David Silver, Satinder Singh, Doina Precup, and Richard S Sutton. "Reward is enough". In: *Artificial Intelligence* 299 (2021), p. 103535. DOI: link.

[113] Nicholay Topin, Stephanie Milani, Fei Fang, and Manuela Veloso. "Iterative bounding MDPs: Learning interpretable policies via non-interpretable methods". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 11. 2021, pp. 9923–9931. DOI: link. ArXiv: link.

[114] Dweep Trivedi, Jesse Zhang, Shao-Hua Sun, and Joseph J. Lim. "Learning to Synthesize Programs as Interpretable and Generalizable Policies". In: *Advances in Neural Information Processing Systems*. Vol. 34. 2021. ArXiv: link. URL: link. Code: link.

[115] Carlos Zednik. "Solving the black box problem: A normative framework for explainable artificial intelligence". In: *Philosophy & Technology* 34.2 (2021), pp. 265–288. DOI: link. ArXiv: link.

[116] Sam Bond-Taylor, Adam Leach, Yang Long, and Chris G Willcocks. "Deep generative modelling: A comparative review of VAEs, GANs, normalizing flows, energy-based and autoregressive models". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44.11 (2022), pp. 7327–7347. DOI: link. ArXiv: link.

[117] Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, et al. "Toy Models of Superposition". In: *Transformer Circuits Thread* (2022). ArXiv: link. URL: link.

[118] K. Kawaguchi, Y. Bengio, and L. Kaelbling. "Generalization in Deep Learning". In: *Mathematical Aspects of Deep Learning*. Cambridge University Press, Dec. 2022, pp. 112–148. DOI: link. ArXiv: link.

[119] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in Neural Information Processing Systems*. Vol. 35. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022, pp. 27730–27744. DOI: link. ArXiv: link.

[120] Daniel A. Roberts, Sho Yaida, and Boris Hanin. *The Principles of Deep Learning Theory*. Cambridge University Press, 2022. DOI: link. ArXiv: link.

[121] Sanjit A. Seshia, Dorsa Sadigh, and S. Shankar Sastry. "Toward Verified Artificial Intelligence". In: *Communications of the ACM* 65.7 (2022), pp. 46–55. DOI: link. ArXiv: link.

[122] Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: link.

[123] Jiaming Ji, Tianyi Qiu, Boyuan Chen, Borong Zhang, Hantao Lou, Kaile Wang, et al. *AI Alignment: A Comprehensive Survey*. 2023. ArXiv: link.

[124] Kenneth Li, Aspen K. Hopkins, David Bau, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. "Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task". In: *The Eleventh International Conference on Learning Representations*. 2023. ArXiv: link. URL: link.

[125] Emanuele Marconato, Andrea Passerini, and Stefano Teso. "Interpretability is in the Mind of the Beholder: A Causal Framework for Human-interpretable Representation Learning". In: *Entropy* 25.12 (2023), p. 1574. DOI: link. ArXiv: link.

[126] Ulisse Mini, Peli Grietzer, Mrinank Sharma, Austin Meek, Monte MacDiarmid, and Alexander Matt Turner. *Understanding and Controlling a Maze-Solving Policy Network*. 2023. ArXiv: link.

[127] Randall Munroe. *K-Means Clustering*. xkcd, no. 2731. 2023. URL: link.

[128] RAPIDS. *RAPIDS cuML: GPU-Accelerated Machine Learning*. 2023. URL: link.

[129] Maxime Alaarabiou, Nicolas Delestre, and Laurent Vercouter. "Explicabilité en Apprentissage par Renforcement : vers une Taxinomie Unifiée". In: *18èmes Journées d'Intelligence Artificielle Fondamentale*. La Rochelle, France, July 2024. URL: link.

[130] Leonard Bereska and Efstratios Gavves. "Mechanistic Interpretability for AI Safety – A Review". In: *Transactions on Machine Learning Research* (2024). ArXiv: link. URL: link.

[131] Ryan Greenblatt, Carson Denison, Benjamin Wright, Fabien Roger, Monte MacDiarmid, Sam Marks, et al. *Alignment Faking in Large Language Models*. 2024. ArXiv: link.

[132] René Haas, Inbar Huberman-Spiegelglas, Rotem Mulayoff, Stella Graßhof, Sami S Brandt, and Tomer Michaeli. "Discovering interpretable directions in the semantic latent space of diffusion models". In: *2024 IEEE 18th International Conference on Automatic Face and Gesture Recognition (FG)*. IEEE. 2024, pp. 1–9. DOI: link. ArXiv: link.

[133] Adam Jermyn, Adly Templeton, Joshua Batson, and Trenton Bricken. *Tanh Penalty in Dictionary Learning*. Transformer Circuits Thread. 2024. URL: link.

[134] Adam Karvonen. "Emergent World Models and Latent Variable Estimation in Chess-Playing Language Models". In: *First Conference on Language Modeling*. 2024. ArXiv: link. URL: link.

[135] Alexander Meinke, Bronson Schoen, Jérémy Scheurer, Mikita Balesni, Rusheb Shah, and Marius Hobbhahn. *Frontier Models are Capable of In-context Scheming*. 2024. ArXiv: link.

[136] Stephanie Milani, Nicholay Topin, Manuela Veloso, and Fei Fang. "Explainable Reinforcement Learning: A Survey and Comparative Review". In: *ACM Computing Surveys* 56.7 (2024), pp. 1–36. DOI: link. ArXiv: link.

[137] Zhihong Shao et al. *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models*. 2024. ArXiv: link.

[138] Gabriel Siegel and Carl F. Craver. "Phenomenological Laws and Mechanistic Explanations". In: *Philosophy of Science* 91.1 (2024), pp. 132–150. DOI: link.

[139] Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, et al. "Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet". In: *Transformer Circuits Thread* (2024). ArXiv: link. URL: link.

[140] Xiuxi Wei, Zhihui Zhang, Huajuan Huang, and Yongquan Zhou. "An overview on deep clustering". In: *Neurocomputing* 590 (2024), p. 127761. DOI: link.

[141] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. "Mastering diverse control tasks through world models". In: *Nature* 640.8059 (2025), pp. 647–653. DOI: link. ArXiv: link.

[142] Lukas Miklautz, Timo Klein, Kevin Sidak, Collin Leiber, Thomas Lang, Andrii Shkabrii, Sebastian Tschiatschek, and Claudia Plant. "Breaking the Reclustering Barrier in Centroid-based Deep Clustering". In: *The Thirteenth International Conference on Learning Representations*. 2025. ArXiv: link. URL: link.

[143] Yazhou Ren, Jingyu Pu, Zhimeng Yang, Jie Xu, Guofeng Li, Xiaorong Pu, Philip S. Yu, and Lifang He. "Deep Clustering: A Comprehensive Survey". In: *IEEE Transactions on Neural Networks and Learning Systems* 36.4 (2025), pp. 5858–5878. DOI: link. ArXiv: link.

[144] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. "Gymnasium: A Standard Interface for Reinforcement Learning Environments". In: *Advances in Neural Information Processing Systems*. Vol. 38. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2025, pp. 163114–163129. DOI: link. ArXiv: link.

[145] Maxime Alaarabiou, Nicolas Delestre, and Laurent Vercouter. "A Unified Taxonomy for Explainability in Reinforcement Learning". KES 2026 – 30th International Conference on Knowledge Based and Intelligent Information and Engineering Systems. 2026.

[146] Maxime Alaarabiou, Nicolas Delestre, and Laurent Vercouter. "Clustering Agent Representation for Explaining Reinforcement Learning Policies". KES 2026 – 30th International Conference on Knowledge Based and Intelligent Information and Engineering Systems. 2026.

Institutional Acknowledgements

This thesis was made possible by several institutions. We thank the French State for its support of public research. We thank the Normandy Region, which funded this thesis. We thank Normandie Université, to which the doctoral school of this thesis belongs. We thank the doctoral school ED 590 MIIS, which ensured the administrative management of this thesis. We thank the Université de Rouen Normandie, which employed the author throughout this thesis. We thank INSA Rouen Normandie, which hosted this work in its premises. We thank the LITIS laboratory, whose members provided the scientific supervision of this thesis. Finally, the experiments presented in this thesis were performed using the computing resources of the CRIANN, which are gratefully acknowledged.

















Mechanistic Explainability in Reinforcement Learning

Maxime Alaarabiou

Résumé

Les agents d'apprentissage par renforcement profond atteignent de hautes performances sur un large éventail de tâches. Cependant, les réseaux de neurones qui implémentent leurs politiques sont difficiles à interpréter en raison de leur nature connexionniste. Cette difficulté vient de ce que l'information est distribuée dans l'ensemble du réseau, ce qui rend difficile de relier ses composants internes aux décisions de l'agent.

Dans cette thèse, l'explicabilité de ces agents est obtenue par l'analyse de leurs espaces latents. Cette démarche s'inscrit dans une perspective mécaniste, qui cherche à expliquer les décisions des agents en analysant les représentations internes apprises par leurs réseaux de neurones.

La thèse commence par passer en revue les méthodes de l'état de l'art pour l'analyse des espaces latents, afin de donner une vue d'ensemble du domaine. Elle précise ensuite ce qui constitue une explication dans le contexte de l'apprentissage par renforcement explicable. Sur ces bases, elle passe en revue l'état de l'art de l'apprentissage par renforcement explicable à l'aide d'une taxonomie spécifiquement adaptée à l'apprentissage par renforcement.

En s'appuyant sur ces deux revues, la thèse introduit *Clustering Agent Representation*, une méthode d'explicabilité qui analyse l'espace latent des politiques d'apprentissage par renforcement afin d'identifier des structures significatives dans leurs représentations internes.

Pour évaluer objectivement les structures obtenues, trois métriques sont proposées. *Clustering Universality* mesure leur cohérence entre des politiques entraînées indépendamment, ce qui établit que les structures identifiées ne sont pas dues au hasard. *Abstraction Score* mesure leur écart à la structure des observations, à celle des actions et à celle d'un réseau non entraîné, ce qui montre que ces structures émergent de l'apprentissage. *Adjusted Clustering Universality* combine ces deux mesures pour fournir un critère de choix des représentations à analyser.

Les expériences menées sur six environnements Atari montrent que des politiques entraînées indépendamment convergent vers des structures de clustering similaires. Comme cette similarité ne s'explique ni par les observations, ni par les actions, ni par l'architecture du réseau, elle indique une convergence des concepts appris par ces politiques. Enfin, une analyse qualitative montre que les clusters obtenus correspondent à des aspects interprétables de la tâche.

Ces résultats montrent que l'analyse des espaces latents est une voie prometteuse pour expliquer les agents d'apprentissage par renforcement, et ouvrent de nouvelles perspectives pour leur compréhension mécaniste.

Mots-clés : Intelligence Artificielle Explicable, Apprentissage Profond, Apprentissage par Renforcement, Explicabilité, Explicabilité Mécanistique, Interprétabilité Mécanistique, Espace Latent, Clustering, Regroupement.

Mechanistic Explainability in Reinforcement Learning

Maxime Alaarabiou

Abstract

Deep reinforcement learning agents achieve strong performance across a wide range of tasks. However, the neural networks that implement their policies are difficult to interpret due to their connectionist nature. This difficulty stems from the fact that information is distributed throughout the network, making it hard to relate internal components to the agent's decisions.

In this thesis, explainability for such agents is achieved through the analysis of their latent spaces. This places the approach within a mechanistic perspective, which seeks to explain agent decisions by analyzing the internal representations learned by their neural networks.

The thesis first reviews state-of-the-art methods for analyzing latent spaces, providing an overview of the field. It then clarifies what constitutes an explanation in the context of explainable reinforcement learning. With these foundations in place, it reviews the state of the art in explainable reinforcement learning through a taxonomy specifically adapted to reinforcement learning.

Building on these two reviews, the thesis introduces *Clustering Agent Representation*, an explainability method that analyzes the latent space of reinforcement learning policies to identify meaningful structures in their internal representations.

To objectively evaluate the recovered structures, three metrics are proposed. *Clustering Universality* measures their consistency across independently trained policies, establishing that the identified structures are not due to chance. *Abstraction Score* measures their departure from the structure of the observations, the actions, and an untrained network, showing that these structures emerge from learning. *Adjusted Clustering Universality* combines these two measures to provide a criterion for selecting which representations to analyze.

Experiments conducted on six Atari environments show that independently trained policies converge toward similar clustering structures. Since this similarity is explained neither by the observations, nor by the actions, nor by the architecture of the network, it points to a convergence of the concepts learned by these policies. Finally, a qualitative analysis shows that the recovered clusters correspond to interpretable aspects of the task.

These results show that analyzing latent spaces is a promising path toward explaining reinforcement learning agents, opening new perspectives for their mechanistic understanding.

Keywords: Explainable Artificial Intelligence, Deep Learning, Reinforcement Learning, Explainability, Mechanistic Explainability, Mechanistic Interpretability, Latent Space, Clustering.